

Agent Harness：实践者教材

这是一份基于事实的 *agent harness* 工程教材。关键事实的引用放在正文对应位置，而不只集中在参考文献中。

24 sections · generated from Markdown sources

前言

本书讨论的是：当语言模型真正开始执行任务时，围绕它运行的多层系统。接下来的章节会把 agent harness 资料串联成一条工程主线。关键事实会在正文对应位置给出来源，方便读者核对证据及其适用范围。

这个领域建立在一个简单的前提之上。LangChain 的 Vivek Trivedy 将其概括为：“Agent = Model + Harness。如果你不是模型，那你就是 harness。”(LangChain - The Anatomy of an Agent Harness)。这句话适合划分责任，但粒度有意保持得很粗。本书会进一步区分紧邻模型的 agent harness、持久化运行时、面向用户的产品、fleet platform 或 control plane，以及独立的 evaluation harness。Anthropic 也采用了这种较窄的区分：agent harness 负责处理输入和编排工具调用，evaluation harness 则负责运行 trial、记录过程、执行评分并汇总结果 (Anthropic - Demystifying Evals for AI Agents)。

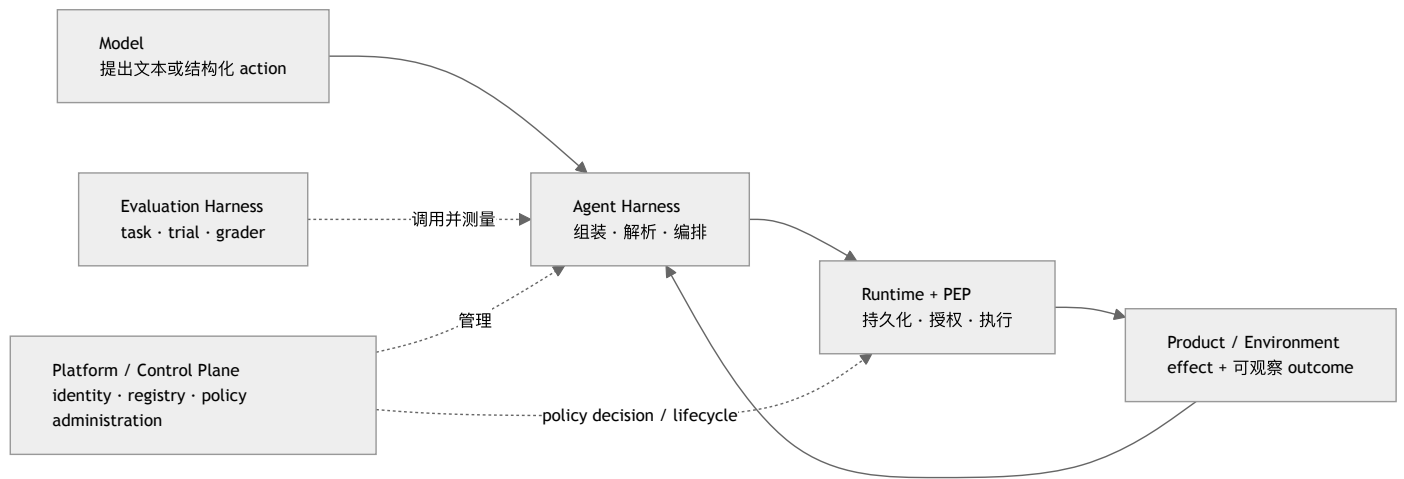
阅读本书时，可以把模型理解为一个接收当前输入表示、再生成文本或结构化输出的组件。工具调用只是提议，不是已经发生的副作用：应用代码负责验证并执行指定工具，再把 tool result 返回给模型 (Anthropic - How Tool Use Works)。正因为模型输出不等于现实世界中的实际效果，后续章节才会重点讨论上下文、工具、状态、权限、测试、沙箱和评估。

配套的第一卷已经介绍了本书所依赖的模型行为，包括 token、attention、单次请求内的 KV cache、provider prompt caching、context window、采样、检索，以及 tool calling 的模型侧边界。第一卷最后给出了六类责任边界：模型负责预测；外部系统负责上下文选择、持久状态、工具执行、权限、验证和后果 (《LLM Foundations》第 14 章)。本卷负责实现这些外部责任，不再重新推导模型机制。

开始之前

本书是两卷系列中的第二卷。我们假定读者已经读过 LLM Foundations for Harness Engineering，并熟悉 token、attention、context window、采样、post-training、检索、模型生成的 tool call、prompt injection，以及 pass@k/pass^k。遇到第一卷已经讲过的概念时，本书会指回对应章节，而不再从头推导。没有读过第一卷的读者仍可理解本书主线，但应把这些简短回顾视为进一步阅读的入口，而不是完整讲解。Provider 价格、缓存生命周期、模型能力和 benchmark 分数都会变化；它们在本书中出现时，都应被理解为带日期的案例，而不是永久定义。

核心公式



要点

- 单次语言模型调用不拥有持久化应用状态，不执行真实工具，也不强制权限；这些能力由外部软件提供。
- Harness engineering 不等同于 prompt engineering：它改进的是整个系统，而不只是单个提示词。
- “Harness” 是一种宽泛的责任缩写；当架构差异重要时，后续章节会区分 agent harness、runtime、product、platform 和 evaluation harness。
- 时变的 provider 与产品事实会标成带日期的案例，而不是写成永久定义。
- 关键事实在章节正文中直接引用来源，方便读者检查原始适用范围。

延伸阅读

- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Anthropic, *How Tool Use Works*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>
- *Awesome Harness Engineering* reading list: <https://github.com/walkinglabs/awesome-harness-engineering>

第 1 章：什么是 Agent Harness?

1.1 从有用的简称到精确的边界

配套的《LLM Foundations》把 **harness** 有意用作模型外部系统的粗粒度简称。在这个粒度上，下面的核心公式很有用：

Agent 的行为 = 模型的行为 + 周边系统的行为

模型接收为当前一次推理组装好的表示，并生成 token 或结构化输出。模型本身不会保存持久的应用状态、授予权限、执行 shell 命令、写入数据库记录，也不会验证所请求的现实世界变化是否真的发生。这些责任位于模型之外，属于**模型—系统责任边界**。

但在设计生产系统时，“模型之外的一切”范围太宽，因此本书需要更细的词汇。例如，Anthropic 把 **agent harness**（也称 scaffold）定义为使模型能够作为 agent 行动的系统：它处理输入、编排工具调用并返回结果；同时又单独定义了围绕该系统创建并运行测试的 **evaluation harness**（[Anthropic - Demystifying Evals for AI Agents](#)）。本书采用这一区分，并在下文继续划分 runtime、product 和 platform 层。

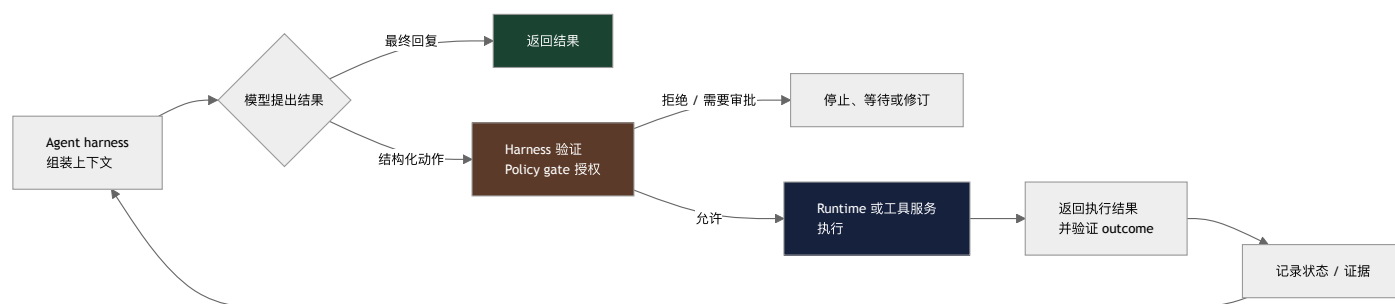
因此，本书用 **agent harness** 指紧邻模型的循环控制系统；当一项论断涉及更大范围时，则使用**周边系统（surrounding system）**。“模型 + harness”这一粗粒度公式仍然是有用的教学工具，但它并不表示所有外部责任都由同一个进程、软件库或团队承担。

1.2 最小 Agent 循环：提议不等于效果

《LLM Foundations》已经介绍了模型侧的工具使用与最小循环（[Foundations 第 12 章](#)）。这里需要进一步明确工程边界：

1. Agent harness 从指令、筛选后的历史或记忆、工具 schema、检索数据和当前状态中**组装上下文**。
2. 模型**提出**最终回复，或提出工具调用等结构化动作。
3. Harness **解析并验证**这一提议，再把它送入所需的授权或审批关卡。
4. Runtime 或工具服务**执行**获准的动作。
5. 周边系统接收执行结果；如果动作后果重要，还要检查最终的环境状态或业务 outcome。
6. 系统记录状态并组装下一次模型输入，或者结束本次 run。

这是一项实际的接口契约，而不只是比喻。在 Anthropic 文档描述的客户端工具循环中，模型发出结构化的 `tool_use` 请求，应用代码执行操作，再把 `tool_result` 返回对话。由 provider 执行的工具只是把 executor 移到了 provider 的基础设施上，并没有让模型本身变成 executor（[Anthropic - How Tool Use Works](#)）。



真正关键的边界位于**提议（proposal）与效果（effect）**之间。通过 schema 验证的请求并不自动拥有执行权限，成功的 API 响应也不一定能证明预期 outcome 已经发生。模型可以解释、请求或建议一项动作，但不能自行授予权限。后续章节会把 validation、authorization、execution 和 outcome confirmation 分别落成具体设计。

1.3 上下文不必单调增长

在最简单的 append-only 实现中，每轮循环都会向 transcript 加入模型请求和工具结果，因此 transcript 会单调增长。但生产 harness 不必在每次调用时重新发送全部原始 transcript。它可以裁剪消息、把 artifact 移出上下文、压缩旧内容、重置 context、检索部分 memory，或只在需要时暴露工具。Anthropic 把 context engineering 描述为整理推理期间可用的 token 集合，而不只是不断累积对话文本（[Anthropic - Effective Context Engineering for AI Agents](#)）。

因此，准确的不变量不是“上下文始终增长”，而是一次 run 会持续产生需要管理的信息。下一次 context 是对这些信息筛选和序列化后的视图。筛选与压缩可能丢失证据或改变行为，所以系统必须在模型输入之外保存持久状态、artifact、provenance 和尚未解决的不确定性。Context 是一次调用的有限输入，而不是系统数据库或 event history；其底层机制见 [Foundations 第 9 章](#)。

1.4 本书使用的六个系统层级

下列层级是本书采用的架构约定。真实产品可能把几个层级合并到一个服务中，也可能把一个层级拆给多个 vendor，但这些责任仍然可以区分。

层级	本书中的定义	典型责任
Model	接收当前表示并生成 token 或结构化输出的概率组件	语言与多模态推理、动作提议、分类、计划
Agent harness	组装输入、解析输出并驱动一个或多个 agent loop 的模型邻近系统	Prompt/context 组装、工具暴露、循环控制、handoff 协调
Runtime	让工作可持久执行，并负责运行获准操作的执行基础	调度、队列、checkpoint、retry 语义、沙箱、worker 生命周期
Product / application	Agent 所嵌入的用户侧 workflow 与领域系统	UX、领域逻辑、业务状态、review surface、用户沟通
Platform / control plane	管理多 tenant、多版本、多 agent 和多 runtime 的管理层	Registry、identity、policy 管理与决策、rollout、fleet 生命周期
Evaluation harness	创建 trial、调用被测系统、采集证据并应用 grader 的测试系统	任务环境、隔离、多次 trial、transcript、outcome、报告

在每一种部署中，这些层级都不一定表现为同心结构。例如，授权决策可能来自 platform 服务，但由 runtime 中的 enforcement point 阻止不允许的动作。同样，由 provider 执行的工具可以提供一部分 runtime 能力，而 workflow 仍归 product 所有。

Framework 与这些层级不同：它是用于实现一个或多个层级的一组抽象或软件库。把某个组件称为 framework，说明的是开发者如何用它构建系统，而不是它拥有哪项生产责任。

1.5 六类外部责任分别归谁所有？

Foundations 卷在结尾给出了必须位于模型预测之外的六类责任：context、state、tools、permissions、verification 和 consequences。按照本书采用的更细粒度，它们的归属如下：

责任	周边系统中的主要 owner	模型的合理角色
Context	Agent harness 筛选并序列化模型可见的视图	使用所提供的上下文；指出可能还需要哪些信息
State	Runtime 保存 execution state；product 拥有领域和业务状态	提议状态变更；绝不能充当唯一的持久记录
Tools	Agent harness 暴露契约；runtime 或工具服务执行	选择工具并提出参数
Permissions	Product/platform policy 作出决策；runtime 中不可绕过的 enforcement point 负责落实	请求访问或解释意图；绝不能自行授予权限
Verification	Runtime、product 和 evaluation harness 采用测试、环境检查、grader 或人工 review	提供 critique 或假设，但只作为一种可能出错的信号
Consequences	Runtime 与接入的外部系统产生并记录实际效果；product 负责解释其业务含义	预测或描述预期效果

在 Foundations 教材的粗粒度边界上，说“harness 拥有这些责任”是正确的。本书通过这张表进一步避免这个简称掩盖真正需要实现某项保证的子系统。

1.6 Agent Harness 与 Evaluation Harness 是不同系统

Agent harness 是被评估系统的一部分。**Evaluation harness** 则在测试期间包围这个系统：它配置任务与环境，调用 model 与 agent harness 的组合，记录一次 trial，检查 transcript 或最终环境状态，应用 grader，再汇总结果。Anthropic 的评估术语也作出了相同区分，并指出评估一个“agent”时，实际评估的是模型与其 agent harness 的组合（Anthropic - Demystifying Evals for AI Agents）。

这一区分在运营中很重要。生产 trace 可以为评估提供证据，但生产 observability 并不会自动构成 evaluation harness。反过来，测试 runner 可以模拟工具与环境，却不一定是为用户提供服务的 runtime。把二者分开后，团队才可能在保持测试基础设施足够稳定的同时，更换 agent harness 并测量改动效果。

1.7 两张有用的地图，但都不是通用标准

业界使用多种彼此重叠的地图来描述周边系统。它们回答的问题不同，不应被误认为规范层级。

Birgitta Böckeler 为 coding agent 提出了三层同心范围：模型；由模型或 agent vendor 提供的 **builder harness**；以及由采用方团队加入 repository 指令、hooks、skills 和 review agents 的 **user harness** ([Thoughtworks / Martin Fowler - Harness Engineering](#))。这是一张描述 ownership 与 customization 的地图。Builder harness 和 user harness 都可能分别包含上文所定义的 agent-harness、runtime、product 与 evaluation 层组件。

2026 年的论文稿 *Agent Harness Engineering: A Survey* 提出了 **ETCLOVG**，用七个方面组织工程关注点：Execution environment、Tool interface、Context、Lifecycle、Observability、Verification 和 Governance ([OpenReview - Agent Harness Engineering: A Survey](#))。本书把它当作覆盖范围 checklist，而不是行业标准或部署架构：

- **Execution environment**：获准动作在何处转化为实际效果。
- **Tool interface**：schema、协议、registry 和选择策略。
- **Context**：组装、检索、memory 注入与压缩。
- **Lifecycle**：启动、checkpoint、恢复、handoff 与终止。
- **Observability**：trace、telemetry、延迟、成本与失败证据。
- **Verification**：outcome check、测试套件、grader 与发布门。
- **Governance**：identity、权限、审批、policy 与审计证据。

六个系统层级回答的是责任位于哪里。ETCLOVG 问的是需要覆盖哪些关注点。Builder/user 同心范围问的是谁提供或定制组件。同一个组件可以处于某个层级、覆盖多个关注点，也可以由多个参与方共同定制。

1.8 Framework、Runtime、Harness 与 Product 标签

Vendor 和 practitioner 文献并不统一使用这些标签。LangChain 的一种分类把 framework 描述为开发抽象，把 runtime 描述为持久执行基础设施，再把 harness 描述为更有明确取舍、功能更完整的 agent 系统；同时也承认存在相互重叠的案例 ([LangChain - Agent Frameworks, Runtimes, and Harnesses, Oh My!](#))。这套分类有助于比较产品，但它不是协议规范。

因此，本书按照一个组件实际承担的责任对其分类，而不是照搬网站上的产品标签。名为“agent runtime”的软件库可能包含 agent-harness 逻辑；作为“agent platform”销售的产品也可能同时打包 runtime、control-plane、evaluation 和 application 功能。六层词汇让我们可以直接讨论这些功能，而无需强行把每个 vendor 放进唯一类别。

1.9 Prompt、Context、Harness 与 Loop Engineering

Prompt engineering、context engineering、harness engineering 和 loop engineering 更适合作为四种互补视角：

- **Prompt engineering** 设计模型可见的指令和示例。
- **Context engineering** 筛选并维护一次推理可用的信息；Anthropic 明确把它描述为整理推理期间使用的 token 集合 ([Anthropic - Effective Context Engineering for AI Agents](#))。
- **Harness engineering** 设计紧邻模型的控制与接口，把提议转换为受治理的工作。
- **Loop engineering** 是一个 practitioner 术语，关注重复调用、反馈、验证、budget 和 stop rule 的设计 ([Addy Osmani - Loop Engineering](#))。

这些词并不表示所有系统都必须沿着同一条历史阶梯演进，彼此之间也不存在替代关系。可靠系统通常需要同时在四个范围内作出设计决策。

1.10 模型变强后，这条边界为什么仍然存在？

能力更强的模型可以减少失败，也可以吸收一些过去需要复杂 prompt 或 orchestration 才能实现的行为。但模型能力不会消除概率性提议与已授权、已记录外部效果之间的区别。只要 agent 能够花钱、修改数据、对外沟通或操作基础设施，某个周边系统就仍然必须负责 credential、policy enforcement、execution、durable state、verification 和 recovery。

能够变化的是这些机制的位置与复杂度。Provider 可能在服务端执行工具；product 可能把原先由显式 workflow 代码完成的 planning 移进模型；runtime 也可能让恢复过程对上层透明。实现边界可以移动，但责任边界仍应清晰可见。这就是本书其余章节的组织原则。

要点

- “模型 + harness”是粗粒度公式：它把模型预测与周边系统分开，但生产设计需要更细的层级。
- **Agent harness 紧邻模型**：它组装输入、解释提议并驱动循环，但并不自动等于完整产品或 platform。
- 提议不等于效果：validation、authorization、execution 和 outcome confirmation 都发生在模型之外。
- **Context** 是组装出来的，并非注定永远追加：单调增长描述的是朴素 transcript，而不是所有 harness。
- **Runtime、product、platform 和 evaluation harness** 各有不同职责，即使同一 vendor 把它们打包出售。

- **Context**、**state**、**tools**、**permissions**、**verification** 和 **consequences** 都有外部 **owner**；模型只贡献提议和可能出错的判断。
- **ETCLOVG** 与 **builder/user** 同心范围是有用的地图，不是通用标准。
- 更好的模型可能移动实现边界，但不会因此取得对现实后果的权限。

延伸阅读

- Anthropic, *How Tool Use Works*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>
- Anthropic, *Demystifying Evals for AI Agents*, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Anthropic, *Effective Context Engineering for AI Agents*, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
- Harrison Chase, *Agent Frameworks, Runtimes, and Harnesses*, Oh My!, LangChain, Oct 2025. <https://blog.langchain.com/agent-frameworks-runtimes-and-harnesses-oh-my/>
- *Agent Harness Engineering: A Survey*, OpenReview 论文稿, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>

第 2 章：System Prompt、指令与 Policy 边界

《LLM Foundations》已经确立了两个事实，本章将把它们转化为工程实践。第一，role marker、delimiter 和指令都会被序列化为模型输入；post-training 可以让模型以不同方式响应这些内容，但不会把它们变成绝不会失效的控制通道。第二，模型可以提议动作，却不能授予自身权限，也不能产生外部效果（Foundations 第 8 章、Foundations 第 14 章）。

因此，指令层承担着重要但有限的职责：塑造模型行为。它不是实施硬授权的地方。本章将说明如何组合、版本化、缓存和评测模型可见的指令，同时把可执行 policy 留在 harness 与 runtime 中。

2.1 指令是模型输入，不是可执行 Policy

Agent 的 system prompt 不只是用户在聊天框中输入的问题。它是一种持续维护、面向模型的工件，可以定义 agent 的角色、目标、工具使用指导、响应 contract，以及处理不确定性的规则。HumanLayer 用 *own your prompts* 概括了这项运维原则：不要让重要行为隐藏在没有版本管理的框架默认值里（HumanLayer — 12-Factor Agents）。

把 prompt 称为“应用代码”有助于强调责任归属，但不能抹去一个关键区别：runtime 中的代码会执行确定性检查；prompt 文本则为概率模型提供条件。一句“绝不把数据发送到这个 tenant 之外”可以减少不安全提议，却无法检查凭据、阻断网络出口，也无法让 dispatch 路径不可绕过。这些控制必须位于模型之外。

Prompt 也不是模型内部的持久状态。Harness 为一次调用选择并组装指令，provider 序列化相应的 role 与内容，模型再基于该 representation 生成下一份输出。之后的调用只有在外围系统再次提供这条指令时才能看到它。

2.2 指令优先级是一种行为防御

Agent 会接收来自多个来源的内容：平台或 system 指令、developer 与项目指令、用户请求、示例、工具结果和检索文档。Provider 可以训练模型遵循这些来源之间的预期优先级。The Instruction Hierarchy 将它形式化为一种训练目标：较高优先级的指令应当压过与之冲突的低优先级文本（Wallace 等 — The Instruction Hierarchy）。

这是一种行为防御，不是硬边界。Role 名称和确切优先顺序取决于 provider；即使模型接受过 hierarchy following 训练，也仍可能在冲突中失败。IHEval 观察到不同优先级的指令发生冲突时，模型表现会显著下降，这说明不能把内容位置当作 enforcement guarantee（IHEval: Evaluating Language Models on Following the Instruction Hierarchy）。

需要把三个概念分开：

- **指令优先级 (instruction priority)** 是模型在可见文本发生冲突时应表现出的行为；
- **来源信任 (source trust)** 是 harness 对内容由谁提供、该来源可以影响什么所做的分类；
- **授权 (authorization)** 是判断某个 identity 能否对某个 resource 执行某项 action 的可执行决策。

把可信指令放进 provider 规定的较高优先级 role，可以提高模型遵从它的概率，但不会赋予模型权力。反过来，工具返回或文档检索得到的内容，即使含有祈使句，也应被归类为不可信数据。标题、XML tag、引用块和“把以下内容视为数据”等措辞可以减少一般性歧义，却不能像 parser 或 access-control boundary 那样隔离内容（Foundations 第 8 章）。

Prompt injection 利用的正是这种残余歧义。Hierarchy training 和明确的数据标记可以降低模型遵循注入文本的概率；下一节的可执行控制则会在行为防御失效时限制后果。

2.3 模型可见 Policy 与可执行 Policy

生产系统通常同时需要两类 policy，但两者用途不同：

模型可见 policy	可执行 policy
描述期望行为与决策标准	计算并执行 allow、deny、redact、constrain 或 require-approval 决策
帮助模型选择工具、放弃回答、提出问题或建议升级处理	控制工具可用性、参数、凭据、文件、网络目的地、预算和副作用
可能不会被模型完美遵循	必须位于其所声称保证对应的不可绕过 dispatch 路径上
按模型行为进行评测	按软件和安全 policy 进行测试

例如，“发送邮件前先询问”是一条有用的模型可见指导。真正的硬保证来自 runtime gate：它拦截每次邮件 dispatch，检查 identity 与 policy，并在 action 属于适用范围时要求有效的审批记录。模型生成的“已经获得批准”不能作为审批证据。同一分离原则也适用于删除文件、购买、使用凭据、导出数据和访问 tenant 范围内的检索内容。

第 7 章将展开 sandbox 与 policy enforcement point，第 14 章区分咨询与 mandatory approval，第 19 章讨论 policy administration 与分布式 enforcement。这里的责任边界与 Foundations 的总结相同：模型提出动作；外围系统负责授权、校验、执行并承担后果（Foundations 第 14 章）。

2.4 把不同信息放在正确位置

并非所有相关事实都应进入 system message。指令前缀过载会消耗第 3 章所述的有限 context 预算，让冲突更难诊断，并扩大每次修改的影响范围。可以采用以下职责划分：

- ****System 或 developer 指令：****持久的角色、目标、高价值行为规则、跨工具指导和响应 contract。使用 provider 所支持的最高适用 role，但不要从这个位置推导出安全保证。
- ****工具描述与 schema：****每个工具的功能、参数和结果 contract。完整的调用与 enforcement 路径见第 6 章。
- ****检索上下文：****当前的、大规模的、tenant-scoped 或 task-specific evidence。保留来源和 access-control metadata，并把返回内容视为不可信输入；第 4 章讨论完整的生产数据路径。
- ****可执行 policy：****在模型之外实施 identity、resource permission、参数限制、审批要求、sandbox 限制和 egress rule。

在四个位置重复同一条规则，会产生逐渐分叉的多个事实来源。对于硬控制，应优先保留一份规范的可执行 policy；在可行时，再由同一个 policy reference 派生简洁的模型可见指导和面向人的解释。

2.5 动态组装需要 Provenance

大多数生产级 agent 并不使用一段固定 prompt 字符串。Harness 会组装一组有序 segment：provider 或 builder 指令、应用 policy 指导、项目指令、task-scoped skill、检索 evidence、用户请求和先前 observation。如果系统在模型 context 之外记录一份 assembly manifest，动态组装会更容易调试。

对于每个 segment，manifest 至少应记录：

- 稳定的 segment ID，以及内容版本或 hash；
- 来源和 owner；
- scope，例如 global、tenant、repository、directory、task 或 turn；
- 适用的 model 与 instruction role；
- freshness 相关时的创建或检索时间；
- 该 segment 派生自哪个 policy、artifact 或 retrieval record。

Manifest 可以回答 raw prompt text 无法可靠回答的问题：当时启用了哪个版本？该 segment 为什么在 scope 内？发生冲突时采用了哪个来源？是否把某个 tenant 的内容混入了另一个 tenant 的调用？它还允许 trace 引用 instruction version，而不必把 secret 或个人数据复制进每条 observability record。

组装过程应拒绝或显式暴露有歧义的冲突，而不是默默依赖偶然的字符串顺序。较窄的 scope 可以细化宽泛约定，但除非可执行 policy 明确允许，否则不能覆盖更高信任级别的限制。Timestamp 和 request ID 通常应留在 manifest 或易变后缀中；把它们放在靠前位置会破坏可复用前缀，却不会增加任何权威性。

2.6 稳定前缀与 Provider Prompt Cache

当 provider 提供跨请求 prompt caching 时，指令布局会影响成本和延迟。这里讨论的是 **provider prompt cache**，不同于一次 generation 内使用的 per-request KV cache。不同 provider 对 eligibility、cache breakpoint、retention period、data control 和价格的规定并不相同，因此必须根据当前 API contract 查询这些属性，不能把它们写成通用事实（[OpenAI — Prompt Caching](#)、[Anthropic — Prompt Caching](#)）。

如果复用依赖前缀匹配，就把稳定、可复用的内容放在易变内容之前。Base instruction 和稳定示例可以放在靠前位置；当前任务、新鲜检索结果、请求特定 identifier 与 timestamp 应放在后面。工具目录需要明确策略：稳定的完整 catalog、deferred loading 或 tool search，以及稳定的 meta-tool surface，在缓存、context 和授权上各有取舍，详见第 3 章和第 6 章。

可缓存不等于正确。逐字节相同的前缀仍可能包含过时或互相冲突的 policy。会改变行为的版本更新应主动使相关缓存前缀失效；解释缓存 telemetry 时，则应遵循 provider 记录在案的语义。

2.7 把指令作为版本化、可评测的工件

修改指令可能改变工具选择、拒答行为、输出结构、延迟和成本。因此，prompt 变更需要 owner、版本控制、review，并用有代表性的任务进行评测。比较 outcome 和失败类别，而不只判断新措辞是否看起来更清楚。第 11 章定义 trial 与 grader；第 17 章说明 trace 如何帮助定位造成故障的 segment 或 interaction。

被评测的单位是完整 configuration，而非孤立的 prompt：instruction version、model 与 provider setting、tool schema、retrieval configuration 和可执行 policy 都会相互影响。为一个模型调优的 prompt 可能使另一个模型出现回退。每次 trial 和生产 trace 都应记录 configuration identity，同时对敏感 prompt 内容实施适当的 access control 与 redaction。

2.8 选择合适的高度和可验证输出

Anthropic 把指令设计描述为选择合适的 *altitude*（高度）。指令过于低层，会变成不断膨胀的脆弱特例清单；过于抽象，则不能为行动提供足够信号（[Anthropic — Effective Context Engineering for AI Agents](#)）。合适的中间位置会说明目标、约束、决策标准和可观察的完成条件，同时为模型处理变化保留空间。

Trace 中出现一次故障时，再加一句话只是可能的修复方式之一。更合适的 intervention 可能是更清楚的 tool schema、确定性 validator、不同的检索 evidence、一条 sandbox rule，或一个新的 eval case。只要稳定 invariant 能用确定性方式表达，就应把它移入可执行检查。

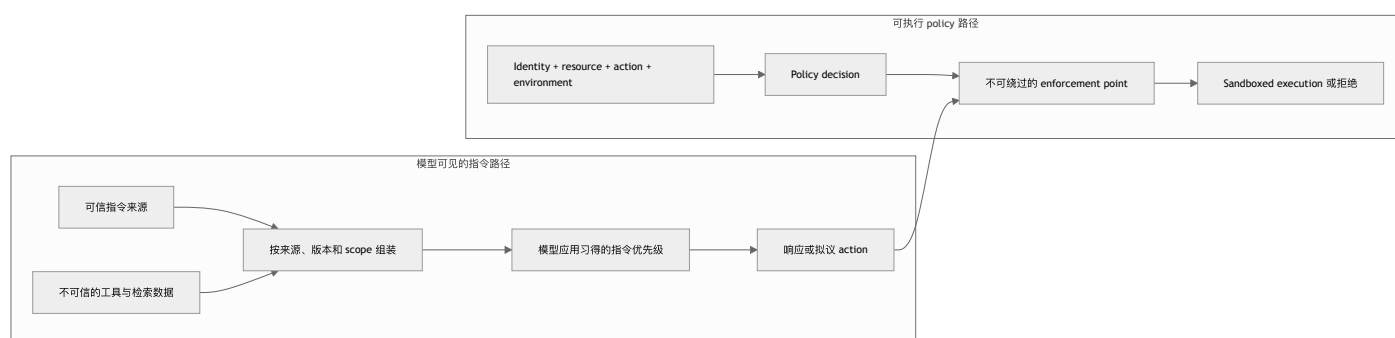
Output contract 应要求人和软件能够检查的工件：plan、当前 status、简洁 rationale、带引用的 evidence、拟议 tool argument、test result 或结构化 uncertainty field。不要把访问隐藏 chain-of-thought 设为 correctness 或 audit 要求。可见推理仍是生成文本，不能证明答案正确，也不保证忠实反映模型的私有计算；应独立验证 claim 与 outcome（[Foundations 第 8 章](#)）。

2.9 分层且有 Scope 的指令

Coding agent 通常会组合 agent 产品提供的 builder instruction、组织或项目指令（如 `AGENTS.md`）、目录范围内的约定，以及 task-specific skill。这些层共同构成模型可见指令集，但它们的名称本身并不能建立信任。Harness 应先解析 scope 与 provenance，再进行组装，最后衡量最终 configuration 是否改善 outcome。

Scoped review rule 是一个有用的例子。高信号规则会指出一种具体缺陷、说明适用范围，并解释如何识别它——例如“标记绕过 `authorize()` 的新 endpoint”，而不是“遵循安全最佳实践”。目录级规则可以让指导贴近其所治理的代码；linter 和 test 则负责实施确定性 invariant（[OpenAI — Custom Code Review Rules for Codex](#)）。更多指令文本并不自动意味着更好：重叠规则会消耗 context、制造冲突，并可能增加 false positive。

图：行为指导与可执行 Enforcement



上方路径塑造概率性的模型行为，下方路径实施真正允许发生的动作。标签和优先级有助于第一条路径，只有第二条路径才能提供执行保证。

要点

- **指令优先级是行为机制，不是权威来源：训练形成的 role 优先级和 delimiter 可以减少混淆，但不能授予权限或实施 policy。
- **把指导与保证分开：模型可见 policy 塑造提议；可执行 policy 对 dispatch 进行授权和约束。
- **记录组装 provenance：版本、来源、scope、tenant 和 policy reference 使动态 prompt 可调试、可审计，同时无需暴露隐藏推理。
- **准确命名缓存：跨请求复用属于 provider prompt-cache contract，不是 per-request KV cache；匹配、保留和计费都取决于 provider。
- **版本化并评测完整 configuration：prompt 会与 model、tool、retrieval 和 runtime policy 共同作用。
- **要求可验证输出：plan、rationale、evidence、action 和 result 都是有用接口；隐藏 chain-of-thought 不是 security 或 audit primitive。

延伸阅读

- Eric Wallace et al., *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*, OpenAI, Apr 2024. <https://arxiv.org/abs/2404.13208>

- Zhihan Zhang et al., *IHEval: Evaluating Language Models on Following the Instruction Hierarchy*, 2025. <https://arxiv.org/abs/2502.08745>
- OpenAI, *Prompt Caching*. <https://developers.openai.com/api/docs/guides/prompt-caching>
- Anthropic, *Prompt Caching*. <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>
- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- OpenAI, *Custom Code Review Rules for Codex*, Jul 2026. <https://developers.openai.com/blog/custom-code-review-rules-for-codex>

第 3 章：上下文是一种有限资源

本章负责单次调用的上下文选择与缓存生命周期；后续章节分别负责检索流水线、记忆和持久状态。

3.1 上下文是预算，不是记忆存储

上下文 (context) 是一次模型调用可见的 token 与多模态表示。它容量有限，针对特定调用组装，不能与持久执行状态或记忆混为一谈。《LLM Foundations》第 9 章解释了模型侧的限制；本章讨论 harness 应当把什么放进这个限制之内。可用上下文更多有时会有帮助，但把窗口填满并不天然有益。Anthropic 把上下文描述为一种“注意力预算”：新增材料可能稀释相关信号，检索准确率也会随模型、任务、长度和信息位置而变化 (Anthropic — Effective Context Engineering for AI Agents)。因此，应把这种上下文退化 (context rot) 当作概率性的工程风险，而不是固定的轮数限制，也不是“任何 prompt 变长都会变差”的定律。

一个朴素的 agent 循环会让问题变得很直观：

- 1. 组装指令、工具、对话记录和当前任务数据；
- 2. 让模型提出一个动作；
- 3. 执行获得授权的动作，并追加其结果；
- 4. 使用扩大的对话记录重复上述过程。

只有这个只追加的基线才会单调增长。生产 harness 可以裁剪低价值材料、清除体量过大的工具结果、卸载 artifact、压缩早期轮次、重置上下文、按需检索信息，或延迟加载工具定义。每种变换都会改变下一次调用可见的内容，因此需要明确的正确性标准和可恢复引用；第 5 章会深入讨论这些损失和 provenance 问题。

3.2 组装足够使用的最小上下文

实际目标是为下一项决策提供足够使用的最小高信号输入。Anthropic 把这个目标描述为：找到能最大化预期结果概率的最小 token 集合 (Anthropic — Effective Context Engineering for AI Agents)。一种实用的组装顺序是：

- 稳定的指令和行为指导；
- 只包含模型可能需要发现的工具；
- 以紧凑结构表示的任务状态；
- 局部连续性所需的近期动作和观察；
- 与当前步骤相关的检索证据和 artifact 引用。

结构有助于导航，但不会产生安全边界。Markdown 标题、XML 标签和清晰的标记可以区分指令与证据，而授权与强制策略检查仍然属于 harness 和 runtime 的责任 (见第 2、7 章)。工具描述之间也应有足够清晰的区别，让模型能够做出选择；重叠或含糊的接口既会占用上下文，也会增加选错工具的概率 (Anthropic — Effective Context Engineering for AI Agents)。

Few-shot 示例也应采用同样的取舍原则。优先选择少量、有代表性并能展示预期决策边界的示例。把所有边界情况都列成目录，可能会挤占实时任务所需的空间，却不能保证模型泛化。

3.3 三个不同对象：上下文、KV cache 与 prompt cache

三个相互关联的概念有不同的生命周期：

对象	生命周期与所有者	对 harness 的含义
上下文	一次模型调用；由 harness 组装	本次调用中模型可见的完整输入。它不是持久状态。
单请求 KV cache	一次生成；由推理系统管理	在该次生成中，为已经处理的 token 复用 attention key/value 状态。
跨请求 provider prompt cache	跨符合条件的请求；由 provider 契约暴露	复用完全匹配或按 provider 规则匹配的前缀。准入条件、匹配、断点、保留时间、数据控制、指标、延迟和计费都依 provider 而定。

第三种机制在内部可能复用由 KV 派生的状态，但这个实现细节并不会让它与单请求 KV cache 成为同一个产品契约。OpenAI 文档说明会自动缓存符合条件且精确匹配的 prompt 前缀，并在响应 usage 中暴露 cached-token 信息；Anthropic 文档则说明了显式 cache breakpoint、前缀匹配和可配置的缓存生命周期 (OpenAI — Prompt Caching; Anthropic — Prompt Caching)。因此，在讨论独立 API 请求之间的复用时，代码和正文都应使用 provider prompt-cache 命中。

Agent 循环往往会让跨请求 prompt cache 很有价值，因为一个较大的前缀可能保持稳定，而每轮只追加很短的动作和观察。但这取决于工作负载，并非普遍规律。Yichao “Peak” Ji 在 2025 年 7 月关于 Manus 的报告中称，其平均输入输出

token 比接近 100:1，并引用了当时 Claude Sonnet 的价格：每百万缓存输入 token 为 0.30 美元，未缓存输入 token 为 3 美元（[Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)）。这些数字只是一个具名的历史案例，不代表当前价格、跨 provider 比率，也不能预测其他工作负载。

3.4 设计与调试可缓存前缀

Cache-aware 设计必须从 provider 契约出发。OpenAI 和 Anthropic 都要求前缀内容匹配才能复用，但各自的准入规则、缓存控制、生命周期和核算方式并不相同；应用应查阅所选模型的当前文档，而不是硬编码一个通用阈值或价格（[OpenAI — Prompt Caching](#)；[Anthropic — Prompt Caching](#)）。

如果 API 的匹配规则会奖励前缀稳定性，就应把稳定内容放在易变内容之前。在语义允许时，把时间戳、request ID、用户专属状态和当前 observation 放在可复用边界之后。确定性序列化同样重要：即使底层对象看起来等价，字段顺序或空白变化也可能改变序列化后的前缀。

Cache 命中率下降时，在改变 agent 设计之前检查以下项目：

- **前缀边界**：哪些确切的字节或 token 位于可复用前缀中？
- **动态字段**：时间戳、nonce、request ID、tenant 值或轮换指令是否进入了该前缀？
- **工具定义**：描述、顺序、schema 或可用性是否发生变化？
- **序列化**：prompt 和工具 JSON 是否以确定方式生成？
- **缓存控制**：该 provider 是否要求或支持显式 breakpoint，其位置是否正确？
- **Provider 指标**：缓存输入用量和延迟是否来自 provider 响应，而不是根据总 token 数推测？
- **当前契约**：模型、region、retention mode 和 data-control 设置是否仍符合 provider 的当前文档？

3.5 工具目录：三种有效策略

Agent 并不必须把每个工具定义永久保留在上下文中，这不是通用规则。MCP 明确允许 server 通知 client 工具列表发生变化，因此协议支持动态目录；只是特定模型或缓存设计可能让这种变化代价很高（[MCP — Tools Specification](#)）。常见模式有三种：

1. **稳定目录加 action masking**。为了保持模型连续性和 prompt-cache locality，让工具定义保持稳定，但由确定性 runtime 拒绝当前状态下不可用的动作。Manus 报告称，它在自己的 provider 和工作负载配置中采用了这种模式，并为同类动作使用一致的名称前缀（[Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)）。它提出的“屏蔽而非删除”是一个案例，不是协议要求。
2. **延迟加载或工具搜索**。不在初始模型输入中放入大型目录，而是在发现相关工具时才加载定义。Anthropic 的 tool-search 接口支持 `defer_loading`，并说明延迟定义如何在选中后展开，同时与其 prompt-caching 设计兼容（[Anthropic — Tool Reference](#)）。
3. **稳定的 meta-tool 或代码界面**。暴露一个小而稳定的搜索、registry 或代码执行界面，在其背后解析更大的 capability 目录。这样可以减少 schema 体积，但会增加一个发现步骤，也不会取消对最终 capability 进行 dispatch validation 和授权的要求。

应当用评测数据在这些模式之间做选择。相关变量包括目录大小、模型的工具选择质量、provider 缓存行为、延迟、token 成本、权限范围以及陈旧工具引用的风险。动态工具列表发生变化时，绝不能因为旧版本曾经暴露过某个工具就沿用授权；第 6 章会定义调用与授权生命周期。

3.6 把工作材料卸载到 Artifact

大体量 observation、源文档、中间计算和生成输出不必全部留在实时上下文里。文件系统或 artifact store 可以为它们提供持久、可寻址的位置。上下文只需保留短摘要，加上能够在需要时恢复底层证据的路径、URI、版本或内容哈希。

这些存储是工作材料，不是自动形成的记忆。如果没有有意义的名称、索引、provenance、访问检查和检索约定，agent 仍可能找不到 artifact，也可能从错误的任务或 tenant 中取回内容。LangChain 把文件系统描述为基础性的 harness primitive，因为它支持渐进卸载与协作，同时仍要求 harness 决定哪些内容应返回上下文（[LangChain — The Anatomy of an Agent Harness](#)）。

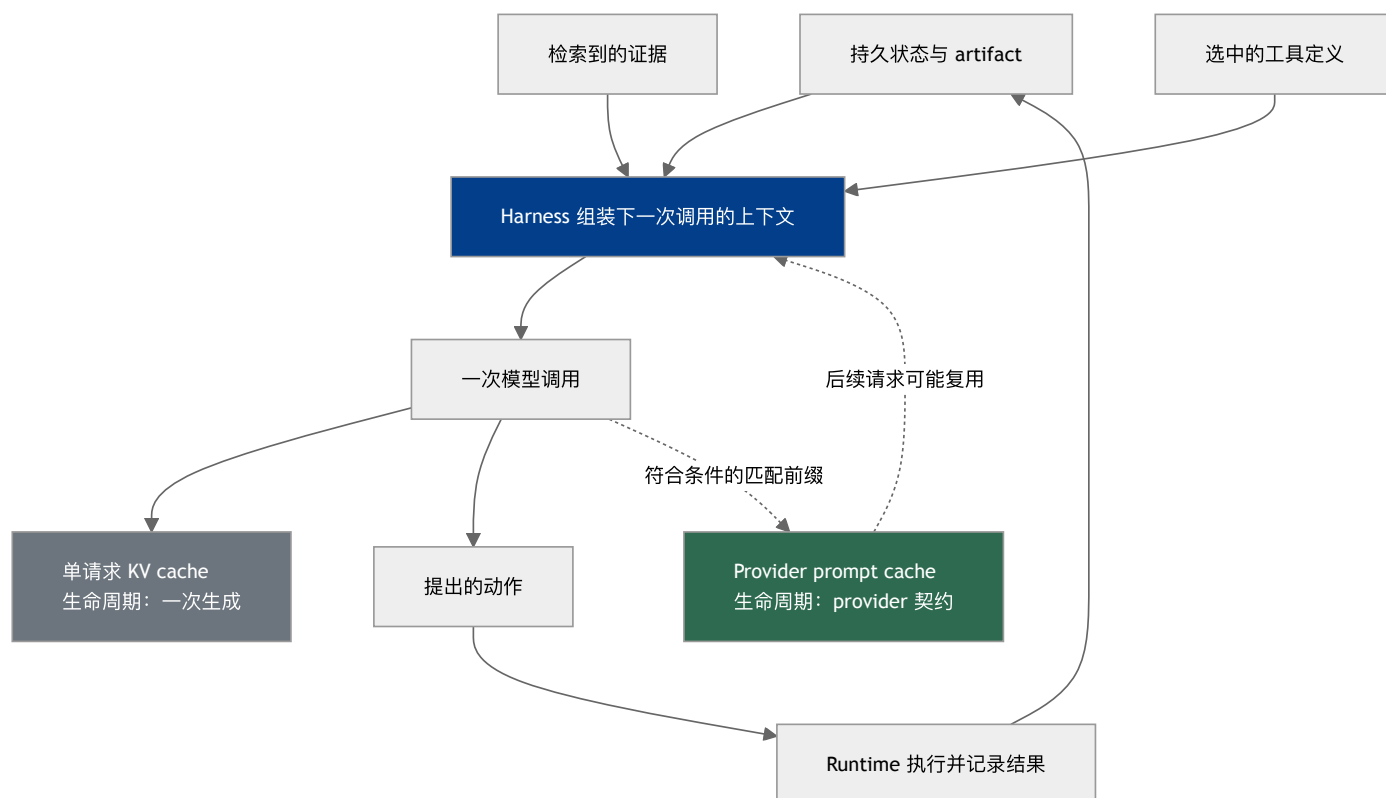
对于 coding agent，代码仓库通常是项目事实的权威来源。OpenAI 的 Codex harness 指南建议，把导航指令、设计知识和可机械检查的约束放在代码附近，让新的 session 能自行发现系统的工作方式与改动验证方法（[OpenAI — Harness Engineering](#)）。这一原则倾向于短小、局部、易发现的文档和标准验证命令，而不是一个不断增长的单一 prompt。

3.7 即时选择，而不是完整检索流水线

Just-in-time retrieval 是一种上下文选择技术：先保留路径、链接和查询等轻量标识符，只在下一项决策需要时加载底层材料（[Anthropic — Effective Context Engineering for AI Agents](#)）。它是对预计算检索的补充，而不是替代。混合系统可以预加载少量高价值指令，为任务检索索引证据，并让 agent 按需检查 artifact。

本章止步于上下文边界：加载什么、何时加载，以及卸载后保留什么引用。第 4 章会沿着生产数据路径继续讨论 ingestion、parsing、chunking、ACL-aware indexing、freshness、hybrid retrieval、reranking、provenance 和 retrieval evaluation。《LLM Foundations》第 11 章提供了这条流水线依赖的 embedding 与 retrieval 概念。

图：上下文选择与缓存生命周期



要点

- 上下文是单次调用的输入，而不是持久记忆：harness 从状态、artifact、检索证据和近期交互中选择上下文。
- 只有朴素的只追加对话记录才会单调增长：生产循环可以裁剪、卸载、压缩、重置、检索或延迟工具。
- **KV cache 与 provider prompt cache 有不同生命周期**：跨请求复用应使用 provider 术语，并把规则和价格视为 provider-specific。
- 稳定工具只是一种策略，不是定律：应使用工作负载评测比较 masking、延迟加载/tool search 与稳定 meta-tool。
- **Artifact 让上下文可恢复**：路径和 provenance 使大体量材料能够离开窗口，而不至于无法找回。
- **Just-in-time retrieval 是上下文选择**：第 4 章负责生产检索流水线。

延伸阅读

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- OpenAI, *Prompt Caching*. <https://developers.openai.com/api/docs/guides/prompt-caching>
- Anthropic, *Prompt Caching*. <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>
- Model Context Protocol, *Tools Specification*, Jun 2025. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
- Anthropic, *Tool Reference*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-reference>
- Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>

第 4 章：生产检索与 Grounding

《LLM Foundations》第 11 章解释了 retrieval embedding、lexical 与 dense search、chunking、reranking 以及 retrieval metrics。本章从生产边界接手：harness 如何把持续变化、受权限约束的源数据变成模型可以安全使用、operator 又能追溯到来源的 evidence。

Retrieval-augmented generation 改变的是模型输入，而不是模型权重：系统检索外部信息，并把它提供给当前 generation (Lewis et al. — [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#))。到了生产环境，这项看似简单的操作同时是 data lifecycle、authorization decision、context-selection step 和 evaluation problem。

4.1 Retrieval、Grounding 与 Outcome 是三种不同的主张

本书有意区分以下三个术语：

- **Retrieval**：针对特定 query、identity 与 task，从受治理的 corpus 中选择 candidate evidence。
- **Grounding**：检查生成结果中的 claims 是否得到实际提供给模型的 evidence 支持。
- **Task outcome**：检查整个 workflow 是否产生了正确的外部结果。

系统可能检索到 relevant chunk，但模型没有使用它；模型可能忠实总结了一段已经过时或本身错误的 retrieved chunk；一份引用完备的回答仍可能没有完成用户任务。反过来，回答也可能依靠参数知识得出正确结论，却没有得到 retrieved corpus 的支持。这些是不同的 failure modes，需要不同的 graders。

因此，harness 应把 retrieval output 当作结构化 evidence product，而不是一段匿名字符串：

```
EvidenceItem {
  source_id, source_version, chunk_id,
  title, location, content_hash,
  retrieved_at, index_version,
  acl_scope, retrieval_scores,
  text
}
```

具体 schema 取决于 application。不可省略的原则是：文本必须始终与 source identity、version、location、authorization scope 以及生成它的 index 绑定。这些字段使后续 citation check、删除、incident review 和可复现 evaluation 成为可能。

4.2 从受治理的 Source Record 构建 Index

Indexing pipeline 应从明确声明的 system of record 开始，而不是从 crawler 偶然发现的某个副本开始。一条实用的 source record 包含稳定的 source ID、source version 或 modification marker、tenant、content type、owner、access policy 或 ACL reference，以及 deletion status。Ingest job 还要记录自己何时、以何种方式观察到该状态。

生产路径如下：

1. **Ingest**：枚举或接收 source changes，同时保留 source identity 与 permissions。
2. **Parse**：提取文本和结构，并保留 page、heading、table、code symbol 或 timestamp 等 location。
3. **Chunk**：创建可检索单元，同时避免把 claim 与理解它所必需的 qualifier 拆开。
4. **Annotate**：附加 provenance、tenant、ACL、language、timestamp、document type 和其他可过滤 metadata。
5. **Deduplicate**：用 content hash 识别完全相同的副本，用经过评测的规则识别 near-duplicate；即使正文只索引一份，也要保留所有 source relationships。
6. **Represent**：创建所选 retrievers 需要的 lexical fields 与 retrieval embeddings。
7. **Publish**：把 immutable records 写入具名的 index version；只有通过 validation 后，该版本才可用于 serving。

Chunk size、boundary、overlap 和 document context 都会影响 retrieval behavior；它们是调优变量，并不是中性的 preprocessing detail。例如，Anthropic 的 Contextual Retrieval 报告会在创建 embedding 与 BM25 index 前加上 document-specific context，但这只是一个经过评测的 preprocessing design，并非普遍要求 (Anthropic — [Contextual Retrieval](#))。

记录 parser、chunker、metadata schema、embedding model 与 mode、lexical analyzer，以及 contextualization prompt 或 model 的版本。没有这份 manifest，就无法把“index 发生变化”拆解为 corpus change、preprocessing change 或 retrieval-model change。

Deduplication 不能抹掉 provenance。如果同一段 policy paragraph 出现在三个受管控的 sources 中，index 可以避免把三份副本都发送给模型，但仍需知道哪些 source versions 包含这段文本，以及各自适用什么 access rules。Content hash 只能证明按某种 normalization 得到的 bytes 相等；它不能证明两条 source records 拥有相同的 authority 或 lifecycle。

4.3 执行 ACL-Aware Query Path

在 query time，harness 组装的不只是一条 search string。它要把 request 与 tenant 和 principal 绑定，推导 mandatory filters，生成一条或多条 retrieval queries，并让当前 index version 与 policy version 贯穿结果。

一条稳健的 query path 包含以下阶段：

1. **Formulate the query**：使用用户 request，以及 retrieval 所必需的最少 workflow state；任何 rewrite 都要与 original query 一起保留。
2. **Apply scope**：在暴露 evidence 前，选择 tenant、corpus、time range、document type 与 authorization filters。
3. **Generate candidates**：根据 workload 运行 dense 和/或 sparse retrieval。
4. **Fuse**：融合 rank 或经过归一化的 scores；不同 retriever 的 raw scores 并不会天然可比。
5. **Rerank**：当 evaluation 证明额外工作值得时，用成本更高的 relevance model 处理有上限的 candidate set。
6. **Deduplicate and diversify**：当 task 需要更广的 evidence 时，避免让等价 passages 浪费 result slots 与 context。
7. **Assemble context**：在 token budget 内选择 evidence，保留 source labels，并让 quoted evidence 与 instructions 明确分离。
8. **Return evidence metadata**：让 model-facing prompt 与下游 citation validation 都能获得 source 和 version identifiers。

Dense 与 lexical retrieval 捕获不同信号；reranking 也只能重新排列第一阶段 retrieval 已经提供的 candidates。Anthropic 发布的实现按这一顺序结合 embeddings、BM25、rank fusion、deduplication 与 reranking；其测得结果只属于报告中使用的 corpora、models、candidate counts 与 metric，不能代表所有 RAG 系统 ([Anthropic — Contextual Retrieval](#))。

Query rewriting 同样可能失败。记录 original query、每条 derived query、filters、candidate IDs、各阶段 score 以及最终选择的 evidence。一次 rewrite 如果悄悄丢掉 identifier 或 negation，search 表面上可能完全健康，结果却已经变得不相关。

4.4 Freshness、Deletion 与 Index Migration

Index 是派生 view，不是 source of truth。它的 lifecycle 应让每条 source record 的四种状态可以被观察：current、pending update、tombstoned 或 failed。“Ingestion job 已运行”弱于“截至 source watermark W 的所有变更均已体现在 serving index V 中”。

按 source class 定义 freshness service level。Product catalog 也许能容忍几分钟延迟；被撤销的访问权限或已删除的机密文档，则可能需要在较慢的文本和 vector 清理完成前就退出 retrieval。安全模式是先让 tombstone 或 deny marker 在 serving path 生效，再异步执行物理删除与 compaction。

当 parser、chunker、embedding model、lexical analyzer 或 schema change 让新旧 records 不再兼容时，应进行 re-index。构建并验证新的 immutable index version，在固定 eval set 上把它与当前版本对比，然后切换 serving pointer。Elasticsearch index aliases 是一个 vendor-specific 示例：它可以切换 application 使用的 index，并支持无停机 reindex；这里可迁移的架构经验是 versioned publication，而不是依赖该 API ([Elastic — Aliases](#))。

每次 migration 都要显式处理：

- **Backfill**：新 schema 能否重建每条仍然 live 的 source record？
- **Dual writes 或 change capture**：backfill 期间到达的 edits 如何应用到两个版本？
- **Validation**：document counts、ACL coverage、抽样 content hashes 与 retrieval evals 是否通过？
- **Cutover 与 rollback**：每条 request 由哪个 serving version 处理，流量是否可以切回上一版本？
- **Retirement**：何时可以删除旧 index 及其中的敏感副本？

Source deletion、ACL revocation 或 legal hold 不只是另一种 relevance update。要让它带着可审计的 completion state，贯穿 ingestion、index publication、caches、replicas、backups 和 citation surfaces。

4.5 在 Retrieval 阶段强制执行 Permission

Permission check 应位于 retrieval path，而不能只放在显示最终回答的界面上。把 authenticated principal、tenant、groups 或 roles、resource scope、适用时的 purpose，以及 policy version 绑定到 search request。只有获得授权的 candidates 才能进入 context assembly。

Microsoft 文档中的 security-filter pattern 展示了这种机制：系统把 principal identifiers 存入可过滤 metadata，再在 query 中提供这些标识，以排除不匹配的 documents。Microsoft 同时提醒，这种 pattern 本身只是基于字符串的 filtering，并不等于 authentication；application 仍必须验证 caller，并正确构造 filter ([Microsoft — Security filters for trimming results in Azure AI Search](#))。

把以下各项视为 correctness invariants：

- 每个 chunk 都继承其 source 当前的 permission semantics；
- ACL changes 通过可测量、高优先级的路径传播；
- 所有 retrieval modes——包括 dense、sparse、hybrid、reranking、cache hits 与 direct document lookup——都应应用等价 scope；
- reranker 永远不会收到 calling service 无权披露的 candidates；
- evidence cache 除了 query 与 index version，还必须按 tenant 和 authorization scope 区分；
- citation 在 view time 通过 authorization check 解析，而不是变成永久有效的 capability URL。

对一份已经组装好的 top- k list 做 post-filtering，并不等于 permission-aware retrieval：它可能只剩下过少的 authorized candidates，也可能在 intermediate services、logs 或 models 中暴露 restricted text。如果 search backend 无法直接执行 policy，就应在任何不受信任的 consumer 看见 candidate content 前部署不可绕过的 policy enforcement point，并在该设计下检索足够多的 authorized candidates。

Tenant isolation 是系统属性，不只是一个 metadata field。根据风险，它可能要求 separate indexes、encryption keys、service identities、network boundaries 或 retention policies。无论采用哪种设计，cross-tenant negative tests 都应进入每个 retrieval release suite。

4.6 Retrieved Content 是不受信任的 Content

Retrieval relevance 不会赋予 instruction authority。Web page、uploaded PDF、ticket、code comment 或 internal document 都可能包含试图改变模型方向或触发 tool use 的文字。NIST 把 prompt injection 定义为利用 untrusted input 与 higher-trust prompt 拼接的攻击；这正是 retrieval system 会建立的边界 ([NIST AI 100-2e2025 — Adversarial Machine Learning Taxonomy](#))。

Harness 应当：

- 把 retrieved spans 标记为 evidence，而不是 system 或 developer instructions；
- 让 source 与 trust metadata 贯穿 query rewriting、reranking 和 context assembly；
- 使用 delimiter 帮助模型导航 evidence，但不声称 delimiter 构成安全边界；
- 把 content 最小化为当前问题真正需要的部分；
- 对 retrieved text 建议的任何 action 继续要求常规 authorization 与 mandatory approval gates；
- 即使 passage 提出请求，也不能把 secrets 和 high-impact tools 交给模型控制；
- 记录 action 发生前有哪些 evidence，以便重建 incident。

第 2 章讨论的 instruction priority 是 behavioral defense。第 7 章中的 sandbox、credential broker、allowlist、policy enforcement point 与 approval gate 才提供 executable boundary。Retrieval filtering 可以减少系统接触已知 malicious material 的机会，但 classifier 或 blocklist 不能证明剩余文本一定安全。

4.7 保留 Citation 与 Provenance

Citation 至少包含两个相互独立的 correctness condition：

1. **Attribution**：citation 能解析到实际提供给模型的 source version 与 location。
2. **Support**：该 passage 支持与之绑定的 claim。

关于 citation-enabled generation 的研究会把 citation correctness 与 completeness 同整体 answer quality 分开评估，这说明仅仅加入 source markers 并不够 ([Gao et al. — Enabling Large Language Models to Generate Text with Citations](#))。

给模型提供稳定的 evidence IDs，而不是要求它复现任意 URL。Generation 完成后，解析 claim-to-evidence links，检查每个 ID 是否来自 authorized evidence set，再从已存储的 source record 渲染 human-facing citation。在风险值得时，使用 entailment check 或 human review 检验 support；不能因为某个 retrieved passage 排名很高，就把它重新标记成“verified”。

Provenance 应跨越 answer generation。Stored output 可以指向 source_id + source_version + location + content_hash，interface 则另行解析最新、仍获授权的 view。这样 operator 才能区分“模型当时看到了什么”和“source 现在写了什么”。

4.8 评测完整 Pipeline，包括 No-Answer Cases

Retrieval 调优应使用 end-to-end trials，而不是只看一个 aggregate relevance number。Evaluation harness 应固定 corpus snapshot、index version、identity 与 ACL set、query transformations、model versions 和 grader versions。Anthropic 的 agent evaluation 指南也把 environment、trial、transcript、outcome 与 graders 视为必须分别定义并保持稳定的组件（Anthropic — Demystifying Evals for AI Agents）。

使用分层 scorecard：

层级	问题与示例 measurements
Corpus 与 index	预期 sources 是否存在、最新、已去重且 permissions 完整？Deletion 与 ACL propagation lag 是多少？
Candidate retrieval	Recall@ k 、Precision@ k 、MRR、NDCG，以及不同 query cohort 下的 misses；《Foundations》第 11 章定义了这些 metrics。
Selection	Fusion 或 reranking 是否在 context budget 内改善有用 evidence？结果有多大比例彼此重复或冲突？
Abstention	当 corpus 缺少支持，或所有 candidates 都不在 authorized scope 内时，系统是否返回经过校准的 no-evidence state，而不是捏造支持？
Grounding 与 citation	哪些 claims 得到支持、缺少支持、受到反驳或缺少 citation？Citation 是否解析到实际使用的精确 source version？
Task outcome	回答或 agent workflow 是否解决了任务，并避免 disallowed actions 或 data disclosure？
Operations	Ingestion lag、分阶段 query latency、failure rate、tokens、reranker calls，以及每个 successful outcome 的 cost。

数据集应包含 answerable、unanswerable、stale-source、conflicting-source、deleted-source、revoked-ACL、cross-tenant、exact-identifier、paraphrase 和 adversarial-document cases。No-answer decision 是 task definition 的一部分，不是应该从 dataset 中省略的例外。

Candidate count、最终 top- k 、fusion weights、reranker cutoff 与 no-evidence threshold 应联合调优。更大的 candidate set 可能改善 recall，但会增加 search 与 reranking 工作；更多 final passages 会消耗 model context，也可能增加干扰。Anthropic 在自己的 reranking 配置中报告了这种 latency/cost/quality tradeoff，并明确建议针对具体 workload 做实验（Anthropic — Contextual Retrieval）。Raw similarity 或 reranker scores 不应被当作可移植概率；应针对精确的 model、corpus、index 与 query cohorts 校准 threshold。

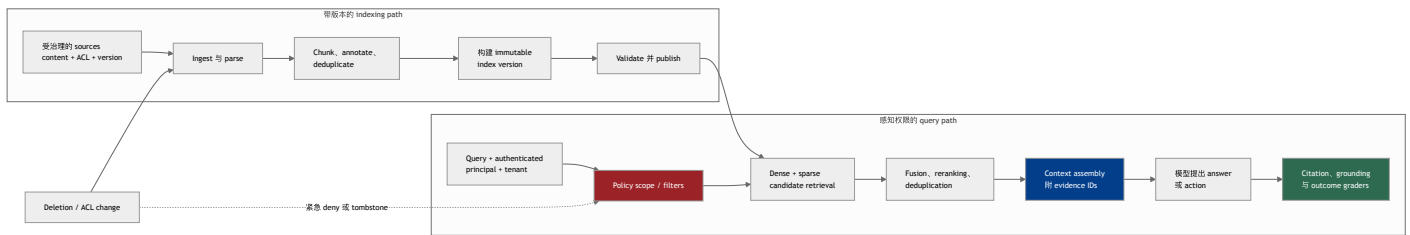
同时报告 stage latency 与 end-to-end latency。至少拆分 query formulation、embedding、sparse 与 dense search、fusion、reranking、context assembly 和 generation。优化目标应是每个 successful、policy-compliant outcome 的 cost，而不只是 cost per query 或 Recall@ k 。

4.9 Case Study：Contextual Retrieval，而不是通用 Baseline

Anthropic 2024 年的 Contextual Retrieval 报告是 combined pipeline 的一个有用具名案例。它在 embedding 与 BM25 indexing 前加入 chunk-specific context，融合 lexical 与 dense candidates，进行 deduplication，并测试了 reranking stage。报告中的 improvement 是在特定 corpora、embedding configurations、candidate counts、基于 top-20 recall 的 failure rate 和特定 reranker 下测得的（Anthropic — Contextual Retrieval）。

可迁移的经验是实验方法：定义 corpus 与 metric，改变一个 pipeline component，测量 retrieval 与 downstream behavior，并把 cost 与 latency 纳入考量。报告中的 improvement percentages 和推荐 top- k 并不是其他 corpus 的默认值。生产团队应先在自己的 documents、permissions、questions 与 outcome graders 上复现对比，再决定是否采用这种设计。

图：受治理的 Retrieval Path



要点

- **Retrieval** 是一条受治理的数据流水线： ingestion、parsing、chunking、metadata、ACL、index version 与 deletion 都属于 answer correctness。
- **Permission** 必须在 **evidence** 暴露前生效： 每种 retrieval mode、cache、reranker 与 citation resolver 都要绑定 identity 和 tenant。
- **Retrieved text** 不受信任： relevance 不会赋予 passage 指令权，也不会允许它造成 side effect。
- **Evidence** 需要持久 **provenance**： 保留 source version、location、content hash、index version 与 authorization scope。
- **Retrieval、grounding、citation 与 outcome** 需要不同 **graders**： 一层成功并不能证明下一层成功。
- 调优整个系统： top-`k`、threshold、reranking、abstention、latency 与 cost 都是 workload-specific decisions。
- 具名结果仍然只是具名案例： Anthropic 的 Contextual Retrieval 结果可以启发实验，但不是通用 performance guarantee。

延伸阅读

- Patrick Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, 2020. <https://arxiv.org/abs/2005.11401>
- Anthropic, *Introducing Contextual Retrieval*, Sep 2024. <https://www.anthropic.com/engineering/contextual-retrieval>
- Microsoft, *Security filters for trimming results in Azure AI Search*. <https://learn.microsoft.com/en-us/azure/search/search-security-trimming-for-azure-search>
- Elastic, *Aliases*. <https://www.elastic.co/guide/en/elasticsearch/reference/current/aliases.html>
- NIST, *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*, NIST AI 100-2e2025. <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-2e2025.pdf>
- Tianyu Gao et al., *Enabling Large Language Models to Generate Text with Citations*, 2023. <https://arxiv.org/abs/2305.14627>
- Anthropic, *Demystifying Evals for AI Agents*. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>

第 5 章：压缩、记忆与上下文交接

长周期任务可能持续得比一次模型调用更久，最终也会超出任何单个上下文窗口。因此，连续工作不能等同于“不断追加 transcript”。Harness 必须变换下一次调用能够看到的内容，同时保留足以恢复被省略信息的证据。《LLM Foundations》第 9 章已经说明，context 是有限且临时的输入，而非持久状态；本章讨论这一边界对 harness 的直接要求（《LLM Foundations》第 9 章）。

核心设计问题不只是怎样缩短 context，而是：

哪些信息现在可以丢失，哪些信息必须在其他位置保持权威，以及后续调用如何恢复证据？

5.1 不应混为一谈的五类对象

如果把所有形式的连续性都叫作“记忆”，context engineering 就很容易变得不安全。本书区分五类对象：

对象	含义	权威来源
Context（上下文）	一次模型调用可见的 token 与多模态 representation	由 harness 为本次调用组装；调用结束后，它不再作为模型可见输入存在
Memory（记忆）	为未来调用检索或注入的信息	具有明确 scope、provenance、更新、访问与遗忘规则的 memory store
Execution state（执行状态）	当前步骤、待执行动作、预算、审批状态等结构化 workflow 事实	应用或 runtime state machine
Artifact（产物）	文件、diff、报告、dataset、build 或已记录工具输出等可寻址工作产物	Artifact store 或底层 system of record，以路径、URI、版本或 content hash 标识
Event history（事件历史）	用于恢复 durable workflow execution 的有序记录	Runtime 的持久 history；如果声称能够 replay，就必须复用已记录的模型与工具结果，而不是悄悄重新计算

临时 context 与持久状态之间的模型侧边界来自 Foundations 第 9、14 章（《LLM Foundations》第 9 章；《LLM Foundations》第 14 章）。Temporal 展示了一种具体的 durable-execution 设计：系统利用持久化的 workflow-history events，在故障后重建 workflow state（Temporal — History Service）。这只是一个实现案例，并不要求每个 harness 都使用 Temporal。

一份压缩摘要可以成为下一次调用的 context、供检查的 artifact，以及以后可检索的 memory 候选项，但它不会因此自动成为权威 execution state 或 event history。如果摘要说审批已经通过，而 runtime state 仍显示待审批，应以 runtime state 为准。第 10 章负责 execution state、checkpoint 与 event history；本章只深入 context transformation、memory 与 handoff artifact。

5.2 不同变换具有不同的损失模型

Harness 有多种方法让任务留在 context budget 内，它们不能互相替代：

技术	变换	可能丢失或出错的内容	恢复路径
Tool-result clearing（工具结果清理）	用简短结论和引用替换体积很大的 observation	原始字段、顺序、warning 或错误细节可能消失	重新加载已记录的结果或 artifact
Recitation（复述）	在当前 context 尾部重述目标、计划或约束	不会有意删除内容，但可能放大过时或错误的信息	与权威 task state 和源证据核对
Compaction（压缩）	把选定的早期 context 总结为更短的 representation	被省略的 rationale、少数证据、精确措辞、provenance 或 uncertainty	沿 citation 和 artifact pointer 回到一手材料
Context reset + handoff（上下文重置与交接）	用刻意构造的 handoff package 启动新 context	任何未进入 package 的事实都不再对模型可见	按需读取权威 state、artifact、event history 或 memory
Retrieval（检索）	选择外部材料并注入后续调用	可能遗漏相关证据，或选中过时、越权、误导性材料	重新检索、检查 provenance，并评测 retrieval pipeline

Compaction 和 clearing 减少已经进入 context 的材料；reset 改变连续性边界；retrieval 把选定的外部材料带回来；recitation 改变显著性，却不会创造持久性。第 4 章讨论检索质量和权限感知的生产 pipeline；第 3 章讨论单次调用预算与 provider prompt cache 行为。

这些选择对缓存的影响也不同。改写前部 prefix 可能导致跨请求的 provider prompt cache 无法命中，但匹配、保留和计费都是 provider-specific contract，而不是 correctness guarantee（OpenAI — Prompt Caching；Anthropic — Prompt Caching）。绝不能为了维持 cache hit 而保留过时或不安全的内容。

5.3 压缩是一份有损且可测试的契约

当 context 接近设定预算时，compaction 会总结较早的交互，再把摘要作为后续 context 的一部分。Anthropic 建议在真实、复杂的 trajectory 上调优 compaction prompt：先提高 recall，确保继续任务所需的信息得以保留；再提高 precision，删除不再有帮助的材料（[Anthropic — Effective Context Engineering for AI Agents](#)）。根据 token 数量或 context utilization 触发压缩是一种实现策略，而不是通用阈值；太晚触发可能耗尽窗口，太早触发则会造成原本可以避免的信息损失。

生产系统中的 compaction 应有明确 schema，至少保留：

- 已经作出的 **decision**；如果理由会约束后续工作，也要保留 rationale；
- **open task** 和下一项预期动作，不能把未完成工作标成完成；
- **constraint**，包括用户要求与适用 policy check 的引用；
- **artifact pointer**，最好包含版本、commit 或 content hash；
- **provenance**，把重要结论连回源 observation 或文档；
- **unresolved uncertainty**，包括相互冲突的 hypothesis 和仍然缺少的证据。

近期且可操作的失败也可能需要进入下一段 context，但必须具有生命周期。如果准确的错误消息、失败参数或 stack trace 能改变下一次尝试，就保留最近一次错误；如果同一种失败反复发生，应把重复输出替换为 diagnosis、retry count 和原始日志指针。Manus 提倡保留有用的失败，让模型据此调整；12-Factor Agents 则建议把错误压缩进下一段 context（[Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)；[HumanLayer — 12-Factor Agents](#)）。两种实践都不意味着应把无限日志留在 live context 中。

评估 compaction 时，应把它当成一次 transformation，而不是根据摘要文笔是否流畅来判断。有效的测试会从 compacted package 恢复一组有代表性的任务，并衡量 agent 是否保留必需约束、识别下一步、找到被引用的 artifact、保留经过校准的不确定性，以及避免重复已经解决或放弃的工作。在条件允许时，应与未压缩 baseline 比较，并加入 adversarial case，例如重要 caveat 只出现过一次，或者与 trace 中的多数内容冲突。

5.4 记忆是一种受治理的信息产品

结构化笔记会把选定信息变成可复用的产品。Anthropic 曾描述 agent 如何在数千个游戏步骤中维护进度笔记、地图和策略，并在 context reset 后重新加载这些笔记（[Anthropic — Effective Context Engineering for AI Agents](#)）。关键机制不是文件名或行文格式，而是围绕笔记建立的 write-retrieve lifecycle。

每条 memory 都应回答：

- **Scope**：哪个 task、user、tenant、repository 或 agent 可以使用它？
- **Provenance**：哪次 observation、哪个来源、哪种 identity 和哪个 tool 产生了它？
- **Type**：它是 observation、用户偏好、派生结论，还是 instruction candidate？
- **Freshness**：它何时创建、何时必须重新验证，什么内容可以替代它？
- **Authority**：谁授权了写入，未来 reader 是否有权访问底层证据？
- **Forgetting**：如何让它过期、删除、回滚或更正？

Memory 不会因为持久化就自动成为事实。Retrieval 可能选错条目，consolidation 可能合并不相容的事实，曾经正确的条目也可能过时。MemGPT 探索了 virtual context management：在容量有限的模型 context 与外部存储之间移动信息（[Packer et al. — MemGPT: Towards LLMs as Operating Systems](#)）。Mem0 探索了提取、整合和检索 memory，而不是回放完整的多 session history（[Chhikara et al. — Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory](#)）。这些是有用的架构案例，并不说明每个短任务都需要 memory subsystem。

持久记忆也是一道信任边界。从网页复制到共享笔记中的不可信文本，在原始 context 消失很久以后仍可能影响后续运行。应把 observation 当作数据；绝不能因为模型总结过某段检索内容，就自动把它提升为 policy；读取和写入时都需要重新检查 authorization。Anthropic 把这种跨运行风险称为 persistent memory poisoning，并强调必须对 agent 可访问状态实施 containment（[Anthropic — How We Contain Claude](#)）。

5.5 复述与清理服务于局部连续性

Recitation 会刻意在当前 context 尾部重写一份简短任务表示，通常是一张 checklist。Manus 报告称，其系统会反复更新 `todo.md`，让长任务的目标在充满 action 的 trajectory 中保持显著（[Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)）。这是一种 attention-management technique，而不是 durable state 或 correctness mechanism。Checklist 应从权威 task state 重新生成，或至少与之核对；否则 recitation 只会让过时计划继续保持显著。

这一动机与“lost in the middle”的实证结果一致：在该研究评估的 retrieval 和 question-answering 场景中，模型表现会随相关信息在长输入中的位置而变化（[Liu et al. — Lost in the Middle](#)）。这一结果不能证明所有模型和任务都有相同的

位置曲线，因此仍应在目标 workload 上验证 recitation。

Tool-result clearing 是范围最窄的有损操作。只有在保存了下一项决策所需的信息后才能清理结果：派生结论、重要 warning 或 unresolved error、据此采取的动作，以及能够恢复原始输出的指针。在 side effect 尚未确认、结果存在争议，或者 audit/debugging 可能需要精确字段时，清理尤其危险。原始输出应该进入 artifact store 或适当记录，而不是无限期留在 live context 中。

5.6 交接是类型化接口，而非可信摘要

只要工作跨过 context boundary，就发生了 context handoff：context reset 之后、session 之间、sub-agent 到 parent，或者一种 model configuration 到另一种配置。应把它视为带有 schema 的接口，而不是一段随意写成的文字。

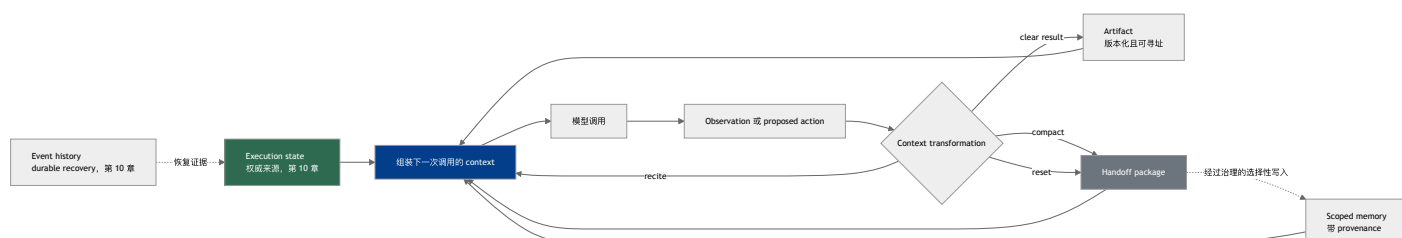
稳健的 handoff package 包含：

1. **Task identity 与 bounded scope**——请求是什么、哪些内容被有意排除，以及完成标准是什么。
2. **Worker identity 与 authority scope**——哪个 agent/model/tooling 执行了工作，它能够访问哪些资源。
3. **Claim 与 decision**——简洁结果，并与 observation、proposal 明确分开。
4. **Evidence 与 provenance**——行内 citation，以及可寻址的 artifact、version、query、command 或 tool-call identifier。
5. **External effect**——尝试过的动作、已确认 outcome、失败，以及任何 pending approval。
6. **Uncertainty 与 open question**——置信限制、冲突、缺失证据和假设。
7. **Verification status**——实际运行过的 test/check、结果，以及完整输出的位置。
8. **Next action**——receiver 下一步应做什么，包括执行高权限动作前需要重新验证的内容。

Sub-agent 是有价值的 context firewall，因为它可以在独立窗口中完成大量工具调用，再把紧凑结果返回给 parent ([Anthropic — Effective Context Engineering for AI Agents](#); [HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#))。这道 firewall 会隔离中间噪声，却不会让结果变得更可信。

尤其需要注意，高权限 parent 不能把低权限 worker 的摘要当作 authorization 或已经验证的事实。Parent 应检查被引用的证据，在 action dispatch 时重新检查当前 policy，并独立确认高影响 outcome。Identity 与 scope 必须随结果一同传递，使 receiver 知道 worker 能看到什么、不能看到什么。Handoff artifact 可以为 execution state 提供输入，但只有 runtime 或 application 才能提交状态转换；第 6、7、19 章将分别讨论 dispatch、enforcement 与 fleet identity。

5.7 上下文连续性循环



图中指向 memory 的虚线是刻意设计的：并非每份摘要都应成为持久 memory。信息升级需要明确的 write policy、provenance、scope 与 lifecycle。同样，event history 支持恢复，却不应被原样塞入每一次模型调用。

要点

- **Context、memory、execution state、artifact 与 event history 是不同对象**：摘要可以引用权威状态，却不能取代它。
- **每种 context transformation 都有自己的损失模型**：clearing、recitation、compaction、reset 与 retrieval 以不同方式失败，也需要不同恢复路径。
- **Compaction 需要 schema 与 evaluation**：应保留 decision、open task、constraint、artifact pointer、provenance 与 unresolved uncertainty。
- **有用错误也有生命周期**：保留近期且可操作的细节，压缩重复噪声，并确保原始证据可恢复。
- **Memory 必须受治理、有限定 scope 且可以修订**：持久化不会让模型生成的笔记自动变为事实或获得授权。
- **Recitation 改变显著性，不改变权威性**：反复重述的计划必须与真实 task state 核对。
- **Context firewall 不是 trust firewall**：Sub-agent 结果必须携带 evidence、identity 与 authority scope、uncertainty、verification status 和可寻址 artifact。
- **高权限 parent 必须在行动前重新验证**：Worker 摘要既不是 authorization，也不是已经确认的 outcome。

- 跨请求复用应使用 **provider prompt cache** 这个名称：不能称作 per-request KV cache，也不能为了命中缓存而牺牲 correctness。

延伸阅读

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- OpenAI, *Prompt Caching*. <https://developers.openai.com/api/docs/guides/prompt-caching>
- Anthropic, *Prompt Caching*. <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>
- Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Charles Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.08560>
- Prateek Chhikara et al., *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.19413>
- Nelson F. Liu et al., *Lost in the Middle: How Language Models Use Long Contexts*, TACL, 2024. <https://arxiv.org/abs/2307.03172>
- Anthropic Safeguards Research Team, *How We Contain Claude*, Anthropic, May 2026. <https://www.anthropic.com/engineering/how-we-contain-claude>
- Temporal, *History Service*. <https://github.com/temporalio/temporal/blob/main/docs/architecture/history-service.md>

第 6 章：工具与调用生命周期

《LLM Foundations》第 12 章停在模型边界：工具调用是模型提出的结构化输出，工具结果则作为可能不可信的上下文返回。本章沿着 harness 继续追踪这个 proposal。核心规则很简单：**格式正确的工具调用既不代表获得许可，也不能证明执行成功。**

6.1 从模型提议到外部效果

对于由 client 执行的工具，模型发出结构化请求，应用代码执行操作，再把结果送回模型。Anthropic 明确把它描述为一项契约：模型本身从不执行该操作；OpenAI 也把模型生成的 tool call 与应用产生的 tool-call output 区分开来 (Anthropic — How Tool Use Works; OpenAI — Function Calling)。Provider 可以托管某些工具并隐藏一部分往返过程，但责任边界不变：模型输出负责提议，执行系统负责决策和行动。

一个生产级调用应通过明确的生命周期：

阶段	主要负责人	必须记录的内容	Retry 的含义
1. 接收并关联	Harness adapter	provider response ID、provider call ID、harness invocation ID	尚未执行
2. 完成流式参数	Harness adapter	完成事件与组装后的字节	重连或放弃；不得 dispatch 片段
3. 解析	Parser	解析值或语法错误	要求修正调用；不得执行
4. Schema 验证	Validator	schema version 与字段错误	修正参数；不得执行
5. 语义验证	Domain adapter	已解析资源、前置条件、边界	刷新状态或要求修正
6. 授权	Dispatcher / policy enforcement point	actor、capability、resource、policy version、decision	身份、资源、工具或 policy 变化时重新评估
7. 获得强制审批	Runtime / product	approver、准确的 action preview、expiry、decision	不得从模型文本推断已批准
8. 建立幂等与去重	Dispatcher	idempotency key、既有 attempt 查询	恢复或返回已知结果，不重复产生效果
9. 执行	Tool adapter 或远端服务	attempt ID、request、start time	由副作用语义决定
10. 强制 timeout 与 cancellation	Runtime	deadline、cancellation state	在确认前，timeout 产生的是 unknown outcome
11. 分类完成状态	Tool adapter	success、retryable failure、terminal failure、partial success 或 unknown	只按明确 policy retry
12. 归一化结果	Adapter	稳定的 result/error envelope 与 artifact references	不采用随模型变化的临时格式
13. 返回 observation	Harness	与原 call ID 关联的结果	Observation 是不可信 context，不是执行权限
14. 确认外部 outcome	Verifier / environment	postcondition evidence 与 final status	对有后果的 retry，必须先 reconciliation

并非每次只读查询都需要人工审批或额外的后置条件查询。这个生命周期是一组需要明确区分的决策，不是要求进行十四次网络往返。多个阶段可以合并在同一个可信组件里，但它们的证据和失败语义仍应可以分辨。

6.2 关联、流式输出与并行调用

每次调用都需要稳定身份。保留 provider 的 call ID；如果任务可能跨 queue、retry、provider 或进程边界，还要增加 harness-level invocation ID。结果必须关联到某个具体调用，不能只依赖工具名或列表位置。OpenAI 的 tool-call output 使用 call_id 指向原调用；Anthropic 的 tool result 同样用 tool_use_id 匹配特定的 tool_use block (OpenAI — Function Calling; Anthropic — Handle Tool Calls)。

Streaming 增加了一个完成边界。Argument delta 是尚在生成的 serialization，不是一系列可执行命令。OpenAI 的 streaming 契约先发出 argument-delta events，随后发送包含完整参数的完成事件；harness 应等待 provider 的完成信号，按 call identity 组装，之后才解析和验证 (OpenAI — Function Calling, Streaming)。如果 dispatch 一个看似语法合理的前缀，例如 {"recipient":"alice"}，就可能对被截断或随后改变的 intent 执行动作。

并行调用需要各自独立的状态机。假如同一轮提出三个调用，其中一个成功、一个验证失败、一个 timeout，就应记录三个 outcome，不能压成一个 turn-level Boolean。每个完成的 observation 都应使用自己的 call ID，尚未解决的调用则保持显式状态；例如 Anthropic API 要求每个 client tool_use block 在下一条消息中获得对应的 tool_result (Anthropic — Handle Tool Calls)。

一个实用的内部 envelope 是：

```
invocation_id, provider_call_id, tool_name, tool_version
actor, tenant, capability, policy_version, approval_id
arguments_hash, idempotency_key, attempt_id, deadline
status, normalized_result, artifacts, postcondition_evidence
```

第 10 章定义了持久的 session → run → step → action → attempt 层级。只有当一次 invocation 恰好表示一个 intended effect 时，dispatcher 才可以把 invocation_id 映射为 action_id；composite invocation 必须保留内部各 action 与 attempt identity，不能把它们藏进一个不透明 ID。

这些字段属于 execution state 和 lineage metadata，没有必要全部放回模型上下文。

6.3 Validity、Authorization 与 Outcome 是不同问题

把“验证”压成一个 Boolean，会隐藏最重要的边界：

- 1. **语法与 schema validity**：参数是否能解析，字段和类型是否符合声明的接口？Provider 的 strict-schema 功能可以改善这一层，但其保证取决于具体 provider 和 API (OpenAI — Function Calling, Strict Mode)。
- 2. **Semantic validity**：标识符是否存在？金额是否为正？目标状态是否允许此操作？引用的 revision 是否仍为最新？这些检查依赖领域状态，无法完全编码进 JSON Schema。
- 3. **Authorization**：这个 actor 此刻是否可以在该 tenant 内、出于当前 purpose、对这个 resource 执行这个 action？工具出现在模型 context 中不等于 capability grant。
- 4. **Approval**：Policy 是否要求某个人批准这一个准确的 action 和 payload？Mandatory gate 必须由 runtime 插入，而不是由模型决定是否调用。MCP 规范建议应用让人看到暴露给模型的工具，并能拒绝敏感调用 (MCP — Tools Specification)。
- 5. **Outcome confirmation**：预期的外部状态是否真的出现？成功的 transport response、工具返回的一段文字和验证过的 postcondition，证据强度并不相同。

以 transfer_funds({from, to, amount}) 为例。合法 JSON 和数值型 amount 只能证明第一层。语义检查还要解析两个账户和业务限额；授权必须把 acting identity 绑定到 from；policy 可能要求审批准确的金额和收款人；执行需要 transaction identity；outcome confirmation 则检查最终交易或账户余额。跳过其中任何一层，都会改变系统可以作出的安全性和正确性声明。

6.4 幂等、去重、超时与重试

Retry policy 的起点应是动作的效果语义，而不是 exception class。RFC 9110 把幂等请求定义为：相同请求执行一次或多次，服务器上的预期效果相同；它还警告，除非 client 知道请求语义是幂等的，或知道第一次请求并未生效，否则不应自动 retry 非幂等请求 (RFC 9110 §9.2.2)。即使工具没有使用 HTTP，tool adapter 也需要同样的纪律。

动作类别	示例	Timeout 后的默认行为	必需保护
纯读取 / read-only	搜索、读取状态	在 budget 内 retry	Deadline、rate limit、request correlation
天然幂等	把目标标签设置为 approved	前置条件仍成立时 retry	Resource version 或 conditional write
通过契约实现幂等	使用 idempotency key 创建付款	按 key 查询，然后恢复或 retry	Durable dedup record 与稳定 key
非幂等且后果重大	发送消息、提交没有 key 的订单	不盲目 retry	Outcome lookup、reconciliation 或 human escalation

Idempotency key 为多次 delivery attempt 中的同一个预期效果命名；**deduplication record** 保存该 key 对应的执行结果。Call ID 标识一项 proposal，attempt ID 标识一次 dispatch。如果每次 retry 都生成新的 idempotency key，去重就失效；如果两个不同的预期效果复用同一个 key，则会发生碰撞。

Timeout 只表示 caller 不再等待，不能证明 callee 已停止或 rollback。应把调用置为 unknown；如果存在 operation-status 或 target-state endpoint，先查询，再决定是否 retry。除非工具契约保证 rollback，否则 cancellation 同样只是一

项停止请求。对于包含多个效果的工具，应返回哪些子操作成功，并暴露补偿动作，而不是只报告一个不可区分的 failure。

Retry logic 不能绕过 authorization 或 approval。当 identity、resource state、tool version、arguments、policy version、approval scope 或 approval expiry 发生变化时，要重新授权。所有 attempts 应计入同一个 invocation budget；budget 耗尽时返回 terminal observation，避免模型意外制造无限 retry loop。

6.5 先归一化结果，再确认 Outcome

不同 provider protocol 采用不同的 tool-result 编码。Harness 应把它们归一化成一套小型内部词汇，同时在需要时把原始响应保存为 artifact：

```
status: succeeded | retryable_error | terminal_error | partial | unknown
summary: compact model-facing observation
data: typed structured result, when available
artifacts: references to large or reviewable outputs
error: stable category, safe message, retry guidance
effect: external operation ID and postcondition evidence
```

MCP 区分 protocol error 与 tool-execution error，也支持带 output schema 的结构化结果；其规范要求 server 返回符合声明 schema 的结构化结果，并建议 client 在把结果交给模型前进行验证 ([MCP — Tools Specification](#))。应保留这种区别。“未知工具”不同于“付款 provider 拒绝扣款”，两者又都不同于“扣款可能成功，但响应丢失”。

结果是 observation，不是 instruction。网页、邮件、仓库文件、peer-agent response 或第三方 API 都可能包含攻击性文本；因此 Anthropic 的 tool-use 文档要求 client 把受外部来源影响的工具结果视为不可信内容 ([Anthropic — Handle Tool Calls](#))。保留 provenance 和 trust label，限制结果大小，只向模型呈现下一步决策所需的信息。对于有后果的动作，confirmation 应优先使用 environment state——transaction status、file hash、deployed version、sent-message ID——而不是模型自己声称任务已完成。

6.6 设计 Agent-Computer Interface

Anthropic 仿照 HCI 使用 *agent-computer interface* (ACI) 一词：工具 schema、description、response 和错误 affordance 都值得仔细设计。它的早期指南建议使用接近训练数据的格式，在模型提交刚性语法之前给它留下规划空间，并避免不必要的格式负担；在一个 SWE-bench agent 中，把文件工具从相对路径改成绝对路径，消除了 agent 离开 repository root 后观察到的大多数路径错误 ([Anthropic — Building Effective Agents](#))。

不要机械地把每个 API endpoint 包装成工具。模型如果必须调用 `list_contacts`、扫描巨大响应，再调用 `send_message`，它获得的接口就不如一个有范围限制的 `search_contacts` 或 `message_contact`。可以合并几乎总是连续执行的步骤，但不要把需要分别授权或分别审查的效果藏进一个巨型工具。`get_customer_context` 可以安全合并读取；`research_and_wire_money` 则会抹掉必要的 policy boundary。

工具描述应明确原本隐含的领域知识：何时使用、何时不用、必需格式、单位、副作用以及含义明确的参数名。Anthropic 报告称，namespace 和描述变更会显著影响模型的 tool-use 表现，因此建议用 eval 持续迭代，而不是假定第一版 schema 已经足够 ([Anthropic — Writing Effective Tools for Agents](#))。Schema 设计可以提高 proposal 质量，但不能替代前述生命周期检查。

6.7 三种工具目录策略

第 3 章介绍了三种有效的 catalog pattern。它们解决的 context 和 continuity 问题不同：

1. ****稳定目录加 action masking。**保持工具定义稳定，以维持模型连续性和 provider prompt-cache locality；同时由确定性 dispatcher 阻止当前状态下不可用的动作。Manus 报告称在自己的系统中使用了名称前缀和 masking；这是特定 provider/workload 的 case study，不是协议规则 ([Manus — Context Engineering for AI Agents](#))。
2. ****Deferred loading 或 tool search。**不把低频定义放进初始 context，在发现后才加载。Anthropic 的 tool-search 接口支持 deferred tools，并说明所选定义如何展开进 conversation ([Anthropic — Tool Reference](#))。
3. ****稳定的 meta-tool 或代码表面。**暴露一个小型 registry、search tool 或 sandboxed code interface，在其背后解析更大的 capability set。这可以减少 schema 体积和模型往返，但 dispatcher 仍必须验证并授权每一项最终 capability。

选择时应依赖 workload-specific eval。需要测量 tool-selection accuracy、context cost、latency、provider prompt-cache behavior、stale-reference failure 和 permission exposure。“Mask, don't remove”不是普遍定律；动态加载也不会因为可见 schema 更少就自动更安全。

6.8 MCP 暴露 Capability；Harness 治理调用

Model Context Protocol 是 client-server protocol：server 暴露工具，client 列出并调用工具。Server 也可以暴露 resources 和 prompts，但协议并不意味着 server 拥有、控制或能够检查每个工具背后的所有下游系统 ([MCP — Tools Specification](#); [MCP — Server Overview](#))。

MCP 标准化的是 discovery 与 transport，并没有消除生命周期责任。规范要求 server 验证输入、实施 access control、限制调用速率并清理输出；它建议 client 对敏感操作进行确认、dispatch 前展示输入、验证结果、设置 timeout 并记录使用情况 ([MCP — Tools Specification](#))。生产 client 还可以增加更严格的 policy、mandatory approval、tenant isolation、deduplication 和 postcondition verification。

部署拓扑同样重要。私有 capability 不必为了被 hosted agent 使用而新增公网入站 endpoint。OpenAI 的 secure MCP tunnel 文档介绍了一种在私有网络中运行 outbound-only client 的方案：client 连接指定目的地，同时保留 authentication 和 streaming ([OpenAI — Connect Private MCP Servers](#))。应把它视为一种部署 pattern，而不是 MCP 本身保证的属性。

6.9 动态 Tool List 需要新的决策

MCP 的 `listChanged` capability 允许 server 通知 client：可用工具列表已经变化 ([MCP — Tools Specification](#))。因此，缓存的 schema 或之前出现在模型 context 中的工具都不代表当前 authority。

收到 list-change notification 后，client 应当：

1. 使用当前 authenticated identity 获取新列表；
2. 比较 tool identity、schema、annotations、server identity 和 version；
3. 使过期的 model-visible definition 和 routing decision 失效；
4. 在下一次 dispatch 前重新运行 capability 与 policy 检查；
5. 如果 action、arguments、resource 或 risk 变化，要求新的 approval；
6. 记录治理该调用的 catalog 和 policy version。

同一规则也适用于 deferred loading 和 meta-tool。Discovery 回答“现在声明提供什么”；authorization 回答“这个 actor 现在允许做什么”。两者是不同操作。

6.10 Namespace 与 Context-Efficient Response

`jira_issues_search`、`asana_projects_create` 之类一致的前缀可以减少名称冲突，也让 harness 对相关 capability 分组。Anthropic 的工具设计 eval 发现，名称以及 prefix/suffix 的选择都会影响表现，最佳选择取决于具体 workload ([Anthropic — Writing Effective Tools for Agents](#))。Namespace 是 usability 和 routing 辅助信息，不能证明两个工具属于同一个 authorization domain。

工具响应应优先提供相关的 typed information，而不是追求最大 payload。应提供 pagination、filter、range、concise/detailed mode 和明确的 truncation marker。面向模型的引用可以使用有含义的 label，同时保留后续调用所需的 canonical ID。Anthropic 报告称，在工具 eval 中用更自然的 identifier 替代不透明 UUID 后准确率得到提升，并建议使用 token-efficient response control ([Anthropic — Writing Effective Tools for Agents](#))。

错误信息应当可操作但安全：给出稳定类别、失败的字段或前置条件、是否允许 retry，以及完整诊断信息保存在哪里。不要用数千行成功日志淹没 context，也不要通过原始 traceback 暴露 secret。大型原始输出应存为 artifact；归一化 observation 则携带可恢复的引用。

6.11 代码执行作为 Meta-Tool

Harness 不必把数百项直接工具定义和每个中间结果都放进模型 context，也可以暴露由 typed wrapper 支持的 sandboxed code surface。Anthropic 的“Code Execution with MCP”描述了通过生成的 TypeScript 文件进行渐进式发现，并报告了一个 Google Drive-to-Salesforce 示例：token 使用量从 150,000 降到 2,000；这是具名示例，不是通用节省比例 ([Anthropic — Code Execution with MCP](#))。

这种 pattern 适合有边界的数据流：filter、join、sort、deduplicate、aggregate 和 validate，然后只返回精简结果。OpenAI 的 programmatic tool-calling 文档描述了同样的主要收益——工具可以从生成的代码中调用，因而不必让全部中间结果经过模型——但 client-side tool 仍由应用执行并返回 output ([OpenAI — Programmatic Tool Calling](#))。

代码执行不会产生 ambient authority。Sandbox、typed proxy 和 dispatcher 必须限制可调用工具、credential、network、filesystem、budget 和 side effect。记录生成的程序、运行时采用的 capability set 与 policy version、每个有后果的 sub-call 和最终 artifact。不要为了减少模型往返，把需要分别审批的动作隐藏在一段程序中。

6.12 同时评估工具与生命周期

工具 eval 不应只测试模型是否选择了正确名称。Anthropic 建议按 prototype、realistic evaluation、programmatic runs、transcript analysis 的循环迭代工具，并根据观察到的 failure 改进工具 ([Anthropic — Writing Effective Tools for Agents](#))。针对本章生命周期，还应包含这些 slice：

- 格式错误、不完整以及 schema-invalid 的参数；
- schema 合法，但 resource 不存在或 version 已过期；
- 跨 tenant 和超出 scope 的 authorization attempt；
- 模型没有主动请求的 mandatory approval；
- duplicate delivery、lost response、effect 发生后的 timeout 与 cancellation race；
- 并行或 multi-effect tool 的 partial success；
- 不可信的 result content 与超大输出；
- proposal 与 dispatch 之间发生动态 catalog change；
- response 名义上成功，但 postcondition 不匹配。

应捕获 proposal、validation decision、policy version、approval、attempt、normalized result 和 outcome evidence。第 11 章进一步定义 eval terminology 和 grader；第 17 章进一步讲生产 tracing。此处的重点是：如果 eval 不检查 environment state，一份干净的 model transcript 仍可能掩盖重复副作用或失败的 authorization check。

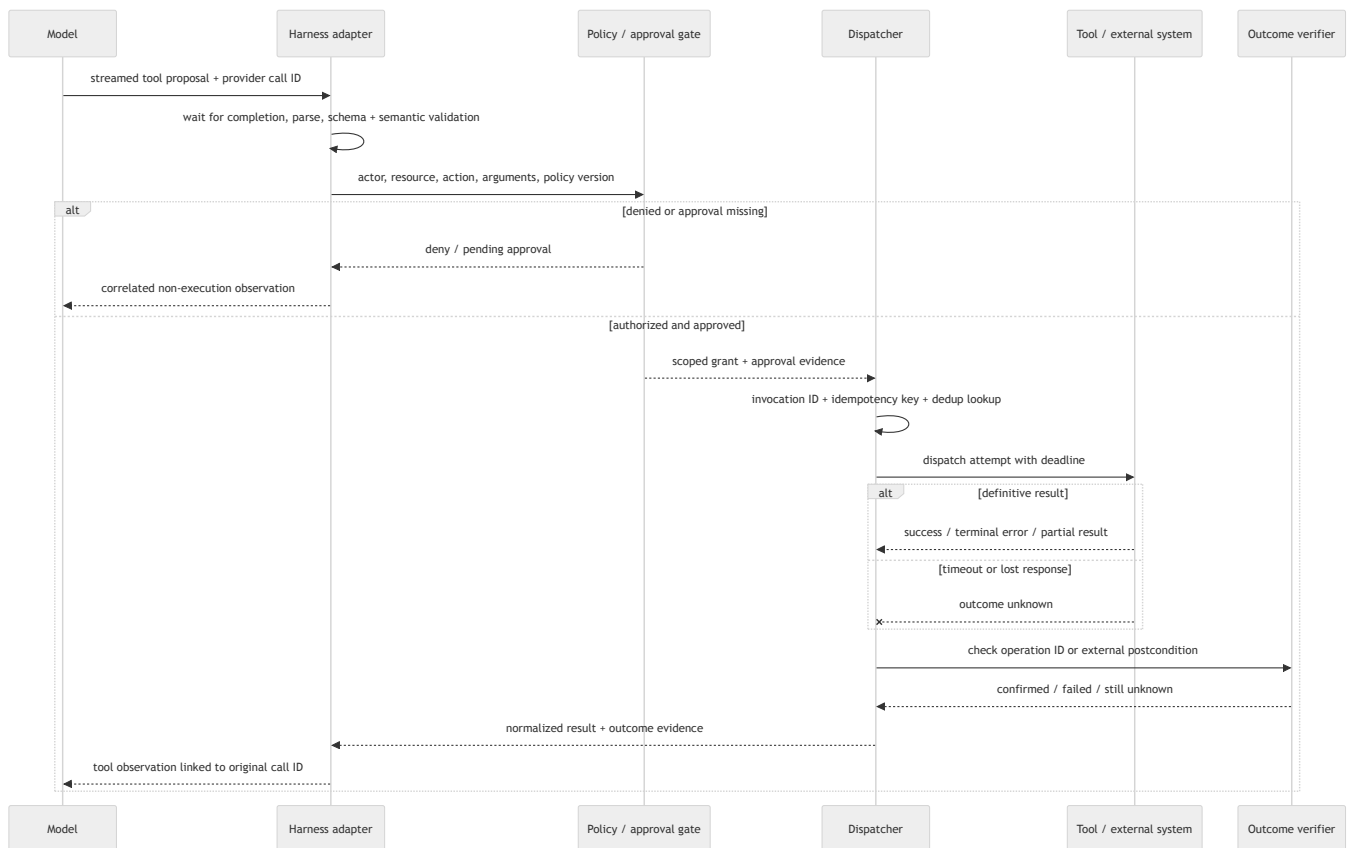
6.13 MCP 与 A2A 跨越不同边界

MCP 把 client 连接到 server 暴露的 capability；A2A 则通过 message 和 task，把 agentic client 连接到另一个 agentic service。当前 A2A 规范定义了用于 discovery 的 Agent Card、task 和 message operation、streaming、authentication/authorization field 以及多种 protocol binding；它不会让 caller 自动检查 peer 内部的 model、memory、tool 或 sandbox ([A2A Protocol Specification](#))。

这种区别改变了契约。MCP tool call 应具有精确的 operation schema，并返回关联到该 call 的结果。A2A delegation 可能长时间运行、交换消息、返回 artifact，并需要 task-level status 或 cancellation。无论哪种边界，本地 harness 都必须建立 identity、delegated authority、data scope、deadline、provenance 和 outcome evidence。Peer-agent output 应视为不可信的外部内容，而不是更高权限的 instruction。

Native multi-agent delegation 最适合范围有界、ownership 清楚且产物可合并的任务。共享可变状态或严格的顺序依赖通常需要单一 owner 加确定性 coordination。A2A 标准化的是 interoperability boundary，并不会消除 distributed-systems 或 trust problem。

图：一次工具调用如何从 Proposal 到 Confirmed Outcome



要点

- **工具调用是一项 proposal： **schema validity 不等于 authorization、approval、execution 或 outcome proof。
- **Call identity 必须贯穿整个生命周期： **关联 streaming fragment、retry、attempt、result 和 confirmation，不能依赖工具名或位置。
- **绝不 dispatch 未完成的流式参数： **先等待 provider 的完成边界，再解析和验证。
- **Timeout 表示 unknown outcome： **对于有后果或非幂等的 action，retry 前先 reconciliation。
- **结果需要归一化和 provenance： **返回精简 observation，同时保留结构化数据、artifact、error 和 postcondition evidence。
- **工具目录有多种有效策略： **稳定目录加 masking、deferred loading/tool search 和稳定 meta-tool 都有适合的 workload。
- **动态 discovery 不会继承 authority： **MCP `listChanged` event 或 deferred tool 都需要新的 capability 与 policy 检查。
- **MCP 负责暴露工具，不保证拥有背后的资源： **client 和 server 仍需明确的 validation、access control、approval、timeout 与 logging。
- **A2A 是 peer-agent boundary，不是 tool-schema substitute： **必须保留 delegated authority、task status、provenance，并把返回内容视为不可信。
- **评估外部状态，而不只看 transcript： **很多 lifecycle bug 在模型可见文字中看起来仍然成功。

延伸阅读

- Anthropic, *How Tool Use Works*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>
- Anthropic, *Handle Tool Calls*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/handle-tool-calls>
- OpenAI, *Function Calling*. <https://developers.openai.com/api/docs/guides/function-calling>
- Model Context Protocol, *Tools Specification*, Jun 2025. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
- A2A Project, *A2A Protocol Specification*. <https://a2a-protocol.org/latest/specification/>
- RFC Editor, *RFC 9110: HTTP Semantics*, Jun 2022. <https://www.rfc-editor.org/rfc/rfc9110.html>

- Ken Aizawa, *Writing Effective Tools for Agents — with Agents*, Anthropic, Sep 2025.
<https://www.anthropic.com/engineering/writing-tools-for-agents>
- Adam Jones and Conor Kelly, *Code Execution with MCP*, Anthropic, Nov 2025.
<https://www.anthropic.com/engineering/code-execution-with-mcp>
- OpenAI, *Programmatic Tool Calling*. <https://developers.openai.com/api/docs/guides/tools-programmatic-tool-calling>

第 7 章：沙箱、护栏与运行时执行

7.1 Agent 安全威胁模型

本章主要讨论各种缓解措施，包括沙箱、hooks 和审批关卡。在介绍这些措施之前，必须先明确它们要防范什么。一个既能读取不可信内容、又能操作外部系统的 agent，面临以下几类特有风险 ([Anthropic - Beyond Permission Prompts](#)):

- **Prompt injection (提示注入)** (见《LLM Foundations》第 8 章 “Prompt Injection as Context Confusion”) —— 模型无法可靠地区分数据与指令。因此，agent 读取的网页、issue 评论、源文件或工具结果等不可信内容，都可能像命令一样影响它的行为。
- **数据外泄** —— 一旦 agent 被操纵，只要它拥有网络访问权，就可能把 SSH key、API token 或专有源码等机密发送到攻击者控制的目的地。
- **破坏性操作** —— 被操纵的 agent 如果拥有文件系统或 shell 访问权，就可能删除或损坏文件，也可能提交并推送有害代码。
- **工具与供应链风险** —— 恶意或遭到入侵的 MCP server、软件包或依赖，可能引入敌对工具或指令，并诱使 agent 信任它们。

实践中最令人担忧的组合，有时被称为 *lethal trifecta* (致命三要素)：能够访问私有数据、会接触不可信内容，并且可以向外通信 ([Simon Willison - The lethal trifecta for AI agents](#))。每项能力单独存在时尚可管理；如果同一个 agent 同时具备三项能力，一条注入指令就可能读取机密并将其发给攻击者。本章的大多数控制措施，都是通过切断其中一环来降低风险：网络隔离切断外部通信，文件系统隔离限制私有数据访问，审批关卡则让人类介入后果重大的操作。

贯穿本章的前提是：模型不是可信组件。它能力很强，却可能受到操纵；harness 的作用，就是在敌对指令和现实后果之间建立一道可执行的边界。

Anthropic 后续的 containment 工作从两个维度进一步细化了这一威胁模型 ([Anthropic - How We Contain Claude](#))。从风险来源看，问题可能来自滥用系统的用户、模型自身的异常行为，也可能来自通过内容操纵模型的外部攻击者；从防御位置看，控制可以部署在模型、执行环境或外部内容边界上。这个矩阵之所以重要，是因为没有任何单层防御能覆盖所有风险来源：alignment 无法保证模型一定忽略提示注入，sandbox 也无法判断一封获准发送的邮件在语义上是否有害。生产环境中的 containment 必须在三个位置同时实施 defense in depth。

7.2 权限疲劳问题

完全无人监督的 coding agent 很危险，但每执行一步都要申请批准的 agent 同样无法实用。Anthropic 把后一种问题称为 approval fatigue：用户频繁点击 approve，不仅会拖慢开发循环，还会逐渐不再认真核对批准的内容，反而降低安全性 ([Anthropic - Beyond Permission Prompts](#))。解决办法不是增加更多弹窗，而是建立清晰的结构边界：agent 可以在边界内自由行动，只有越界时才请求权限。

Anthropic 的内部使用数据显示，在不降低安全性的前提下，沙箱把权限提示减少了 84% ([Anthropic - Beyond Permission Prompts](#))。

7.3 沙箱同时是笼子、重置按钮和许可证

OpenReview 综述指出，sandbox 的作用不止是保障安全。在 agent 系统中，沙箱同时承担三项任务 ([OpenReview - Agent Harness Engineering: A Survey](#)):

- **Security**：限制不可预测的模型动作和 prompt injection 所造成的 blast radius。
- **Reproducibility**：为 eval、训练轨迹和长时间运行的 session 提供可重置的 baseline。容器或 microVM 可以销毁后重建，开发者工作站则不能。
- **Liveness**：划定 agent 可以自主行动的区域，使它不必在每次写文件、安装软件包或发起网络调用时都询问人类。

第三项作用尤其体现了 agent 系统的特点。沙箱不只是限制行为的笼子，也是允许行动的许可证。它把逐项询问权限改为按 session 配置权限，使长周期自治真正可用，同时避免陷入权限疲劳。

Containment 的强度应随任务风险提高。以下三种常见模式构成了一条实用的防护阶梯 ([Anthropic - How We Contain Claude](#)):

- ****临时容器 (ephemeral container) ****可随时丢弃，成本也较低，适合范围明确、且 agent 需要在较小 blast radius 内充分行动的任务。
- **Human-in-the-loop sandbox**适合交互式工作：安全操作可以自动执行，一旦跨越边界，任务就会暂停并请求审批。
- ****密封虚拟机 (sealed VM) ****通过更强的 kernel 和网络边界隔离高风险 workload，但启动和运营成本也更高。

真正应该问的，不只是“是否使用了 sandbox”，而是“哪些资源仍然可以访问、哪些状态会在 reset 后保留、哪些权限能够跨越边界”。即使计算环境可以重置，也无法消除挂载在其中的 credential、写入环境外部的被投毒记忆，或能够传输私有数据的 egress 路径所带来的风险。

7.4 文件系统隔离必须与网络隔离配对

Claude Code 的沙箱同时建立文件系统和网络两类边界，Anthropic 认为缺一不可。文件系统隔离可以防止受到 prompt injection 的 agent 修改敏感文件，网络隔离则可以阻止它泄露数据或下载恶意软件。如果没有网络隔离，遭到入侵的 agent 可能外传 SSH key；如果没有文件系统隔离，它又可能逃出沙箱并获得网络访问能力 (Anthropic - Beyond Permission Prompts)。

这一实现基于 OS 级 primitive，包括 Linux bubblewrap 和 macOS seatbelt；它不仅约束 Claude Code 的直接操作，也覆盖其启动的所有 subprocess。网络流量先通过 Unix domain socket 进入 proxy，再由 proxy 执行域名限制；agent 请求访问新域名时，也由 proxy 负责向用户确认。该 runtime 已经开源 (Anthropic - Beyond Permission Prompts)。

Claude Code on the web 把这一设计扩展到云端沙箱。在那里，git credentials、signing keys 等敏感凭据从不与 agent 一同放入沙箱。Git 交互由自定义 proxy 处理，只有在确认操作符合权限要求后，proxy 才会附加 scoped credentials，例如确保 push 的目标是预先配置的分支 (Anthropic - How We Contain Claude)。

Egress allowlist 本质上是一种能力授予，而不是一份无害的目的地清单。允许访问 package registry，就意味着允许下载可执行代码；允许访问源码托管站点，也可能意味着允许发布内容；允许访问通用 Web endpoint，则可能补齐 lethal trifecta 中的数据外泄一环。因此，网络策略不能只判断域名，还应同时绑定目的地、协议、操作、身份和任务，并记录每条连接是由哪项规则授权的。

Containment 还必须在建立信任之前就生效。用户打开仓库时，系统可能在显示 trust dialog 之前，已经开始加载配置、发现依赖、启动 language server、运行 hooks 或创建本地 listener。因此，project-open 和 config-load 路径都应按敌对输入处理：可以只解析时就不要执行；关闭自动 hooks 和 listeners；在 workspace 被明确设为可信之前，不提供 credentials 和 egress (Anthropic - How We Contain Claude)。

7.5 身份、Policy 决策、执行与证据

沙箱边界必不可少，但仅有沙箱还不够。Runtime enforcement 还要回答：agent 代表谁、哪个 resource 与 action 在 scope 内、哪个 policy version 生效，以及执行后留下什么证据。NIST 把计算访问决策的组件称为 policy decision point (PDP)，把针对受保护资源的请求落实该决策的组件称为 policy enforcement point (PEP) (NIST - Zero Trust Architecture Glossary)。即使具体产品不使用这些名称，这种分离仍适合 agent action。

生产系统中有三项关键设计：

- **身份与 delegated authorization**：agent 应使用 scoped、短期 authority，而不是继承用户完整的 ambient credential。Credential broker 或 proxy 只在操作获准后、并且只针对预定 audience 附加 secret。
- **Policy decision**：PDP 可以综合 identity、tenant、purpose、tool、规范化参数、resource、session state、environment 和 risk。静态 allow/deny list 容易检查但粒度较粗；上下文 policy 只有在输入和版本被记录时才可追溯。
- **Policy enforcement**：dispatch 路径上的 PEP 负责落实 allow、deny、redact、constrain、stronger-sandbox 或 require-approval。Prompt 中的规则不是 PEP；agent 可以绕过的 hook 也不能提供硬保证。
- **证据**：记录 proposed action、identity、decision 与 policy version、必要的 approval、execution result 和 verified outcome。Trace 可以承载其中一部分证据，但只有满足完整性、integrity、retention、identity 和 access-control 要求的记录才能称为 audit record (NIST - SP 800-53 Rev. 5, Audit and Accountability controls)。

供应链攻击也在这里进入 harness 的治理范围。MCP tool poisoning、tool squatting、rug-pull update、幻觉包名和 retrieval-source poisoning，都跨越了 tool interface 与 governance 的边界。安全的 harness 必须检查工具、软件包、数据集和检索来源的 provenance 与 integrity；只在 prompt 中提醒一句“请小心”远远不够。

7.6 Advisory Hook 与 Blocking Enforcement

沙箱是一类程序化护栏，hooks 和 middleware 则可以添加更细粒度的行为。Claude Code 允许用户定义命令或脚本，并在 agent 启动、工具调用后、停止等生命周期事件上自动运行它们 (HumanLayer - Skill Issue)。LangChain 的 middleware 在结构上与此类似。有些 hook 是确定性脚本，另一些只会向模型注入建议。

自动运行并不等于每个 hook 都是 enforcement point。**Advisory hook** 可以发通知、补充上下文或要求模型重新考虑，但模型仍可能忽略它。**Blocking hook** 只有在所有相关动作都必经该处、deny 会阻止 dispatch、失败能够安全处理、而且 agent 没有其他路径访问受保护资源时，才可以支撑硬规则。区别取决于 placement 与 bypass resistance，而不只是 hook 是否由代码编写。

常见用途包括通知、自动批准或拒绝、系统集成和结果验证。例如，hook 可以在 agent 完成时播放声音，拒绝 migration 命令并要求用户手动执行，发送 Slack 消息或创建 PR，也可以在 agent 停止前运行 typecheck 和 build，把错误返回给 agent，要求它修复后才能结束。HumanLayer 的示例 hook 会在 Claude 每次尝试停止时并行运行 Biome 和 TypeScript；检查通过时静默退出，失败时只返回错误，并以 exit code 2 通知 harness 再次启动 agent。

LangChain 报告称，这类 middleware 是 deepagents-cli 在 Terminal-Bench 2.0 上从 Top 30 提升到 Top 5 的关键因素。PreCompletionChecklistMiddleware 会在 agent 退出前拦截它，提醒它按照任务 spec 验证结果；LocalContextMiddleware 会在启动时梳理工作目录和可用工具；LoopDetectionMiddleware 则记录每个文件的编辑次数，如果同一文件被编辑了 N 次，就提示 agent 重新审视当前方案。这样可以打断“doom loop”，避免 agent 围绕一个已经失败的方法反复尝试细微变体 (LangChain - Improving Deep Agents)。

综述中的 governance 分类把这些 hook 纳入一条更完整的执行管线：pre-invocation check 可以拒绝危险的工具调用；post-invocation hook 可以在不可信输出进入上下文之前添加 taint 标记或执行 redact；stop hook 可以要求完成验证；escalation hook 则可以把难以判断的情况交给人类。操作的后果越严重，安全性就越不应依赖模型是否记得某条指令 (OpenReview - Agent Harness Engineering: A Survey)。

7.7 Mandatory Approval 是 Runtime Gate

Agent 可以调用 request_human_approval 之类的工具，这是一种有用的 workflow suspend/resume 模式；但它不足以执行高风险 action，因为被操纵或判断错误的模型可能直接省略这次调用。对于受保护操作，dispatcher 或 PEP 必须识别规范化后的 action，并在执行前强制要求 approval record。MCP tools 规范同样把呈现工具确认、让人能够拒绝调用列为 client 的责任；确认发生在 invocation 周围，而不是因为模型自愿请求 (MCP - Tools Specification)。

Approval 应绑定 reviewer 实际看到的精确 action：agent 与 delegating identity、tenant、resource、规范化参数、policy version、风险解释、expiry 和执行限制。获得批准后，如果 material argument、destination、credential scope 或 policy version 发生变化，gate 必须重新评估新 action。这样可以避免 time-of-check/time-of-use 间隙：reviewer 批准的是无害提议，真正 dispatch 的却是另一项操作。

Retry 和 fallback 也必须经过同一 gate。先前批准不能授权已经改变的请求；先前 deny 也不能被转换成一条可以由模型说服绕过的 instruction。如果 run 等待期间 policy 发生变化，恢复时应重新作出决策，而不是沿用陈旧 authority。第 14 章继续讨论人工 review surface；第 19 章则把 policy administration 与分布式 PEP 放进 fleet 架构。

7.8 前馈与反馈：控制论视角

Thoughtworks 的 Birgitta Böckeler 从更高层次对这些控制进行了分类 (Thoughtworks - Harness Engineering)。外层 harness 的控制可以分为两个方向：

- **Guides（前馈）** 在 agent 行动之前预判并引导其行为，目标是提高首次产出正确结果的概率。AGENTS.md、skills、参考文档和语言服务器提示都属于这一类。
- **Sensors（反馈）** 在 agent 行动之后观察结果，帮助它自行纠正错误。测试、linter、type checker 和 AI code review 都属于这一类。

如果 harness 只有前馈机制，它会不断发布规则，却无法判断规则是否奏效；如果只有反馈机制，它会反复发现同类错误，却无法提前避免这些错误。两者缺一不可。

这两个方向还可以沿另一条轴继续划分：

- **Computational** 控制，例如 linter、type checker 和结构测试，具有较强的确定性，通常可在毫秒到数秒内完成，结果也可靠。
- **Inferential** 控制，例如语义分析、AI code review 和 LLM-as-judge，能够处理细微判断，但速度更慢、成本更高，而且结果具有非确定性。

两条轴相互独立。AGENTS.md 中的编码约定属于 inferential feedforward；提交时检查模块边界的 ArchUnit 测试属于 computational feedback；/code-review skill 属于 inferential feedback；在启动前创建项目结构的脚本则属于 computational feedforward。设计良好的 harness 会组合使用这四类控制。

7.9 三类调节对象

Böckeler 还按照 harness 所调节的对象，把它们分成三类 (Thoughtworks - Harness Engineering)：

- **Maintainability harness**：调节内部代码质量、重复、复杂度、覆盖率和风格。由于已有数十年的工具积累，这是最容易建设的一类。
- **Architecture fitness harness**：调节性能、可观测性和可调试性，覆盖应用中横跨多个模块的“fitness functions”。
- **Behavior harness**：判断应用的功能行为是否符合预期。这一类问题尚未解决。如今，多数团队把功能 spec 用作前馈，把 AI 生成的测试用作反馈，有时再加入 mutation testing；Böckeler 坦率地指出，目前还不能充分信任 AI 生成

的测试。

这套分类有助于评估 harness 的覆盖范围。一个在 maintainability 上很强、在 behavior 上却很弱的 harness，可能会给团队带来虚假的安全感。

7.10 时机：把质量左移

CI 的经验表明，问题发现得越早，修复成本就越低；同样的原则也适用于 harness 设计。速度快的 computational sensors，例如 linter 和快速测试，应在 commit 前运行；成本较高的 computational 与 inferential sensors，例如 mutation testing 和更全面的 code review，可以在 pipeline 中完成 integration 后运行；用于发现持续漂移的 sensors，例如死代码检测、依赖扫描和日志异常 judge，则应脱离单次变更的生命周期持续运行 ([Thoughtworks - Harness Engineering](#))。

Böckeler 指出，OpenAI Codex 团队的 harness 也采用了类似结构：用自定义 linter 和结构测试落实分层架构，再通过周期性的“garbage collection”扫描漂移，并让 agent 提出修复建议。

7.11 Harnessability、Agentic Readiness 与环境可供性

不同代码库搭建 harness 的难度并不相同。强类型语言天然提供 type-checking sensor；清晰的模块边界使架构约束可以转化为可执行规则；Spring 等 opinionated framework 则封装了许多细节，使 agent 不必自行处理 ([Thoughtworks - Harness Engineering](#))。

Böckeler 借用 Ned Letcher 提出的 *ambient affordances*（环境可供性）来概括这一点：环境本身具有某些属性，可以让 agent 更容易理解、导航和处理其中的系统 ([Thoughtworks - Harness Engineering](#))。Greenfield 团队可以从第一天起就有意识地设计这些 affordance；legacy 团队则面临相反的处境——最需要 harness 的地方，往往也最难建立 harness。

展望未来，Böckeler 提出了 *harness templates*：针对不同服务拓扑，把所需的 guides 和 sensors 打包在一起，例如 JVM CRUD service、Go event processor 或 Node dashboard，再随现有 service template 一同分发。她借助 Ashby 的必要变异度定律解释这一设想：调节器至少要拥有与被调节系统同等的变异度。由此可见，主动限制服务拓扑本身就在减少变异度，也让构建完整 harness 更容易实现 ([Thoughtworks - Harness Engineering](#))。

在生产环境中，一个更宽泛的概念是 **agentic readiness**：自治调用方能否安全地理解和调用系统，观察执行状态，在失败后重试，并在必要时撤销操作？某项服务对人类开发者可能很友好，但如果 API 隐藏副作用、错误信息含糊，或没有稳定的 operation ID，它对 agent 来说仍然很难使用。以下设计可以提高 readiness：

- mutation 接受 idempotency key，并提供可查询的 operation status；
- API 明确区分 read、propose、commit 和 compensate，而不是把这些阶段隐藏在一次不透明调用中；
- 把 machine identity 和 delegated authorization 作为一等概念；
- 错误信息说明失败位置、重试是否安全，以及哪些证据可以证明系统已经恢复；
- 状态变化应当可观察、可归因；如果无法真正 undo，则提供补偿操作。

这些设计就是自治软件所需的 ambient affordances。它们减少了必须由模型承担的概率性推理，也为第 19 章控制平面的 policy、lifecycle 和 audit evidence 提供了稳定的执行接口。

7.12 运营安全：熔断器、终止开关、预算与金丝雀

Sandbox、policy 和 enforcement point（第 7.3-7.7 节）约束 agent 可以做什么。另一类控制则限制 agent 或工具在运行时行为异常所造成的后果。这些措施大多直接借鉴了分布式系统的可靠性工程和安全运营实践，也必须由周边系统执行——因为已经受到操纵或陷入循环的 agent，不能被指望自行约束行为。

- **熔断器 (circuit breaker)**。在不稳定或成本较高的依赖外加一层保护，例如工具、下游服务或子代理；失败次数达到阈值后，熔断器会跳闸，使后续调用立即失败，避免长时间挂起或形成重试风暴 ([Fowler - CircuitBreaker](#))。对 agent 来说，这能限制持续报错的工具，或反复重试同一失败操作的 agent 所造成的爆炸半径。它与第 5 章的上下文策略相互补充：让一次仍可行动的失败保持可见，同时防止它无限重复。
- **终止开关 (kill switch)**。由人类或策略触发，能够立即停止一个 agent 或整个 fleet，并且不依赖 agent 自身的控制流。受到 prompt injection 的 agent 可能正在主动违背原有指令，因此终止开关必须存在于 harness 中，例如 supervisor 进程、可撤销凭证或 sandbox 拆除机制；不能只在 prompt 中写一句“收到要求时停止”。
- **动作预算、迭代上限与成本调节器**。为工具调用次数、token、墙钟时间或花费设置硬性上限。达到上限后，循环必须停止并上报，而不能继续失控运行。这是第 13 章循环停止规则和第 18 章单任务预算在运营层面的实现：无界循环既会造成账单失控，也会让爆炸半径失控。
- **金丝雀令牌 (canary token)**。在受到 prompt injection 的 agent 可能读取或外泄的位置放置假机密，例如未使用的 API key、诱饵文件或陷阱 URL。金丝雀触发回调时，会发出一个高可信度警报，表明 agent 已被操纵去接触本不该访问的数据 ([Thinkst - Canarytokens](#))。金丝雀与 sandbox 不同：它不会阻止 lethal trifecta 中的数据外泄环节（第 7.1 节），而是负责发现这类行为。因此，当预防措施并不完美时，它可以成为最后一道检测防线。

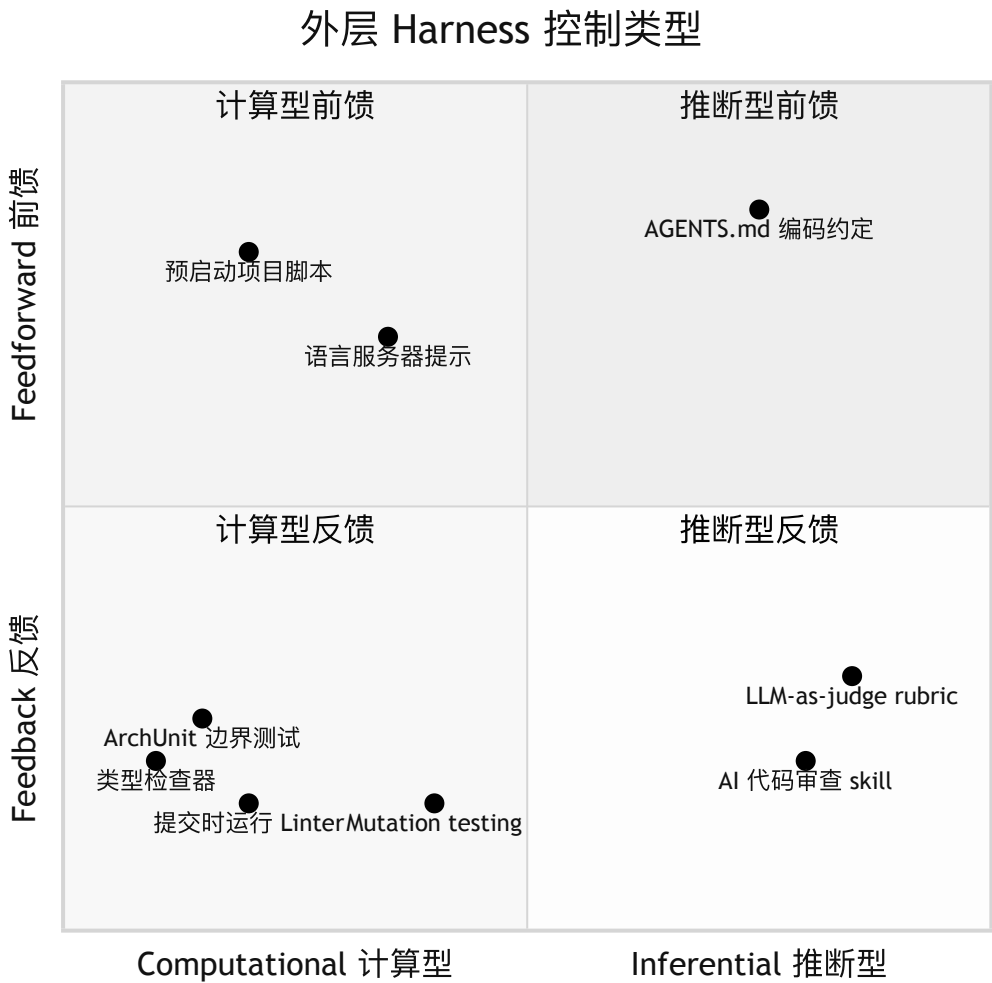
本节遵循的原则与全章一致：失败后果越严重，就越不能依赖模型主动选择避开风险。预防措施，如 sandbox 和策略，应与检测措施，如金丝雀和第 18 章的漂移告警，配合使用；任何一类措施单独使用都不充分。

图：从提议到受控效果



模型可以提出并解释 action；只有外部决策与执行路径才能授权 credential、跨越 sandbox 边界，或产生受保护的副作用。

图：前馈/反馈 x 计算/推断



要点

- 先明确威胁模型：当私有数据、不可信内容和外部通信组成“致命三要素”时，prompt injection、数据外泄、破坏性操作和供应链风险会尤其危险。
- Anthropic 在内部 sandbox rollout 中报告权限提示减少了 84%：这是带日期的 case study，不是通用缩减比例。
- 沙箱承担三项任务：security、reproducibility 和 liveness。
- 文件系统与网络隔离必须配对：二者对应不同攻击向量，单独使用都不足。
- Containment 是一个矩阵，而不是单个 sandbox：应同时在模型、环境和内容边界上防范用户滥用、模型异常与外部攻击，并根据风险选择临时容器、交互式 sandbox 或 sealed VM。
- Egress 代表真实权限：允许访问一个目的地，就赋予了 agent 一项实际能力，因此网络访问必须绑定操作、身份和任务；仅仅打开项目，不应自动获得 ambient trust。
- **决策与执行不同：**PDP 计算 policy，动作路径上不可绕过的 PEP 负责落实。

- ****Policy** 要求时 approval 必须强制执行：**模型主动调用 approval tool 是有用的编排方式，但不能成为唯一的高风险 gate。
- ****Hook** 的强度不同：**advisory hook 可以引导或通知；只有 blocking、不可绕过的 hook 才能执行硬规则。
- ****Trace** 不会自动成为 audit：**应在明确的 integrity 与 retention 要求下保存 identity、policy decision、approval、execution 和 outcome。
- **前馈与反馈都需要**：guide 没有 sensor 就没有学习回路；sensor 没有 guide 只能事后反应。
- **Harness 覆盖三类对象**：maintainability、architecture fitness 和 behavior，其中 behavior 仍然最难解决。
- **环境可供性很重要**：强类型语言和 opinionated framework 能降低搭建 harness 的难度。
- **Agentic readiness 是 API 的属性**：幂等性、明确的 operation status、machine identity、重试语义、可观察状态和补偿操作，都能让系统更适合自治调用方。
- **运行时安全还需要运营控制**：熔断器、终止开关、动作与成本预算、金丝雀令牌可以在异常发生时限制后果。它们必须存在于 harness 中，因为不能指望受到操纵的 agent 主动停止自己。

延伸阅读

- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts*, Anthropic, Oct 2025.
<https://www.anthropic.com/engineering/claude-code-sandboxing>
- Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026.
<https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026.
<https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026.
<https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- Martin Fowler, *CircuitBreaker*, martinfowler.com, Mar 2014. <https://martinfowler.com/bliki/CircuitBreaker.html>
- Thinkst, *Canarytokens* (free tripwire tokens). <https://canarytokens.org/>
- Anthropic Safeguards Research Team, *How We Contain Claude*, Anthropic, May 2026.
<https://www.anthropic.com/engineering/how-we-contain-claude>
- NIST, *Zero Trust Architecture Glossary*. <https://pages.nist.gov/zero-trust-architecture/glossary.html>
- NIST, *SP 800-53 Rev. 5: Security and Privacy Controls for Information Systems and Organizations*.
<https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
- Model Context Protocol, *Tools Specification*, 2025 年 6 月 18 日。
<https://modelcontextprotocol.io/specification/2025-06-18/server/tools>

第 8 章：模型选择、路由与推理预算

Agent = Model + Harness。模型选择属于 harness configuration，但“把它发送给另一个模型”并不是一种单一操作。Router、quality cascade、reliability fallback、hedged request 与 human escalation 解决的是不同问题，也有不同的 correctness conditions。

本章只有在目标配置能够履行当前调用的 input、output、tool、state、policy 与 data-handling contracts 时，才把一次 route 视为安全。姊妹卷 *LLM Foundations* 解释模型行为；本章负责运行层面的选择与切换。

8.1 选择 Model-Plus-Harness Configuration

选择单位通常是一次 model call 或 workflow step，而不是整个 application。如果 evaluation 表明较小配置仍能保持所需 outcome，一个 loop 可以把 planning 交给能力较强的模型，把范围明确的 classification 或 extraction 交给较小模型。Anthropic 同样建议采用能够成功的最简单 workflow，只在 model-driven complexity 产生可测价值时才增加它 ([Anthropic — Building Effective Agents](#))。

被选中的对象不只是 model ID：

```
RouteTarget {
  provider, model_snapshot, endpoint,
  modalities, context_and_output_limits,
  tool_protocol, tool_schemas,
  structured_output_contract,
  reasoning_configuration,
  policy_profile, data_region,
  timeout, price_schedule
}
```

例如，OpenAI model catalog 会按 model 发布 context window、maximum output、modalities、tools、function calling 与 structured-output support 等 capability，而不是把它们视为某种通用 API model 的属性 ([OpenAI — Models](#))。Deployment 必须固定实际验证过的 capability snapshot；即使 application code 没有变化，可变的 provider alias 也可能发生变化。

Per-call selection 有一个重要例外：stateful assistant turn 或 tool-use continuation 可能把多次 API call 绑定到同一个 provider-specific protocol。除非 harness 有明确的 translation 或 restart strategy，否则在中途切换并不安全。第 8.5 节会回到这条 reasoning-state boundary。

8.2 五种不同的 Routing Mechanism

不要用 routing 统称所有模型切换。

Mechanism	Decision point	Purpose	Required signal
Router	Attempt 之前	为 request 或 step 匹配合适 target	执行前可获得的 features，以及经过评测的 routing policy
Quality cascade	一次较便宜的 attempt 之后	首次结果未达到质量门槛时升级	能够接受或拒绝该结果的 verifier
Reliability fallback	Infrastructure 或 availability failure 之后	在兼容 target 上继续提供服务	Failure classification 与预先验证的 fallback target
Hedging	并发执行或短暂延迟后执行	让等价 attempts 竞速，以降低 tail latency	可安全重复的 operation、winner rule 与 loser cancellation
Human escalation	Automated attempt 之前或之后	把 ambiguity、authority 或 high-impact judgment 移交给人类	Stop condition、review package 与 durable wait/resume path

RouteLLM 是 learned pre-execution router 的一个研究案例：它使用 preference data 训练，在强弱模型之间做选择，并在自己的 benchmark suite 上评测 cost-quality tradeoff ([Ong et al. — RouteLLM](#))。它测得的节省属于那套 router、model pair、prices 与 evaluation set，并不能保证 difficulty routing 会在另一个 workload 上获得收益。

FrugalGPT 是 quality cascade 的一个研究案例：它评估某个 service 首先给出的答案，再有条件地升级到另一个 service，并在所研究的数据集上执行 budget-constrained optimization ([Chen et al. — FrugalGPT](#))。没有可靠 acceptance signal 的 cascade 只是一条更慢的 route；schema validity 本身既不能证明 semantic correctness，也不能证明 task success。

Hedging 不是 fallback。The Tail at Scale 描述了在大型 online services 中通过发送 redundant requests 降低 tail latency，并把额外工作作为这种方法的代价 (Dean and Barroso — The Tail at Scale)。在 agent system 中，只有当重复 attempts 均符合 policy，且不会各自造成 external side effects 时，才可以 hedge model generation。Harness 只能选择一份结果，并阻止 losing proposals 进入 tool dispatch。

Human escalation 不是“更强的 model tier”。当缺少的是 authorization、responsibility、domain judgment 或 clarification，而不是更多 inference compute 时，应当使用它。Workflow 必须在等待时保存 state，并向 reviewer 展示相关 evidence、proposed action、uncertainty 与 alternatives。

8.3 Fallback 之前必须满足 Compatibility

Fallback target 只有在通过当前调用的 compatibility gate 后才具备资格。应测试精确的 model snapshot、endpoint 与 harness adapter；provider family 和“OpenAI-compatible”transport 都不能证明 semantic equivalence。

Dimension	Compatibility question	Failure if ignored
Modalities	Target 能否接收每种 input type，并以要求的 fidelity 产生所需 output type？	Image、audio、file 或 tool result 被丢弃或错误转换。
Context and output limits	序列化 input、预留 reasoning/output budget、tool schemas 与预期 result 是否都能放入限制？	Truncation、request 被拒绝或缺少 final output。
Tool protocol and schema	Target 是否支持相同的 call/result protocol、parallelism、tool-choice modes、schema subset 与 call-ID continuation？	Invalid call、丢失 correlation 或 dispatch behavior 改变。
Structured output	Target 是否强制执行所需 schema，以及约定的 refusal/error representation？	Downstream code 接收到 prose 或另一种 failure shape。
Reasoning and continuation state	Provider-specific reasoning items、signatures、response IDs 与尚未完成的 tool turn 能否继续，或者能否被有意重启？	Request 被拒绝，或者 continuation 悄悄丢失先前工作。
Safety and policy	Target 是否获准用于该 tenant、risk class、content、tool set 与 action scope，harness 是否统一处理 refusal？	Fallback 绕过 model allowlist 或改变 policy behavior。
Data region and retention	Endpoint、model、tools、storage、processing region、retention mode 与 subprocessors 是否都获得许可？	Route 违反 residency 或 data-handling requirements。

Capability difference 是具体的 API fact，并非理论边界情况：model catalog 会公开不同的 modality、tool、structured-output 与 context support (OpenAI — Models)。Data-region support 也可能随 endpoint、model、tool、storage 与 processing mode 变化；例如 OpenAI data-control 文档会区分 regional storage 与 regional processing，并按 service 列出 eligibility (OpenAI — Data Controls)。应把 provider 当前文档当作 compatibility registry 的输入，而不是 local contract tests 的替代品。

Latency、price、capacity 与 rate limits 属于 selection constraints，而不是 semantic compatibility。一个 target 可以兼容，却对 request 的 service tier 而言太慢或太贵。反过来，如果 fast target 无法履行 output 或 policy contract，它就没有资格被选中。

为已知 targets 预先计算 compatibility graph，再在 dispatch 时重新检查 request-specific values：实际 token count、attached modalities、selected tools、tenant policy、data region 与 provider 当前状态。失败时应关闭到另一个 eligible route 或 human escalation；不能为让 target 接受调用而悄悄剥离 unsupported input。

8.4 Fallback 与 Retry 必须尊重 Side Effect

Model call 提出 output；第 6 章的 dispatcher 决定是否执行 external action。Retry 是低风险还是危险，取决于是否越过这条边界。

重试前先对 attempt 分类：

- 1. 尚未提出 action：retry 或 fallback 可能重复 inference cost，但不会重复 external effect。
- 2. 已提出 action，但尚未 dispatch：丢弃旧 proposal，重新 route，并对新 proposal 正常执行 authorization。
- 3. Dispatch outcome 已知：记录 outcome；只有 tool 文档中的 semantics 允许时才 retry。
- 4. Dispatch outcome 未知：在决定 repeat、compensate 或 escalate 之前，先通过 action key 或 idempotency key 查询 target system。

HTTP Semantics 规定，除非 client 已知 request semantics 确实具有 idempotency，或者可以判断 original request 没有被应用，否则不应自动 retry non-idempotent request (RFC 9110 §9.2.2)。即使 tool transport 不是 HTTP，agent retry 也需要同样的纪律。

对于 side-effecting action，使用能跨越 model fallback、gateway retry、worker restart 与 timeout 的稳定 action ID 或 idempotency key。在 dispatch 前持久化 proposed operation，并把 external resource ID 或 outcome 与之一起保存。如果 target system 既不提供 idempotency，也不提供 status lookup，ambiguous timeout 就属于 human-review 或 compensation case，而不是再次调用 action 的许可。

Retry 不会仅仅因为第一次 attempt 获准，就自动继承 authorization。当 identity、resource state、action arguments、policy version、route target 或 approval validity 发生变化时，应重新评估 policy。Fallback 也不能把 refusal、deny 或 mandatory approval 重新解释成 infrastructure error。

8.5 Reasoning Budget 是 Provider-Specific Control

Reasoning effort 是一条有用的 routing axis，但并不存在名为“一个 reasoning token”或“high effort”的跨 provider 通用单位。每个 provider 都会定义自己的 parameters、token accounting、supported models、continuation objects，以及它们与 tools 和 caching 的交互。

OpenAI 当前文档按 model 定义 reasoning.effort levels，并通过 reasoning.context 与 response continuation 管理 persisted reasoning；其指南要求在 representative workloads 上比较不同设置，而不是假设最高 level 一定最佳

([OpenAI — Model Guidance](#))。Anthropic 则按 model 记录 manual/adaptive thinking mode、budget_tokens 或 effort controls，以及 tool-use continuation 中保留 thinking blocks 的规则 ([Anthropic — Extended Thinking](#))。这是两套不同的 API contracts，而不是同一个通用 control 的两种拼写。

Harness 应暴露 provider-neutral objective，例如：

```
ReasoningPolicy {
  quality_tier,
  max_end_to_end_latency,
  max_call_cost,
  allow_persisted_reasoning,
  data_handling_profile
}
```

Provider adapter 把这个 objective 映射到受支持 parameters，并记录 resolved settings 与 usage。不能只按名称把 high 翻译成 high。应在同一 task slice 上校准每个 provider/model configuration。

Reasoning state 同样属于 compatibility。Anthropic 要求某些 tool-use continuation 完整、原样传回 thinking blocks；OpenAI 则通过 provider-specific response items 与 IDs 记录 continuation ([Anthropic — Extended Thinking](#)；[OpenAI — Model Guidance](#))。Harness 不能把一个 provider 的 opaque state 序列化进另一个 provider 的 protocol，就把它称为 continuation。系统必须留在原 target，从 explicit context summary 与 durable tool outcomes 重启，或者使用拥有独立 eval 的 translation。

Visible 或 summarized reasoning 不是 verifier。Provider 可能省略或总结 internal reasoning，返回的 representation 也只属于各自的 API contract ([Anthropic — Extended Thinking](#))。应当评判 answer、evidence、tool outcomes 与 environment state。

8.6 先应用 Policy，再优化 Cost

Routing 是 constrained optimization problem。首先确定 eligible set，然后才在集合内进行优化。

Eligibility 可能取决于：

- tenant 与 user entitlements；
- approved provider、model snapshot 与 endpoint；
- data classification、storage/processing region 与 retention mode；
- 所需 safety 或 domain policy profile；
- allowed modalities、hosted tools 与 third-party subprocessors；
- action risk、approval status 与当前 incident controls。

Eligible set 之外的 cheap route 并不是节省。Provider data-residency 文档表明，eligibility 为什么必须覆盖完整 request path：regional storage 不一定意味着 regional processing，third-party tools 也可能有各自的 policy ([OpenAI — Data Controls](#))。

为每次 routing decision 记录 policy version 与 eligibility result。如果 provider error 后不存在 eligible automated target，应返回显式 fail-closed result 或 human escalation，而不是继续落到未经批准的 default。

8.7 评测 Routing System，而不只是各个 Model

Routing policy 即使让每个 candidate 单独看都很好，也可能因为选错 candidate 而失败。应评测完整的 model-plus-harness configurations、router decisions、escalation signals 与 failure handling。Anthropic 的 evaluation 指南会区分 task、trial、grader、transcript、outcome、agent harness 与 evaluation harness；这种区分有助于把 routing failure 归因到 decision，而不只是 selected model ([Anthropic — Demystifying Evals for AI Agents](#))。

使用 representative slices，而不是一个混合平均数：

Slice	What to measure
Quality	Task success、required evidence、verifier false accept/reject，以及 easy/hard 与 domain cohorts
Cost	每个 successful outcome 的 input、output、reasoning、cache、tool、hedge 与 escalation cost
Latency	Queue、router、provider、tool、verifier 与 end-to-end p50/p95/p99；各阶段 timeout
Failure class	Rate limit、timeout、transport error、context overflow、schema failure、refusal、tool mismatch、ambiguous side effect
Policy	Ineligible-route rate、region/retention/model-allowlist violations，以及 approval 与 tenant-isolation tests
Routing behavior	Route distribution、misroutes、cascade escalation rate、fallback success、hedge win/cancel rate、human escalation rate

在 router eval 中，同时标注代价较高的 false escalation，以及更严重的 false economy：被发送给 cheap target、最终却违反 task 或 policy 的 cases。在 cascade eval 中，独立评估 first answer，避免 weak verifier 让 cascade 显得高效。在 fallback test 中，注入每类 failure 并断言哪些 transition 合法。在 hedging test 中，统计 duplicate tokens，并验证 losing outputs 无法 dispatch action。

在 trial manifest 中固定 model snapshots、adapters、prompts、tools、policy versions、prices 与 regional endpoints。Current price 应作为带日期的 configuration data 报告，而不是 model family 的永久属性。只有 quality、cost、latency、failure 与 policy slices 全部满足 gate，一条 route 才算 ready。

8.8 AI Gateway 是部署位置选项，而不是架构角色

AI gateway 可以集中 provider adapters、credentials、quotas、routing、retries、telemetry，有时也会承载 policy checks。它可以是有用的 implementation location，但产品名称并不会让它自动成为 control plane 或 policy enforcement point。

Kubernetes 用 *control plane* 指管理整个 cluster overall state 的 components，并把它们与运行 workloads 的 node components 区分开来 ([Kubernetes — Components](#))。在本书中，fleet control plane 同样负责跨多个 agents 管理 desired configuration、identity、policy administration/decisions、versions 与 lifecycle。即使另有 control service 对 gateway 进行配置，位于同步 model-call path 上的 gateway 通常仍是 data-path component。

NIST 按 function 定义 policy enforcement point (PEP)：subject 请求 protected resource 时，它负责强制执行 access-policy decision ([NIST — Zero Trust Glossary](#))。Gateway 只有在不可绕过的路径上实际执行某项 decision 时，才是该 decision 的 PEP。记录 policy result、增加 guardrail callback 或托管 routing logic，本身并不会使 gateway 成为 tool execution、file access 或 downstream side effects 的 PEP。

集中化也会带来 correlated risk。围绕 gateway 设置明确 timeout 与 circuit breaker，保留 end-to-end call/action IDs，并保证 gateway retry 与 application retry 使用同一套 idempotency/outcome rules。把 route policy 保存为 versioned configuration，并在脱离具体 gateway product 的情况下独立测试它。

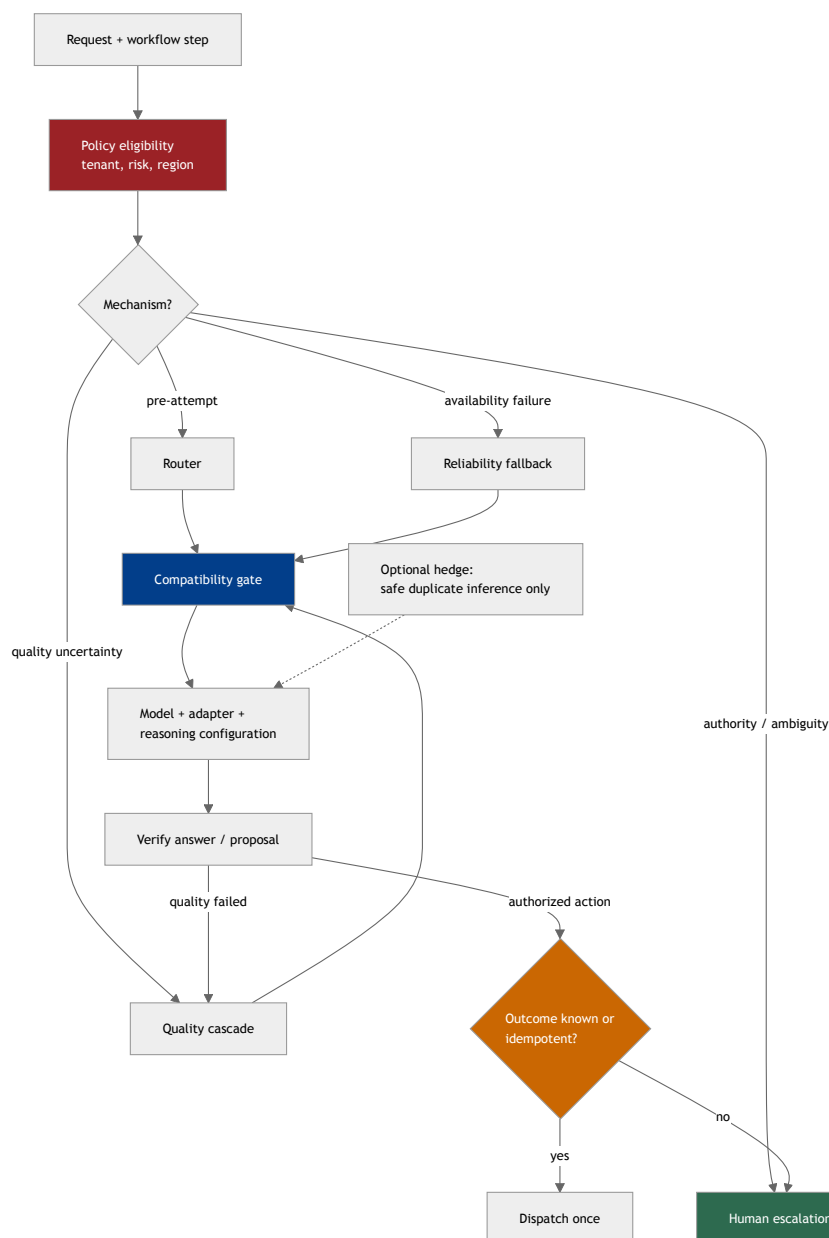
8.9 把每次 Route Change 当作一次 Release

Model upgrade、provider swap、reasoning-setting change、新的 cascade verifier、region change 或 gateway policy edit，都会改变 model-plus-harness configuration。应使用与代码相同的 eval 与 rollback discipline 发布它们。

只有在 data policy 允许，而且 duplicate request 不可能触发 action 时，才使用 shadow traffic。在有边界的流量上 canary eligible configuration，比较第 8.7 节中的 slices，并在 trace 中保留 route target 与 decision reason。Alias 便于 provider update，但如果 provider 暴露 resolved model snapshot，production reproducibility 就要求将其记录下来。

Rollback 同样受 compatibility gate 约束。较旧 target 可能已经无法支持新的 tool schema、output contract 或 policy requirement。“Last known good”应指相对于当前 harness contract 的 last known good，而不只是以前部署过的 model ID。

图：经过 Policy 与 Compatibility Gate 的 Route



要点

- 选择 **configuration**，而不是 **model name**：endpoint、modalities、limits、tools、output contract、reasoning state、policy 与 region 必须一起迁移。
- 明确 **mechanism** 名称：router、quality cascade、reliability fallback、hedging 与 human escalation 各有不同 trigger 与 proof。
- **Compatibility** 先于 **fallback**：绝不能仅仅为了让 backup target 接受调用，就剥离 input、tool、state 或 policy requirement。
- **Retry** 不等于重复 **side effect**：再次 dispatch 前使用稳定 action ID、idempotency 与 outcome verification。
- **Reasoning control** 依赖 **provider**：通过经过测试的 adapter 映射中立的 quality/latency/cost objective；opaque continuation state 通常不可移植。
- **Policy** 定义 **eligible set**：只能在 tenant、risk、data path 与 action 允许的 targets 中优化 cost 与 latency。
- 把 **routing** 当作系统评测：分别切分 quality、cost、latency、failure class、policy 与 routing behavior。
- **Gateway** 不自动等于 **control plane** 或 **PEP**：应按它实际承担的 management 与 enforcement function 分类。

延伸阅读

- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024.
<https://www.anthropic.com/engineering/building-effective-agents>

- Isaac Ong et al., *RouteLLM: Learning to Route LLMs with Preference Data*, 2024. <https://arxiv.org/abs/2406.18665>
- Lingjiao Chen, Matei Zaharia, and James Zou, *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*, 2023. <https://arxiv.org/abs/2305.05176>
- Jeffrey Dean and Luiz André Barroso, *The Tail at Scale*, 2013. <https://research.google/pubs/the-tail-at-scale/>
- OpenAI, *Models*. <https://developers.openai.com/api/docs/models>
- OpenAI, *Model Guidance*. <https://developers.openai.com/api/docs/guides/latest-model>
- Anthropic, *Extended Thinking*. <https://platform.claude.com/docs/en/build-with-claude/extended-thinking>
- IETF, *RFC 9110: HTTP Semantics*, §9.2.2. <https://www.rfc-editor.org/rfc/rfc9110.html#section-9.2.2>
- OpenAI, *Data Controls in the OpenAI Platform*. <https://developers.openai.com/api/docs/guides/your-data#data-residency-controls>
- Anthropic, *Demystifying Evals for AI Agents*. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Kubernetes, *Kubernetes Components*. <https://kubernetes.io/docs/concepts/overview/components/>
- NIST, *Zero Trust Glossary*. <https://pages.nist.gov/zero-trust-architecture/glossary.html>

第 9 章：Agentic 工作流程模式

Workflow pattern 描述系统如何排列模型调用、工具、检查和状态转换。它们不会改变第 6 章建立的责任边界：模型可以提出内容、route、任务分解或下一步动作，而 harness 或 runtime 负责验证 proposal、执行经过授权的 effect、记录状态，并返回 observation。对于由 client 执行的工具，Anthropic 的 API contract 同样要求模型发出 tool-use request，再由应用代码执行（[Anthropic — How Tool Use Works](#)）。

本章比较 control-flow shape。第 10 章负责 durable execution state 与 event history，第 11 章负责 system evaluation，第 13 章展开 verifier hierarchy。仅靠一张 workflow diagram 无法获得其中任何一种能力。

9.1 三种控制形态

Anthropic 把 **workflow** 定义为沿预设代码路径调用模型和工具，把 **agent** 定义为由模型动态决定 process 和 tool use，并建议从能够满足需求的最简单架构开始（[Anthropic — Building Effective Agents](#)）。落到实现上，更准确的做法是区分三种形态：

形态	谁选择下一步？	谁执行并持有状态？	最适合的任务
Deterministic workflow （确定性工作流）	代码选择预定义 transition；模型可以填充某个 node 的输出	Workflow controller/runtime	分支和检查可以预先确定的稳定流程
Model-directed loop （模型引导循环）	模型根据当前 observation 提出下一步 action	Harness 负责验证和 dispatch；runtime 持有 budget、status 和 stop rule	下一步有效动作无法预先可靠枚举的开放任务
Hybrid workflow （混合工作流）	代码固定外层 graph，模型在有限 node 或 loop 中进行选择	外层 controller 保持权威；内层循环具有明确 budget 和 handoff contract	同时需要可预测边界与局部灵活性的生产任务

Augmented LLM 是 Anthropic 对接入 retrieval、tool 和 memory 的模型所用的术语；它可以出现在上述任何一种形态中（[Anthropic — Building Effective Agents](#)）。在一个 node 中加入模型，不会让 deterministic workflow 自动变成 autonomous agent。反过来，允许模型选择 action，也不意味着向它授予 credential，或把 transcript 变成 execution state 的权威来源。

Routing 这个词也出现在两个层次。Workflow routing 选择 processing branch；model routing 选择由哪个模型或 reasoning budget 执行一次调用。第 8 章负责第二种决策。一个 workflow 可以同时使用两者，但应分别记录和评测。

9.2 每种模式都需要运行契约

只有方框和箭头的图并不完整。每种 pattern 都必须定义六个字段：

- Applicability**（适用条件）：什么样的任务结构值得引入额外调用和协调？
- State owner**（状态所有者）：哪个外部组件记录当前 node、attempt、artifact、budget 和 pending work？
- Stop condition**（停止条件）：哪个 success、failure、abstention、escalation、cancellation 或 exhaustion event 会结束执行？
- Failure propagation**（失败传播）：如何表示无效输出、工具故障、timeout、partial effect 与 downstream contamination？
- Evaluation unit**（评测单元）：质量是在 call、route、branch、iteration、完整 run、最终 artifact 还是 external outcome 上打分？
- Effect boundary**（影响边界）：模型 proposal 在哪里得到验证、授权、执行与确认？

Anthropic 的 evaluation 术语区分 task 与重复 trial、transcript 与最终 outcome，以及 agent harness 与 evaluation harness（[Anthropic — Demystifying Evals for AI Agents](#)）。这一区分对本章很重要：一种模式可能生成看起来很漂亮的 transcript，同时 route 到错误分支、重复执行 side effect、遗漏 subtask，或接受质量不合格的最终 artifact。Pattern eval 必须检查相关 external state 和 outcome，而不能只看模型文本。

所有模式的 effect boundary 都相同。第 6 章定义了从 proposal 到 outcome 的完整 lifecycle。以下各节重点讨论另外五个字段，并默认任何有实际后果的 tool call 仍须经过该 lifecycle。

9.3 组合式工作流程模式

Anthropic 把 prompt chaining、routing、parallelization、orchestrator-workers 和 evaluator-optimizer 列为常见组合模式；它们在 latency、cost 和 predictability 上各有取舍（[Anthropic — Building Effective Agents](#)）。根据 selection 发生的位置不同，这些模式可以是 deterministic、model-directed 或 hybrid。

9.3.1 Prompt Chaining (提示链)

Prompt chaining 把一个 stage 经过验证的输出交给后续 stage。即使每个 stage 都包含模型调用，chain 通常仍是 deterministic 的：选择下一 node 的是代码，而不是模型。

- **适用条件：** 任务可以拆成稳定序列，而且 intermediate interface 能够清楚定义。
- **状态所有者：** Workflow controller 记录当前 stage、经过验证的 intermediate value、artifact 和 attempt。
- **停止条件：** 最后一个 stage 通过 acceptance gate，或者任一 stage 进入 terminal failure 或 escalation path。
- **典型失败：** 早期遗漏或格式错误的 intermediate value 污染后续 stage；串行执行也会累积 latency。
- **评测单元：** 同时评测每个 stage contract 与 end-to-end outcome。最终分数很高，也不能掩盖某一步发生了数据泄漏或错误 effect。

如果可以使用 typed intermediate representation，就不要在 stage 之间传递任意 prose。Gate 可以在无效 proposal 变成下一 stage 的假定事实之前拒绝或修复它。

9.3.2 Routing (路由)

Routing 把输入分配到专门 branch。Deterministic rule、传统 classifier 或模型都可以提出 route；harness 应用 routing policy 并执行 dispatch。

- **适用条件：** 输入能够划分为具有运行意义的类别，各类别有不同 handler，而且类别边界可以被评测。
- **状态所有者：** Router/controller 记录候选 label、confidence 或 abstention signal、policy version、选中 branch 与 fallback。
- **停止条件：** 一个获准 branch 接受该输入，或者 router 按明确规则 abstain、escalate 或 reject。
- **典型失败：** 含糊输入被高置信度地分错，并在看似合理但实际错误的 branch 中静默失败。
- **评测单元：** 在带 label 的 slice 上评测 routing decision，并按 route 条件评测 downstream task outcome；同时纳入 abstention 与 fallback cost。

应把 workflow routing 与第 8 章的 model routing 分开。“送到 billing branch”和“使用 model B”是不同的决策，即使两者由同一组件计算。

9.3.3 Parallelization (并行化)

Parallelization 会在 aggregation 之前启动多个 branch。**Sectioning** 分派彼此独立的 subtask；**voting** 让多个 worker 处理同一个 decision ([Anthropic — Building Effective Agents](#))。模型可以提出拆分方式或 vote，但必须由外部 coordinator 创建 branch record、执行限制，并聚合已经完成的结果。

- **适用条件：** 各 branch 足够独立，能够并行执行；或者多次 sample 对某个 decision 的价值已经得到测量。
- **状态所有者：** Coordinator 持有 branch set、call ID、deadline、partial result、cancellation status 和 aggregation rule。
- **停止条件：** 所有必需 branch 完成、达到指定 quorum，或 deadline/failure policy 选择 partial-result 或 escalation path。
- **典型失败：** 把部分完成误认为整体成功；worker 重复工作或共享同一错误前提；aggregation 丢弃少数证据。
- **评测单元：** 同时评测 branch 和 aggregate outcome，包括 coverage、agreement calibration、wall-clock latency 与总资源消耗。

一致不等于证明。Voting 需要测量 diversity 和 calibration；sectioning 需要显式 coverage check，防止拆分时遗漏必要工作。

9.3.4 Orchestrator-Workers (编排者—工作者)

在 orchestrator-workers 中，模型动态提出任务分解，worker 完成有边界的 subtask，最后由 synthesis step 合并结果。Anthropic 在自己的研究系统中使用了这种模式：lead agent 把搜索工作委派给并行 subagent，再综合调查结果 ([Anthropic — How We Built Our Multi-Agent Research System](#))。这是一个有文档记录的系统与 workload，并不能证明该模式会改善每种任务。

- **适用条件：** 必要的 subtask shape 取决于具体输入；subtask 具有清晰 ownership；结果能通过显式 artifact 或 evidence 合并。

- **状态所有者：** Coordinator/runtime 持有 task graph、worker identity 与 scope、budget、artifact、completion status 和 synthesis attempt。
- **停止条件：** 所有必需 subtask 都已交代清楚，且 synthesized outcome 通过 task-level gate；或者达到 fan-out、time、cost、no-progress limit。
- **典型失败：** 无限制 fan-out、coverage 重复或遗漏、冲突写入、provenance 丢失，或 synthesis 夸大 worker 的发现。
- **评测单元：** 评测完整 task outcome，同时评测 decomposition coverage、worker handoff quality、synthesis faithfulness、latency 与 cost。

第 5 章的 handoff contract 在这里同样适用：worker 结果需要 evidence、artifact pointer、identity 与 authority scope、uncertainty 和 verification status。更高权限的 orchestrator 不能把未经验证的 worker summary 直接转化成高权限 effect。

9.3.5 Evaluator-Optimizer（评估者—优化者）

Evaluator-optimizer 在 candidate generation、feedback 与 revision 之间循环。Anthropic 建议在 evaluation criteria 清楚，而且迭代确实带来可验证收益时使用该模式（[Anthropic — Building Effective Agents](#)）。模型生成的 critique 是 controller 的一项 observation；它本身既不会执行下一轮 iteration，也不能宣布 external task 已经完成。

- **适用条件：** Candidate 可以迭代改善；acceptance criteria 可重复；收益足以抵消额外调用。
- **状态所有者：** Loop controller 持有 candidate version、feedback、score、iteration count、budget 与 accepted artifact。
- **停止条件：** Acceptance rule 通过、进展停滞、达到 iteration/budget cap，或 evaluator abstain 并升级处理。
- **典型失败：** Evaluator 和 generator 共享同一 blind spot；score 来回振荡；candidate 迎合 rubric 却没有改善真实 outcome；循环永不收敛。
- **评测单元：** 检查每次 iteration 和最终 accepted outcome；测量相对初始 candidate 的改善、false accept/reject、资源消耗，以及优化标准以外的 regression。

这一模式只规定 evaluation checkpoint 放在哪里。第 11 章解释 task、trial、grader 和 outcome measurement；第 13 章解释 deterministic check、self-check 和 independent evaluator 分别适用于何种 assurance。这里不规定普遍适用的“maker 绝不能是 checker”规则。

9.4 模型引导循环与有边界的混合代理

ReAct 风格的循环交替进行模型推理、action proposal、由外部产生的 observation 和下一次模型 decision（[Yao et al. — ReAct](#)）。在生产系统中，harness 必须让这层抽象具有可运行语义：

- **适用条件：** 下一项有效 action 在很大程度上取决于 observation，无法在固定 graph 中预先预测。
- **状态所有者：** Runtime 持有 run/step/action identity、context reference、budget、approval、tool outcome 和 current status。Transcript 是面向模型的视图，而不是权威状态。
- **停止条件：** 外部检查确认 success、没有剩余工作、task blocked、需要 approval 或 human input，或者 iteration/time/cost/no-progress/cancellation limit 被触发。
- **典型失败：** Drift、反复执行无效 action、在 outcome unknown 时 retry、过早自行宣布完成，或者 context 增长导致早期 constraint 消失。
- **评测单元：** 评测完整 run 与 external outcome，再在 action 和 step 层诊断；同时纳入 side effect、recovery behavior、latency 与 resource use。

Hybrid 会把这种 loop 放进 deterministic envelope。代码可以固定 ingest → analyze → review → publish，而有边界的 agent 在 analyze node 内选择 search 和 analysis action。外层 workflow 持有 entry condition、allowed capability、budget、handoff artifact 与 exit gate。这种结构能把模型不确定性限制在局部，同时不假装 static DAG 可以枚举每个有效的内部步骤。

HumanLayer 的 12-Factor Agents 与 LangChain 的 deploybot 描述都主张把职责聚焦的 agentic component 放进更确定的控制结构（[HumanLayer — 12-Factor Agents](#)；[LangChain — The Anatomy of an Agent Harness](#)）。应把它们建议的形态视为 practitioner case study，而不是通用 turn-count limit；loop boundary 应由 workload-specific eval 决定。

对于 hybrid pattern：

- **状态所有者：** 外层 workflow 保持权威；内层 loop 接收 scoped task state，并返回 typed handoff。
- **停止条件：** 必须同时满足内层 loop 的 budget/exit contract 与外层 node 的 acceptance gate。

- **失败传播：** 内层 failure 应变成显式 node result——retryable、terminal、partial、unknown 或 escalated——而不是虚构 success。
- **评测单元：** 既测试内层 loop 处理有边界任务的能力，也测试完整 workflow outcome，包括 handoff correctness。

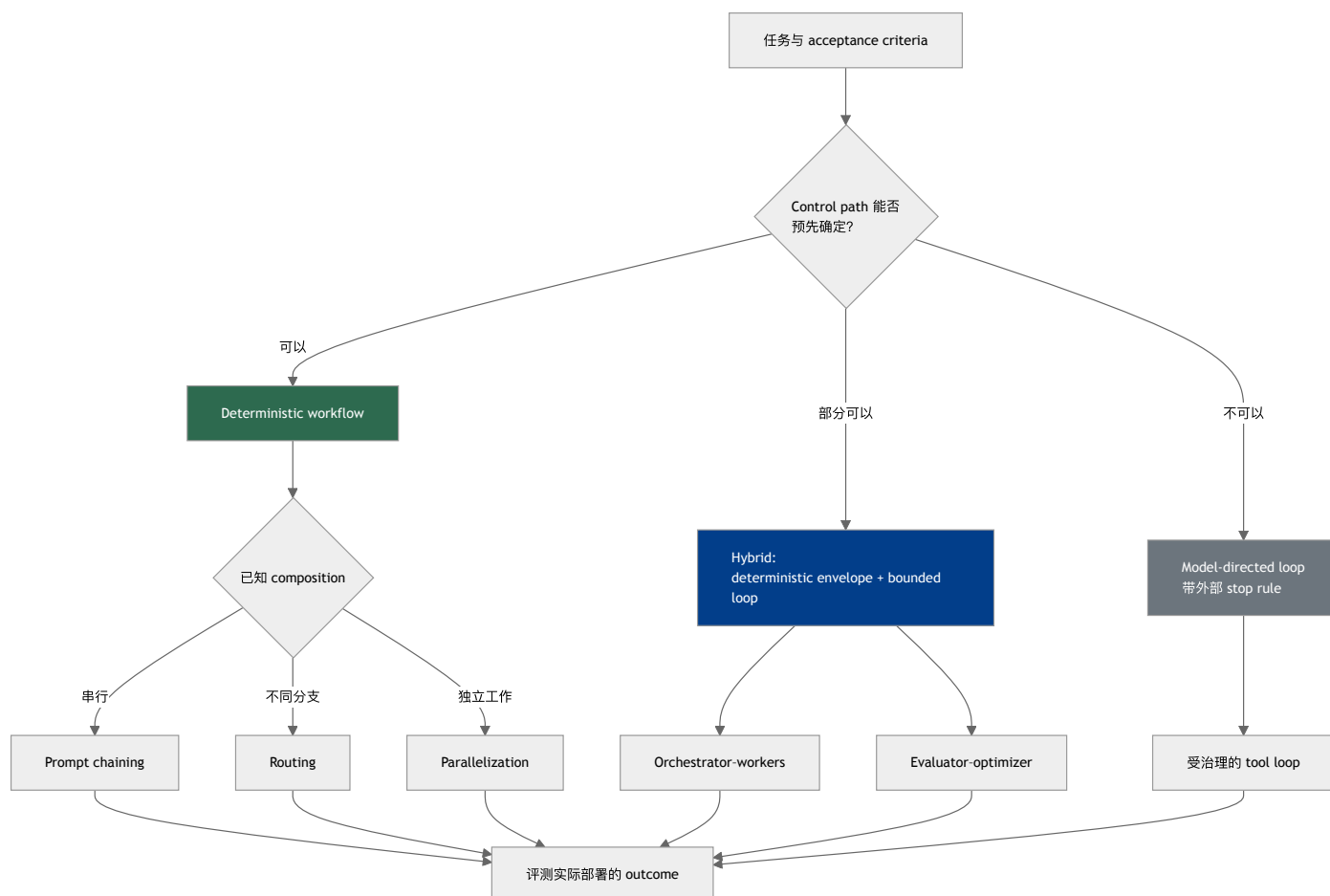
9.5 研究模式代表设计谱系，而非生产保证

多篇重要论文探索了 search、reflection 与 planning structure。它们提供了有价值的设计语言，但论文中针对特定 task、model、tool 和 budget 的结果，并不能证明这些模式具有同等的生产成熟度或能够普遍带来改善。如果实现这些模式，其 search frontier、reflection、candidate set、tool result 和 termination status 仍必须由外部 harness 或 runtime 持有。

模式	研究的机制	生产环境中的解释
Reflexion	在收到反馈后保存自然语言 reflection，并在后续 attempt 中复用 (Shinn et al. — Reflexion)	应把 reflection 当作带 provenance 和 expiry 的易错 memory，而非已经验证的 diagnosis
Self-Refine	使用模型生成的 feedback 反复修改模型之前的输出 (Madaan et al. — Self-Refine)	Controller 仍需设置外部 cap 和 acceptance rule；模型自我满意不等于 outcome confirmation
CRITIC	在 critique 和 correction 过程中使用外部工具 (Gou et al. — CRITIC)	Tool evidence 可以加强检查，但每次调用仍须经过第 6 章的 validation 和 authorization lifecycle
Tree of Thoughts (ToT)	对候选“thought”branch 执行带 lookahead 和 backtracking 的搜索 (Yao et al. — Tree of Thoughts)	Harness 持有 frontier、branching budget、value record 和 termination；文本 branch 不是 durable execution state
Language Agent Tree Search (LATS)	使用 value estimate 和 reflection，对 language-agent trajectory 运行 Monte Carlo tree search (Zhou et al. — LATS)	Tree growth、environment effect 与 rollout budget 都需要显式 isolation 和 accounting
ReWOO	分离预先制定计划的 Planner、执行工具的 Worker 与组合 observation 的 Solver (Xu et al. — ReWOO)	在论文所研究的设置中，预先规划可以减少重复 planning；但 worker 必须上报使计划失效的 observation，不能盲目执行陈旧步骤

这些机制可以嵌套进前述组合 pattern。例如，ToT 可以在一个 deterministic node 内搜索，Reflexion 可以在 failed trial 后更新有限 scope 的 memory，ReWOO 可以定义 hybrid 的 plan-execute-synthesize workflow。应评测实际部署的组合，而不是把质量归因于 pattern 名称。

9.6 选择与组合模式



Pattern 可以沿不同轴线组合。Router 可以选择一条 chain；chain 可以包含 parallel stage；orchestrator 可以依据第 8 章的 policy 把 worker route 到不同模型；evaluator-optimizer 可以运行在一个 bounded node 内。在每个边界，都要保留稳定的 task identity、typed state transition、显式 budget、failure status 与 outcome evidence。

应优先选择能够满足任务的最静态形态。当 observation 会改变路径时，更强的 model-directed control 可能有价值，但它也会扩大 stop rule、recovery 和 evaluation 必须覆盖的状态空间。Framework 可以封装这些 pattern，却不能消除检查真实 prompt、call、state transition 与 effect 的需要。Anthropic 同样提醒，framework abstraction 在早期开发中可能遮蔽底层 prompt 和 response ([Anthropic — Building Effective Agents](#))。

要点

- 模型负责提出，外部系统负责执行和记忆：Workflow pattern 永远不会把 authority 或 state ownership 转移给模型。
- 区分 **deterministic**、**model-directed** 与 **hybrid control**：包含 LLM 的 workflow 不会自动变成 autonomous agent。
- 为每种模式定义运行契约：Applicability、state owner、stop condition、failure propagation、eval unit 与 effect boundary 都必须明确。
- 选择正确的评测单元：Call、route、branch、iteration、run、artifact 与 external outcome 回答的是不同问题。
- 把 **workflow routing** 与 **model routing** 当作两个决策：第 8 章负责模型与 reasoning-budget selection。
- 并行和委派需要 **coverage accounting**：一致不等于证明，worker summary 也不是 authorization。
- **Evaluator-optimizer** 不定义 **verifier** 强度：Grader design 属于第 11 章，verifier independence 属于第 13 章。
- 研究模式的名称不是成熟度声明：Reflexion、Self-Refine、CRITIC、ToT、LATS 与 ReWOO 必须在实际部署配置中评测。
- **Hybrid pattern** 往往是实用的中间形态：Deterministic outer boundary 可以容纳局部灵活的 model-directed work。

延伸阅读

- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024.
<https://www.anthropic.com/engineering/building-effective-agents>

- Anthropic, *How Tool Use Works*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>
- Anthropic, *Demystifying Evals for AI Agents*, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Shunyu Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, ICLR 2023. <https://arxiv.org/abs/2210.03629>
- Noah Shinn et al., *Reflexion: Language Agents with Verbal Reinforcement Learning*, NeurIPS 2023. <https://arxiv.org/abs/2303.11366>
- Aman Madaan et al., *Self-Refine: Iterative Refinement with Self-Feedback*, NeurIPS 2023. <https://arxiv.org/abs/2303.17651>
- Zhibin Gou et al., *CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing*, 2023. <https://arxiv.org/abs/2305.11738>
- Shunyu Yao et al., *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, NeurIPS 2023. <https://arxiv.org/abs/2305.10601>
- Andy Zhou et al., *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models*, 2023. <https://arxiv.org/abs/2310.04406>
- Binfeng Xu et al., *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*, 2023. <https://arxiv.org/abs/2305.18323>

第 10 章：状态、事件历史与生产要素

如果 agent 唯一的状态就是下一次 prompt，它就无法从 crash 中恢复、无法安全等待 approval，也无法解释重复的副作用。本章定义后续 evaluation、long-running task、tracing、AgentOps 与 fleet 所依赖的持久数据模型。HumanLayer 的“12 Factor Agents”仍是一份有价值的具名从业者 manifesto，但它不是正式标准，也不是完整架构 (HumanLayer — 12-Factor Agents)。

10.1 不能合并成一个概念的六种对象

History 一词常被用于几种不同的数据产品。本书采用以下定义：

对象	规范含义	主要用途	它不是什么
Execution state	结构化的当前 workflow state：status、current step、pending action、budget、retry count、approval state 及相关业务引用	决定下一步允许发生什么	模型 context 或无结构 transcript
Event history	按顺序持久记录、且在声明的 replay contract 下足以恢复的已接受 workflow events	重建 state 并 resume	自动成为合规 audit log
Checkpoint	可恢复的 state snapshot，加上继续执行所需的 history position 和引用	限制恢复时间并提供 resume point	完整因果历史
Artifact	文件、diff、报告、dataset、build 或大型原始结果等可寻址工作产物	在 live context 之外保存输出和证据	仅因为模型可读取就成为 execution state
Trace	用 span、timestamp、attribute、event、link 与 status 表示执行路径的 observability data	调试 latency、causality 与 failure	天然就是 durable recovery source 或 audit record
Audit record	为 accountability 保存的受保护记录，对 content、identity、timestamp、integrity、access 与 retention 有明确要求	Investigation、governance 与 compliance	每一行 log 或 sampled trace

Temporal 把 Event History 定义为 append-only log，用于 workflow 在 crash 或 failure 后恢复 (Temporal — Events and Event History)。OpenTelemetry 用 spans 定义 trace，并允许 sampling，因此 trace 可以有意省略某些执行或细节 (OpenTelemetry — Tracing API; OpenTelemetry — Tracing SDK)。NIST SP 800-53 的 AU control family 则分别规定 audit-record content、generation、protection、review 与 retention (NIST SP 800-53 Rev. 5)。这些资料说明，一种产品可以成为另一种产品的数据来源，但不会因此成为它的同义词。

Production trace 可以链接到 event ID，event 可以引用 artifact，选定 events 也可以复制进受保护的 audit store。这种 lineage 很有价值，但仍不能推出 trace = history = audit。Context 和 memory 又是另外两类对象：context 是一次 model call 可见的 representation；memory 是被选择、供未来调用使用的信息。两者都不应成为权威 execution state。

10.2 ID 层级：Session、Run、Step、Action、Attempt 与 Call

Durability 需要能够跨越 process 和 model-context boundary 的 identifier。本书采用一套本地层级，而不假设某个 vendor 的命名：

Identifier	标识什么	生命周期与关系
session_id	持久的用户或工作流，包括其 conversation 与 artifact	可以包含多个 run、resume 或 branch
run_id	在固定 objective 与 configuration lineage 下的一次逻辑 workflow execution	属于一个 session，并能跨 worker replacement 保持不变
step_id	一个逻辑 state transition 或 orchestration position	属于一个 run；只有在明确 loop semantics 下才能再次访问
action_id	一个预期的外部效果	属于一个 step，在 delivery retry 间保持稳定
attempt_id	对 step 或 action 的一次执行或 delivery 尝试	是同一逻辑 step/action 下的 retry 维度
call_id	一次 model-call 或 tool-call protocol exchange 及其相关 output	属于 step 或 attempt；不能代替 action 的 idempotency identity

典型包含关系是 session → run → step → action → attempt；model 和 tool 的 call_id 则挂在发起它们的 step 或 attempt 上。Branch 获得新的 run 或 branch identity，并记录 parent history position。Worker ID、sandbox ID、model context ID 和 trace ID 都是关联信息，不能替代这些 durable ID。

有些 dispatcher 使用 `invocation_id` 表示内部 tool-dispatch record。应公开它与本章层级的映射，而不是增加一个有歧义的同义词：当一次 invocation 只表示一个预期效果时，它可以与 `action_id` 相同；composite invocation 则可以包含多个 action 和 attempt。

这种划分能够回答实际问题。Crash 的 worker 可以替换，而不改变 `run_id`。Timeout 的 payment retry 使用新的 `attempt_id`，但保留 `action_id` 与 idempotency key。Model retry 使用新的 `call_id`，不能悄悄重新定义已获 approval 的 action。Temporal 自身也会在 workflow-execution chain 中区分 durable Workflow ID 与各个 Run ID，这说明逻辑工作与一次执行实例需要不同身份 ([Temporal — Workflow Execution](#))。

10.3 Reducer、Event Sourcing、Checkpoint 与 Workflow Engine

四种相关技术解决的是不同问题：

1. **Reducer。**类似 `state_next = reduce(state_previous, event)` 的确定性函数负责计算 projection。它可以处理完整 history、checkpoint 之后的 suffix，或从其他 store 复制来的 events。Reducer 是代码，不是 persistence。
2. **Event sourcing。**已接受的 domain change 以 append-only event stream 保存，并成为被 sourcing state 的权威来源。Microsoft 把该 pattern 描述为持久保存一系列 action，再通过 replay 重建 current state；它同时警告这个 pattern 会增加复杂度，应选择性采用，而不是用于所有 CRUD subsystem ([Azure Architecture Center — Event Sourcing](#))。
3. **Snapshot 或 checkpoint。**位于已知 history position 的 materialized state image 可以减少 rehydration cost。Azure 的 event-sourcing 指南把 snapshot 作为避免 replay 完整 stream 的优化 ([Azure Architecture Center — Event Sourcing](#))。在本书中，*checkpoint* 还包含继续执行所需的引用与 resume contract。
4. **Workflow engine。**Runtime infrastructure 负责 scheduling、持久 timer 与 wait、task dispatch、retry policy、ownership 以及 failure 后恢复。例如 Temporal 会记录 command 和 event，使 Workflow Execution 能从最近的 durable state 恢复 ([Temporal — Workflow Execution](#); [Temporal — History Service Architecture](#))。

这些选择可以组合，并不要求成套采用。系统可以在普通 append-only workflow log 上使用 reducer，而不对 customer database 采用 event sourcing；可以创建 checkpoint，而不删除之前的 event history；workflow engine 可以编排 activity，而 domain data 仍留在传统 transaction store 中。反过来，event-sourced business aggregate 本身也不提供 timer、worker lease、approval wait 或 activity retry。

应明确记录 source-of-truth map。例如 event history 可以负责 workflow position 和 approval transition；payment system 负责 transaction outcome；artifact store 负责生成文件；current-state projection 负责 query。“Unify state” 应表示共享 identity 与有意设计的 derivation，而不是把每个 byte 强塞进同一个 prompt、table 或 event stream。

10.4 Durable Execution 与诚实的 Replay

Durable execution 表示 progress 能够跨 process、worker、network 或 infrastructure failure 保存，并从已记录 state 恢复。Temporal 的 replay 机制会针对既有 Event History 运行 workflow code，把新生成的 command 与 history 对照，然后从最后一个记录事件继续 ([Temporal — Workflow Execution](#))。这是一个具体实现，而不是唯一可选 engine。

Agent workflow 增加了一条关键边界：model call 和外部 tool call 都是非确定性 activity。Temporal 会把已完成 Activity result 写入 Event History；它的 Side Effect primitive 在 replay 时返回已记录 result，而不是重新执行 function ([Temporal — Activities](#); [Temporal — Events and Event History](#))。Agent 应采用同一规则：

- 在对应 `call_id` 下记录完整 model response 或 structured proposal；
- 记录每个 tool result、error、external operation ID 与 outcome status；
- replay 时把这些已记录 outputs 交回 reducer 或 workflow code；
- 不要再次调用模型或重新运行有副作用的工具，却把新 trajectory 标记成 replay；
- 如果需要新答案，应创建具有新 identity 和 lineage 的显式 retry、fork 或 new run。

Replay event history 是重建原来的 execution。使用相同 user input 重新运行，则是在采样新的 execution。后一种操作对 eval 和 recovery experiment 很有价值，但不是 deterministic replay。

Workflow-code change 同样需要 version discipline。如果新的 reducer 或 workflow version 针对同一份旧 history 产生不同 command sequence，replay 就可能不兼容。保存 run 对应的 code、model、prompt、tool-schema、policy 和 reducer version；使用 migration、version gate 或 new run，而不是静默重新解释旧 events。Checkpoint 必须注明生成它的 history position 与 reducer/schema version。

10.5 Event 需要明确契约

Event 应描述已经接受的事实，不能把尚未提交的 intention 伪装成过去式。一个实用 envelope 包括：

event_id, event_type, schema_version, occurred_at, recorded_at
session_id, run_id, step_id, action_id, attempt_id, call_id
actor, tenant, source, causation_id, correlation_id
payload_or_artifact_ref, policy_version, previous_sequence

并非每个 event 都需要所有 ID。approval_received 需要 approver 和 action；worker_replaced 需要 run 和 worker identity；model_call_completed 需要 call 和 step。Envelope 应让字段省略显得有意且明确，而不是产生歧义。

Append 时应携带 expected sequence 或 version，防止两个 worker 都提交互不兼容的 next event。Inbound command 和外部投递 event 应使用稳定 deduplication identity。Microsoft 指出，ordering 与 per-entity event identifier 是 event sourcing 的核心问题，event store 也不能与 message broker 互换 (Azure Architecture Center — Event Sourcing)。Queue 可以 delivery work；已接受的 workflow history 才决定什么真正成为 run 的一部分。

大型 payload 通常应存为 immutable 或 versioned artifact，并用 content hash 或 durable URI 引用。保留足够 metadata，以便授权访问并验证引用版本。Redaction 应保留 event 确实发生过的事实与适用 lineage；从 model-facing context 删除字节，与对 durable store 实施 retention 或 erasure policy 不是同一操作。

10.6 Recovery 是状态机，不是“全部 Retry”

Runtime 应为每一种 interruption 定义恢复 transition：

中断	恢复前的 durable record	安全的下一步 transition
Process 或 machine crash	最后 committed event、当前 checkpoint、ownership lease	使用已记录 history rehydrate；只发出尚未 accepted 的工作
Worker replacement 或 deploy	Run ID、worker/config version、in-flight step 与 lease	获得 ownership，replay recorded output，在明确兼容 version 下继续
Tool timeout 或响应丢失	Action ID、attempt ID、idempotency key、external operation reference、unknown status	对有后果 action，retry 前先 reconciliation external state
Duplicate delivery	Command/event dedup key 与 accepted sequence	返回已记录 disposition；不重复 append 或 execution
Approval wait	准确的 pending action、policy decision、所需 approver scope、expiry	释放 compute；只从 authenticated approval/deny event 恢复
Cancellation request	Requester、scope、reason、time、current action state	停止调度新工作，传播 cancellation，再记录实际 terminal 或 partial outcome

Temporal 的 workflow status model 区分 cancellation request 与成功处理后的 Cancelled terminal state；其 Activity history 也分别记录发出取消请求和接受取消 (Temporal — Workflow Execution; Temporal — Events and Event History)。必须保留这一区别。Cancellation 不是 rollback，timeout 也不能证明没有发生 side effect。

Approval wait 应采用 durable event，而不是让 web process sleep。HumanLayer manifesto 正确强调 launch/pause/resume API 和结构化 human-input event (HumanLayer — 12-Factor Agents)。不过，对于高风险 action，第 7 章不可绕过的 policy gate 和第 14 章的 approval semantics 仍然适用：模型可以请求 consultation，但 runtime 决定 approval 是否 mandatory，并验证 approver。

每条 recovery path 都需要 budget 和 terminal state。反复 transient failure 可以进入 failed 或 needs_human；不兼容 workflow code 可以进入 migration_required；尚未解决的外部效果可以维持 unknown，而不是被错误标为 failed。Event history 应保留系统如何得出该决定。

10.7 十二要素：通用原则还是实现选择？

HumanLayer 把以下十二项作为 production LLM software 的实践建议 (HumanLayer — 12-Factor Agents)。应把它们看作需要评估的 manifesto，而不是 standards checklist：

HumanLayer factor	值得保留的 durable principle	依赖具体情境的 implementation choice
1. Natural Language to Tool Calls	把 model proposal 与 deterministic execution 分开	不是每个任务都需要工具；wire format 不一定是 JSON
2. Own Your Prompts	对 model-visible configuration 进行 versioning 与 eval	只要 version 和 rendered input 可检查，团队可以使用 provider/framework template
3. Own Your Context Window	让 context assembly 显式可见	XML 和 single-message event rendering 是格式选择
4. Tools Are Just Structured Outputs	Tool call 是 output，不是 effect	Provider-native、MCP 或 custom serialization 都可以表达它
5. Unify Execution and Business State	使用共享 ID 与明确 source of truth	是否从一个 event stream 推导所有 execution state 是可选的；外部系统可以拥有 business fact
6. Launch/Pause/Resume With Simple APIs	Long-running work 需要显式 lifecycle operation	Endpoint shape 与 workflow engine 属于实现选择
7. Contact Humans With Tool Calls	Human request 和 response 应成为结构化 durable event	Model-requested consultation 不能替代 mandatory runtime approval gate
8. Own Your Control Flow	Harness 必须拥有 stopping、waiting、retry 与 dispatch	Custom loop、graph runtime 和 managed workflow engine 都是可选方案
9. Compact Errors Into Context	为下一步决策保留 actionable error information	完整诊断属于 history/artifact；具体 compaction 与 escalation threshold 需要 eval
10. Small, Focused Agents	限定 scope，并在 model capability frontier 附近评估可靠性	“3-10，最多约 20 步”是 manifesto heuristic，不是通用上限
11. Trigger From Anywhere	把 external trigger 归一化成 authenticated、idempotent event	Slack、email、SMS、webhook 与 cron support 是产品选择
12. Make the Agent a Stateless Reducer	在可行时从 durable input 重建当前 execution state	Pure reducer 不要求对每个 domain system 采用 event sourcing

这种解读保留了 manifesto 最重要的洞见——agent system 主要是包围概率调用的软件——同时不会把其中的示例提升为通用事实。尤其是，“stateless reducer”描述的是 worker behavior：可替换 worker 可以从 durable input 进行计算。整个系统则是有意保持 stateful 的。

10.8 从单一 Program 到 AgentOps 与 Fleet

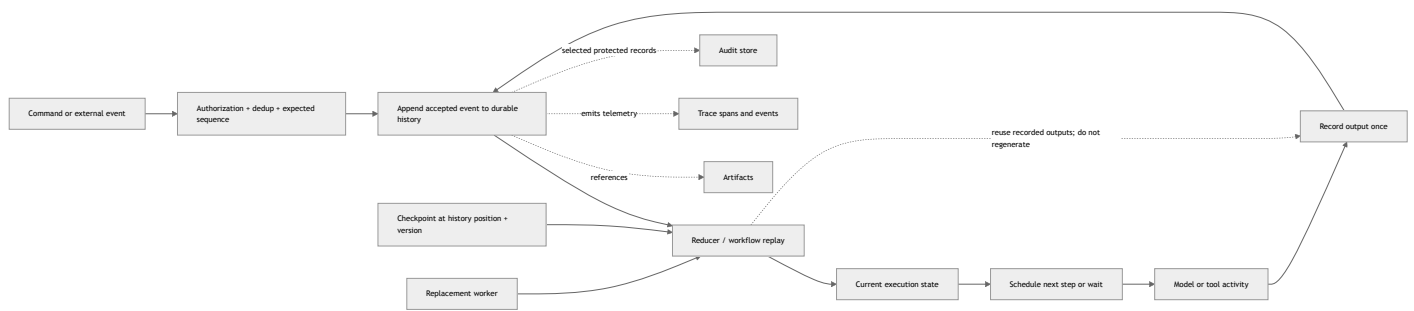
本地 prototype 可以把 execution state 和 artifact 放在 repository 里。共享服务还需要 tenant boundary、schema migration、retention、quota、worker ownership、configuration rollout，以及跨多个 active run 的 reconciliation。这是 operational scope 的变化，不能据此把一个 event log 变成 universal control plane。

第 17 章把 trace 作为 iteration 所需的 observability input。第 18 章把本章的 identity 和 data boundary 应用于 cost attribution、privacy、retention、deployment 与 incident response。第 19 章再加入 fleet identity、policy administration、distributed policy enforcement、lineage 与受保护的 audit record。这里的交接是有意设计的：

- 本章负责 execution state、recovery history、checkpoint 与 replay semantics；
- AgentOps 负责在规模化环境中安全运营和改变这些系统；
- fleet/control-plane 章负责跨 run identity、governance 与 enforcement topology。

这条边界也阻止一种常见捷径：event history 可以向 AgentOps 或 audit pipeline 提供选定证据，但它不会自动具备每项 audit requirement 所需的 completeness、access control、immutability 或 retention。这些属性必须单独设计并验证。

图：Durable State、Replay 与 Evidence Product



本章要点

- **Execution state、event history、checkpoint、artifact、trace 与 audit record 是不同产品：**通过 lineage 关联它们，不要混用其保证。
- **采用 durable identity：**session、run、step、action、attempt 和 call ID 回答不同的 recovery 与 deduplication 问题。
- **Reducer、event sourcing、checkpoint 与 workflow engine 不是同义词：**每一种技术都可以独立采用。
- **Replay 必须复用已记录的 model 和 tool output：**再次调用会产生新的 attempt 或 branch，而不是 replay 旧 execution。
- **Recovery 是明确的 state transition：**crash、worker replacement、timeout、duplicate delivery、approval wait 与 cancellation 分别需要不同证据和下一步 action。
- **Timeout 或 cancellation request 不是已确认的 external outcome：**对有后果的效果进行 reconciliation。
- **HumanLayer 十二要素是一份具名 manifesto：**保留通用 ownership 和 lifecycle 原则，同时把 XML format、step count、channel 与全面 event sourcing 标为选择。
- **Stateless worker 运行在 stateful system 之上：**durable state 位于可替换 compute 之外。
- **本章为 AgentOps 和 fleet governance 提供基础：**它不会把 trace 重定义为 history，也不会把 history 重定义为 audit record。

延伸阅读

- Temporal, *Workflow Execution*. <https://docs.temporal.io/workflow-execution>
- Temporal, *Events and Event History*. <https://docs.temporal.io/workflow-execution/event>
- Temporal, *Activities*. <https://docs.temporal.io/activities>
- Temporal, *History Service Architecture*. <https://github.com/temporalio/temporal/blob/main/docs/architecture/history-service.md>
- Microsoft Azure Architecture Center, *Event Sourcing Pattern*. <https://learn.microsoft.com/en-us/azure/architecture/patterns/event-sourcing>
- OpenTelemetry, *Tracing API*. <https://opentelemetry.io/docs/specs/otel/trace/api/>
- NIST, *SP 800-53 Rev. 5: Security and Privacy Controls*. <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>

第 11 章：评估

《Foundations》第 13 章建立了 model-behavior layer 的基础：representative tasks、repeated trials、grader types、pass@k、pass^k，以及 metrics 作为 proxies 的局限。它明确把 tool execution、permissions、side effects、agent trajectories、environment-state grading、operational reliability 与 release gates 交给本卷（[LLM Foundations — 第 13 章](#)）。

本章评测用户真正遇到的系统：model 加上 agent harness、runtime、tools、policies 与 environment。它的输出不只是一项分数，而是支持某条有限主张的 evidence，例如：“Configuration C 在 budget B 与 policy P 下，已经可以用于 task distribution D。”

11.1 Evaluation Contract

Evaluation 要先陈述 claim，再选择 task 或 metric。OpenAI 的可信评测指南区分 capability elicitation、controlled comparison 与 safeguard-performance claims，因为三者需要不同的 harness 和不同的 supporting evidence（[OpenAI — Trustworthy Third-Party Evaluations](#)）。

面向 release 的 contract 应当说明：

- task distribution 与 excluded cases；
- 接受测试的 model-plus-harness configuration；
- 可用的 environment、tools、permissions 与 data；
- token、time、attempt、cost 与 side-effect budgets；
- success、failure、abstention 与 policy-violation criteria；
- graders 与 human adjudication policy；
- 做出 decision 所需的 uncertainty 与 per-slice reporting。

缺少这份 contract 的 headline score 很容易被过度泛化。它只描述一个 setup 下观察到的 behavior，而不是 model 或 agent 的内在属性。

11.2 Agent Eval 的规范词汇

Anthropic 定义了 agent evaluation 的主要组件，并明确区分 evaluation harness 与 agent harness（[Anthropic — Demystifying Evals for AI Agents](#)）。本书使用以下规范词汇：

术语	本书定义
Task	一条 test case，包含声明过的 inputs、initial environment、available capabilities、budgets 与 success criteria。
Trial	一套 tested configuration 从已恢复 baseline 出发，对一个 task 进行的一次 attempt。
Grader	依据声明的 evidence 判断一项明确 property，并返回 verdict 或 score 的 versioned logic。
Eval transcript / trajectory	为某次 trial 判分而完整保存的 model、tool 与 observation-level record。
Outcome	Trial 产生的最终 relevant state，包括 environment 与 external-effect state，而不只是 response text。
Evaluation harness	负责 provision baseline、调用 agent harness、记录 evidence、运行 graders、聚合 trials 并生成报告的 test infrastructure。
Agent harness	被测试的 model-facing system，负责组装 context、驱动 loop、验证 proposals，并协调 tools 与 runtime。
Evaluation suite	用于支持某项 capability、regression、reliability 或 policy claim 的 versioned task collection。

Anthropic 有时把 transcript、trace 与 trajectory 当作完整 eval run 的近义词。本书把 **trace** 专门用于由 span、attribute、event、link、timestamp 与 status 构成的 observability data；OpenTelemetry 也允许 tracing layer 进行 sampling（[OpenTelemetry — Trace API](#)）。只有当 evaluation harness 保证每项 grader-required event 与 artifact 都完整时，才能从 trace instrumentation 派生 eval trajectory。经过采样的 production trace 并不会自动成为 eval transcript。

11.3 分别评估四层 Evidence

Agent success 包含多个层级。不能把所有 grader 都压缩为同一种“correct”。

Evidence layer	能够证明什么	Typical graders	单独使用时不能证明什么
Process / trajectory	Required/forbidden actions、approval use、tool arguments、turn count、budget 与 recovery behavior	Event assertions、policy checks、trajectory rubric	Final artifact 或 environment 是否正确
Artifact	Produced file、diff、report、dataset、build 或 citation set 的属性	Unit tests、static analysis、schema/content checks、expert rubric	Artifact 是否已部署或造成预期 external effect
Environment state	Database row、filesystem state、UI/backend state、message sent、permission change 与 cleanup completion	State queries、integration tests、independent API checks	User 是否体验到或认可结果
User / business outcome	Resolution、acceptance、satisfaction、conversion、loss avoided 或其他 product-level effect	User confirmation、business event、controlled study、delayed outcome join	在缺少更多 evidence 时进行细粒度归因

Anthropic 的订票案例具体说明了 environment distinction：agent 声称机票已经预订，属于 transcript content；environment 中是否存在 reservation row 才是 outcome ([Anthropic — Demystifying Evals for AI Agents](#))。同一原则也适用于代码：可信的 final message 不等于 passing build，passing unit test 也未必等于真正可用的 end-to-end user flow。

如果 path 本身属于 contract，例如 mandatory approval、prohibited data access 或 cost ceiling，那么 process grading 是合适的。不能仅仅因为作者预期某条路径，就要求一种精确 tool sequence；Anthropic 报告过 rigid path grading 会拒绝通过另一条路径实现目标的有效解法 ([Anthropic — Demystifying Evals for AI Agents](#))。

保留四层之间的 links：task_id → trial_id → trajectory events → artifact versions → environment snapshot → user/business event。这条 lineage 让 reviewer 能区分 process violation、artifact defect、environment failure，以及 trial 之后才发生变化的 outcome。

11.4 从隔离、可恢复的 Baseline 运行多个 Trial

一次成功 trial 只能证明可能性，不能证明可靠性。Agent output 与 tool path 可能在运行之间变化，因此当 product claim 依赖 repeatability 时，evaluation harness 应对每个 task 运行多个 trials ([Anthropic — Demystifying Evals for AI Agents](#))。

每个 trial 都从一份声明过的 baseline 开始：

- immutable environment image 或 reproducible fixture version；
- 干净的 filesystem、database、queues、caches、browser profile 与 external-resource namespace；
- task-specific credentials 与 tenant identity；
- 必要时使用 controlled clock、network stubs 或 recorded external responses；
- 已知 resource quotas，且不存在早先 trials 遗留的 artifacts；
- teardown verification，以及 setup 或 cleanup 失败时的 recovery path。

Anthropic 记录过 shared state 同时会造成 failure correlation 和虚假的分数膨胀，其中一个内部案例是 agent 检查了早先 trials 遗留的 git history ([Anthropic — Demystifying Evals for AI Agents](#))。因此，isolation 保护的是 measurement validity，而不只是 test hygiene。

区分三种结果：

- Task verdict**：tested system 获得 pass、fail 或 partial credit。
- Infrastructure-invalid trial**：setup、dependency、grader 或 cleanup 失败，因此该 attempt 没有测量 agent claim。
- System reliability failure**：在声明的 production-like contract 内出现 in-scope timeout、tool error、recovery failure 或 resource exhaustion。

不能把第 3 类作为“eval noise”丢弃。也不能在不单独报告的情况下，把第 2 类算成 agent failure。只有根据被记录的 policy 才能 retry invalid trial；否则重复 retry 会掩盖不稳定的 evaluation harness。

11.5 固定 Tested Configuration，并按 Slice 报告

Eval result 属于完整的 tested configuration。需要固定或记录：


```
EvalConfiguration {
  model_snapshot, provider_endpoint, reasoning_settings,
  agent_harness_version, prompt_and_context_policy,
  tool_catalog_and_schemas, runtime_and_environment_image,
  retrieval_and_index_versions, policy_and_approval_rules,
  retry_timeout_and_budget_rules,
  task_suite, graders, adjudication_policy
}
```

OpenAI 的 evaluation playbook 同样要求报告披露 system、harness、tool access、elicitation method、budgets、safeguards 与 validity checks，因为这些选择可能显著改变 score 能够支持的 claim ([OpenAI — Trustworthy Third-Party Evaluations](#))。

先报告 per-slice result，再给 blended aggregate。至少包括：

- **task 与 quality**：domain、difficulty、language、input shape，以及 ordinary/edge/historical failure；
- **routing**：第 8 章定义的 route target、reasoning tier、cascade/fallback/hedge/human escalation；
- **cost**：input、output、reasoning、cache、retrieval、tool、grader 与 human-review cost；
- **latency**：router、model、tool、grader、end-to-end p50/p95/p99、timeout 与 queue time；
- **failure class**：model、transport、schema、tool、environment、recovery、grader、policy 与 ambiguous side effect；
- **policy**：tenant、identity、data region、model allowlist、approval、restricted-content 与 cross-tenant tests。

Aggregate 可能在 high-risk slice 回归的同时得到改善。因此，release gate 应作用于 critical slices 与 hard policy checks，而不只是一项 weighted average。

11.6 正确使用 pass@k、pass^k 与 Uncertainty

令 p 表示单次 trial 的成功概率。如果 k 次 trials 相互独立且同分布，那么：

- **pass@k** 是至少成功一次的概率： $1-(1-p)^k$ ；
- **pass^k** 是所有 attempts 都成功的概率： p^k 。

Pass@k 适用于真正会生成多个 candidates，并能识别 successful candidate 的产品；pass^k 表示重复一致性。

Anthropic 强调，应由 product contract 决定使用哪项 metric ([Anthropic — Demystifying Evals for AI Agents](#))。不能仅仅因为 pass@k 数字更大，就为只允许一次 attempt 的 user experience 报告它。

当 n 个 sampled candidates 中有 c 个正确时，HumanEval/Codex 论文在 $n \geq k$ 时使用估计量 $\frac{c}{n} \frac{\binom{n-c}{k}}{\binom{n}{k}}$ 计算 pass@k ([Chen et al. — Evaluating Large Language Models Trained on Code](#))。该估计量回答的是 sampling question；它不会使 correlated trials 变得独立，也不能证明 selector 能找到 passing candidate。

汇总结果必须同时报告 counts 与 uncertainty：task 数量、每个 task 的 trial 数、successes、invalid trials 和 interval method。NIST 介绍了用于 binomial proportion 的 Wilson 等 confidence interval；与把 observed fraction 当作精确事实相比，这类区间能表达估计不确定性 ([NIST/SEMATECH — Confidence Intervals for a Proportion](#))。Binomial interval 假设用于计算的 observations 符合声明的 binomial sampling model。

Agent trials 往往按 task 或 environment 聚类。如果 attempts 共享 task、outage、cache、account 或 fixture defect，把每次 attempt 当作独立 observations 会夸大 precision。应视情况使用 task-level paired comparison、task-level bootstrap 或 hierarchical model，并披露 sampling unit。第 16 章会分析一个 infrastructure configuration 显著改变 agentic coding 结果的实测案例 ([Anthropic — Infrastructure Noise](#))。

Uncertainty 不只是一条 confidence interval。还要报告对 task-set version、grader version、invalid-trial policy 与 infrastructure configuration 的 sensitivity。即使统计区间很窄，misconfigured eval 仍然是无效 evidence。

11.7 把 Grader 作为 Measurement Instrument 校准

对于能够确定的 property 使用 deterministic grader；对于 rubric-based semantic judgment 使用 model grader；对于 expert 或 consequential ambiguity 使用 human。Anthropic 记录了这三类 grader，并建议用 human judgment 校准 model-based grader ([Anthropic — Demystifying Evals for AI Agents](#))。

每个 grader 都要有 contract：

```
GraderResult {
  grader_version, evidence_ids,
  verdict: pass | fail | partial | abstain,
```

```
score, reason_code, explanation,
confidence_or_supporting_checks
}
```

构建经过 adjudication 的 calibration set，其中包含 clear positives、clear negatives、boundary cases、valid alternative solutions、insufficient-evidence cases，以及企图利用 grader 的 adversarial attempts。对每个 grader 与 critical slice 测量：

- **false positive**：grader 让已经 adjudicate 为 failure 的 case 通过；
- **false negative**：grader 把已经 adjudicate 为 success 的 case 判为失败；
- **abstention rate**：grader 正确或过度地拒绝下结论；
- **coverage**：grader 能对 intended cases 中多少比例给出有 evidence 支持的 verdict；
- **agreement by slice**：model 与 human judgments 在哪些位置有分歧，而不只是 aggregate agreement；
- **cost 与 latency**：grader 是否适用于 live loop、CI gate 或 offline audit。

这里的 *positive* 指 grader 的 `pass` label。如果 safety classifier 把“violation”定义为 positive，这些名称就会反转；因此应发布带 label 的 confusion matrix，而不能只依靠术语名称。

错误代价并不对称。Destructive-action safety check 中一个 false positive 可能比许多 false negatives 更重要；creative-writing rubric 中的 false negative 可能主要只是浪费 review time。Threshold 与 escalation rule 应根据 consequence 设置，而不是使用一个通用 target accuracy。

为 semantic grader 提供明确的 **abstain** 或 **insufficient evidence** 选项。Anthropic 报告 model grader 需要仔细 calibration，并建议在 evidence 不足时提供 “Unknown” path ([Anthropic — Demystifying Evals for AI Agents](#))。其 motivated-mislabeling 研究还发现，向 model evaluator 透露后果可能改变 labels；abstention 与更严格 rubric 能减轻、但不能消除观察到的效应 ([Anthropic — Agentic Misalignment in Summer 2026](#))。当 candidate identity 与 deployment consequence 与 rubric 无关时，应向 judge 隐藏它们。

在运行 release eval 前定义 conflict resolution：

1. deterministic evidence 只对该 check 真正能证明的狭窄 property 优先；
2. mandatory policy failure 不能被 quality graders 的分数平均掉；
3. grader disagreement 或 abstention 连同全部 evidence 进入 review queue；
4. human adjudicator 使用 versioned rubric，并记录 reason code；
5. adjudicated cases 返回 calibration set，但 rubric 发生变化时必须创建新的 grader version。

Human judgment 不会自动成为一致的 ground truth。对 high-impact dispute 使用多位 reviewer 或 escalation reviewer，测量 disagreement；当 experts 对 rubric 有不同理解时，应改进 rubric。

11.8 测量 Agent Loop 使用的 Verifier

第 13 章会在 schema check、deterministic test、environment check、same-agent critique、independent model grader 与 human review 之间做选择。本章为该选择提供 evidence。

对每个 proposed verifier 测量：

- 它能检测和遗漏哪些 failure classes；
- false-positive、false-negative、abstention 与 disagreement rates；
- 它在实际运行位置上的 latency 与 cost；
- 它的 failure 是否与 generator failure 相关；
- 它是否容易受到 leaked answer、reward hacking 或 path overfitting 影响；
- 接受 bad result 或拒绝 good result 的 consequence。

Live loop 中使用的 verifier 与 evaluation harness 中使用的 grader 可以共享代码，但二者扮演不同角色。Live verifier 会改变 trajectory，也可以向 agent 提供 repair information；holdout grader 则测量最终系统。如果 agent 可以检查 hidden tests、reference answer 或 release grader，eval 测到的可能是 grader exploitation，而不是 task success。

不要因为“maker must not be checker”就直接选择 independent model judge。Independence 可以减少一种 correlated failure，但它是否有用，取决于测得的 error profile、consequence、latency 与 cost。第 13 章会使用这些 evidence 构建 verifier hierarchy，而不是规定一条普遍 checker rule。

11.9 把 Eval Result 转化为 Release Evidence

Release packet 应当能够从 immutable artifacts 中重建：

- claim 与 decision owner;
- tested configuration 与 comparison baseline;
- task-suite manifest 与 slice coverage;
- environment baseline、isolation 与 reset evidence;
- trial counts、invalid-trial accounting 与 pass/fail/partial results;
- product semantics 确实需要时的 pass@k 或 pass^k, 以及 uncertainty;
- grader calibration results、conflicts、abstentions 与 human adjudications;
- policy 与 safety gate results;
- cost、latency、reliability 与 routing slices;
- representative trajectories、artifacts、environment diffs 与 known limitations;
- release decision、exception owner、expiry、rollback trigger 与 post-release monitors。

OpenAI 建议 evaluation report 说明 claim、tested system、harness/tools、budgets、elicitation choices, 以及对 broken task、reward hacking、contamination、refusal 与 sandbagging 等 hazard 的检查 ([OpenAI – Trustworthy Third-Party Evaluations](#))。同样的纪律能让 internal release decision 接受 review, 而不是让一项 dashboard number 变成原因不明的批准。

Release gate 应结合 hard constraints 与 statistical evidence。即使 aggregate task success 得到改善, 一项得到确认的 cross-tenant disclosure、unauthorized action 或 broken mandatory approval 仍然属于 gate failure。轻微 quality regression 则可能需要 uncertainty-aware decision, 而不是自动 block。应记录由哪条 rule 产生 decision。

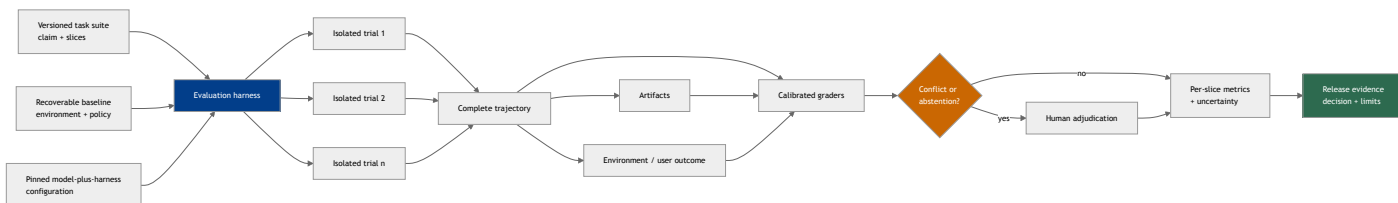
11.10 维护 Suite, 并闭合 Production Loop

每次重要 evaluation run 都要阅读 trajectories 与 grader disagreements。Anthropic 报告过 transcript review 会发现 aggregate score 看不见的 ambiguous tasks、overly rigid graders、shared-state shortcuts 与其他 defects ([Anthropic – Demystifying Evals for AI Agents](#))。Failure 应显得公平: task 可以解决, environment 正常工作, verdict 也能从声明的 evidence 推导出来。

Evaluation suite 是 versioned product。加入逃逸到生产环境的 failures, 保留重要 regressions, 轮换 held-out cases, 退役不合理的 tasks, 并保留历史, 而不是原地编辑 old scores。如果 task 或 grader 发生变化, 就不能把新旧 score 当作只有 agent 发生变化来比较。

Offline eval 不能替代 production monitoring、user feedback、controlled experiment 或 incident review。Production 会带来新的 task distribution 和延迟出现的 user/business outcomes; evaluation harness 则把合适案例变成隔离、可重复的 tests。第 17 章会使用 trace 寻找 failure pattern, 但本章的 release evidence 仍以完整 trial trajectories、artifacts、outcomes 与 declared graders 为基础。

图：从隔离 Trial 到 Release Evidence



要点

- **Foundations 止步于 model-behavior eval; Harness 负责 system evidence:** trajectory、tool、permission、side effect、environment outcome、reliability 与 release gate 都属于本章。
- **使用精确 eval objects:** task、trial、grader、eval trajectory、outcome、evaluation harness 与 agent harness 各不相同。
- **Trace 不会自动成为 trajectory:** observability data 可能被 sampling; grader evidence 必须完整。
- **分别评估 evidence layers:** process、artifact、environment state 与 user/business outcome 回答不同问题。
- **运行重复、隔离的 trials:** recoverable baseline 与 invalid-trial accounting 是 reliability claim 的前提。
- **报告 tested configuration 与 slices:** quality、routing、cost、latency、failure 与 policy result 必须与 aggregate 一起呈现。

- 根据 **product semantics** 使用 **pass@k** 与 **pass^k**: 说明 independence assumption、counts、intervals 与 sampling unit。
- 校准 **graders**: 测量 false positive、false negative、abstention、coverage、conflict 与 human adjudication。
- 在 **verifier** 进入 **loop** 前先测量它: 第 13 章会使用这些 error、consequence、latency 与 cost profiles。
- 发布 **release evidence**, 而不是裸分数: 保留 claim、versions、artifacts、uncertainty、exceptions、rollback rules 与 post-release monitors。

延伸阅读

- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026.
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- OpenAI, *A Shared Playbook for Trustworthy Third-Party Evaluations*, May 2026.
<https://openai.com/index/trustworthy-third-party-evaluations-foundations/>
- OpenTelemetry, *Tracing API*. <https://opentelemetry.io/docs/specs/otel/trace/api/>
- Mark Chen et al., *Evaluating Large Language Models Trained on Code*, 2021. <https://arxiv.org/abs/2107.03374>
- NIST/SEMATECH, *Confidence Intervals for a Proportion*.
<https://itl.nist.gov/div898/handbook/prc/section2/prc241.htm>
- Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026.
<https://www.anthropic.com/engineering/infrastructure-noise>
- Anthropic Safeguards Research Team, *Agentic Misalignment in Summer 2026*, 2026.
<https://alignment.anthropic.com/2026/agentic-misalignment-summer-2026/>

第 12 章：长时运行 Agent 与跨上下文任务

长时运行 Agent 工程的本质是**延长任务跨度 (horizon extension)**：保存足够多且经过验证的意图、进度和操作上下文，让有效工作可以跨越单个模型上下文或 worker session 的有效生命周期继续推进。它不是让同一条 transcript 无限延长。

本章以第 10 章建立的持久化 runtime 为前提。事件历史、replay、checkpoint、retry、action identity、approval state 的持久化和崩溃恢复由第 10 章负责；人类审批契约则由第 14 章负责。本章回答另一个问题：**每次跨越边界时，哪些语义工作状态必须保留下来，下一份上下文才能正确续接任务？**

12.1 先定义要延长的“跨度”

“长”可能指几种不同的量：

- **Wall-clock duration (墙钟时长)**：一次 run 保持开启的时间。
- **Model-context span (模型上下文跨度)**：一次模型调用或一份上下文可见的输入量。
- **Session span (会话跨度)**：一个 worker 或 sandbox 被保留的时间。
- **Task horizon (任务跨度)**：完整的模型加 harness 系统能以某一可靠性完成多困难的连贯端到端任务。

这些量不能互换。一个六小时进程可能大部分时间都在等待；一次 reset 可以替换模型上下文，却不结束 run，也不替换 sandbox。反过来，一次很短的调用也可能完成一个人类专家需要数小时才能完成的任务。

METR 的 **task-completion time horizon (任务完成时间跨度)** 是一个有用但范围更窄的能力指标。截至 METR 在 2026 年 5 月 8 日更新的 Time Horizon 1.1，50% time horizon 指：以人类专家任务时长为自变量拟合成功率后，预测 Agent 成功率为 50% 时对应的任务时长。METR 用 logistic curve 拟合 Agent 成功率与人类任务时长；对大多数任务，人类时长取在相似说明和工具条件下、受邀专业人员成功尝试时长的几何平均值。当前 suite 包含一百多个可自动评分、相对自包含的任务，来自 RE-Bench、HCAST 和一组较短的软件任务，主要分布在软件工程、机器学习和网络安全领域。METR 同时警告，当前 suite 对 16 小时以上的估计并不可靠 ([METR – Task-Completion Time Horizons](#))。

这个指标**不是** Agent 能连续自主运行多长时间，也不表示它能完成低于该时长的所有任务。该任务分布相对干净、上下文依赖较低，而真实的长期项目往往需要协作、隐性知识和与人互动。METR 在 2026 年 1 月 22 日的方法说明中，仍将 2019–2025 年的长期趋势概括为大约每 6–7 个月翻一番；但同一说明也警告，任务分布并没有被非常精确地定义，把结果外推到数月或数年的工作并不稳健 ([METR – Clarifying Limitations of Time Horizon](#))。

这里的方法论结论是：引用任何 horizon 数字时，都要记录日期、任务分布、harness 配置、成功阈值和拟合方法。除非用合适的任务分布重新评测完整系统，否则不要声称某个新 handoff 机制“延长了 METR horizon”。

12.2 把语义连续性与持久化执行分开

长时运行系统需要两个互补层次：

层次	它回答的问题	是否由本章负责？
持久化执行	哪个 action 已经发生、记录了什么结果，以及从哪里恢复才不会重复副作用？	否——见第 10 章
任务跨度延长	已经达成了什么、证据是什么、还有哪些风险、下一份上下文该做什么？	是

这个区分可以避免两个常见的类别错误：

1. Progress document 是一种**投影 (projection)**，不是完整的因果事件历史。它可能过时、不完整或被覆盖。
 2. Event log 也不会自动成为好的 handoff。继任者不应为了找到当前目标和下一步动作，被迫重读数千条底层事件。
- 持久历史使恢复安全；handoff 使恢复可理解。生产系统通常两者都需要。

12.3 Initializer 建立一个可续接的工作世界

Initializer 是首次运行时的角色，不一定必须使用独立模型。它把请求和环境转化为一套后续上下文能够检查和操作的工作系统。在 Anthropic 的长时编码实验中，initializer 创建了启动脚本、进度文件、初始 commit 和完整 feature list；之后的 coding session 逐步推进，并给下一份上下文留下工件 ([Anthropic – Effective Harnesses for Long-Running Agents](#))。

通用 initializer 应建立：

- **目标 outcome 与非目标**：满足请求时应出现什么可观测状态，哪些内容不在范围内。
- **Acceptance map**：milestone、依赖和验证方法。
- **可复现的进入路径**：用于检查、运行和测试当前环境的命令或流程。
- **Baseline evidence**：相关测试、服务、数据或外部资源的初始状态。

- **Progress state**: 对 milestone 状态、决策、风险和后续工作的版本化投影。
- **Artifact namespace**: 代码、报告、数据集、截图、日志等工作产物的稳定位置或 ID。
- **Policy 与 permission envelope**: 适用限制、已授予能力、approval 状态，以及权威 policy 的位置。

对软件工作而言，仓库内、版本化的工件尤其有效，因为继任者能够发现并验证它们。OpenAI 介绍过一种实践：使用简短的仓库地图、结构化文档、版本化 execution plan、进度与决策日志、lint 和文档维护任务，而不是一份巨大的 instruction file ([OpenAI — Harness Engineering](#))。这是一种实现模式，并不要求每个领域都使用 Git；不变条件是：权威工件必须可寻址，在需要时可版本化，并且下一位 worker 能访问。

Initializer 也必须能够明确失败。如果它无法建立可运行 baseline、无法确定目标 outcome，或无法取得必要权限，就应记录 blocker，而不是在未知状态上生成一份自信的计划。

12.4 用 Milestone 把大目标变成有界工作

Milestone 是可验证的中间 outcome，而不是“处理后端”这类活动标签。实用的 milestone record 包含：

字段	用途
milestone_id	跨上下文保持稳定的身份
outcome	应当存在的可观测状态
dependencies	所需先决状态和工件
verification	可以证伪完成状态的检查
evidence	已执行检查的结果和 artifact pointer
status	not_started、in_progress、blocked 或 verified
risks	已知不确定性、技术债或验证盲区

Anthropic 的实验使用 JSON feature list，并要求每个 coding session 一次只处理一个 feature。选择 JSON、包含 200 多个 feature 的案例，以及 one-feature slice，都是那套特定编码实验中的观察，不是普遍常数 ([Anthropic — Effective Harnesses for Long-Running Agents](#))。Slice 大小应通过实验选择：足够小，能在边界前完成、验证并干净交接；也足够大，能产生有意义的进展。

只有证据仍然有效时，milestone state 才应保持单调。如果依赖变化，或 regression 使原证据失效，已经 verified 的项目也可以退回 in_progress 或 blocked。勾选框只是导航工具，不是证明。

12.5 把每次 Handoff 变成 Artifact Contract

Handoff 是让继任者无需猜测即可继续工作的语义包。无论继任者是 reset 后的同一模型、不同模型、人类还是另一个 worker，每次 handoff 都必须包含：

1. **Completed work**（已完成工作）——具体变更和 milestone 转换，而不是“有一些进展”。
2. **Verified evidence**（验证证据）——实际执行过的检查、结果、必要的时间戳，以及检查覆盖范围。
3. **Open risks**（未决风险）——失败、不确定性、未测试路径、过时假设和 blocker。
4. **Next action**（下一步动作）——一个有界、可执行的步骤，以及它的预期结果。
5. **Artifact pointers**（工件指针）——稳定的路径或 ID；在漂移会影响正确性时，还要带 version、commit 或内容身份。
6. **Permission context**（权限上下文）——有效 grant、denial、待审批项、过期时间或 scope，以及必须重新授权的操作。

一种可行的 schema 是：

```
handoff_version: 1
objective: "observable target state"
completed_work:
  - milestone_id: M-03
    change: "what changed"
verified_evidence:
  - check: "command, query, or inspection"
    result: "pass/fail plus relevant measurement"
    artifact: "artifact://run/.../evidence/..."
open_risks:
```

```
- "known uncertainty or blocker"
next_action:
  step: "one bounded action"
  expected_result: "observable result"
artifact_pointers:
  - uri: "artifact://project/..."
    version: "commit, digest, or revision"
permission_context:
  grants: ["scoped capabilities"]
  denials: ["known restrictions"]
  pending_approvals: []
```

不要把 secret 复制进 handoff。只记录 capability reference、scope 或 approval ID；让 runtime 在第 7 章描述的执行边界重新附加凭据。

旧上下文被丢弃前，handoff 必须已经持久化。它的更新与 milestone 转换应使用 runtime 定义的一致性机制——最好是原子 commit 或版本检查——以免两个 worker 静默发布互不兼容的后继状态。该 commit 的因果记录仍然属于第 10 章的事件历史。

12.6 有意识地选择上下文转换

第 5 章区分了 context、compaction、memory 和 handoff。对长时任务，harness 必须明确选择转换方式，不能把所有上下文压力都视作同一个问题：

转换	保留什么	适用情况	主要风险
Continue	当前可见上下文	相关证据仍可容纳且保持连贯	噪声累积和过时假设
Compact	持续上下文内的摘要表示	连续性有价值，且可信摘要能容纳	遗漏或扭曲
Reset + handoff	新上下文中的精选语义状态	干净推理界面值得付出重建成本	Handoff 状态缺失
Worker handoff	跨 worker 的工件和执行状态	ownership、专业分工或 runtime placement 改变	版本与权限不匹配
Durable recovery	已记录结果与执行位置	故障、取消、部署或 lease 丢失	由第 10 章定义，而不是由 prompt text 定义

Anthropic 明确区分了 compaction 与 context reset：前者在原位总结历史，后者启动新上下文并依赖结构化 handoff。在一项与模型版本绑定的实验中，reset 对 Sonnet 4.5 表现出的“context anxiety”很重要，而 Opus 4.5 使该 reset 机制可以移除；同一后续研究又在 Opus 4.6 上移除了 sprint decomposition (Anthropic — Harness Design for Long-Running Application Development)。这些证据说明应按模型和任务测试 scaffolding，而不是证明 reset、compaction 或 sprint 永远必要。

Context reset 不得隐式重置 durable run、sandbox、permission 或外部资源；这些生命周期拥有不同身份。反过来，sandbox 因受损或遭到入侵而被替换，也不能被描述为一次普通 context reset。

12.7 恢复时重新验证，不盲信进度说明

继任者应把 handoff 当作高价值索引，但仍需将其中的主张与权威状态进行 reconciliation。

Resume protocol 应：

- 1. 确认 objective、run identity、配置版本、artifact revision 和 permission context。
- 2. 读取最新已提交的 handoff 与 progress state。
- 3. 检查被引用的工件，而不是只依赖粘贴的摘要。
- 4. 复现最小但有用的 baseline：启动服务、查询外部状态、打开文档或执行聚焦检查。
- 5. 把观测状态与已完成 milestone 和证据对照；对证据过期或被反驳的项目重新打开。
- 6. 选择一个有界的 next action，并说明其预期可观测结果。
- 7. 执行动作后，更新 evidence、progress state、risk 和下一份 handoff。

Anthropic 的 coding session 也会在实现下一个 feature 前，先检查工作目录、Git 历史、progress file、feature list 和基础端到端 baseline (Anthropic — Effective Harnesses for Long-Running Agents)。这条规则不限于编码：从工件恢复，但对照环境验证。

如果 handoff 指向缺失工件、不同版本、已过期授权或失败 baseline，继任者应进入 reconciliation 或 blocked 状态。不能为了维持“进度感”，从想象出来的状态继续。

12.8 让自验证停留在 Action-Local 层

Self-verification 是 action loop 内的即时反馈。例如：编辑后运行聚焦测试、打开生成文件、实际走一遍浏览器路径、写入后查询数据库，或把输出与 schema 对照。Anthropic 的长时编码实验发现，明确要求端到端浏览器测试，可以发现代码检查和较窄验证遗漏的失败（Anthropic — Effective Harnesses for Long-Running Agents）。

因此 self-verification 很有价值，但它不能让 acting agent 成为全局完成状态的最终裁决者。局部检查可能覆盖不全，Agent 可能误读结果，主观质量也可能需要经过校准的判断。

是否使用独立 evaluator 取决于具体条件，并非每个 milestone 都必须使用。Anthropic 在 2026 年的后续实验中发现，独立评价对主观任务以及接近 generator 能力边界的任务尤其有用；但对新模型已经可以可靠独立处理的工作，它也可能只是额外开销。随着模型和 harness 变化，更新后的 harness 还把 evaluator 从每个 sprint 执行改成了末尾单次执行（Anthropic — Harness Design for Long-Running Application Development）。

应根据第 11 章的 evaluation design 和第 13 章的 loop control 决定何时加入 evaluator。相关条件包括：影响高、标准主观或对抗性强、确定性检查薄弱、任务靠近模型可靠性边界，以及有证据表明 evaluator feedback 对 outcome 的改善足以抵偿 latency 和 cost。独立性只是设计杠杆之一，不能替代经过校准的 grader 和基于环境的证据。

12.9 依据 Outcome 与环境状态完成任务

Agent 说“完成了”只是 transcript event，不是 outcome。Anthropic 的评测术语给出了具体区分：订票 Agent 可以说 reservation 已经创建，但真正的 outcome 是环境数据库里是否实际存在这条 reservation（Anthropic — Demystifying Evals for AI Agents）。

因此，finalization 应针对权威工件和环境状态，执行与任务对应的成功谓词：

```
finalize only if
    required_outcomes_hold(environment, artifacts)
    and required_evidence_is_current
    and no_blocking_risk_is_open
    and required_approvals_are_satisfied
```

例如：数据库中存在字段正确的记录、已部署 endpoint 通过健康与行为检查、文件的内容与格式符合要求，或外部 transaction 已提交且有可验证 receipt。由 acceptance map 决定哪些 outcome 是必要的；不是每个任务都需要同一套检查。

Checklist 可以帮助 Agent 导航，progress file 也可以概括之前的工作，但两者都不能成为唯一 completion oracle。如果 outcome 无法验证，应报告 unverified 或 blocked，同时说明缺少的证据与 next action，而不是基于自信报告 complete。

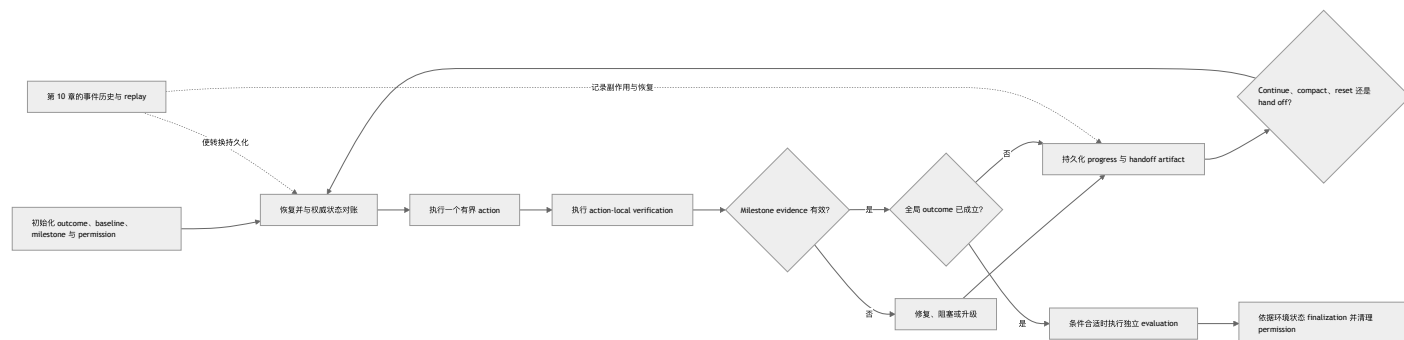
Finalization 还要生成 terminal artifact：已达成 outcome、验证证据、残留风险、artifact pointer，以及权限清理或移交状态。清理阶段应撤销临时 capability，并标明仍保持运行的资源；但不能删除 audit 或 recovery 仍需要的持久证据。

12.10 失败模式与控制措施

失败模式	可观测症状	Harness 控制
One-shot overreach	大片变更只完成一半，没有干净停止点	有界 milestone 与 slice budget
Premature completion	把局部可见进展误认为完整 outcome	Acceptance map 加基于环境的 finalization
Stale progress state	Handoff 与工件不一致	版本化指针与 resume-time reconciliation
Evidence laundering	声称“测试通过”，却没有命令、结果或覆盖范围	结构化 evidence record
Half-finished boundary	离开上下文时 baseline 已损坏	边界健康检查与明确 blocker state
Permission amnesia	继任者重复被拒操作，或假定自己拥有 grant	强制 permission context 与重新验证
Artifact loss	摘要引用了无法访问的工作	可寻址 artifact store 与 retention policy
Reset cargo cult	频繁 reset 增加成本，却不改善 outcome	按模型/任务做 ablation，并记录 reset telemetry
Self-certified finish	Agent 文字或 checklist 是唯一证明	对权威状态执行 outcome predicate
Infinite local repair	verification-fix loop 反复执行却没有净进展	第 13 章的 loop budget 与 escalation

长期项目的维护负担也会增长。OpenAI 介绍过把仓库 “golden principles” 编码为机械规则和周期性清理任务的实践，原因是 Agent 会复制仓库中既有的模式，其中也包括坏模式（OpenAI — Harness Engineering）。通用控制原则是：让质量约束可执行，并把维护安排成明确工作，而不是期待每位继任者重新发现架构意图。

12.11 参考生命周期



图中的 loop 是语义流程。崩溃恢复、replay、幂等性和并发 ownership 仍然是第 10 章的 runtime 职责。

12.12 设计检查表

启用跨上下文执行前，确认：

- 目标 outcome 与非目标可观测。
- Initializer 能建立 baseline，或明确报告失败。
- Milestone 具有稳定 ID、依赖、verification 和 evidence 字段。
- 每次 handoff 都包含 completed work、verified evidence、open risks、next action、artifact pointers 和 permission context。
- Artifact pointer 稳定，并在漂移重要时带版本。
- Resume 会把说明文档与权威工件、环境状态进行 reconciliation。
- Context reset、sandbox replacement 与 durable recovery 是不同操作。
- Self-verification 保持 action-local；evaluator 的使用由任务风险与测得的收益证明。
- 全局完成状态由 outcome 与环境状态决定，而不是 Agent 文字或 checklist。
- 第 10 章的 runtime 能持久化、replay、去重并恢复底层执行。
- Finalization 会记录证据，并清理临时权限与资源。

要点

- 长时运行设计延长的是任务跨度，不是 transcript。
- 第 10 章提供持久恢复，第 12 章提供语义连续性。
- Initializer 用 baseline、milestone、artifact、progress state 和 permission 建立可续接世界。
- 每次 handoff 都需要六项内容：completed work、verified evidence、open risks、next action、artifact pointers 和 permission context。
- Reset 与 compaction 是与模型和任务相关的选择。应测试，而不是把历史 scaffolding 固化。
- Self-verification 提供局部反馈。独立 evaluation 有条件地使用，也不能替代基于环境的证据。
- 只有 outcome 能结束任务。Agent 自述与 checklist 是有用投影，但永远不足以证明完成。
- Time-horizon 数字必须附带测量上下文。包括日期、任务分布、可靠性阈值、harness setup 和方法。

延伸阅读

- Justin Young et al., *Effective Harnesses for Long-Running Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
- Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>
- Anthropic, *Demystifying Evals for AI Agents*, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>

- METR, *Task-Completion Time Horizons of Frontier AI Models*, updated May 8, 2026. <https://metr.org/time-horizons/>
- Joel Becker, *Clarifying Limitations of Time Horizon*, METR, Jan 22, 2026. <https://metr.org/notes/2026-01-22-time-horizon-limitations/>

第 13 章：Loop Engineering 与 Verifier 层级

$Agent = Model + Harness$ 。第 1 章介绍了 agent 的内层循环：harness 组装上下文，模型提出动作，runtime 执行动作，环境返回 observation，然后循环继续。本章把这个循环外围的控制结构视为工程对象，明确什么会启动 run、单次迭代允许做什么、什么证据算作进展或成功，以及 run 为什么停止。

Loop engineering 是有用的实践者术语，但不是正式标准或参考架构。Addy Osmani 用它描述一类能够发现工作、分发工作、检查结果、持久化状态并决定下一步的系统（[Addy Osmani — Loop Engineering](#)）。本章保留这一运维洞见，但使用全书既有的 harness、runtime、policy 与 evaluation 术语来定义 loop。

13.1 研究根源与实践者谱系

底层动作循环早于这个标签出现。ReAct 交错生成 reasoning trace 与特定任务动作，使动作能从外部环境获取信息，也使 observation 能够更新计划（[Yao et al. — ReAct](#)）。生产 harness 不必暴露私有 reasoning trace，但必须实现可观察的控制循环：提出动作 → 执行策略 → 运行 → 记录 observation → 更新 execution state。

后来出现的两个实践者标签，描述了运行这类循环的特定方式：

- **Ralph** 描述一种面向仓库的实践：单一进程每轮执行一个任务，并在每次迭代中重新载入 specification 与 plan（[Geoffrey Huntley — Ralph Wiggum as a “software engineer”](#)）。
- **Loop engineering** 强调 trigger、隔离工作区、可复用指令、connector、sub-agent，以及位于单次对话之外的状态（[Addy Osmani — Loop Engineering](#)）。

这些属于实践者谱系，而不是一致性目标。它们都没有确立“每个 agent 都必须在每轮 reset 上下文、使用多个 agent、按 schedule 运行，或逐步走向无人自治”。每种模式都只是设计选项，其价值必须在具体任务上得到证明。

13.2 从 Loop Contract 开始

Loop contract 是使重复运行有边界且可测试的 harness 配置。启用重复执行之前，应定义：

- **Trigger**：人类请求、事件、schedule 或父 run，以及去重语义。
- **Goal 与 acceptance contract**：请求的结果、允许范围、约束和判定成功所需的证据。
- **Execution state**：当前 task、attempt、预算消耗、policy 状态、artifact 和最近一次已接受的 checkpoint。
- **Iteration boundary**：单轮最多可以尝试并提交的工作单元。
- **Action envelope**：可用 tool、identity、resource、副作用类别与 mandatory approval gate。
- **Verifier plan**：要运行的检查、顺序、阈值、abstention 行为和 escalation 目标。
- **Progress function**：区分有效变化与“看似忙碌但没有进展”的度量。
- **Stop 与 suspend rule**：终止原因、重试规则和可恢复的等待状态。

这份 contract 属于 harness 与 control plane，而不属于模型生成的文字。模型可以提出“目标已完成”，但由 harness 判断配置要求的证据是否支持 success。Anthropic 的生产模式指南也把 agent 描述为在循环中使用环境反馈、在 checkpoint 或 blocker 处暂停并请求人类输入，同时通过迭代上限等停止条件保留控制（[Anthropic — Building effective agents](#)）。

一个有边界的迭代可以表示为：

载入 execution state

- 选择一个符合条件的工作单元
- 授权并执行有边界的动作
- 收集 artifact 与环境 observation
- 运行配置好的 verifier plan
- checkpoint 已接受的进展
- 继续、挂起或带原因终止

迭代次数本身不等于进展。有效的 progress function 取决于任务：失败测试数量下降、必填字段通过校验、目标环境进入预期状态、证据覆盖率提高，或者由人类解决一个具名歧义。应持久化这些度量和 artifact identity，使后续 run 能判断是否真的发生了变化。

13.3 默认 Verifier 层级

先使用能够为当前后果提供充分证据的最低成本 verifier；如果低层证据仍留下重大不确定性，再向上叠加。默认升级层级如下：

层级	Verifier	最适合提供的证据	主要局限
1	Schema check	必填字段、类型、范围和协议形状	形状合法不代表结果正确
2	确定性测试与 linter	可复现的行为断言、不变量和静态规则	测试可能不完整，也可能与某种实现方式耦合
3	环境 outcome check	外部系统确实达到预期状态	良好终态可能掩盖不安全或不允许的路径
4	同一 agent 的 critique	低成本、上下文充分地发现并修复明显缺陷	Critique 可能沿用 maker 的假设与盲点
5	独立 model grader	对开放式或细腻属性进行 rubric 判断	独立身份无法消除模型错误或相关失败
6	Human review	意图、问责、专家判断与歧义处理	慢、昂贵，而且不会自动保持一致

这是升级层级，不是要求每个任务都采用全部六层，也不是说后一层在所有方面都优于前一层。人类不应替代 checksum，独立模型也不应替代可执行测试。Agent eval 通常组合 code-based、model-based 和 human grader：代码检查快速且可复现；模型 grader 能处理细腻判断，但需要用 human grader 校准；人工评分更慢且成本更高 ([Anthropic — Demystifying evals for AI agents](#))。

“Maker must not be the checker” 这句旧口号过于绝对。当后果较低且缺陷在局部可见时，同一 agent 的 critique 可以是高效的修复步骤。反过来，一个名义上独立的 checker 可能与 maker 共用相同的基础模型、训练分布、prompt 假设、tool 或不完整 rubric。只有当角色分离带来的错误下降经过测量、足以抵偿成本，而且相关失败得到理解时，这种分离才有价值。

13.4 根据证据选择 Verifier，而不是根据口号

第 11 章定义了本章使用的 verifier profile：失败类别、false positive 与 false negative、abstention、disagreement、coverage、latency、cost 和 correlated failure。应根据五项任务属性选择和组合层级：

- Consequence**。后果越重大、越难逆转，越需要更强证据、更严格的 false-accept 目标，而且往往需要 policy gate 或由可问责的人进行 review。
- Task ambiguity**。精确输出适合 schema 与确定性检查；开放式质量或意图问题可能需要经过校准的模型 rubric、专家 review 或明确 abstain。
- Capability boundary**。应根据 eval 结果而不是直觉，判断 maker 能否可靠处理该任务类别。接近测得的能力边界时，额外 critique 或 evaluator 可能带来价值；远在边界以内时，它可能只增加 latency 与 cost。Anthropic 在一个长时运行 coding harness 中报告了这种与任务相对的效果：随着模型能力提升，对于 generator 已能可靠完成的工作，evaluator 变成了额外开销；但在能力边缘，它仍然有用 ([Anthropic — Harness design for long-running application development](#))。
- Latency 与 cost**。先运行便宜且信号强的检查，把较慢 grader 留给尚未消除的不确定性。Verifier 的 cost 与 latency 应和 maker 的 cost 与 latency 分开记录。
- Correlated failure**。估计 maker 与 checker 是否会在相同样本上失败。不同角色、prompt 或模型本身并不能证明存在有用的独立性；应使用第 11 章的 joint-error 与 disagreement 度量。

一条实际策略可以是：对每个 tool result 进行 schema check；对每个候选 artifact 运行测试与 linter；在宣称外部 outcome 之前检查真实环境；用同一 agent critique 进行低成本修复；仅对剩余的 rubric 属性调用独立 grader；当后果或未解决歧义超过阈值时要求 human review。

每个 verifier 都应至少定义三种结果，而不是强制二元判断：pass、fail 和 abstain；版本化 contract 也可以定义 partial。Abstention 必须路由到另一个 verifier 或交给人类解决，绝不能被静默转换为通过。对于研究等主观、开放式任务，应根据专家人类判断持续校准模型 grader ([Anthropic — Demystifying evals for AI agents](#))。

13.5 将在线验证与发布评测分开

Live verifier 会影响 trajectory：它选择重试、提供反馈、接受 checkpoint 或停止 loop。**Holdout grader** 在不引导 trajectory 的前提下衡量完成后的系统。把 release grader 的隐藏答案或完整 rubric 作为迭代反馈，会形成 evaluator leakage 的路径，使系统针对检查而不是底层任务进行优化。

因此：

- 只暴露有助于修复所必需的 acceptance evidence；
- 把 holdout case 和 release threshold 放在 maker 无法写入的状态之外；
- 为每个 schema、test suite、rubric、grader model 和 human-review protocol 建立版本；
- 记录哪个 verifier result 导致了每次 transition；
- 定期把自动 verifier decision 与 blind human label 比较；
- 调查 disagreement 与 joint error，而不只看整体 pass rate。

Outcome check 与 trajectory check 回答不同问题。前者问预期状态是否已经达到；后者问达到该状态的路径是否遵守 policy、budget 与 tool constraint。Anthropic 的 eval 指南明确区分对 outcome 与 transcript 的评分，并给出了组合 state check、required tool call、turn limit 与 quality rubric 的示例 ([Anthropic — Demystifying evals for AI agents](#))。当正确结果可能通过不可接受的路径获得时，loop 同时需要两类检查。

13.6 Stop、Suspend 与 Resume 是不同状态

每个 run 都必须以机器可读的 stop_reason 终止，或者进入具名的非终止挂起状态。最小终止集合是：

Stop reason	必要条件	重试语义
success	配置要求的 acceptance contract 已通过；自我宣称不足以成立	只有在出现新 goal 或证据失效时才开始新 run
budget_exhausted	达到任一 token、time、money、action 或 iteration 上限	只有获得新授权预算后才恢复
no_progress	Progress function 在配置的窗口内持续低于阈值	改变证据、plan、tool 或任务表述后再恢复
repeated_failure	同一个归一化 failure signature 超过允许重试次数	应改变可疑原因后再恢复，不能只重置计数器
policy_denied	Policy decision point 拒绝动作，而且不存在允许的替代方案	模型重试不能覆盖该决定；只能通过授权路径改变 policy 或 scope
human_cancelled	获得授权的人取消 run	不得自动恢复；必须有新的明确请求
environment_terminal	环境报告吸收态，例如已删除、已过期、已关闭或不可恢复失败	只有环境和 task contract 允许时才开始新 run

approval_wait、缺少输入、rate-limit backoff 和依赖暂时不可用，通常属于挂起，而不是成功或失败的终止状态。应持久化 checkpoint、尚未解决的请求、过期时间以及有权解决它的 identity。恢复时应从已记录的 execution state 继续，而不是让模型从对话文字重建等待状态。

Policy enforcement 必须位于模型 loop 之外。NIST 把 policy decision point 定义为计算访问决策的组件，把 policy enforcement point 定义为对受保护资源执行访问决策的组件 ([NIST — Zero Trust Architecture Glossary](#))。因此，在本书架构中，policy_denied 是 harness/runtime outcome，而不是模型可以靠辩解绕过的异议。

优先级也必须明确。例如，在调度下一次 retry 前应先检查 human_cancelled、policy_denied 与 environment_terminal；success 必须有当前有效的证据；预算消耗的提交应具有足够的原子性，避免并发 worker 各自花掉同一份剩余额度。

13.7 嵌套 Loop 需要所有权边界

一种实用拓扑，是把快速的内层动作循环与较慢的外层控制循环分开：

- **Action loop** 执行一个有边界的工作单元，并消费即时环境反馈；
- **Repair loop** 根据 verifier failure 选择下一次尝试；
- **Task loop** 选择下一工作单元，并检查 goal-level completion；
- **Release loop** 应用 holdout evaluation、policy、approval 或 deployment control；
- **Product loop** 使用真实世界 outcome 修改 goal 与 acceptance contract。

不要让内层 loop 改写父级 success criteria。Worker 可以提出修改 plan，但必须由父 contract 的所有者接受。同样，子 run 可以返回 artifact 与 evidence；父级仍负责判断这些证据是否满足自己的 verifier plan。

Evaluator–optimizer loop 适用于 evaluation criteria 清晰且迭代改进能够产生可测量价值的情况；它不是每次模型调用都应套用的默认 wrapper ([Anthropic — Building effective agents](#))。每个嵌套 loop 都应拥有自己的 budget、state owner、stop reason 与 escalation route，避免一次“retry”扩展成无边界的委派树。

13.8 Context Reset 只是一种恢复策略

Ralph 实践有意让每个 loop 只执行一个仓库任务，并在每轮重新加载 plan 与 specification artifact ([Geoffrey Huntley — Ralph Wiggum as a “software engineer”](#))。这样做可以减少长交互造成的污染，也使 durable artifact 更重要。但它不能证明“memory 必须存在磁盘上”这一更强主张，也不能证明每轮都应丢弃上下文。

应根据失败模式选择 context policy：

- 当近期 observation 和未解决的局部状态仍然有时，继续使用同一上下文；
- 当相关 trajectory 可以被压缩为带可追溯引用的摘要时，执行 **compaction**；
- 当陈旧假设或 context pressure 导致 repeated failure 时，执行 **reset and rehydrate**；

- 当 capability、authority 或 responsibility 发生变化时，**handoff** 到另一个 run。

把 canonical execution state、artifact、checkpoint 和 event history 持久化在 harness 管理的存储中。仓库文件可以是其中一种 artifact，但它不会自动成为 identity、budget、approval 或 policy state 的事实来源。Anthropic 的长时运行 harness 报告介绍了使用结构化 artifact 在 session 间 handoff，同时也表明：当更强模型能力使某些 scaffolding 成为多余开销时，应移除这些结构 ([Anthropic — Harness design for long-running application development](#))。

13.9 用 Capability-Control Profile 取代自治阶梯

单一成熟度阶梯会错误暗示所有产品都应从人工运行进化到 auto-merge。设计成熟度是多维的。至少应记录以下维度：

维度	示例值	设计问题
Trigger autonomy	manual · event · schedule · parent run	谁可以启动工作，如何避免重复？
Action scope	read-only · draft · reversible write · irreversible effect	单轮可以改变什么？
Verifier strength	各层级及其测量 profile	什么证据足以支持继续或成功？
Runtime durability	ephemeral · checkpointed · replayable	中断后什么能够保留？
Policy 与 approval	allowlist · dynamic policy · mandatory gate	哪些决策不属于模型权限？
Observability	final artifact · trace · event history · lineage	运维者能否重建决策路径？
Recovery	retry · compensate · rollback · human repair	如何控制部分失败？
Human responsibility	operator · reviewer · approver · accountable owner	哪些位置仍保留判断与问责？

一个系统可以在 durability 与 observability 上高度成熟，同时有意保持 read-only 与 manual trigger。另一个系统可以定时执行低后果、可逆动作，但在 release 时要求 human approval。目标 profile 应由产品价值、失败后果与证据决定，而不是普遍追求最大自治。Anthropic 同样建议只在能够证明改善 outcome 时增加 agentic complexity，并指出 latency、cost 与错误累积方面的取舍 ([Anthropic — Building effective agents](#))。

13.10 把 Loop 作为可测量的控制系统运行

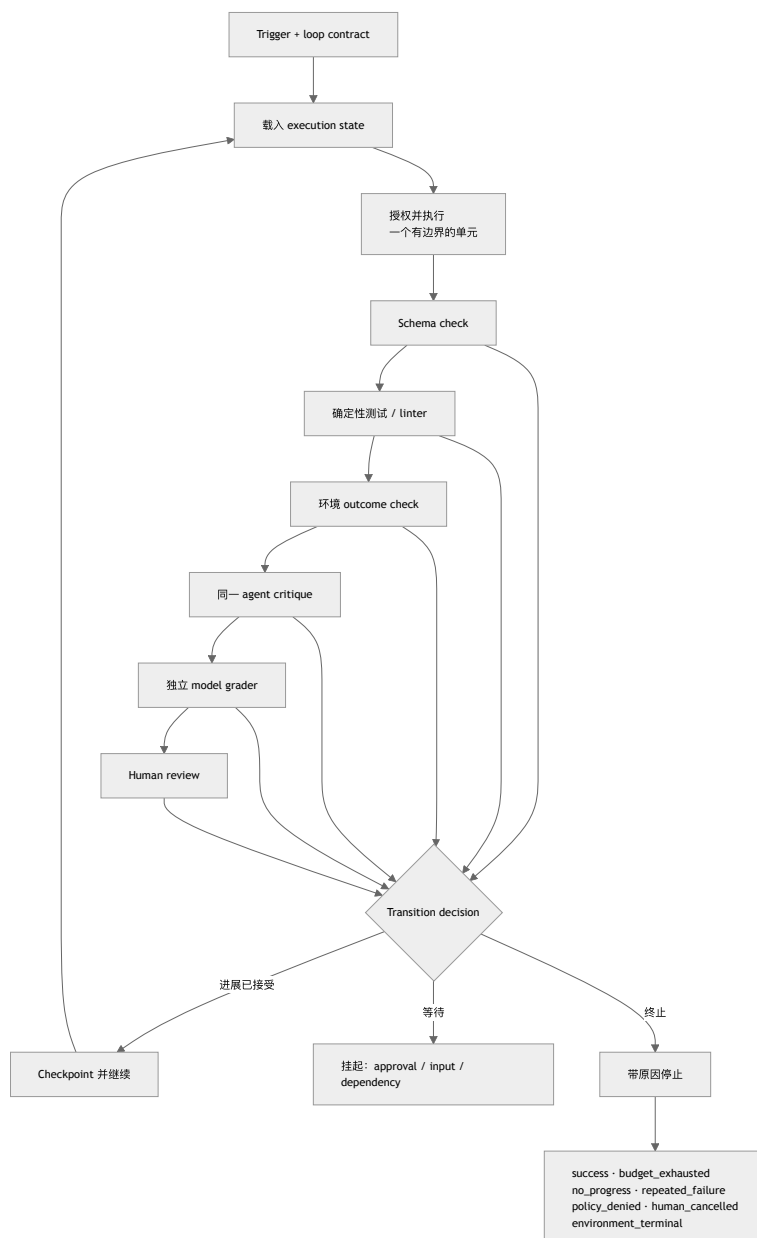
每个 run 都应生成 loop control record，其中包含 contract version、trigger identity、iteration 与 retry count、已用 budget、verifier version 与 result、progress measurement、artifact、policy 与 approval event、最终 checkpoint 和 stop_reason。第 10 章提供 durable execution 语义；第 11 章提供 verifier calibration；第 12 章提供局部 self-verification 模式；第 14 章提供 human review 与 approval 语义；第 17–19 章把 trace、budget、policy enforcement、lineage 和 fleet operation 连接起来。

使用这些记录回答运维问题：

- 哪些 failure signature 消耗了最多 retry？
- 哪些 verifier 层真正改变 decision，而不只是增加 cost？
- Maker 与 checker 在哪些地方共同失败？
- success 有多大比例能通过 holdout 与 human review？
- 哪些任务因为预算、无进展、策略拒绝、人类取消或环境终态而停止？
- Context reset 是否改善了目标 failure class 的恢复表现？

只有当证据支持某项具体变化时，才扩大 loop 的 action scope、trigger autonomy 或 retry allowance。当 continuation 由测得的进展证明合理、termination 由可审计原因证明合理时，这个 loop 才称得上经过工程化。

图示：使用递进证据的有界 Loop



本章要点

- “Loop engineering”与“Ralph”是实践者谱系，而不是标准。应选择性地采用其中的模式，并测量结果。
- **Loop contract** 使重复运行有边界。它定义 trigger、state、action envelope、evidence、progress、budget，以及 stop 或 suspend 语义。
- 使用 **verifier** 层级。从 schema check 开始，依次经过确定性证据、环境证据、critique、独立模型评分和 human review；只有不确定性与后果要求时才逐层升级。
- **Maker** 可以 **critique** 自己的工作。只有当独立检查经过校准的错误 profile 与相关失败下降足以证明成本合理时，它才有价值；这不是无条件规则。
- 每个 run 都需要明确的终止原因。必须覆盖成功、预算耗尽、无进展、重复失败、策略拒绝、人类取消与环境终态。
- 等待不等于停止。Approval、缺少输入与暂时性依赖等待属于持久化挂起，并具有明确的恢复权限。
- 不存在普遍适用的自治阶梯。成熟系统会选择适合自身任务与风险的 capability-control profile。

延伸阅读

- Shunyu Yao 等, *ReAct: Synergizing Reasoning and Acting in Language Models*. <https://arxiv.org/abs/2210.03629>
- Anthropic, *Building effective agents*. <https://www.anthropic.com/engineering/building-effective-agents>

- Anthropic, *Harness design for long-running application development*. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- Anthropic, *Demystifying evals for AI agents*. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Geoffrey Huntley, *Ralph Wiggum as a “software engineer”*. <https://ghuntley.com/ralph/>
- Addy Osmani, *Loop Engineering*. <https://addyosmani.com/blog/loop-engineering/>
- NIST, *Zero Trust Architecture Glossary*. <https://pages.nist.gov/zero-trust-architecture/glossary.html>

第 14 章：人-Agent 交互

人在 agent system 中扮演多种角色：提供 intent、回答问题、批准或拒绝有重要后果的动作、引导进行中的工作、取消 run，以及 review 证据。这些角色需要 product 与 runtime 的支持，不能只靠一条含糊指令来安全实现，例如要求模型“在适当的时候询问”。

Mixed-initiative interface 研究早在现代 agent 出现之前就提出了底层设计问题：代人行事的系统必须判断何时行动、何时让人决定，如何衡量打断用户的成本，以及如何保留交互 context ([Horvitz — Principles of Mixed-Initiative User Interfaces](#))。Agent system 又加入了 external side effect 与 long-running state，因此接口还必须明确表示 authority、evidence、timing 与 recovery。

14.1 咨询与强制审批是两条不同路径

两种人机交互模式在 chat UI 中看起来可能相似，但它们具有不同的安全语义：

模式	触发方式	含义	Enforcement owner
Agent-requested consultation (Agent 主动咨询)	模型或 workflow 因 intent、事实或判断不清楚而提出问题	人的输入成为一项 observation，可以指导下一次模型调用	Harness/runtime 记录问题与回答；回答不会自动授予无关 capability
Runtime-mandated approval gate (Runtime 强制审批门)	Policy 判定 proposed action 在 dispatch 前必须获得批准	经过认证且限定 scope 的 decision，在明确条件下允许或拒绝该准确动作	动作路径上不可绕过的 policy enforcement point (PEP)

HumanLayer 提出的“contact humans with tool calls”是第一条路径中很有用的 orchestration pattern：把问题表示成结构化工作，挂起执行，再把回答作为 observation 返回 loop ([HumanLayer — 12-Factor Agents](#))。模型可以请求澄清，也可以建议由人 review 某个选择，但它也可能根本没有提出请求。

因此，第二条路径不能依赖模型自愿调用。NIST 把 PEP 定义为执行 access decision 的组件，其 access-control 指南还要求 enforcement mechanism 不可被绕过 ([NIST — Zero Trust Architecture Glossary](#)；[NIST IR 7987](#))。MCP tools specification 同样建议应用展示 tool input，并对 sensitive operation 请求确认 ([MCP — Tools Specification](#))。即使模型从未发出 request_human_approval，runtime 仍必须拦截动作。

Consultation answer 可以促使 policy 重新评估某个动作，但除非回答是由 authenticated gate 按所需 scope 收集的，否则它本身不是 mandatory approval。反过来，runtime 也可以要求审批 agent 认为很普通的动作。

14.2 根据风险分配人的注意力

每个动作都要求审批会让人形成习惯并拖慢有用工作；review 太少又会鼓励盲目信任。Anthropic 描述了怎样把文件与网络操作放进明确 sandbox boundary，从而减少重复 permission prompt，同时继续要求确认越过边界的动作 ([Anthropic — Beyond Permission Prompts](#))。一般原则是：把人的注意力用在 policy-relevant consequence 足以证明其必要性的地方。

至少应沿以下维度对 action 分类：

- **Reversibility (可逆性)**：能否恢复准确的先前状态，rollback 是否已经测试？
- **Blast radius (影响半径)**：影响一个 scratch file、一个 user、一个 tenant、production service，还是公开 audience？
- **External communication (外部通信)**：动作是否会向 working boundary 之外发送、发布、提交或共享内容？
- **Money (金钱)**：动作是否会产生购买、转账、订阅、合同或按量计费支出？
- **Production (生产环境)**：动作是否会改变 live infrastructure、data、access 或 availability？
- **Sensitive data (敏感数据)**：动作是否会读取、转换、暴露、保留或传输 secret、personal data、regulated data 或 tenant-confidential content？

下面是一张起始矩阵：

动作画像	示例	默认交互	必需证据
可逆、位于 sandbox、影响半径小	读取允许的文件；运行测试；编辑一次性 workspace	在现有 authorization 下执行，并按需报告	Action/result record；sandbox boundary
可逆且 scope 有限，但会改变持久工作	创建 draft 或 branch；更新非生产 artifact	执行并报告，或按 policy 抽样 review	Diff 或 versioned artifact；rollback path；已运行的 check
外部通信	发送邮件；发表评论；提交表单	预览准确 audience 与 payload；除非有限 standing policy 已覆盖，否则通常需要审批	Recipient/destination、normalized payload、identity、policy reason
金钱或有约束力的承诺	购买、转账、扩大付费 API 使用、接受合同	必须审批，并显示 amount、recipient/vendor、limit 与 expiry	准确 transaction proposal；budget 与 authorization evidence
生产环境或大影响半径	部署、修改 access、变更 live data、轮换 credential	必须审批，或者进入更严格的 change-management gate	Diff/plan、affected resource、test、rollback、current health
敏感数据披露或移动	导出客户数据；发送 secret；改变 sharing scope	除非得到明确授权，否则拒绝；policy 允许时仍需 mandatory approval	Data class、source、destination、purpose、minimization/redaction evidence
不可逆或破坏性动作	永久删除；不可撤回地发布；破坏性 migration	必须审批，并提供 recovery 或 exception plan	准确 target、impact analysis、backup/recovery evidence、expiry

这不是通用 allowlist。可逆性可能具有迷惑性：如果文件受版本控制，删除本地文件或许没有危险；而发送一条“可编辑”的消息仍然可能造成不可逆的信息披露。Policy 应结合这些维度与 identity、purpose、environment 和当前 resource state 作出判断。

14.3 强制审批门把批准绑定到准确 Proposal

Mandatory approval 应针对 immutable 或 content-addressed 的 action preview 发放。Pending record 至少包括：

proposed_action
normalized_arguments
resource_identity_and_version
acting_user_agent_and_delegation_identity
policy_id_and_version
risk_class_and_policy_reason
approval_scope_and_expiry

Decision record 还应加入 authenticated approver identity、approver authority scope、decision、time、condition 与 proposal hash。UI 可以展示友好的 label，但 grant 必须绑定 normalized argument 与 stable resource identifier，而不能只绑定 natural-language prose。

发生 **material change（实质变化）** 后，先前批准立即失效，proposal 必须重新通过 policy。典型变化包括 amount、recipient、destination、target resource 或 version、data class、external audience、acting identity 或 delegation、tool semantics、side-effect scope、policy version，或者执行时间已经超过 grant expiry。Policy 应针对具体 domain 定义 materiality；模型不能自行判断两个动作“足够接近”。第 6 章对 retry 采用相同规则：approval、authorization 与 outcome status 的输入改变后，必须重新检查。

批准只允许系统尝试执行有限 scope 的动作。它不能证明工具已经运行，不能证明 timeout 没有产生 effect，也不能证明目标 outcome 已经发生。Approval ID、dispatch attempt 与 outcome evidence 必须保持分离。

14.4 审批事件支持恢复；审计还需要更多保证

Consultation 与 approval 应通过显式 transition 进入 workflow event history，例如：

consultation_requested -> consultation_answered
approval_required -> approval_granted | approval_denied | approval_expired
approval_granted -> approval_revoked | action_dispatched

每个 event 都携带 run/action identity、sequence、actor、time、proposal 或 artifact reference、policy version 与 previous state。这样，durable runtime 可以在等待期间释放 compute，并在之后重建是否允许恢复执行。Temporal 的 event-history model 是一个具体案例：系统用持久化 event 在故障后重建 workflow state ([Temporal — Events and Event History](#))。第 10 章定义本书的 event 与 replay contract。

Event history 不会自动成为 audit record。NIST SP 800-53 把 audit-record content、generation、review、protection、access 与 retention 分别列为明确 control ([NIST SP 800-53 Rev. 5](#))。只有当系统能够建立所需的 **completeness**、**identity**、**timestamp**、**integrity**、**retention** 与 **access** 属性时，approval event 才能成为 audit evidence。Sampled trace 或可修改的 application log 可以链接到 event，却不能悄悄继承这些保证。

Audit representation 应保留展示过什么、谁作出 decision、其 authority 是什么、决定之后发生了哪些变化、dispatch 了哪个 action，以及确认了什么 outcome。同时还应尽量减少不必要的 prompt、personal data 与 secret；可审计性并不意味着获准永久保留所有内容。

14.5 围绕决策与证据设计 Review Surface

有效的 review surface 应帮助人理解正在发生什么、需要作出哪项决定，以及哪些证据支持该决定。Human-AI interaction guideline 建议界面清楚说明系统 capability、展示 status、支持 correction，并让人能够轻松 dismiss 或 override 错误输出 ([Amershi et al. — Guidelines for Human-AI Interaction](#))。

应根据情况展示：

- **Plan 与 current status**：预期 milestone、已完成工作、pending work 与 blocker；
- **Tool action**：Operation name、normalized argument、resource、acting identity、预期 side effect，以及 idempotency/outcome status；
- **Diff 或 artifact**：准确 version、content hash、rendered change、destination 与 rollback option；
- **Test 与 evidence**：实际运行的 check、result、citation、environment state 与完整输出链接；
- **Uncertainty**：assumption、冲突 evidence、缺失 check、unknown outcome，以及解决这些问题需要什么；
- **Policy reason**：为什么 action 被 allow、deny、redact、sandbox 或送去 approval，适用哪个 policy/version；
- **Decision consequence**：Approval 允许什么、何时 expiry，以及哪些 material change 会要求重新决定。

这不要暴露 hidden chain-of-thought。OpenAI Model Spec 指出，部分模型会生成 hidden chain-of-thought，但除可能提供摘要外，它不会暴露给 developer 或 user ([OpenAI — Model Spec, Hidden Chain of Thought](#))。Review system 应依赖简洁 plan、可观察 action、artifact、check、source、policy decision 与 environment outcome，而不是 private reasoning text 或模型事后生成的 rationale。

应区分 **pre-action approval** 与 **post-action review**。Dispatch 之前，重点展示准确 proposed effect 与 approval scope；执行以后，重点展示实际变化、verification evidence、剩余 uncertainty 与 recovery option。无论在哪个界面，只显示一句“done”都不充分。

14.6 Steering 与 Cancellation 需要事件语义

Steering 不能只是追加一条消息。应记录 `steering_requested` event，其中包含 requester identity、scope、instruction reference、reason 与 time。Controller 随后记录该请求是 accepted、rejected 还是 superseded，以及它从哪个 sequence 或 step 开始生效。通常，它可以影响下一次模型调用，并阻止尚未 dispatch 的后续 action；却不能倒过来修改已经 dispatch 的 action argument。

Cancellation 同样至少具有两种状态：**cancellation requested**，以及最终的 **cancelled**、**partially completed**、**failed** 或 **unknown** outcome。Temporal 的 workflow model 区分取消请求与最终 terminal state，而且 activity cancellation 采用协作语义，并不证明 rollback 已经发生 ([Temporal — Workflow Execution](#)；[Temporal — Events and Event History](#))。

收到 steering 或 cancellation event 后：

1. 停止在其 scope 内安排新 action；
2. 记录当前 model/tool call，以及是否已经发生 dispatch；
3. 如果 in-flight operation 的 contract 支持取消，就发出 cancellation request；
4. 不得假定已经 dispatch 的 effect 被回滚；
5. 对 external state 进行 reconciliation，并记录 succeeded、partial、cancelled、failed 或 unknown outcome；
6. 如果新方向对 pending approval 造成 material change，就让该批准失效；
7. 在按新 instruction 恢复之前建立 checkpoint 或 handoff。

一次长时间、不可中断的 model call 可能只能在返回后看到 steering。Remote tool 可能忽略 cancellation，也可能在收到请求前已经完成。UI 必须如实显示这种时序：“request received”并不等于“action stopped”。第 6 章负责单次调用的 cancellation 与 unknown-outcome handling；第 10 章负责 durable state transition。

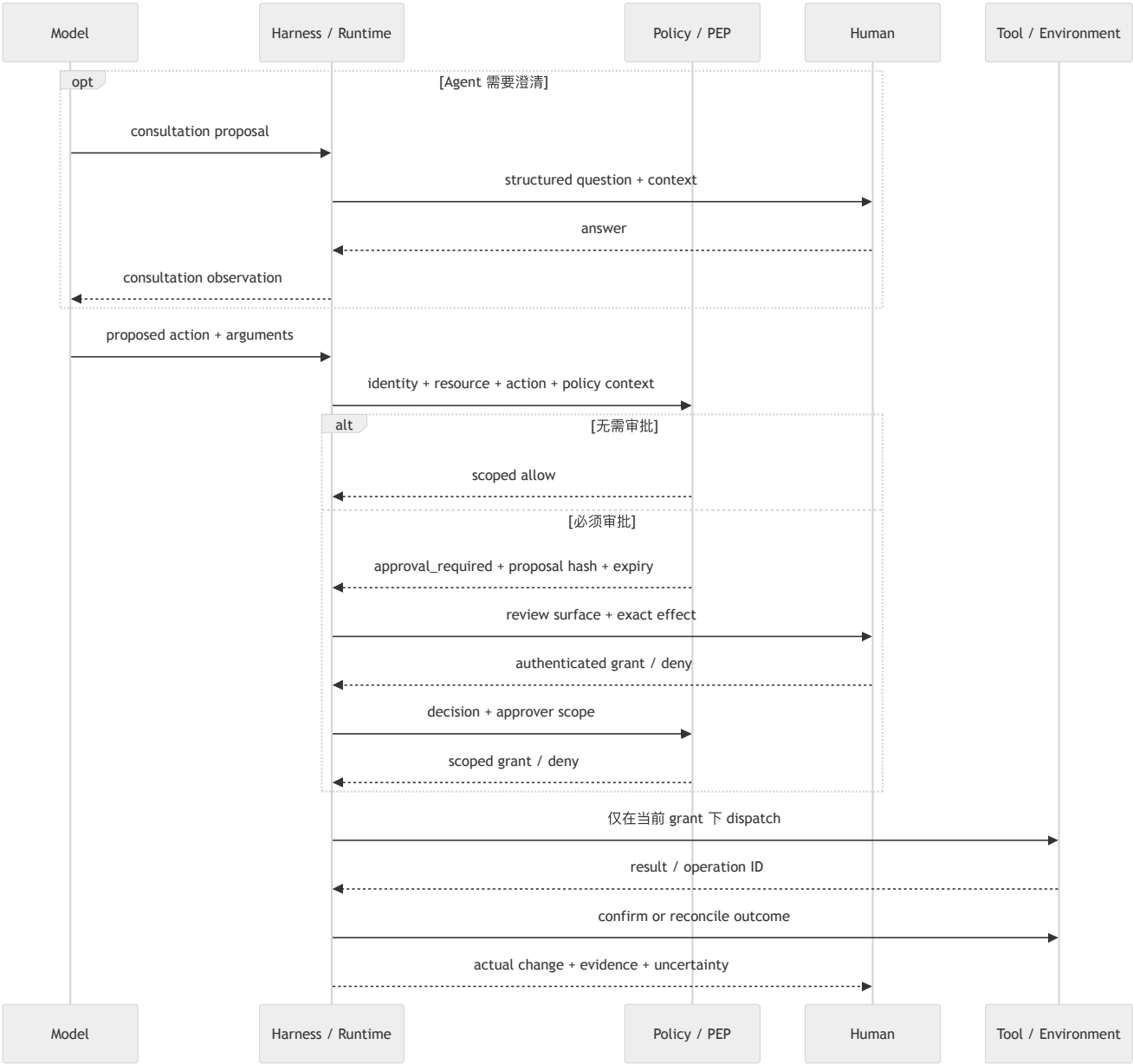
14.7 从一次 Review 到 Fleet Queue

对于一个 run，reviewer 可以直接检查 pending action；面对整个 fleet，queue 应按风险与证据排序，而不是只按 arrival time 或 transcript length。值得升级的信号包括即将 expiry 的 mandatory approval、sensitive-data 或 production action、广泛 blast radius、external communication 或 spend、deterministic check 失败、artifact 缺失、evidence 冲突、unknown side effect、新的 tool/policy combination，以及反复发生的 human override。

验证充分的低风险工作可以根据 policy 只展示摘要或抽样；有重要后果的工作则应保留准确 proposal 与 review artifact。Queue status 必须链接到 run/action ID 和当前 event-history state，防止一张过时 card 批准已经变化或完成的 action。

跨 run identity、queue ownership、policy administration、revocation 与 fleet lifecycle 属于第 19 章。本章提供 fleet control plane 必须保留的人工 decision 与 review contract。

14.8 围绕一个动作的两条人工路径



Consultation path 改善 intent 与 judgment；mandatory path 执行 policy。两条路径可以出现在同一个 run 中，却不能互相替代。

要点

- **区分咨询与审批**：模型可以向人询问信息，但 runtime-mandated gate 不能依赖模型自愿调用。
- **把批准绑定到准确动作**：记录 normalized argument、resource、identity、policy/version、risk、scope 与 expiry；material change 必须重新决定。
- **根据风险分配注意力**：综合考虑 reversibility、blast radius、external communication、money、production 与 sensitive data。
- **批准是权限，不是 outcome evidence**：Dispatch 与 postcondition confirmation 仍是不同的 lifecycle stage。
- **Event history 不会自动成为 audit**：只有具备明确 completeness、identity、integrity、timestamp、retention 与 access，approval event 才能成为 audit evidence。
- **Review 可观察工作，而不是隐藏推理**：展示 plan/status、tool action、diff 或 artifact、test 与 evidence、uncertainty、policy reason 和 decision consequence。
- **Steering 与 cancellation 是状态转换**：记录它们何时生效，并 reconcile 任何已经 dispatch 的 action。
- **Fleet review 属于 control plane**：第 19 章负责跨 run queue、identity、policy administration、revocation 与 lifecycle。

延伸阅读

- Eric Horvitz, *Principles of Mixed-Initiative User Interfaces*, CHI 1999. <https://www.microsoft.com/en-us/research/publication/principles-of-mixed-initiative-user-interfaces/>
- Saleema Amershi et al., *Guidelines for Human-AI Interaction*, CHI 2019. <https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts: Making Claude Code More Secure and Autonomous*, Anthropic, Oct 2025. <https://www.anthropic.com/engineering/claude-code-sandboxing>
- Model Context Protocol, *Tools Specification*, Jun 2025. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
- NIST, *Zero Trust Architecture Glossary*. <https://pages.nist.gov/zero-trust-architecture/glossary.html>
- NIST, *IR 7987: Policy Machine*. <https://nvlpubs.nist.gov/nistpubs/ir/2014/NIST.IR.7987.pdf>
- NIST, *SP 800-53 Rev. 5*. <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
- Temporal, *Workflow Execution*. <https://docs.temporal.io/workflow-execution>
- Temporal, *Events and Event History*. <https://docs.temporal.io/workflow-execution/event>
- OpenAI, *Model Spec*, Oct 2025. <https://model-spec.openai.com/2025-10-27>

第 15 章：Computer-Use 与多模态 Agent

Computer use 是工具调用生命周期的一种特化，而不是另一套抽象。模型接收一个或多个用户界面 observation，提出结构化 computer action，再由 application 或 harness 决定是否执行以及如何执行。随后，screenshot、结构化状态、错误或执行记录作为相关联的 tool result/observation 返回。OpenAI 的当前契约使用 `computer_call` action 和 `computer_call_output`；Anthropic 的 client-executed computer tool 则在同一条“模型提出、应用执行”的边界下使用 `tool_use` 与 `tool_result`（[OpenAI — Computer Use](#)；[Anthropic — Computer Use Tool](#)；[Anthropic — How Tool Use Works](#)）。

Computer use 的特殊难点，不是图像不再属于 tool result，也不是点击不再类似 function proposal，而是 GUI observation 局部且容易过期、grounding 涉及几何映射、action 依赖未必可见的界面状态，而且一次成功的 input event 不一定产生预期的 environment outcome。

15.1 结构化 Computer-Tool Loop

这条专门化 loop 扩展了第 6 章的 invocation lifecycle：

1. 捕获 **observation bundle**。记录 screenshot 和/或结构化 UI state，同时保存 identity、时间、几何信息和 surface metadata。
2. 让模型提出 **action proposal**。输出是结构化动作，例如 `click(x, y)`、`type(text)`、`scroll(direction, amount)`、按键组合，或自定义 DOM/role action。
3. 标准化并验证。把 proposal 绑定到它使用的 observation；验证 action type、参数、坐标变换、当前 application/origin、目标 scope 和 policy。
4. 授权并审批。不可绕过的 enforcement point 应用 allowlist、sandbox 约束和必要的 mandatory human gate。
5. 由应用执行。Browser automation、OS input adapter 或其他 UI runtime 实施动作，而不是模型实施。
6. 返回 **execution result** 与新 **observation**。把二者与 proposal 关联，并根据事实报告 `succeeded`、`failed`、`partial` 或 `unknown`。
7. 验证 **environment outcome**。判断预期的 UI 和 backend postcondition 是否成立；否则重新观察、恢复或停止。

OpenAI 当前的内置 loop 会在一个 `computer_call` 中返回有序 action，由应用代码执行，再要求应用用相关联的 `computer_call_output` 返回更新后的 screenshot。Anthropic 同样要求应用代码把 tool request 转换成 computer action，并通过 `tool_result` block 返回 screenshot 或 command output（[OpenAI — Computer Use](#)；[Anthropic — Computer Use Tool](#)）。Provider payload 不同，但责任边界相同。

15.2 区分六类记录

稳健的 harness 不能把这些对象压缩成一个笼统的“screen state”：

对象	它表示什么	最少应有的 metadata	它不能证明什么
Screenshot observation	某一时刻捕获的渲染像素	observation_id、捕获时间、image digest、像素尺寸、viewport、crop、scale/device-pixel ratio、window/tab、当前 URL 或 application identity	Dispatch 时仍然存在同样的像素
DOM/accessibility state	UI automation surface 暴露的结构化 element、role、label、value、bounds 和关系	snapshot/version ID、frame 或 window、origin、提取方法、element reference	精确外观、视觉显著性、可见性或可点击性，除非另外检查
Coordinate grounding	把预期目标或 element reference 映射到某一 observation 的坐标空间	source observation、target reference、bounding box/point、置信或歧义、coordinate transform	已经派发 input event
Action proposal	模型请求的 UI operation 及参数	proposal/call ID、source observation ID、action type、normalized arguments、expected effect	授权、执行或成功
Execution result	Executor 对 dispatch 和 input-event processing 的已知事实	attempt ID、逐 action status、时间戳、active surface、error、partial/unknown status	已经到达预期 product state
Environment outcome	交互后经过验证的状态	UI assertion、URL/origin、backend/API state、durable object ID、external-effect evidence	该路径符合 policy，除非还检查了 process evidence

Screenshot 可以是 tool result 的 payload；DOM 或 accessibility data 可以随同一个 result 返回，也可以通过单独工具返回。Click 可以编码成 provider 定义的 computer action、自定义 function call 或生成代码（[OpenAI — Computer Use](#)）。这些是 protocol 与 interface choice，不是互斥类别。

应保持整条链路的 identity：

```
observation_id
-> grounding_id
-> proposal_id / provider_call_id
-> dispatch_attempt_id
-> execution_result_id
-> post_action_observation_id
-> outcome_evidence_id
```

有了这条 correlation，eval 或 incident review 才能判断：是模型选择了错误目标、坐标映射错误、executor 在过期状态上行动，还是应用接收了输入却没有更新 backend。

15.3 根据决策选择 Observation Channel

Harness 可以提供几种互补的 observation channel：

- **Screenshot**：保留捕获时的渲染布局、样式、overlay 和空间关系。
- **DOM 或 application tree**：对支持的应用提供结构化内容与 element identity。
- **Accessibility state**：在应用正确发布信息时，提供语义 role、label、value、checked/expanded state 与 hierarchy。
- **OCR 或 extracted text**：在结构化状态不可用时提供文字；如果还要用于 grounding，则应保留 bounding box 与 source-image identity。
- **Direct environment query**：查询权威 backend 或 application state，用于 verification，而不是 perception。

OpenAI 当前指南明确支持视觉 computer action，也支持自定义 Playwright/Selenium/VNC/MCP harness，以及能把 screenshot 与 DOM workflow 组合起来的 code-execution harness（[OpenAI — Computer Use](#)）。这支持一种混合设计：用结构化状态获得稳定 element identity 和高效阅读，用像素判断几何与视觉条件，再用 direct state query 验证 outcome。

没有哪一种 channel 会自动成为所有问题的 ground truth。DOM node 可能存在，但点击被 overlay 接收；screenshot 可能显示文字，却不提供 element role 或稳定 identity。Playwright 因此会在受支持的 action 前检查 visibility、stability、element 是否接收 event、enabled state 和 editability 等属性（[Playwright — Auto-Waiting and Actionability](#)）。Computer-use harness 应在 automation stack 允许时暴露同类 precondition。

Observation channel 应按任务和模型做评测。记录每次决策使用了哪个 channel 和 extraction version；如果模型 context 只接收紧凑 projection，则把完整 observation 作为 artifact 保存。

15.4 Grounding 是带版本的几何契约

Coordinate grounding 把预期目标转换到 executor 的坐标空间。它可以使用：

- 从 screenshot 直接预测的 raw pixel coordinate；
- 通过 DOM 或 accessibility state 解析的 element reference；
- Set-of-Mark 之类的编号 overlay，由系统给候选区域添加离散 label（[Yang et al. — Set-of-Mark Prompting](#)）；
- 先按语义选择 element、再检查当前 bounds 并点击验证点的 hybrid lookup。

每次映射都必须声明其 geometry：

```
captured_pixel_size, model_visible_size, viewport_size
crop_origin, scroll_container_and_offset, browser_zoom
device_pixel_ratio, resize_scale, coordinate_origin
window_or_frame_id, element_or_mark_id
```

不能让 provider 在不可见的情况下缩放 screenshot，再把返回坐标直接应用到原始 display。Anthropic 当前 computer-use 文档警告：过大图像可能在模型看到前被缩小，因此模型坐标对应缩小后的图像；它建议在 client 侧用已知 scale 缩放，并特别说明 Retina device-pixel-ratio 的处理（[Anthropic — Computer Use Tool](#)）。OpenAI 同样建议当前 GPT-5.6 computer-use screenshot 使用 detail: "original"，如果 client 做了 downsize，则必须重新映射坐标（[OpenAI — Computer Use](#)）。

Grounding 会随 source observation 一起过期。Dispatch 前，应重新解析 element reference，或确认相关 geometry 和 surface identity 仍然有效。坐标永远不能作为按钮的 durable identifier。

15.5 Observation/Action Race 是常态

从捕获到 dispatch 之间，界面可能变化。核心 race 是：

observe state S0 -> propose action for S0 -> environment becomes S1 -> execute against S1

以下机制都会制造 race：

机制	失败示例	控制措施
Stale screenshot	模型响应延迟时页面已经跳转，click 仍落在旧页面位置	把 action 与 observation age、surface identity 绑定；超过 freshness deadline 就重新观察
Layout shift 或 animation	目标移动，坐标落到另一个 control	要求 bounds 稳定，或在 dispatch 前立刻重新 grounding
Focus change	输入进入 address bar、terminal 或错误字段	在敏感输入前检查 active window/frame 与 focused element
Scroll drift	滚动了错误 container，或目标离开 viewport	记录 container 与 offset；scroll 后验证 target 仍在 viewport
Modal/overlay	Consent dialog 或 popup 截获 click	检测最上层 dialog/overlay，并暂停或交给 policy
Latency/navigation	Click 已被接收，但下一个 action 仍假定旧页面	等待声明的 navigation/network/UI condition，再捕获新 observation
Partial batch execution	Action 1 成功，action 2 失败，继续 action 3 已不安全	记录逐 action status；停止依赖 action，返回 partial 和新 observation

OpenAI 当前内置契约可以在一个 computer_call 中批量返回有序 action，并在 batch 后捕获 screenshot (OpenAI — Computer Use)。这是 latency optimization，不表示每个 batch 都是原子的。因此，本章建议只批处理 precondition 可以共同保持有效的 action；对 consequential、会触发 navigation、依赖 focus 或不易逆转的 action，应插入 observation barrier。

不要通过盲目 replay click 或 submit 来“修复”race。响应丢失或 timeout 时，即使 executor 没有观察到成功，环境也可能已经变化。应使用第 6 章的 partial 与 unknown state，检查 UI 和 backend，再决定 deduplicate、escalate 或是否允许下一次 consequential attempt。

15.6 在 Dispatch 时验证 Action

Computer-use dispatch 是第 6 章 validation stack 的特化，但不能绕过它。执行前，policy enforcement path 应检查：

- proposal 与 observation identity、freshness 和 correlation；
- action schema 与 semantic bounds；
- 适用的 application、process、window、tab、frame、URL、domain 和 origin；
- 当前 focus、target visibility/stability、overlay state 与 coordinate transform；
- actor、tenant、credential scope、policy version 和 action allowlist；
- action 是否传输数据、对外通信、花费资金、改变访问权限、修改 production，或难以撤销；
- 与精确 normalized proposal 绑定的 mandatory approval；
- retry、deduplication、deadline、cancellation 和 action-budget state。

即使 provider 批量提出 action，result envelope 也应逐 action 记录：

status: denied | not_executed | succeeded | failed | partial | unknown
proposal_id, attempt_id, source_observation_id
surface_before, surface_after
normalized_action, executor_error
post_action_observation, postcondition_evidence

Executor 应在 state-changing action 或 batch 后捕获新 observation，并验证预期转换。“Mouse event delivered”是 execution result；“表单只提交一次，而且 order 以预期字段存在”才是 environment outcome。

15.7 安全边界必须包围模型

渲染后的网页、文档、email、chat、PDF 和 tool output 都是不可信输入。OpenAI 的 computer-use 指南明确说 on-screen instruction 不是 user permission，并建议使用隔离环境以及 domain/action allowlist；Anthropic 则警告网页或图像中的内容在某些情况下可能覆盖 instruction，并建议使用最小权限 VM/container、限制 domain，以及对有现实后果的决策做人类确认（OpenAI — Computer Use；Anthropic — Computer Use Tool）。

对高风险 UI action，应组合：

- 1. **Action allowlist**：只暴露或 dispatch 已批准的 action type 和 scoped target；默认拒绝意外 keyboard shortcut、download、shell launch、clipboard read 或 credential entry。
- 2. **Sandbox**：使用独立 browser profile、container 或 VM，并最小化 filesystem、network、credential、device 和 host integration，具体边界见第 7 章。
- 3. **Domain 与 origin check**：在 navigation 和 dispatch 时检查实际 current origin，包括 redirect、iframe、popup 和新 tab；上一次 screenshot 中记住的 domain 不是当前 authority。
- 4. **Mandatory approval**：在发送/发布、传输敏感数据、购买、删除、修改 access、接受条款或其他由 policy 分类的 consequence 之前，立刻插入第 14 章的 runtime gate。Approval 绑定准确 target、payload、identity、origin 和 expiry。
- 5. **Post-action verification**：在报告成功或尝试 retry 之前，独立检查新 UI，并在可能时检查权威 backend state。

Instruction priority 仍然是一层行为防御：它可以要求模型把页面内容当作 data，并报告可疑 instruction；但它不是 allowlist、sandbox、origin check、approval record 或 PEP。第 2 章定义了这条边界。Provider-side prompt-injection classifier 同样只是 defense in depth；Anthropic 明确说明，即使有 classifier layer，computer-use precaution 仍然必要（Anthropic — Computer Use Tool）。

15.8 Visual Accounting 必须绑定 Provider 与 Model

“一张 screenshot”等价于“一页文字”不存在可移植换算。Image preprocessing、detail mode、resize、patch/tile rule、token multiplier 和价格都会随 provider、model、snapshot 与 API contract 变化（Anthropic — Vision；OpenAI — Images and Vision）。

下面两个带日期的例子说明了原因。它们彼此不等价：

2026 年 7 月 31 日核验的契约	输入示例	文档给出的 accounting basis
Anthropic Vision, Claude Opus 5 high-resolution tier	1000×1000 image	$\text{ceil}(\text{width}/28) \times \text{ceil}(\text{height}/28) = 1,296$ visual tokens；同一契约对 Claude 4.7 及更新 high-resolution model 列出 2,576-pixel long-edge 与 4,784-visual-token native limit（Anthropic — Vision）
OpenAI Vision, GPT-5.6 , detail: "original"	1000×1000 image	$\text{ceil}(\text{width}/32) \times \text{ceil}(\text{height}/32) = 1,024$ original patches；当前契约说明 GPT-5.6 original 不会为了 pixel-dimension 或 patch-budget limit 缩放图像（OpenAI — Images and Vision）

每一项 visual cost 或 context claim 都应记录：

provider, endpoint, model and snapshot/tool version
verification date, image detail/fidelity mode
source and model-visible dimensions, crop and resize transform
patch/tile rule and multiplier, observed input usage, price contract

应根据实测任务效果与实际 usage 优化。Crop、downscale、structured UI state、zoom 或减少 observation barrier 都可能降低 cost 和 latency，但也可能降低 legibility 或增加 race risk。根据第 5 章，把旧 screenshot 存为 artifact，在模型 context 中只保留与决策相关的 observation；但不能删除 evaluation 或 incident review 所需证据。

Round-trip latency 也依赖具体契约。Anthropic 把它的 screenshot-based computer tool 描述为较慢，因为 action 需要 screenshot round trip；而 OpenAI 当前工具可以在下一次 screenshot 前返回有序 action batch（Anthropic — Tool Combinations；OpenAI — Computer Use）。应分别测量 capture、upload、inference、approval wait、dispatch、application settle 与 verification。

15.9 Recovery 与 Stop Condition

遇到以下条件，computer-use loop 应停止或 escalation：

- 经过验证的 environment outcome 已成立；

- Policy 拒绝 next action, 或 mandatory approval 被拒绝/过期;
- 当前 origin、account、application 或 target 超出 scope;
- Modal、CAPTCHA、authentication challenge、可疑 instruction 或 sensitive-data request 需要人类处理;
- Execution 为 partial 或 unknown, 而且 reconciliation 无法确定状态;
- Observation、action、token、cost 或 wall-time budget 已耗尽;
- Loop 重复执行却没有 environment progress;
- 用户 steering 或 cancellation。

仅仅因为模型输出 prose 或不再请求工具, 就停止并报告完成, 并不能提供充分完成证据。Finalization 前, 应对任何 in-flight action 做 reconciliation, 并应用 outcome predicate。第 6 章和第 14 章定义 unknown effect、steering、cancellation 与 approval expiry; 第 13 章定义通用 loop stop hierarchy。

15.10 按最终 UI 与 Backend State 评测

Computer-use eval 应分别评价这些层次:

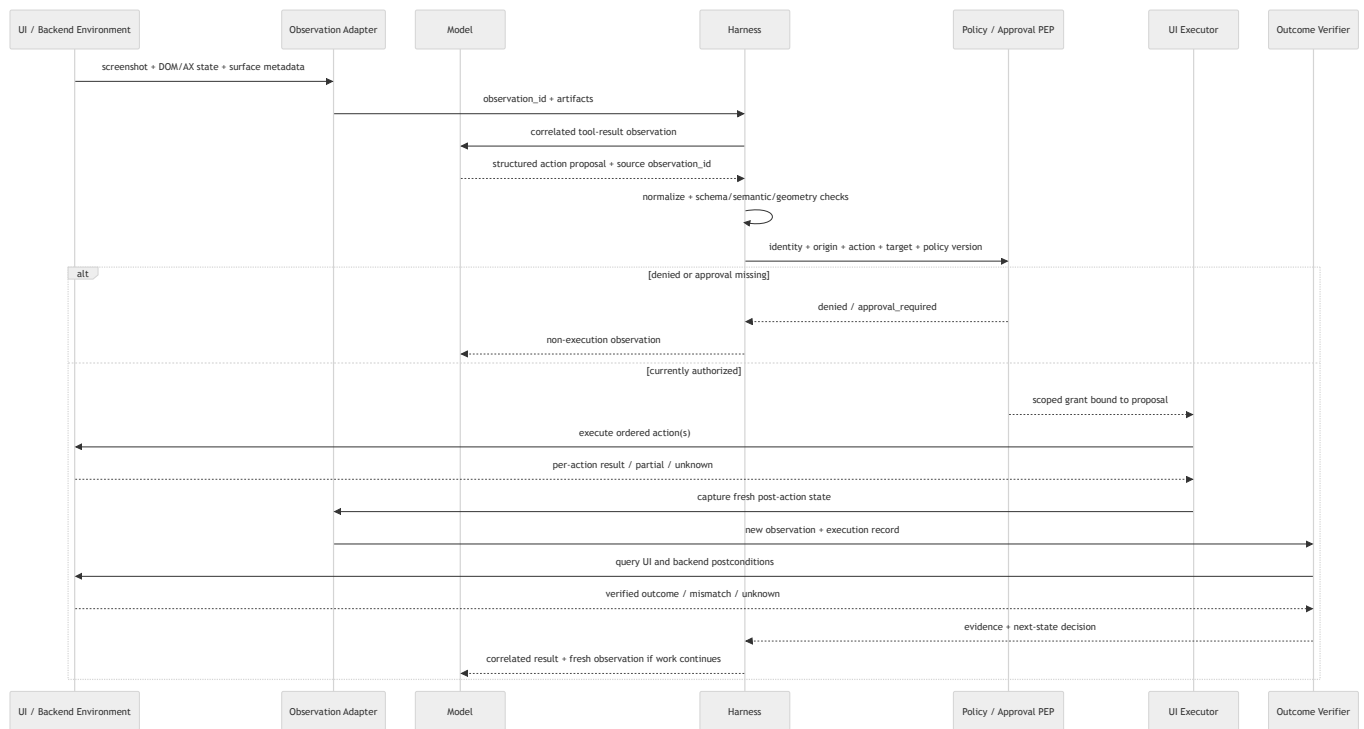
- **Process**: 允许的 origin、禁止 action、approval 使用、sensitive-data handling、budget 和 race recovery。
- **Artifact**: 下载或编辑的文件、screenshot、export report 或其他可寻址输出。
- **UI environment state**: 可见 dialog state、selected value、current page、application state 或渲染 confirmation。
- **Backend/external state**: database record、submitted form、sent message、account permission、order 或其他 durable effect。
- **User outcome**: 预期任务是否实际得到满足。

Final screenshot 是证据, 但不一定就是 outcome。Confirmation banner 可能过时或误导; backend state 也可能在没有可见 confirmation 的情况下变化。Anthropic 的 Agent eval 指南专门描述了使用 URL/page-state check 与 backend verification 评价 browser task, 并区分 transcript 与最终 environment outcome ([Anthropic — Demystifying Evals for AI Agents](#))。

WebArena 提供带 functional correctness evaluator 的 self-hosted web task; OSWorld 则提供使用 execution-based evaluation script 检查 application 和 system state 的 desktop task ([WebArena](#); [OSWorld](#))。应把它们视作 benchmark design 的一手资料, 而不是实际 workload task 的替代品。应用第 11 章要求的 isolated baseline、repeated trial、complete trajectory、configuration pinning、grader calibration 与 per-slice reporting。

报告 slice 时, 应包含 observation channel、screen resolution/detail contract、grounding method、application/origin、action type、modal/navigation presence、latency、partial/unknown execution、policy gate 与 outcome-verification method。否则 aggregate score 可能掩盖 Agent 只会处理静态页面, 或只在缺少 backend verification 时表现成功。

15.11 完整 Computer-Use Loop



Screenshot 与 structured state 是 observation payload；UI operation 是 structured proposal；应用执行它，只有 postcondition evidence 才能建立 environment outcome。

15.12 设计检查表

发布 computer-use workflow 前，确认：

- 每个 proposal 都指明它依赖的 observation 与 coordinate transform。
- Screenshot、DOM/accessibility state、grounding、proposal、execution result 和 environment outcome 分别拥有 identity。
- Executor 会在 dispatch 时验证 surface、origin、focus、scroll、overlay 和 geometry。
- Batched action 拥有兼容 precondition，并产生逐 action result。
- Partial 与 unknown execution 会停止依赖 action，并触发 reconciliation。
- 高风险 action 会经过 allowlist、sandbox、origin check、mandatory approval 和 post-action verification。
- On-screen instruction 被视作不可信内容；instruction priority 不被当作 enforcement。
- Visual accounting 记录 provider、model、version、resolution、detail mode、date、usage 和 price contract。
- Completion grader 检查最终 UI 与 backend state，而不只检查 trajectory 或 screenshot。
- Eval 会恢复 isolated baseline，并报告 race、grounding、resolution、policy 和 outcome slice。

要点

- **Computer use** 是结构化 tool loop。模型提出，应用负责验证、授权、执行和返回 observation。
- **Pixel** 与 **tool result** 不是对立项。Screenshot 通常正是相关联 tool result 的 payload。
- **Gesture** 与 **function call** 不是对立项。Click、type、scroll 和 key action 都是 provider 或 custom schema 下的结构化 proposal。
- 必须区分 **perception**、**grounding**、**execution** 与 **outcome**。它们的失败模式和证据不同。
- **Observation/action race** 是预期内问题。Staleness、layout、focus、scroll、modal、latency 和 partial execution 都需要明确状态处理。
- **安全边界包围模型**。Allowlist、sandbox、origin check、mandatory approval 和 verification 构成边界；instruction priority 只辅助模型行为。
- **Visual accounting** 绑定具体契约。绝不能把 screenshot 换算成通用文本页数。
- **Environment outcome** 决定成功。评价最终 UI 与 backend state，并对 policy-sensitive path 检查 process。

延伸阅读

- OpenAI, *Computer Use*. <https://developers.openai.com/api/docs/guides/tools-computer-use>
- Anthropic, *Computer Use Tool*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/computer-use-tool>
- Anthropic, *How Tool Use Works*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>
- Microsoft Playwright, *Auto-Waiting and Actionability*. <https://playwright.dev/docs/actionability>
- Jianwei Yang et al., *Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V*, 2023. <https://arxiv.org/abs/2310.11441>
- Anthropic, *Vision*. <https://platform.claude.com/docs/en/build-with-claude/vision>
- OpenAI, *Images and Vision*. <https://developers.openai.com/api/docs/guides/images-vision>
- Anthropic, *Tool Combinations*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-combinations>
- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Shuyan Zhou et al., *WebArena: A Realistic Web Environment for Building Autonomous Agents*, 2023. <https://arxiv.org/abs/2307.13854>
- Tianbao Xie et al., *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*, 2024. <https://arxiv.org/abs/2404.07972>

第 16 章：Agent Eval 中的基础设施噪声

第 11 章把 evaluation result 定义为关于完整、版本化 model-plus-harness configuration 的证据。本章分析其中一个很容易被隐藏的部分：运行 trial 的基础设施。对于会安装依赖、调用服务、运行测试并在多轮中改变环境状态的 agent，CPU、内存、存储、网络、并发和时间限制既会影响 run 能否完成，也会改变哪些策略可行。Anthropic 的基础设施实验具体展示了这条边界：资源预算或时间限制不同的两个 agent，参加的并不是同一场测试 ([Anthropic — Quantifying infrastructure noise in agentic coding evals](#))。

Infrastructure noise 这个词取决于目标 claim。如果 eval 声称隔离 model 或 harness change，未受控的基础设施变化就是 nuisance variable 或 confounder；如果 eval 声称测量产品在已声明 production contract 下的端到端表现，那么 rate limit、timeout 和 resource ceiling 就属于被测系统，其失败应计入 reliability。在决定固定或排除什么之前，先明确自己要提出哪一种 claim。

16.1 在运行 Trial 前定义 Estimand

Estimand 是实验计划估计的量。基础设施研究通常会问三个不同问题之一：

- 1. **固定环境下比较 model-plus-harness**：保持基础设施不变，估计系统 A 与 B 的 outcome 差异。
- 2. **基础设施敏感性**：保持 model、harness、task 与 grader 不变，估计改变一项基础设施因素后的变化。
- 3. **生产系统可靠性**：对用户实际会遇到的基础设施条件采样，估计完整系统的 outcome、latency、cost 与 failure distribution。

这些问题需要以不同方式处理 failure。意外的集群设置错误可能让 model-capability trial 无效；由已声明 product contract 施加的 timeout 则是 system reliability failure，不能作为“噪声”移除。应保留第 11 章对 task verdict、infrastructure-invalid trial 和 in-scope reliability failure 的区分。

查看分数之前，先写下目标 claim 与 treatment contrast：

Claim: 把 X 从 A 改为 B 会改变 metric Y
Population: 已声明的 task suite 与 slice
Fixed configuration: model、harness、grader 与具名 infrastructure field
Experimental unit: 一个 trial
Pairing/blocking key: task × replicate × environment block
Primary effect: outcome rate 的绝对差
Uncertainty: 已声明的 interval 或 model, 在 task/block 层聚类
Invalid-trial policy: 版本化的纳入、排除与重试规则

缺少这份 contract 时，“同一个 benchmark”可能掩盖完全不同的实验。

16.2 Anthropic 资源实验是一个具名 Case Study

Anthropic 在 Google Kubernetes Engine 集群上，以六种资源配置运行 Terminal-Bench 2.0；这些配置从把每个 task 的资源规格同时作为 allocation floor 与 hard ceiling (1×)，一直延伸到 uncapped resource。实验使用同一个 Claude model、harness 和 task set，目标 treatment 是资源配置 ([Anthropic — Quantifying infrastructure noise in agentic coding evals](#))。

报告的结果只属于这个实验设置：

比较	实验设置	报告结果
Terminal-Bench 1× → 3× ceiling	同一 model、harness、task set；在每个 task 的资源规格之上增加 headroom	Infrastructure error 从 5.8% 降至 2.1% (p < 0.001)；success score 在该实验的噪声范围内波动 (p = 0.40)
Terminal-Bench 3× → uncapped	同一个六配置实验	Infrastructure error 又下降 1.6 个百分点，success 则增加将近 4 个百分点
Terminal-Bench 1× → uncapped	同一实验的两个极端	Success 相差 6 个百分点 (p < 0.01)
SWE-bench 1× → 5× RAM	对 227 个 problem 各运行 10 个 sample 的 crossover experiment	Score 增加 1.54 个百分点

Anthropic 把 Terminal-Bench 曲线解释为两个经验区间。增加到约 3× 之前，额外 headroom 主要减少 container failure；超过这一点后，更多资源启用了大型依赖、昂贵 subprocess 或高内存测试等方案。bn-fit-modify 例子显示，

一些模型会在宽松限制下安装 Python data-science stack，却会在严格限制下于安装阶段耗尽内存；与此同时，使用标准库的解法依然存在（[Anthropic — Quantifying infrastructure noise in agentic coding evals](#)）。

这个 case 也说明为什么 allocation 与 enforcement 必须分别报告。Kubernetes 区分用于调度的 resource **request** 与 runtime 执行的 resource **limit**；CPU limit 与 memory limit 的执行行为也不相同（[Kubernetes — Resource Management for Pods and Containers](#)）。因此，“8 GB machine”这样的标签不是完整的执行 contract。

16.3 三个百分点不是通用阈值

Anthropic 的结论是：在资源方法尚未标准化时，对于所研究的 **agentic-coding** 设置，在配置得到记录并匹配之前，应谨慎看待 leaderboard 上低于 3 个百分点的差异。原文把这条建议与三项结果联系起来：Terminal-Bench 中等资源区间的 observed spread 略低于 2 个百分点，naive binomial interval 约为 1-2 个百分点，而资源分配两端的 spread 达到 6 个百分点（[Anthropic — Quantifying infrastructure noise in agentic coding evals](#)）。

不要把这条特定 case 的诊断规则转换成以下任何说法：

- 低于 3 个百分点的差异永远不真实；
- 超过 3 个百分点的差异就是显著的；
- 3 个百分点是可以接受的 regression budget；
- 每个 benchmark 都含有 3 个百分点的 infrastructure noise；
- 只要 RAM 相同，就消除了所有基础设施混淆。

通用 cutoff 会忽略 sample size、paired structure、task mix、baseline rate、variance、multiple comparison、grader error，以及实际发生变化的基础设施变量。对于新的 eval，应在自己的实验设计下估计 effect size 与 uncertainty。NIST 关于 binomial proportion interval 的指南让 interval 随 observed count 与 sample size 变化，而不是使用固定 percentage-point 规则（[NIST/SEMATECH — Confidence Intervals for a Proportion](#)）。

16.4 固定并发布 Infrastructure Manifest

为第 11 章的 EvalConfiguration 增加 infrastructure manifest。记录配置值，并在可用时记录观测值：

变量类别	至少应固定或记录的字段
Hardware 与 resource	Cloud/provider、region/zone、node 或 VM type、CPU architecture、vCPU request 与 limit、memory request 与 limit、accelerator、ephemeral-storage request 与 limit、scheduler class、autoscaling behavior
Disk 与 filesystem	Local/remote storage type、capacity、IOPS/throughput policy、filesystem 与 mount option、workspace location、起始 free space、snapshot/reset mechanism
Network	Allowed destination、DNS/proxy configuration、egress path、bandwidth/connection limit、到各 dependency 的 region、network emulation、outage 或 incident status
Container image	Image registry、repository、immutable digest、base-image digest、OS/runtime version、entrypoint、sandbox/runtime version
Dependency	Lockfile 与 package-manager version、repository/index snapshot、system package、compiler、test runner、browser/driver、installation policy
Service 与 rate limit	Model/provider endpoint 与 region、account tier、quota、每个 interval 的 request/token 限制、connection pool、retry 与 backoff policy、observed throttling
Cache state	Image、package、build、repository、DNS、retrieval 与 application cache；cold/warm treatment；priming sequence；reset 或 sharing scope
Parallelism 与 contention	Evaluation concurrency、每个 node 的 worker、agent subprocess limit、neighboring workload、task scheduling/order、queue policy
Timeout 与 clock	End-to-end、model、tool、command、install、test、network、queue、idle 与 cleanup timeout；retry budget；clock/timezone control

不要只用可变 tag 标识 image。Docker 文档说明 tag 之后可能解析到不同 image，而 digest 可以固定 image version；它还说明 build cache 与未固定 package 可能改变后续 build 实际安装的内容（[Docker — Building best practices](#)）。除 image digest 外，还应记录 dependency lock hash 与 repository snapshot，因为 trial 可能会在启动后继续安装软件。

“Warm”并不是单一状态。应明确哪个 cache 是 warm、由谁 prime、是否跨 trial 共享，以及 agent 能否观察到其他 trial 的 artifact。如果 cache state 不是 treatment，就应在 variant 之间 reset 或平衡；如果它就是 treatment，则应精确定义 cold 与 warm protocol。

16.5 尽可能按 Task 与 Environment 配对

进行 A/B comparison 时，让两个 variant 在相同 task_id 和相同 declared baseline 上运行，并将它们放入匹配的 environment block：相同 image 与 dependency、hardware/resource class、network policy、cache treatment、timeout policy、grader version，以及足够窄的运行时间窗口。每个 task 需要重复足够多的 trial 以表示模型非确定性，并在 block 内 randomize 或 counterbalance A/B 顺序。

这些 observation 是 paired data，因为每个 A result 都能与同 task、同 block 的 B counterpart 自然匹配。NIST 对 paired observation 的定义正是如此：一个 sample 中的第 *i* 个 measurement，与另一个 sample 中的第 *i* 个 measurement 自然配对，并分析 pair 内 difference ([NIST/SEMATECH — Analysis of Paired Observations](#))。NIST 的实验设计指南同样建议在 block 内保持可控 nuisance factor 不变，并对无法控制的因素进行 randomization ([NIST/SEMATECH — Randomized Block Designs](#))。

“Same environment”不等于复用已经污染的 state。每个 trial 都应从同一个 immutable 或 recoverable baseline 开始，但使用隔离的 namespace、filesystem、credential 和 external-resource scope。Matched block 应让预期条件相同，同时避免一个 trial 的 artifact 帮助另一个 trial。

如果 pairing 不可行，例如 variant 使用不兼容 hardware、provider 或 time window，应把结果报告为 unpaired comparison，并列出 confound。Region、time of day、account tier、image、concurrency 或 provider incident 都可能仍是替代解释；Anthropic 具体列出了 cluster health、hardware、concurrency、egress bandwidth 与时变 API 条件，同时注明其 time-of-day observation 尚未经过正式量化 ([Anthropic — Quantifying infrastructure noise in agentic coding evals](#))。此时，不要把 observed difference 描述为 model 或 harness 本身的 causal effect。

16.6 保存 Raw Trial Ledger

Leaderboard point estimate 不是充分证据。应为每个 trial 保存一条 immutable record：

```
InfrastructureTrial {
  task_id, trial_id, replicate_id, pair_or_block_id, variant,
  start_time, run_order, model_and_harness_version,
  task_suite_and_grader_version, infrastructure_manifest_hash,
  node_or_worker_id, image_digest, dependency_lock_hash,
  configured_resources, observed_peak_resources,
  cache_treatment, concurrency, rate_limit_events,
  phase_timings, retries, timeout_reason,
  task_verdict, validity, failure_category,
  artifact_ids, trajectory_id, environment_outcome_id
}
```

在隐私与安全控制允许的范围内，应把 raw trial row 与 aggregate 一起发布或保存。至少报告 task 数、每个 task 与 variant 的 trial 数、pass/fail/partial count、invalid trial、retry 和 missing record。Anthropic 建议运行多个 trial，因为 agent output 会变化；其指南也强调 coding eval 需要稳定环境 ([Anthropic — Demystifying evals for AI agents](#))。

使用版本化 failure taxonomy：

- model/agent substantive failure；
- CPU、memory、accelerator、process 或 disk exhaustion；
- storage 或 filesystem failure；
- network、DNS、proxy 或 egress failure；
- container-image、dependency、package-index 或 setup failure；
- provider error、rate limit 或 backoff exhaustion；
- timeout、queue delay 或 cancellation；
- concurrency contention 或 worker eviction；
- cache contamination 或 failed reset；
- harness、grader、fixture、cleanup 或 logging failure。

分类取决于 claim。已声明 production contract 内的 rate limit 是有效 reliability failure；被测系统之外的 grader outage 则可能让 trial 无效。应在应用 retry 或 exclusion policy 之前记录 reason code，避免基础设施错误在重跑中消失。

16.7 同时报告 Effect Size 与 Uncertainty

对于 binary outcome，先报告以百分点表示的绝对 pass-rate difference，并附 raw numerator 与 denominator。Relative lift 或 rate ratio 可以补充，但必须同时给出 baseline：“1.5x”在 2% 和 60% baseline 下代表完全不同的绝对变化。

对于 paired repeated trial，先计算 task-level difference，避免保留 attempt 更多的 task 静默占据更大权重：

```
d_task = mean(outcome_B for task and matched blocks)
          - mean(outcome_A for task and matched blocks)
primary effect = mean(d_task across the declared task population)
```

应选择尊重实验设计的 interval 或 statistical model：paired task bootstrap、适合 matched binary data 的 procedure，或者在 trial 嵌套于 task 与 environment block 时使用 hierarchical model。报告 sampling unit、confidence 或 credible level、method、assumption，以及对 invalid-trial handling 的 sensitivity。第 11 章解释了为什么把 correlated attempt 当作独立观测会夸大 precision。

每张结果表都应包含：

- 每个 variant 的 raw task 与 trial count；
- absolute effect size，以及有用时带 baseline 的 relative effect；
- uncertainty interval 与任何预先指定的 hypothesis-test result；
- failure-category count 与 delta；
- latency、cost 与 resource-use distribution；
- 按 reason 列出的 invalid、missing、retried 与 excluded trial；
- per-task 或 per-slice result，而不只是 pooled score；
- 完整 configuration 或其 content-addressed manifest。

p-value 不能替代 effect size 或 interval。同样，窄 interval 也不能修复未控制的 confound 或损坏的 grader。如果比较许多 infrastructure variant 或 slice，应在查看结果前声明 primary contrast，并在解释中考虑探索性 multiple comparison。

16.8 诊断 Score 为什么改变

至少区分四种机制：

1. **Reliability stabilization**：到达 grader 的无效或 in-scope infrastructure failure 变少。
2. **Strategy enablement**：Agent 现在能够安装、搜索、编译、测试或 brute-force，而严格配置之前会阻止这些方案。
3. **Latency-budget interaction**：更快 hardware、warm cache、更少 contention 或不同 rate limit，使更多工作可以在同一个 timeout 内完成。
4. **Uncontrolled confounding**：另一个 configuration 或时变条件与目标 treatment 同时发生变化。

同时比较 outcome 与 trajectory evidence：failure reason、resource peak、install 与 test 行为、subprocess count、service throttling、queue time 和 phase latency。在 Anthropic 的 Terminal-Bench case 中，1x – 3x 区间主要表现为 infrastructure error 下降；而超过 3x 后，score 增幅超过了该错误降幅，并伴随资源密集型策略 ([Anthropic – Quantifying infrastructure noise in agentic coding evals](#))。这是关于该实验的证据，而不是通用 breakpoint。

不要把每个对基础设施敏感的 success 都重新标为 score inflation。如果产品确实提供更大的资源预算，善用它可能是有效能力。报告应说明测量目标究竟是约束下的效率、预算内最大 task success，还是 production condition 下的 reliability。

16.9 运行 Sensitivity Matrix，而不是只用一种“干净”配置

固定 baseline 能支持比较，却不能展示 robustness。在 release 或 publication 前，测试重要 configuration boundary：

因素	控制比较示例	要检查的证据
Hardware/resource	Declared request/limit floor 与校准后的 ceiling	Outcome、OOM/throttling、peak use、strategy change
Disk/network	Local 与 remote disk；normal 与声明的 bandwidth/latency condition	I/O error、install/test latency、timeout interaction
Image/dependency	相同 image digest；然后进行一次计划内 dependency upgrade	Reproducibility、setup failure、behavioral delta
Rate limit	Declared quota tier 或 controlled throttling	Retry/backoff behavior、queue time、incomplete task
Warm cache	精确定义的 cold 与 warm treatment	Hit rate、latency/cost、contamination check
Parallelism	固定 per-trial limit 下的低并发与 production concurrency	Contention、tail latency、eviction、outcome
Timeout	Production timeout 与用于诊断的更长 timeout	Near-boundary completion、hung phase、strategy change

当目标是诊断时，可以一次筛查一个因素；当 interaction 重要时，使用预先指定的 factorial 或 blocked design。不要同时改变 model、harness、image、resource、concurrency 与 timeout，然后把全部 score delta 归因于单一组件。

16.10 发布可重建的结果

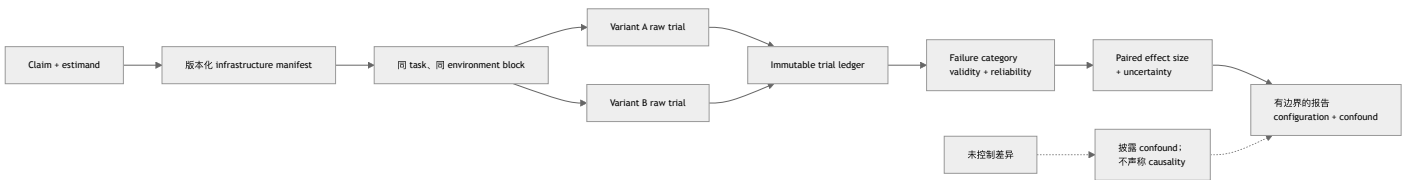
一份能够体现基础设施影响的报告应包含：

- claim、estimand、task population、slice 与 decision owner；
- model、agent harness、evaluation harness、tool、grader 与 policy version；
- experiment design、pairing/blocking、order randomization、run date 与 location；
- 完整 infrastructure manifest，包括 resource allocation 与 **enforcement**；
- raw trial ledger，或带 checksum 的 privacy-preserving derivation；
- point estimate、raw count、effect size、uncertainty 与 failure category；
- invalid-trial、retry、missing-data 和 exclusion policy，以及 sensitivity analysis；
- 具名 confound 与 causal/cross-lab comparison 的边界；
- release decision、exception owner，以及 monitoring 或 rerun trigger。

OpenAI 的 evaluation playbook 同样要求报告披露 tested system、harness 与 tool access、elicitation method、budget、safeguard 和 validity check，因为这些内容决定了结果能够支持什么 claim（[OpenAI — A shared playbook for trustworthy third-party evaluations](#)）。

跨实验室复现不要求 vendor 名字完全相同，但要求实验 contract 等价，或者明确说明差异。如果无法建立等价性，应分别发布结果，而不是通过 normalization 隐藏差别。当其他 reviewer 能够重建运行内容、分类 failure，并理解哪些 uncertainty 来自 sampling、哪些来自 configuration 时，benchmark score 才会成为有用证据。

图示：从基础设施变量到有边界的 Claim



本章要点

- **Infrastructure noise** 取决于 **claim**。Model comparison 中的 nuisance variable，在 product eval 中可能是 in-scope reliability condition。
- 三个百分点建议是具名 **Anthropic case result**。它不是通用 significance threshold、regression budget 或所有 benchmark 的噪声估计。
- 发布完整 **execution contract**。Hardware/resource、disk/network、image 与 dependency、rate limit、cache state、parallelism 和 timeout 都属于 manifest。

- 尽可能按 **task** 与 **environment** 配对。使用 matched baseline 与 block，再分析 pair 内 difference；否则应披露 unpaired confound。
- 保存 **raw trial** 与 **failure category**。在重试或排除前区分 substantive failure、in-scope system reliability failure 与 infrastructure-invalid trial。
- **Effect size** 必须与 **uncertainty** 一起报告。附 raw count、sampling unit、per-slice effect、interval、invalid-trial sensitivity、latency、cost 与 resource use，而不是只给 leaderboard point estimate。
- 所有 **multiplier** 与 **score shift** 都必须附带实验设置。3×、6-point 与 1.54-point 结果描述的是 Anthropic 的特定实验，不是可移植常数。

延伸阅读

- Gian Segato, *Quantifying infrastructure noise in agentic coding evals*, Anthropic。
<https://www.anthropic.com/engineering/infrastructure-noise>
- Mikaela Grace 等, *Demystifying evals for AI agents*, Anthropic。
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- NIST/SEMATECH, *Randomized Block Designs*。 <https://www.itl.nist.gov/div898/handbook/pri/section3/pri332.htm>
- NIST/SEMATECH, *Analysis of Paired Observations*。
<https://www.itl.nist.gov/div898/handbook/prc/section3/prc311.htm>
- NIST/SEMATECH, *Confidence Intervals for a Proportion*。
<https://itl.nist.gov/div898/handbook/prc/section2/prc241.htm>
- Kubernetes, *Resource Management for Pods and Containers*。
<https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>
- Docker, *Building best practices*。 <https://docs.docker.com/build/building/best-practices/>
- OpenAI, *A shared playbook for trustworthy third-party evaluations*。 <https://openai.com/index/trustworthy-third-party-evaluations-foundations/>

第 17 章：基于 Trace 的迭代

Production trace 可以展示一次被观测执行路径中的耗时分布、组件交互与错误，因此很适合用于调试和发现 regression case 候选项。但这不会让 trace 自动成为其他所有用途的 system of record。

第 10 章把 **trace** 定义为由 span、timestamp、attribute、event、link 与 status 组成的 observability data。OpenTelemetry 通过 tracing API 暴露这些对象，并允许实现对记录或导出的内容进行 sampling ([OpenTelemetry — Trace API](#)；[OpenTelemetry — Trace SDK](#))。本章讨论如何安全使用这种易错且可能不完整的信号，改进经过版本管理的 model-plus-harness system。

17.1 Trace 是一种证据产品，而不是四种

以下产品必须保持分离：

产品	必需属性	主要用途	Sampled trace 为什么不能取代它
Trace	在声明的 sampling policy 下，可查询的 span 与 causal/correlation metadata	Observability、performance diagnosis、failure discovery	Sampling 可能丢弃完整执行或单项细节
Event history	在 recovery/replay contract 下，有序且持久的 accepted event	重建 execution state 并恢复运行	Trace order 与 retention 不是权威 workflow state
Eval trajectory	一次 trial 中，declared grader 所需的全部模型、工具、observation、artifact 与 policy event	评测 process behavior	Production instrumentation 可能省略或脱敏 grader 所需证据
Audit record	明确定义的 content、identity、timestamp、completeness、integrity、access、protection 与 retention	Accountability、investigation、compliance	Trace 可能可修改、被采样、使用不同 access scope，或保留时间太短

Temporal event history 是用于恢复的 append-only workflow record，而 OpenTelemetry sampling 可以有意丢弃 attribute、event、status 或完整 span ([Temporal — Events and Event History](#)；[OpenTelemetry — Trace SDK](#))。NIST 把 audit generation、content、protection、review、access 与 retention 分别视为显式 control，而不是任意 log 自动继承的属性 ([NIST SP 800-53 Rev. 5](#))。第 11 章同样要求 eval trajectory 对 grader 消费的证据保持完整。

这些产品应该互相链接，却不能混为一谈。Span 可以携带 event-history ID；eval trajectory 可以由 unsampled instrumentation 与 artifact 构建；audit pipeline 可以复制选定且受保护的 record。正因为各自的保证仍然明确，这种 lineage 才有价值。

17.2 关联持久 ID 与 Trace/Span ID

第 10 章定义了能够跨越 observability backend 生命周期的 application identity：

```
session_id -> run_id -> step_id -> action_id -> attempt_id
call_id 挂在发起 protocol exchange 的 step 或 attempt 上
```

Trace identity 回答的是不同问题。W3C Trace Context 标准化了跨 service boundary 传播 trace-id、parent/span identity 与 trace flag 的方式 ([W3C — Trace Context](#))。不能用这些面向传输的 ID 取代 durable workflow identity。

持久 Identity	Trace 关联方式	重要区别
<code>session_id</code>	作为相关 root span 与 child span 的 attribute	一个 session 可以包含多个 run 和 trace
<code>run_id</code>	作为该 run 每个 span 的 attribute；常与一个 root trace 或相互链接的多个 trace 关联	Long-running run 可能比 trace retention 更久，也可能跨越 trace boundary
<code>step_id</code>	作为表示该 logical transition 的 orchestration span attribute	在显式 loop semantics 下重复某一步，不会使 <code>span_id</code> 成为 workflow position 的来源
<code>action_id</code>	作为 intended-effect span 与所有 delivery attempt 的 attribute	Retry 之间保持稳定；它不是 tool-call transport ID
<code>attempt_id</code>	作为一次 model/tool execution-attempt span 的 attribute	同一 logical step 或 action 下每次 attempt 都会变化
<code>call_id</code>	作为 provider/protocol model 或 tool span 及关联 result 的 attribute	标识一次 exchange；不能为 external effect 提供 idempotency
<code>trace_id / span_id</code>	原生 observability correlation	可能被采样、在边界重新生成，或根本没有进入 durable recovery data

同步 causal nesting 使用 parent-child relation。对于从 queue 接回的工作、fan-out/fan-in branch、与早期 attempt 相关的 retry，或者在原 trace 结束后启动的 operation，应使用 span link；OpenTelemetry 允许 link 指向同一 trace 或另一 trace 中的 span（[OpenTelemetry — Trace API](#)）。Durable event envelope 也应保留 `causation_id` 和 `correlation_id`，因为 trace backend 不是 accepted state 的权威来源。

17.3 围绕问题设计 Span，而不是追求最大 Payload

一个 span 表示一项 operation。OpenTelemetry 允许 instrumentation 设置 attribute、添加带 timestamp 的 event、设置 status、记录 exception，以及链接相关 span（[OpenTelemetry — Trace API](#)；[OpenTelemetry — Exceptions](#)）。每种字段都应有意使用：

- **Attribute** 保存稳定且可查询的事实：durable ID、operation type、component 与 configuration version、model/tool name 与 schema version、tenant-safe classification、token/cost counter、retry number、policy outcome 和 artifact reference。不要把 raw secret、完整 prompt、高 cardinality payload 或可变 prose 当作 primary key。
- **Event** 记录 operation 内某个时点发生的事实：retry scheduled、approval requested、first byte received、timeout observed、cancellation requested、result normalized 或 postcondition checked。Span event 仍是 observability data，不会自动成为 accepted workflow event。
- **Status** 汇总 operation 的 tracing status。OpenTelemetry 定义 `Unset`、`Ok` 与 `Error`，但不会编码 application 的完整 state machine（[OpenTelemetry — Trace API](#)）。`partial`、`cancelled`、`unknown outcome`、`policy denial` 与 `business success` 应继续作为明确 domain attribute/event 和 durable state。
- **Error** 应使用稳定 category 与安全 description，并在需要时附上完整诊断的引用。OpenTelemetry exception convention 会在未处理异常导致 span 以 error 结束时，把 exception 记录为 event（[OpenTelemetry — Exceptions](#)）。如果 exception 被捕获且 recovery 成功，就不应在没有解释的情况下让整个 run 看起来已经失败。
- **Link** 表示非树状关系，而不是把每个异步 dependency 强行塞进错误的 parent-child chain。

OpenTelemetry GenAI semantic convention 定义了新兴的 model、agent 与 tool span 命名，但这些 convention 带有版本，而且其中一部分可能仍标记为 Development（[OpenTelemetry — GenAI Spans](#)）。应固定 convention version、记录 local extension，并在 adapter boundary 映射 provider field。不要把不断变化的 semantic-convention draft 写进 durable business state。

一份最小 span envelope 可以是：

```
trace_id, span_id, parent_span_id, links
session_id, run_id, step_id, action_id, attempt_id, call_id
operation, component_version, model_or_tool_version
started_at, ended_at, status, error_type
policy_decision, token_and_cost_counters, artifact_refs
sampling_policy_version, content_capture_mode
```

17.4 Sampling Policy 是解释的一部分

OpenTelemetry sampling 的作用是降低采集开销与数据量；sampler 可以返回 `DROP`、`RECORD_ONLY` 或 `RECORD_AND_SAMPLE`，而未记录的 span 会丢弃自身 attribute、event 与 status（[OpenTelemetry — Trace SDK](#)）。因

此，除非 estimator 与 inclusion probability 足以支持范围更广的结论，否则根据 exported trace 生成的 dashboard 回答的只是 sampled population 的问题。

应定义 sampling policy 并进行版本管理：

- 普通成功 run 的 baseline probability 或 rule；
- 对 error、policy denial、approval flow、unknown side effect、新 version 与 canary traffic 提高 capture priority；
- head sampling 与 tail sampling 的行为，以及 downstream service 意见不一致时怎样处理；
- content-capture mode、redaction point、storage location、retention 与 access class；
- 必须保持完整计数的 metric 或 ledger 怎样在 sampled trace export 之外生成；
- collector、exporter 或 instrumentation 失效时，operator 如何发现 blind spot。

Priority sampling 会提高诊断覆盖率，却不会自动产生具有统计代表性的 dataset。如果 failure trace 被过度采样，其原始占比就不是 production failure rate。如果 successful run 很少被采样，没有观察到某种失败模式也只是很弱的证据。Billing、hard policy event 与 durable recovery 应使用 unsampled counter 或其他完整 operation record，而不能要求 trace 提供 sampling policy 已经删除的保证。

17.5 不应混淆的三种活动

活动	问题	证据	输出
Trace grader	完整 evaluation trajectory 是否遵循声明的 process rule？	Unsampled trial trajectory、event、tool argument、approval、budget、artifact	一次 trial 的 versioned grader verdict
Outcome grader	Artifact、environment 或 user/business state 是否满足成功标准？	Test、state query、独立 API check、artifact inspection、user/business event	Versioned outcome verdict 或 score
Incident investigation	生产环境中发生了什么、影响了什么、原因是什么，以及系统应怎样 contain 和 recover？	Trace 加 event history、log、audit record、configuration、external-system state、human report	Timeline、impact、containment/recovery action、contributing cause、follow-up

Anthropic 的 agent-eval 指南区分 transcript/trajectory evidence 与 outcome，并警告不能只评判 agent 说了什么 (Anthropic — Demystifying Evals for AI Agents)。NIST incident-response 指南把 preparation、detection、response 与 recovery 视为组织风险管理活动，而不是 grader 的 pass/fail decision (NIST SP 800-61 Rev. 3)。

Production trace analyzer 可能标记“缺少 approval span”。如果相关 record 并不保证完整，这只是一条等待 investigation 的 hypothesis。只有当 evaluation harness 保证捕获所有必需 approval evidence 时，eval 中的 trace grader 才能因此判定 trial 失败。Outcome grader 可以判定最终 artifact 通过，而 process grader 同时判定存在 policy violation；两种 verdict 都不能取代对真实 production event 的 incident triage。

17.6 把生产故障转化为受治理的 Regression Case

Production failure 是 eval task 候选项的重要来源，但把 trace 直接复制进 suite 会带来 privacy、rights、duplication 与 validity risk。应使用受治理的 intake pipeline：

1. 先处理 live event。保留易失证据，通过 incident process 完成 containment、notification 与 recovery；不能为了设计 eval 而延迟响应。
2. 确认使用权。在把 production material 改作其他用途之前，应根据需要检查 user notice 或 consent、contract term、data-processing purpose、third-party content license 与 intellectual-property restriction。NIST AI RMF core 要求在 AI-system risk work 中对 privacy 和 third-party data/software rights 进行有文档记录的管理 (NIST — AI RMF Core)。
3. 最小化并脱敏。删除 secret、credential、personal/tenant data、不必要 payload 与 production identifier；当 investigation 或 legal obligation 需要时，把受保护证据另行保存。
4. 重建完整证据。把 sampled trace 与获准使用的 event-history record、artifact、configuration version 和 external outcome 连接起来。明确标记缺失事实；不能根据 partial trace 虚构 complete trajectory。
5. 创建隔离 fixture。用可复现 environment、synthetic 或 licensed data、scoped identity 与 deterministic reset/teardown 取代 live side effect。
6. 去重与聚类。多个 alert 可能来自同一个底层 failure。保留到每个 source incident 的 lineage，同时避免重复 copy 悄悄增加一个类别的权重。
7. 检查 distribution shift。判断该案例应进入 representative task distribution、historical-regression slice、high-risk policy suite，还是只用于 incident testing。不能在没有 versioning 与 disclosure 的情况下改变 headline

distribution。

8. 分别编写 **process** 与 **outcome assertion**。明确哪些 trace evidence 必须完整，以及哪种 independent environment result 能够证明成功。
9. 审定并版本化。在接纳案例之前，review task fairness、solvability、leakage、rights、grader behavior 与 expected failure。

OpenAI evaluation playbook 把 contamination、broken task、reward hacking、refusal 与 sandbagging 列为 validity hazard，并建议报告 tested system、harness、budget 与 validity check ([OpenAI — Trustworthy Third-Party Evaluations](#))。逃逸到生产环境的 failure 可以增强 regression suite，但一份不断增长且只包含失败的 archive 并不代表 ordinary traffic。第 11 章的 suite versioning、slice、isolation 与 grader calibration 仍然适用。

17.7 Trace Review 产生假设，而不是自动修复

Trace review 应先对可能负责的 boundary 进行分类，再提出修改：

症状	等待验证的候选原因	需要收集的证据
工具选择错误或 call 格式错误	Tool description/schema、context selection、model capability、router choice	Proposal、catalog/version、validation error、相关 context、comparison trial
循环反复或过早停止	Stop state、error normalization、context compaction、no-progress detector、model behavior	Step/action ID、error class、state transition、budget、final outcome
Transcript 正确，external state 错误	Tool timeout、duplicate/partial effect、stale observation、缺少 postcondition check	Action/attempt ID、operation ID、event history、environment query
Policy 或 approval failure	PEP 缺失、policy 过时、identity/delegation mismatch、UI 或 event bug	Policy input/version、decision、approval proposal hash、dispatch evidence
Latency 或 cost regression	Model/routing change、cache miss、retrieval/tool latency、retry fan-out、infrastructure change	Child span、sampling-adjusted metric、configuration diff、controlled eval

模型文本只是一个 component，不是默认 root cause。修复方式可能是修改 prompt，也可能是 typed tool、state transition、sandbox rule、policy gate、context transformation、infrastructure fix 或 grader change。在编辑 harness 之前，应先记录 hypothesis 与能够证伪它的 test；否则团队可能会对一条令人印象深刻的 trace 过拟合，却没有修复 failure class。

17.8 Meta-Harness 修改需要发布 Pipeline

所谓“self-improving agent”应指 production evidence 进入外部且受治理的 improvement process，而不是 deployed agent 无边界地改写自己当前生效的 prompt、tool、memory policy、runtime 或 grader。OpenAI 的 tax-agent case study 描述了 expert correction 怎样变成经过 review 的 finding、targeted eval 与 bounded coding task，而不是直接重写 deployed agent ([OpenAI — Building Self-Improving Tax Agents with Codex](#))。

Human-authored change 与 meta-harness-generated change 都应采用相同 release discipline：

1. 限定 **proposal**。写明 failure class、component、owner、预期 effect，以及允许修改的 file/configuration。
2. 创建 **version**。固定受影响的 model、prompt、tool schema/implementation、retrieval configuration、memory policy、sandbox image、runtime、policy、budget 与 grader。
3. 运行 **offline eval**。要求通过 targeted regression、更广泛的 holdout suite、critical policy slice、grader calibration check，以及 cost/latency/reliability comparison。
4. 获得 **approval**。责任 owner review change、evidence、known limitation、deployment scope、canary plan 与 rollback trigger。Candidate 不能批准自身。
5. 安全执行 **canary**。从有边界的 traffic slice、有限 authority、适当隔离的 tenant 或 task，以及更强 outcome monitoring 开始。
6. **Promote 或 rollback**。使用预先声明的 criteria，保留 decision 与 lineage，并在 trigger 生效时恢复最近的 compatible version。

Production traffic 可以产生 trace、finding、candidate test 或 change proposal，却不能授予某个 process 无边界修改 active production configuration，并立即在同一 traffic 上评价自身的权限。Proposal、evaluation、approval、deployment 与 outcome monitoring 应使用相互分离的 identity；还要防止 candidate 编辑自己的 holdout task 或 release grader。

17.9 Model-Harness 依赖带来测试责任

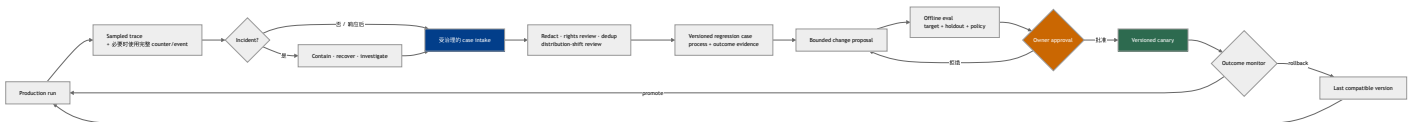
Performance 属于接受测试的 **model-plus-harness configuration**，而不只属于其中一个 component。OpenAI evaluation guidance 明确要求报告公开 harness choice，因为 tool、budget、safeguard 与 elicitation setup 都可能改变 score 所支持的结论（[OpenAI — Trustworthy Third-Party Evaluations](#)）。

这是 dependency claim，而不是预测 model 与 harness 必然沿一个方向 “co-evolve”。新模型可能使 planner 变得多余，也可能需要不同 tool interface，或暴露新的 failure。Harness change 可能改善一个模型，却使另一个模型 regression。Anthropic 描述了一种 ablation-style practice：移除或改变一个 component，再运行 realistic eval，判断它对当前 task 与 model 是否仍然物有所值（[Anthropic — Harness Design for Long-Running Application Development](#)）。

测试责任很具体：

- 每条 trace、eval result 与 release packet 都要把 model 和 harness 一起版本化；
- 任意一方变化后，都要重新运行相关 suite，包括 routing、fallback、tool、context、policy 与 infrastructure；
- 使用 ablation 与 controlled comparison，而不能假设某个 component 永远 load-bearing；
- 报告出现 coupling 的 task slice，不能泛化一项 benchmark result；
- 对使用早期 configuration 创建的 durable run 保持 rollback compatibility 与 migration。

17.10 受治理的迭代循环



要点

- **Trace 是 sampled observability data**：它不能自动充当完整 event history、eval trajectory 或 audit record。
- **关联 ID，而不是混淆 ID**：Session/run/step/action/attempt/call identity 保持持久；trace/span ID 用于 observability propagation 与 correlation。
- **使用完整 span model**：Attribute、event、status、error 与 link 回答不同的诊断问题。
- **版本化 sampling policy**：Priority capture 会改变 observed distribution，也不能取代完整 counter、policy event 或 recovery data。
- **区分三种活动**：Trace grading、outcome grading 与 incident investigation 具有不同的问题、证据和输出。
- **治理 production-to-eval intake**：Review redaction、consent 与 licensing、deduplication、distribution shift、task validity 与 grader evidence。
- **把 trace finding 当作 hypothesis**：修改前应验证负责的 model、context、tool、runtime、policy、infrastructure 或 grader boundary。
- **不能原地自我修改**：Meta-harness proposal 必须经过 offline eval、独立 approval、versioning、canary deployment、monitoring 与 rollback。
- **Model-harness 依赖会产生 regression responsibility**：它是 configuration 的实证属性，而不是必然的历史趋势。

延伸阅读

- OpenTelemetry, *Trace API*. <https://opentelemetry.io/docs/specs/otel/trace/api/>
- OpenTelemetry, *Trace SDK*. <https://opentelemetry.io/docs/specs/otel/trace/sdk/>
- OpenTelemetry, *Exceptions*. <https://opentelemetry.io/docs/specs/otel/trace/exceptions/>
- OpenTelemetry, *Semantic Conventions for Generative AI Spans*. <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>
- W3C, *Trace Context*. <https://www.w3.org/TR/trace-context/>
- Temporal, *Events and Event History*. <https://docs.temporal.io/workflow-execution/event>
- NIST, *SP 800-53 Rev. 5*. <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
- NIST, *SP 800-61 Rev. 3*. <https://csrc.nist.gov/pubs/sp/800/61/r3/final>
- NIST, *AI RMF Core*. <https://airc.nist.gov/airmf-resources/airmf/5-sec-core/>
- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>

- OpenAI, *A Shared Playbook for Trustworthy Third-Party Evaluations*, May 2026. <https://openai.com/index/trustworthy-third-party-evaluations-foundations/>
- OpenAI, *Building Self-Improving Tax Agents with Codex*, 2026. <https://openai.com/index/building-self-improving-tax-agents-with-codex/>
- Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>

第 18 章：AgentOps——成本、隐私与生产运维

前面的章节定义了 model-plus-harness 系统如何执行、恢复并产出评估证据。生产环境增加了另一类责任：真实 run 执行期间，运维人员必须让这套确切配置始终处于成本、隐私、可靠性和发布边界内。

本书使用 **AgentOps** 作为这套运维纪律的本地统称。它不是某个强制产品的名称，也不表示所有 vendor 都采用同一分类。第 10 章区分了执行状态、event history、trace 和 audit record；第 11 章定义了发布证据；第 17 章说明 sampled trace 如何辅助诊断，却不会因此变成 system of record。本章继续保持这些边界。

18.1 成本需要完整账本，而不只是 trace

****成本账本（cost ledger）****是在明确会计范围内，完整且去重的计费事件与可对外支出事件集合。Trace 可以链接这些事件并帮助诊断，但它不会自动成为账本，也不是 audit record。

OpenTelemetry sampling 可以决定不记录 span、不导出已记录的 span，或丢弃其 attribute、event 与 status；collection limit 也可能丢弃 span 字段（[OpenTelemetry — Trace SDK](#)）。因此：

- sampled trace 不能证明支出完整；
- 会丢失或延迟 export 的 trace backend 不能独自完成账期结算；
- 只有**完整且未丢失的成本事件或 cost span**才能进入成本账本；
- 如果使用 cost span，其采集路径在对应会计范围内必须不采样、能检测丢失、支持幂等摄取并经过对账；
- trace 应携带 cost_event_id 或 provider call reference，使运维人员可以把诊断 span 与账本条目关联起来，而不混同两者保证。

一条实用的成本事件至少包含：

```
cost_event_id, session_id, run_id, step_id, action_id, attempt_id, call_id
tenant_id, provider_call_id, model_or_tool_version, billing_item
input_units, output_units, cached_units, reasoning_units
price_catalog_version, amount, currency, external_spend
retry_of, fallback_from, status, occurred_at, recorded_at
```

amount 必须基于固定的价格/目录版本计算，或复制自权威 provider record。迟到事件、重复投递、credit 和价格修正需要显式 adjustment entry，不能通过覆盖历史来处理。把账本总额与 provider invoice 及工具/vendor receipt 对账，并针对缺失 call ID 或无法解释的差额告警。

成本报告还应说明分析单位和统计总体：每次模型调用、工具调用、run、成功 run、tenant、release version 或会计窗口。如果排除被放弃和重试的 run，较低的单次请求成本可能掩盖昂贵的失败。

18.2 在使用处定义延迟分位数

本章针对明确的统计总体和时间窗口采用以下本地定义：

- **p50** 是观测延迟的中位数；
- **p95** 是使至少 95% 观测值不超过它的最小报告阈值；
- **p99** 是对应的 99% 阈值。

每张图表或 SLO 都必须说明测量对象：端到端 run 延迟、模型调用延迟、工具延迟、排队时间、等待批准时间或其他区间；还必须注明时间窗口、release/configuration、样本数、timeout 处理方式，以及是否包含取消或失败的 run。不要对各自独立计算的分位数再求平均并声称得到了合并分布；应聚合兼容的原始观测，或合并适当的 histogram/sketch。

延迟应与 outcome 和成本一起报告。如果一次 run 更快，却更常超时或跳过必要验证，它就不是运维上的改进。

18.3 四种缓存，四套正确性契约

“缓存”并不是一种机制。本书沿用第 3 章引入的规范区分：

缓存	复用边界	复用内容	正确性条件
单次请求 KV 缓存（per-request KV cache）	单次 generation/request 内	已处理前缀对应的模型内部 key/value state	同一 live generation，且 model/runtime state 兼容
Provider prompt cache	不同 provider request 之间	Provider 管理的精确可复用 prompt 前缀计算	满足 provider 特定的精确前缀、模型、保留与资格规则
Application response cache	不同 application request 之间	精确规范化请求的一份已完成响应	请求精确相同，且 authorization、tenant、version、freshness 与 policy 兼容
Semantic cache	不同 application request 之间	含义相似请求的一份已完成响应	similarity 以及 authorization、tenant、version、freshness、provenance 与 risk gate 全部通过

Provider prompt cache 不复用最终响应；provider 仍然为当前请求生成输出。作为带日期的产品案例而非可移植契约，OpenAI 文档在 **2026-07-31** 核验时描述了精确 prompt 前缀命中以及 provider 特定的资格和保留行为（[OpenAI — Prompt caching](#)）。另一个带日期的产品案例是 Microsoft Azure API Management 文档；在 **2026-07-31** 核验时，它描述了可为含义相似 prompt 返回已存响应的 semantic caching，并提供 similarity threshold（[Microsoft — Semantic caching in Azure API Management](#)）。产品行为可能变化；这些案例不会重新定义上述四类缓存。

语义相似绝不是 authorization 或事实等价的证明。Semantic cache 的 key 与准入/查询 gate 必须包含或绑定以下全部信息：

tenant_id
user_id and authorization_scope
model_id and model version
normalized prompt/request and prompt-template version
tool schemas, tool implementations, and relevant tool-result versions
retrieval index, corpus snapshot, filters, and retrieval configuration
policy and safety-rule version
TTL / freshness deadline
invalidation dependencies and invalidation epoch
response provenance: source run, response, citations/artifacts, and creation time

每次 lookup 都必须重新检查当前 caller 的 authorization 和当前 policy。Tenant isolation 是强制要求；共享 embedding index 并不能让跨 tenant 响应复用变得安全。Invalidation 必须覆盖源文档变化、权限变化、删除请求、model/prompt/tool/retrieval/policy release，以及被发现的错误响应。TTL 是 freshness 的最长边界，不能替代 invalidation。

对于敏感、个性化或高影响决策，默认禁用语义响应复用。只有在测试衡量了 false hit、stale answer、跨 scope 拒绝、provenance 展示、invalidation 传播和安全 miss/fallback 后，才能启用具体 use case。缓存命中与未命中仍应产出完整成本事件和 policy 相关记录。

18.4 Run budget 是一个向量

单一 token cap 不是生产预算。每个 run 都需要一套版本化 budget vector，至少包括：

维度	计数示例	强制执行问题
Token	input、output、reasoning、cached read/write unit	下一次调用能否开始，并仍为安全收尾保留预算？
Dollar	模型、检索、存储、grader 与工具费用	计划动作是否会超过 run 或 tenant 的金额上限？
工具调用	尝试过的调用，包括失败调用和分支	loop 或 fan-out 是否达到调用上限？
Wall time	排队、模型、工具、批准、重试和清理时间	剩余 deadline 是否足以完成或安全停止？
External spend	采购、转账、付费 API 动作、云资源 provision	该 side effect 是否在金额和目的地限制内获得单独授权？
Retry/fallback	retry、hedge、模型 fallback、恢复尝试	是否允许再次尝试，且其 side-effect outcome 是否已知？

Runtime 而不是模型文本负责维护 counter，并拒绝会越过 hard boundary 的工作。模型可以收到剩余预算状态用于规划，但不能给自己增加预算。Soft threshold 可以触发 compaction、更便宜的 routing、减少 fan-out 或升级给人；hard threshold 则转入预先声明的终态或暂停态。

为验证、durable state 写入、清理和可用 handoff 保留预算。Retry 和 fallback 既计入各自上限，也计入其消耗的每种资源。早期 side effect 为 unknown 时，不要把 retry budget 用在盲目重复上；应按第 10 章的恢复状态机解决或升级。用

于强制 hard ceiling 的预算事件必须是完整 durable state，不能从 sampled trace 重建。

18.5 隐私是一条端到端数据生命周期

NIST Privacy Framework 是通过 enterprise risk management 识别和管理隐私风险的自愿工具，不是针对某一产品的 telemetry checklist (NIST — Privacy Framework)。对于 agent harness，应把这种风险纪律转化为覆盖整条数据路径的控制：

生命周期位置	必需的 harness 控制	需要保留的证据
收集与 context	最小化字段和文档；优先 handle 或 summary；在模型/工具使用前分类敏感度	purpose、data class、选定字段、source，以及必要时的法律/组织依据
Redaction	在 prompt、tool argument、trace、eval fixture 或 support export 前移除 secret 和非必要个人数据；确需完整数据时保留 access-controlled reference	redaction policy/version、detector result、exception/approval
Retrieval	搜索前执行 tenant 与 user authorization，materialization 前再次执行；把 purpose 与 scope 带入 filter	principal、tenant、query/filter、corpus/index version、allow/deny decision
Trace capture 与 storage	分别声明 sampling 和 content capture；默认只收 metadata；加密、访问控制并隔离 trace store；绝不把 sampling 当作 redaction	capture mode、sampling policy、destination、access log、retention class
Retention	按 data class 与 evidence purpose 指定 TTL/retention；有意地过期 cache、trace、memory、artifact 与派生 eval data	retention decision、expiry，以及适用时的 legal/incident hold
Egress	Allowlist provider/tool destination；必要时限制 region/account；最小化或脱敏 payload；敏感传输需要批准	destination、purpose、发送字段、credential audience、approval/policy decision
Deletion	按 policy 把删除或撤销传播到 application cache、semantic index、retrieval store、memory、trace、artifact 与 downstream processor	tombstone/request ID、受影响对象、完成状态或有记录的 exception

Sampling 会减少 telemetry 体量，但不会让保留内容失去敏感性。Redaction 会减少暴露内容，但不能替代 retrieval ACL 或 egress policy。除非 downstream copy 和 derived artifact 也在处理范围内并记录了完成状态，否则 retention 到期并不能证明删除完成。

18.6 分开监控五类运维问题

不要把所有信号压缩成一个“agent health”分数。应分开问题、数据完整性要求和 owner：

信号类别	它回答什么问题	信号示例
Service health	服务系统是否可用且未超过资源限制？	availability、request/error rate、queue depth、saturation、model/tool latency、p50/p95/p99
Agent outcome	Run 是否取得经过独立验证的任务结果？	outcome-grader pass、environment-state success、incomplete/unknown outcome、recovery success
Safety/policy event	受保护规则是否触发或失效？	denial、approval、sandbox violation、cross-tenant attempt、sensitive egress、policy-version mismatch
Cost	产生了多少完整支出，预算消耗速度多快？	每 run/outcome/tenant/release 的 ledger amount、缺失成本事件、invoice variance、budget exhaustion
Human escalation	人能否及时审查并解决工作？	queue age、approval expiry、未解决的 unknown side effect、override rate、handoff completion

Service uptime 不能证明任务成功；agent 声称完成不能证明外部 outcome；sampled trace count 不能证明总成本或 policy event frequency。每个 SLO 都必须注明权威 source 和完整性假设。Dashboard 可以按 run_id 和 release version 关联五类信号，但 alert 应路由给能够采取行动的 owner。

只有在第 17 章的 intake process 处理 redaction、consent/licensing、deduplication、distribution shift 与 evidence reconstruction 后，生产失败才能成为第 11 章的 regression candidate。Incident investigation 仍需把 trace 与 durable history、ledger record、audit record、configuration 和 external-system state 结合起来。

18.7 发布完整的 model-harness configuration

发布单位是一套版本化配置，而不是“prompt”或“model”。Manifest 至少必须固定：

model/provider and inference settings
prompt/instruction templates and context policy
tool schemas and tool implementations
retrieval index/corpus snapshot and retrieval configuration
memory schema and memory policy
sandbox image, capabilities, network policy, and resource limits
policy/approval rules and runtime/router versions
grader versions and evaluation contract
run budgets, retry/fallback policy, and cache policy

上述任一组件变化都会创建新的 release candidate，并重新运行受影响的 eval slice。OpenAI 的评估指南强调报告确切的 tested system、harness、tool、budget、safeguard 和 validity check，因为这些选择决定一项评估主张能支持什么 ([OpenAI — Trustworthy Third-Party Evaluations](#))。

继续使用第 11 章的 **release evidence packet**：claim 与 scope、immutable configuration、task-suite 与 slice version、isolated trial trajectory、grader calibration、outcome/process/safety result、uncertainty、成本与延迟、known limitation、exception owner/expiry、approval、canary plan、post-release monitor 与 rollback trigger。只有这份 packet 能支持声明的 claim，release gate 才能通过。

把获批单位部署为 canary，并把每个 run 与 ledger entry 关联到 release version；比较 outcome、safety、cost、service health 与 escalation 信号。Rollback 必须恢复兼容的完整配置、保留 durable-run migration rule，并能在声明的 trigger 触发时执行。Candidate 不能批准自己，也不能编辑 holdout grader。

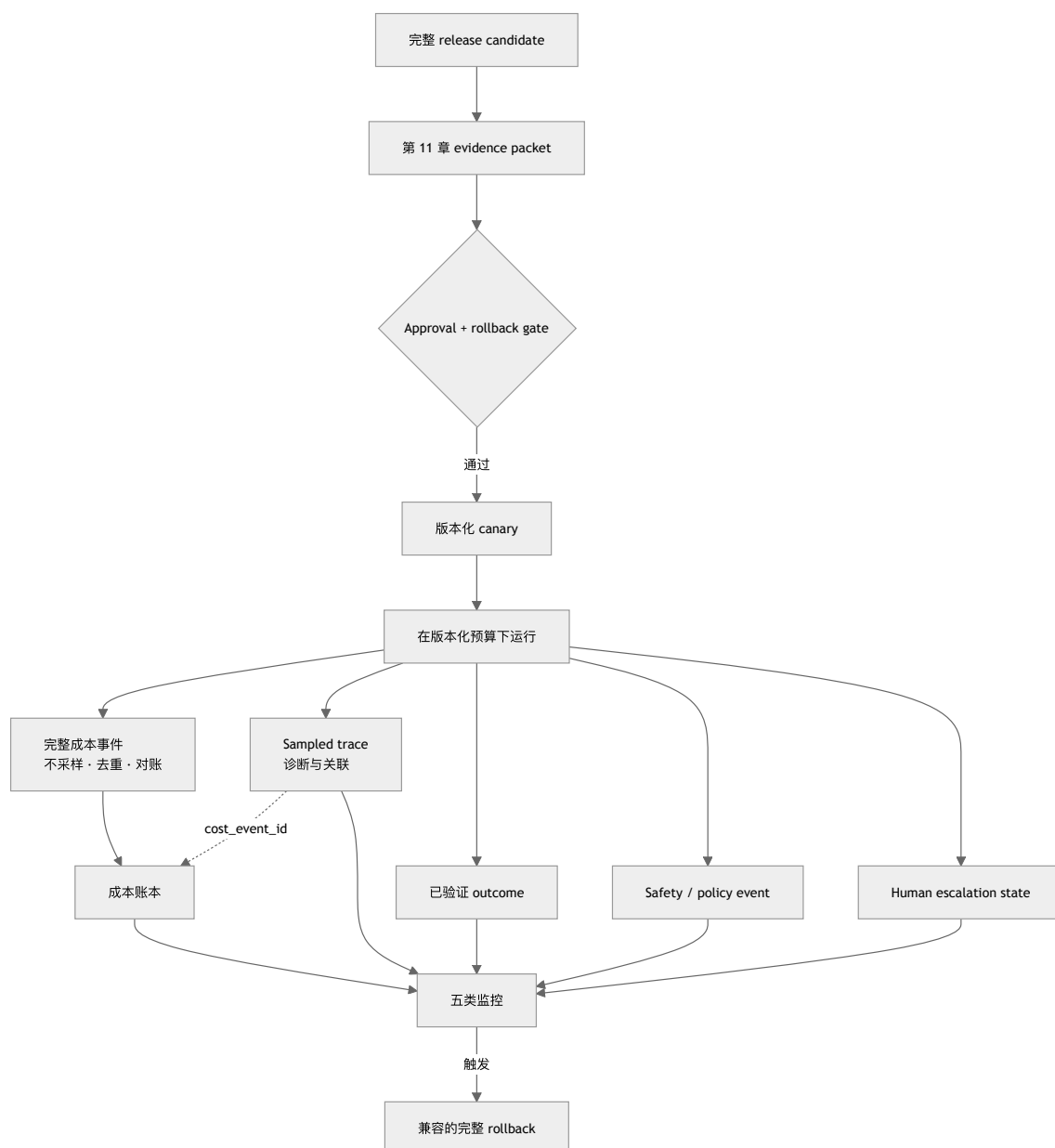
18.8 治理与 vendor 主张需要范围和日期

NIST AI RMF 是自愿采用的风险管理指南，OWASP GenAI 项目发布工程风险指南；二者都不是针对特定 jurisdiction 的法律结论 ([NIST — AI Risk Management Framework](#)；[OWASP — Top 10 for LLM Applications](#))。ISO/IEC 42001:2023 规定 AI management-system standard，而 Regulation (EU) 2024/1689 是 EU AI Act 的官方文本 ([ISO — ISO/IEC 42001:2023](#)；[European Union — Regulation \(EU\) 2024/1689](#))。Certification、framework mapping 或 product feature 本身并不能证明某一具体部署符合所有适用义务。

范围说明——生效日期 2026-07-31。 法律义务取决于部署所在 jurisdiction、provider/deployer role、use case、受影响人群、合同，以及相关日期已经生效的条款。对于实际部署，应核对最新官方文本并咨询合格法律顾问。本章是工程指南，不构成法律建议。

对于快速变化的 vendor “AgentOps”、gateway、observability、cache、evaluation 与 governance 产品，只把它们当作带日期的实现案例。记录 product、documented behavior、region/tier、source URL 与 verification date；让架构和发布契约独立于 vendor label。

图：生产证据与控制回路



Trace 用于解释 run；完整 domain record 用于强制预算、会计、policy、恢复与证据要求。

要点

- **Sampled trace 不是成本账本或 audit record：只有完整、未丢失且已去重的成本事件或 cost span 才能结账，sampling 会破坏这种完整性。
- **在本地定义分位数：为 p50/p95/p99 注明统计总体、窗口、区间、配置、样本数和 timeout/failure 处理方式。
- **区分四套缓存契约：单次请求 KV、provider prompt、application response 和 semantic cache 在不同范围复用不同内容。
- **Semantic cache 复用对 authorization 敏感：绑定 tenant、user/auth scope、model、prompt、tool、retrieval、policy version、TTL、invalidation 与 provenance；敏感、个性化和高影响复用默认禁用。
- **预算是向量，不只是 token：在 durable runtime state 中强制执行 token、dollar、工具调用、wall time、external spend 与 retry/fallback。
- **隐私覆盖完整生命周期：minimization、redaction、retrieval ACL、trace capture/storage、retention、egress 和 deletion 都需要独立控制与证据。
- **分开监控五类问题：service health、agent outcome、safety/policy event、cost 与 human escalation 各有不同权威 source。
- **发布和回滚完整配置：承接第 11 章的 evidence packet，以及第 17 章的 approval、canary、monitoring 与 rollback 纪律。

延伸阅读

- OpenTelemetry, *Trace SDK*. <https://opentelemetry.io/docs/specs/otel/trace/sdk/>
- OpenAI, *Prompt caching* (产品文档; 核验于 2026-07-31) .
<https://developers.openai.com/api/docs/guides/prompt-caching>
- Microsoft, *Enable semantic caching for LLM APIs in Azure API Management* (产品文档; 核验于 2026-07-31) .
<https://learn.microsoft.com/en-us/azure/api-management/azure-openai-enable-semantic-caching>
- NIST, *Privacy Framework*. <https://www.nist.gov/privacy-framework>
- NIST, *AI Risk Management Framework*. <https://www.nist.gov/itl/ai-risk-management-framework>
- OWASP, *Top 10 for LLM Applications*. <https://genai.owasp.org/llm-top-10/>
- ISO, *ISO/IEC 42001:2023*. <https://www.iso.org/standard/42001>
- European Union, *Regulation (EU) 2024/1689*. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>
- OpenAI, *A Shared Playbook for Trustworthy Third-Party Evaluations*. <https://openai.com/index/trustworthy-third-party-evaluations-foundations/>

第 19 章：Agent Fleet 与控制平面

前面的章节治理的是一个 model-harness 系统：它如何获得权限、执行 run、从失败中恢复、请求审批并产出证据。Fleet 带来了另一类协调问题。许多团队可能发布许多版本，不同 tenant 可能共享基础设施却不能共享权限，工作还可能跨 runtime 迁移，或被委派给其他 agent。Operator 需要对以下问题给出一致答案：哪些对象可以运行、代表谁运行、受哪个 release 和 policy 约束，以及如何收回这些权限。

本书把 **agent control plane**（agent 控制平面）作为一种借鉴自分布式系统的架构约定，而不是某个产品的名称，也不声称它是唯一的行业标准分层。例如，Kubernetes 文档中的 control plane 负责管理 cluster state，worker node 则运行 workload；这个类比很有用，但 agent fleet 不是 Kubernetes cluster，也不必照搬其组件边界（[Kubernetes — Components](#)）。本章中，**action plane**（动作平面）执行 model call、retrieval、tool call、代码执行和 agent-to-agent 交互；**control plane**（控制平面）管理 fleet 范围的 registry、identity、policy administration 与 decision、placement、lifecycle、review queue 和 evidence index。真正的 enforcement 仍然发生在 action path 上明确且不可绕过的位置。

19.1 控制平面负责治理，Runtime 负责执行

应区分以下四类责任：

责任主体	负责	不会因此变成
Model	提议：文本、plan、候选 action 和 interpretation	Executor、credential holder、policy authority 或最终事实来源
Harness	模型邻近的 context assembly、tool mediation、validation、loop control 和 evidence capture	Fleet 范围的 release、tenant 或 identity 可信来源
Runtime	Durable execution state、event history、queue、retry、scheduling、worker、sandbox 和 cancellation	Policy author；也不会仅因为保存了 event 就自动成为完整 audit system
Control plane	Registry、identity、policy administration 与 decision、desired state、placement constraint、revocation、review queue 和 lineage index	必须由单个 process 实现的服务、特定 vendor product，或每项 action 都必须经过的集中式 hop

集中式管理可以与分布式执行并存。Control-plane API 可以发布 desired release、signed policy bundle 或 revocation epoch；本地 gateway、sidecar、tool proxy 和 sandbox broker 再靠近受保护资源执行这些状态。反过来，把一个 service 放在网络上并称作“control plane”，并不会赋予它实际权限。只有当每项相关 action 都会到达一个真正能阻断它的 enforcement point 时，它的 decision 才有意义。

控制平面也不会取代第 10 章的 runtime contract。Worker 崩溃后，应由 event history 和 checkpoint 恢复 run。控制平面可以决定它应该在哪里恢复、是否仍被允许恢复，却不应从 dashboard 或 sampled trace 中重建 workflow state。

19.2 Registry 存储第 18 章定义的 Release Unit

不要再造一套更弱的“agent record”。Fleet 注册和 promotion 的不可变对象，就是第 18 章定义的完整 **release manifest**：model/provider 与 inference setting；prompt 与 context policy；tool schema 与 implementation；retrieval snapshot 与 configuration；memory schema 与 policy；sandbox image、capability、network policy 与 limit；policy/approval rule 和 runtime/router version；grader 与 evaluation contract；以及 budget、retry/fallback 和 cache policy。

Registry 只在该 manifest 周围增加 fleet-management binding，而不把它的字段复制到可变列中：

```
release_id, manifest_digest, release_manifest_ref
publisher_id, accountable_sponsor_id, owner_team_id
release_state, approval_ref, evidence_packet_ref
eligible_tenants, eligible_workload_classes, placement_constraints
endpoints_and_protocol_versions, dependency_release_ids
desired_deployments, observed_deployments
created_at, deprecates_at, revoked_at, revocation_reason
```

Run 启动前，应先把 display name 解析成不可变的 release_id，并把该 ID 写入 run。Registration 不等于 approval：draft、evaluated、approved、deployed、suspended、deprecated、revoked 和 retired 是不同的 fleet state。Release-state transition 应指明 evidence packet 和 accountable approver；rollback 应选择另一份完整且兼容的 manifest，而不是只替换 prompt 或 model。

Discovery metadata 很有用，但并不充分。Public capability card 可以帮助其他 agent 找到 endpoint；内部 registry 则判断这个确切 release 是否获准服务于当前 tenant、purpose、workload class 和 environment。Dependency edge 必须可查询，这样在撤销 tool、policy bundle、sandbox image、publisher 或 model 时，才能找出受影响的 release 和运行中工作。

19.3 Identity 是一条链，而不是一把 API Key

Fleet action 涉及多个 identity 和 constraint。“是 agent 做的”过于含糊；只有 user identity 又会丢失实际行动的 release 和 workload。控制平面至少应绑定：

```
release_version_id      # 第 18 章的不可变 release
sponsor_id              # 承担责任的人或组织
delegator_chain         # subject 以及每个 acting/delegating principal
tenant_id               # isolation 与 policy domain
purpose_id              # 有边界的 task/workflow 正当理由
runtime_id              # durable execution service 及其 version
workload_id             # 具体 process/service identity
credential_audience    # 预期 resource/service
issued_at, not_before, expires_at
credential_id, revocation_epoch, revocation_status
session_id, run_id
```

Release identity、runtime identity 与 workload identity 应彼此分离。一个 release 可以有許多并发 workload；一个 runtime 可以托管许多 release；替换 process 不应悄悄改变 release 或 delegator。SPIFFE 可作为一种实现构件，而不是强制标准：它定义了唯一标识 workload 的 SPIFFE ID，以及供 workload 证明身份的 SVID ([SPIFFE — Concepts](#))。它没有提供本书需要的 sponsor、purpose、release manifest 或 delegation semantic，因此这些仍然是显式的控制平面 claim。

Credential 应短期有效、绑定 audience，并且只在受保护边界呈现。Raw credential 不得进入 model context、message、generated code 或通用 artifact storage。PEP 在使用 credential 前，要校验 issuer、subject/actor chain、tenant、purpose、audience、scope、time bound、revocation state、release status 以及 requested resource/action。

19.4 Delegation 只能收窄权限

本书采用单调收窄的 delegation rule：

```
child_effective_authority =
    parent_effective_authority
    n approved_child_release_policy
    n tenant_and_environment_policy
    n delegated_actions_and_resources
    n purpose, audience, budget, and time bounds
```

每一跳都在链上增加一个 actor，并且可以移除 action、resource、destination、budget 或 time，却不能增加 parent 原本不具备的权限。如果确实需要更大的权限，应由有权 principal 通过 policy 与 approval path 作出新授权；child 不能从 task text 中自行推断。

OAuth 2.0 Token Exchange 区分 subject 与 actor，并定义 audience、resource 和 scope 参数；但它明确把 token trust 和大量 deployment policy 留在规范范围之外 ([RFC 8693 — OAuth 2.0 Token Exchange](#))。它可以编码 delegation flow 的一部分，却不会自行保证本书的收窄不变量、revocation propagation 或 purpose binding。这些属性必须由 issuer 和 PDP 执行。

最重要的是，**child 的 output 携带信息，不携带 child 的权限**。Parent A 接受 child B 的 report、plan、artifact 或 tool result 时，A 不会继承 B 的 credential，B 也不会获得 A 的 authority。如果 A 想把该 output 转化为 privileged action，就必须以 A 当前的 identity 和 policy 提交一项新的 action proposal。这是在 fleet 层面防止 trust escalation 和 confused-deputy behavior 的做法。

19.5 分开 Policy Administration、Decision 与 Enforcement

即使由同一产品实现多个部分，以下三项功能也必须清晰可见：

功能	Contract	常见输出
Policy administration	人和受治理的自动化负责 author、review、version、test、approve、distribute 与 retire policy bundle	不可变 policy version、signature、rollout status、revocation epoch
Policy decision point (PDP)	根据 identity、tenant、release、resource、action、purpose、delegation、environment、budget 与 policy 计算结果	allow、deny、redact、require_approval、sandbox、更窄的 scope/budget、reason 和 decision ID
Policy enforcement point (PEP)	截获真实 request，并在受保护 operation 发生前执行 PDP decision	被阻断或转换的 request、approval gate、sandbox route、enforcement event

这里存在一个术语陷阱。NIST Zero Trust glossary 把 **Policy Administrator (PA)** 定义为通过向 PEP 发出命令来执行 policy engine decision 的组件，并把 policy engine 与 PA 合称 PDP；这个发生在 request time 的 PA，不等同于本书更宽泛的 policy authoring 与 lifecycle 功能 ([NIST — Zero Trust Architecture Glossary](#))。因此，本章用 **policy administration** 指 authoring/lifecycle，用 **PDP** 指完整的 request-time decision service。

Gateway、sidecar、tool proxy、network egress proxy、retrieval materializer 或 sandbox broker 可以成为 PEP，但只能针对它不可绕过地介入的 action 才算 PEP。例如，如果 generated code 能绕过 tool proxy 直接访问网络，这个 proxy 就不是 shell egress 的 PEP。NIST IR 7987 把 PEP 描述为执行 reference-mediation function 的组件；本书进一步把完整且抗篡改的 mediation 规定为 fleet invariant ([NIST IR 7987 Rev. 1 — Policy Machine](#))。当系统无法确定当前 identity、policy、approval 或 revocation state 时，有后果的 operation 应 fail closed。

Prompt 不是 PEP。“未经审批绝不转账”可以帮助 model 提出更安全的候选 action，却不能阻止被攻陷或判断错误的 model 输出调用。只有当 PEP 在执行前验证了一项与确切不可变 action proposal 绑定的 approval，[第 14 章](#)的 mandatory approval 才真正成为控制。

19.6 Lifecycle、Placement 与 Kill Switch

Release lifecycle、run lifecycle 和 sandbox/worker lifecycle 彼此关联，却不能互换。撤销 release 会阻止新 run，但已经签发的 credential 和 active run 仍可能继续，除非其各自状态也经过 reconciliation。Controller 应比较 desired state 与 observed state，并明确处理每项后果：

- scheduling 只接纳对当前 tenant、purpose、workload class、region 和所需 isolation 获批的 release；
- run 拥有 queued、active、waiting for approval、suspended、cancel-requested、completed、failed 和 canceled 等 durable state；
- worker 和 sandbox 使用 lease，以便发现废弃执行，并在不复制 logical run 的前提下恢复；
- upgrade 使用完整 release unit，并遵守[第 18 章](#)的 canary、monitoring、rollback 和 durable-run compatibility rule。

Fleet **kill switch** 是一组协调的状态转换，不是一条自然语言“stop”消息。根据 credential、run、release、publisher、tenant、tool 或整个 fleet 等不同 scope，控制平面必须：

1. 推进 revocation epoch，并撤销或使受影响 credential 失效；
2. 向每个相关 PEP 发布 deny rule，并停止接纳新工作；
3. 通过 run lifecycle 请求 suspend 或 cancel，停止续租 lease，并在 policy 要求时终止 sandbox；
4. 对 side-effect outcome 为 unknown 的 in-flight action 做 reconciliation，而不是盲目 retry；
5. 记录 acknowledgement、无法联系的 enforcement point、exception 和最终 observed state。

RFC 7009 标准化了 OAuth token-revocation request：它会使 token 失效，也可能影响相关 token 或底层 grant，但 access-token revocation 支持和传播行为取决于具体实现 ([RFC 7009 — OAuth 2.0 Token Revocation](#))。因此，credential revocation 是 kill switch 必需的 actuator 之一，却不是全部机制。PEP denial 关闭仍然开放的 action path，runtime cancellation 则控制已经开始运行的工作。发送给 model 的消息最多只是一项额外 steering signal。

19.7 Trace、Event History、Audit 与 Lineage 是不同产品

这些记录可以共用 identifier 或 storage infrastructure，但它们的 contract 不能合并：

Evidence product	规范含义	完整性与用途
Trace	含 timing、parent/link relation、attribute、event 和 status 的 span	用于 diagnosis 与 performance evidence；可能被 sampling 或丢弃。OpenTelemetry 明确允许 sampling decision 丢弃 span data (OpenTelemetry — Trace API ； OpenTelemetry — Trace SDK)
Event history	用于重建 workflow state 和恢复进度的有序 durable execution event	声明的 workflow scope 内的 runtime system of record。以 Temporal 为具体例子，其 append-only Event History 会 durable persist，以便在故障后恢复 (Temporal — Events and Event History)
Audit record	针对明确 control scope 的受保护 accountability record	需要必要的 principal、action、object、outcome、timestamp、integrity、retention、access 与 completeness control；NIST SP 800-53 的 Audit and Accountability control family 是相关控制目录，并不意味着普通 log 已经满足它 (NIST SP 800-53 Rev. 5)
Lineage	连接 identity、version、input、artifact、decision、action 与 outcome 的 provenance graph	支持跨产品 causal query；W3C PROV-DM 提供稳定的 entity/activity/agent 与 derivation vocabulary，本章则定义 fleet-specific graph (W3C — PROV-DM)

Temporal 文档也会非正式地把 Event History 称作“用于 debugging 的 audit log” ([Temporal — Events and Event History](#))。这并不会让每份 event history 自动成为本书所指的 protected audit record。同样，unsampled trace 仍不会自动变成 durable workflow history；把 audit row 复制进 graph，如果没有表示 causal edge 和 version identity，也不会得到 lineage。

19.8 最小 Fleet Lineage Graph

Lineage 应通过不可变 identifier 关联已有的 authoritative record，而不是复制其中的可变 prose。对于每项 consequential action，至少保留足以遍历以下路径的 edge：

```
session_id → run_id → step_id → action_id → attempt_id → call_id
run_id → release_id → manifest_digest → component artifacts
run_id → sponsor / delegator chain / tenant / purpose / runtime / workload
action_id → policy_decision_id → policy_version + decision inputs
action_id → approval_id → immutable proposal digest + reviewer + expiry
call_id → input/output artifact IDs → content hashes and source versions
call_id → cost_event_id → 第 18 章 cost ledger
action_id → enforcement_event_id → PEP identity and result
records → protected_audit_record_id + trace_id/span_id where available
action_id → external_effect_id → verified outcome or explicit unknown
```

Link 比收集所有 payload 更重要。Reviewer 应能查询：哪个 release 和 policy 产生了昂贵或有害的 outcome，哪些 run 依赖已被撤销的 artifact，以及每项获批 side effect 是否同时具有 PEP event 和独立验证的 result。保存 content hash 与受保护 reference；对敏感 prompt、memory 和 tool content 则遵循第 18 章的 privacy lifecycle，进行 minimization 或 redaction。

Cost 属于 lineage，但不能因此把 trace 变成 accounting system。把第 18 章完整 ledger 中的 `cost_event_id` 关联到 `call_id`、`run_id`、`tenant_id` 与 `release_id`；绝不能从 sampled span 推断 fleet 总成本。同样，应关联 protected audit record，而不是假定 lineage index 自身满足 audit retention 和 integrity 要求。

19.9 Fleet Review 是跨 Run 的控制界面

第 14 章定义了 single-action approval contract：mandatory approval 是绑定到确切不可变 proposal 的 runtime gate，而不是对话中的一句“看起来可以”。Fleet 控制平面负责把这些 proposal 交给正确 reviewer 的 **cross-run review queue**。

Queue item 应携带 `tenant_id`、`release_id`、`run_id`、`action_id`、risk class、policy reason、evidence link、uncertainty、proposed side effect、resource version、proposal digest、requested approval scope、expiry，以及当前 run/event state。排序依据应是 risk、deadline、reversibility、uncertainty 与 evidence quality，而不是 arrival order 或 transcript length。Routing 则依据 tenant、domain、region、separation-of-duties rule 和 reviewer authority。

Queue 只是 view，不是 authorization source。只有当 PEP 验证了 approval 的不可变绑定，它才有效。如果 action argument、resource version、identity chain、release、policy、risk class、scope 或 expiry 改变，应把 card 标记为 stale，并要求重新 decision。Cancellation 或 steering 更新 durable run state；关闭浏览器中的 card 并不会取消工作。

Queue SLO 应跟踪 age、expired proposal、reassignment、override、未解决的 unknown side effect 和 reviewer workload，且不能把沉默视作批准。

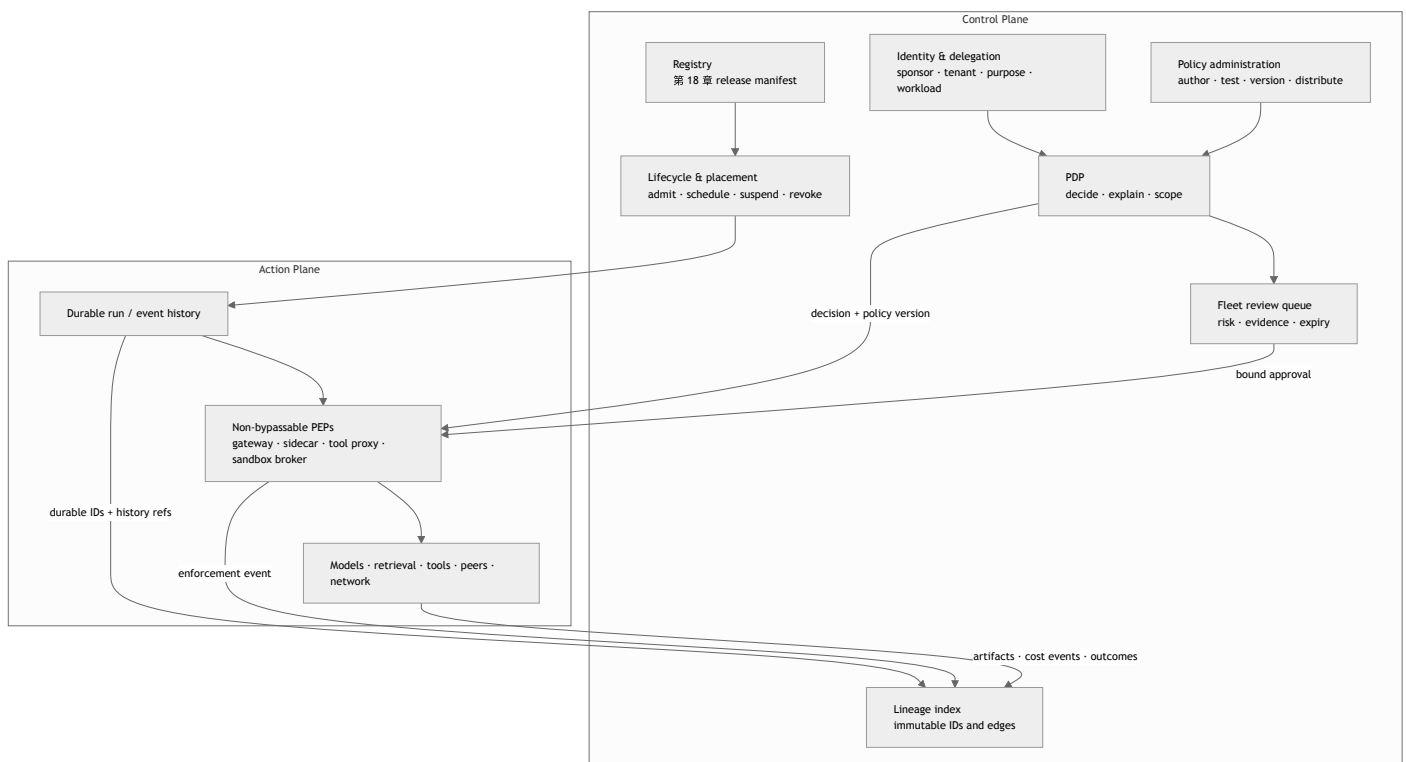
19.10 带日期的标准快照

下面没有任何一个来源定义了本章的完整控制平面。由于这些界面会变化，引用 protocol 或 product 时必须记录状态和日期：

界面	在 2026-07-31 核验的状态	在本章覆盖什么 — 以及不覆盖什么
A2A	页面显示的最新 released specification 为 1.0.0	通过 Agent Card 进行 agent discovery，并定义 message、task、cancellation 与 interoperability。规范明确把 in-task authorization 的 scope、representation、validity 和 revocation semantic 留给实现，因此它不是 fleet authority 或 release registry (A2A — Protocol Specification)
MCP	带版本的 authoritative protocol revision 2025-06-18	在 host、client 与 server 之间暴露 resource、prompt 和 tool。它是 capability/data integration protocol，而不是 release manifest、sponsor/delegator identity、PDP/PEP topology 或 fleet lifecycle 标准 (MCP — Specification 2025-06-18)
W3C PROV-DM	日期为 2013-04-30 的 W3C Recommendation；W3C 将其描述为稳定的参考材料	成熟且 domain-agnostic 的 provenance vocabulary；不定义 fleet-specific node、enforcement guarantee 或 retention policy (W3C — PROV-DM)
NIST ZTA terminology	当前 NIST reference vocabulary，于 2026-07-31 核验	为 access control 提供有用的 PDP/PEP/PA definition；不是 agent-control-plane product specification (NIST — Zero Trust Architecture Glossary)

Vendor registry、gateway、identity broker、runtime 和 observability product 可以实现这套设计的一部分。把某个产品当作证据时，应记录 product name、documented feature、release/tier/region、source URL 与 verification date。不要让不断变化的产品标签反过来重定义架构。

图：Fleet 中的 Decision、Enforcement 与 Evidence



控制平面可以是分布式的。真正的不变量是：approved release 与 identity 到达 PDP，而每项 protected action 都到达能够执行当前 decision 和 revocation state 的 PEP。

要点

- ****Control plane** 是本书的约定，不是通用标准：******它治理 fleet-wide state，runtime 与 harness 则执行单次 run。
- ****复用完整 release unit**：******registry 直接指向第 18 章不可变的 release manifest 与 evidence packet。

- **携带 identity chain: **release、sponsor、delegator、tenant、purpose、runtime、workload、audience、expiry 与 revocation 必须保持可区分。
- **Delegation 只能收窄: **child result 传递的是信息, 绝不是 permission; 高权限后续动作需要新的 policy decision。
- **分开 policy administration、PDP 与 PEP: **gateway 或 proxy 只有在不可绕过地介入某条 path 时才是 PEP; prompt 永远不是 PEP。
- **Kill switch 需要三类 actuator: **撤销 credential、在 PEP deny, 并转换 durable run/worker lifecycle。
- **Trace、event history、audit 与 lineage 的保证不同: **用 durable ID 关联它们, 但不要把其中一个当作另一个。
- **Lineage 关联 authoritative record: **包括 release、identity、policy、approval、PEP action、artifact、cost event、protected audit reference 与 verified outcome。
- **Fleet review 延续第 14 章: **跨 run 工作按 risk 和 evidence 排序, 而确切 approval 仍由 runtime enforcement。

延伸阅读

- Kubernetes, *Components*. <https://kubernetes.io/docs/concepts/overview/components/>
- NIST, *Implementing a Zero Trust Architecture — Glossary*. <https://pages.nist.gov/zero-trust-architecture/glossary.html>
- NIST, *NIST IR 7987 Rev. 1: Policy Machine: Features, Architecture, and Specification*. <https://nvlpubs.nist.gov/nistpubs/ir/2015/nist.ir.7987r1.pdf>
- IETF, *RFC 8693: OAuth 2.0 Token Exchange*. <https://www.rfc-editor.org/rfc/rfc8693.html>
- IETF, *RFC 7009: OAuth 2.0 Token Revocation*. <https://www.rfc-editor.org/rfc/rfc7009.html>
- SPIFFE, *SPIFFE Concepts*. <https://spiffe.io/docs/latest/spiffe-about/spiffe-concepts/>
- OpenTelemetry, *Trace API and Trace SDK*. <https://opentelemetry.io/docs/specs/otel/trace/api/> and <https://opentelemetry.io/docs/specs/otel/trace/sdk/>
- Temporal, *Events and Event History*. <https://docs.temporal.io/workflow-execution/event>
- NIST, *SP 800-53 Rev. 5*. <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
- W3C, *PROV-DM: The PROV Data Model*. <https://www.w3.org/TR/prov-dm/>
- A2A Protocol, *Protocol Specification*. <https://a2a-protocol.org/latest/specification/>
- Model Context Protocol, *Specification 2025-06-18*. <https://modelcontextprotocol.io/specification/2025-06-18>

第 20 章：展望

本卷从一条有意精确定义的边界出发：agent 不是一个仅凭流畅文字便获得权限的模型。它是嵌入外部系统的模型；外部系统选择输入、治理动作、保存状态并检查结果。产品、模型能力、协议和主流脚手架都会变化。这条边界告诉我们：当它们变化时，哪些主张仍然需要由外部保证来成立。

因此，最后一章不预测某种必然胜出的 agent 架构。它收束两卷的论证，区分持久原则和有时效性的实现证据，列出开放的工程问题，并给出一张导航图：从《LLM 基础》中的模型机制，走到本书对应的系统责任。

20.1 不变的责任边界

最终原则与《LLM 基础》第 14 章完全一致：

模型负责预测或提出方案。外部系统负责上下文选择、持久状态、权限、执行、验证和后果。

“外部系统”不是某个必须存在的单一服务。第 1 章把它划分为 agent harness、runtime、产品/应用、平台/控制平面和 evaluation harness。一次部署可以合并这些层，也可以把一部分交给 provider，但仍然必须明确：每项保证究竟由哪个组件提供。

责任	模型的合理角色	外部 owner 与所需保证
上下文选择	解释给定的表示；提示缺失信息	Agent harness 为每次调用选择、限定、序列化输入，并追踪其 provenance
持久状态	提出状态转换或总结进度	Runtime 与产品持久化执行/业务状态；上下文和 memory 不能是唯一事实来源
工具表面与执行	选择工具并提出结构化参数	Harness 暴露契约；dispatcher/runtime 验证、授权、执行、关联调用，并规范化结果
权限	表达意图、请求访问或要求审核	产品/平台 policy 作出决定；不可绕过的 PEP 执行当前决策和强制审批
验证	给出 critique、估计或候选判断	Runtime、产品、evaluation harness 和人检查 artifact、环境状态、流程规则与结果
后果	预测或描述预期效果	Runtime 与集成系统创建、记录、对账、补偿或回滚效果，并继续为其负责

由 provider 执行的工具、托管 memory 功能或模型生成的 critique，可能会移动实现边界。它们不会消除以下问题：谁授权操作、什么状态具有权威性、什么证据能证明结果，以及谁处理失败。流畅度、置信度、schema 有效性，或者模型自称工作已经完成，本身都不能提供这些保证。

20.2 持久的设计原则

以下原则应当能够跨越模型家族和产品名称的变化：

- 把方案与效果分开。把解析、验证、授权、必要时审批、执行和结果确认视为彼此不同的生命周期阶段。
- 把上下文视为经过选择的视图。上下文是单次调用的有限输入；memory、artifact、执行状态、event history、trace、eval trajectory、cost ledger、audit record 和 lineage 各有不同契约。
- 采用满足任务所需的最低动态控制机制。确定性 workflow、有界 hybrid node 和模型主导的 loop 都是应当评估的选择，不是必须逐级攀升的自治阶梯。
- 把硬保证放在无法绕过的路径上。Prompt 指令与模型自我约束属于纵深防御；权限、sandbox、egress、预算和强制审批需要外部强制执行。
- 验证真正相关的结果。API 成功响应、美观的 artifact、整洁的 transcript 或通过的局部检查，只能证明它实际测量到的性质。
- 围绕中断与歧义做设计。持久化 identity、checkpoint、预算、审批状态、幂等记录，以及 partial 或 unknown 结果，使恢复不至于退化成盲目重试。
- 评估并发布完整配置。模型、prompt、工具、retrieval、memory、runtime、sandbox、policy、grader、cache 和预算共同决定一个结果能够支持什么主张。
- 明确区分各种证据保证。关联 trace、event history、eval trajectory、cost ledger、audit record 和 lineage，但不要把它们当作彼此的同义词。

这些是关于责任归属与证据的原则，不承诺某种实现永远最优。模型变强后，团队可以在受控评估的支持下移除 planner、reset、evaluator 或工具选择辅助。模型不能因此批准自己的权限，不能事后让未知副作用变得安全，也不能把采样后的 observability 变成完整记录。

20.3 持久原则与时变主张

读者在依赖一项主张之前，应当先判断它属于哪一类：

主张类别	示例	必要处理方式
持久的责任边界	模型提出方案；外部系统授权并执行	作为长期设计原则陈述，并检验所声称的外部保证
依赖 workload 的设计选择	重置上下文、使用独立 grader、延迟提供工具，或选择 routing cascade	说明任务、模型加 harness 配置、预算、对照和测得结果
协议或标准状态	规范版本、conformance level 或章节成熟度	引用官方版本、发布/状态和核验日期；不要据此推断实现覆盖率
厂商产品行为	Cache 匹配、retention、托管工具、模型限制、价格或区域支持	引用当前官方产品文档，并注明模型/tier/区域和核验日期
研究测量或趋势	Benchmark 分数或拟合的能力时间视野	保留任务分布、harness、metric、不确定性、更新日期和局限；不要把测量改写成预测

下面的快照展示这种标注纪律。它不是排名，也不是预测；每一行均于 **2026-07-31** 核验：

项目	核验时的状态或成熟度	该状态不能证明什么
OpenTelemetry Trace SDK	规范将 SDK 标为“除非另有说明，否则为 Stable ”；个别章节可以有其他状态（ OpenTelemetry — Trace SDK ）	导出的 trace 是完整、未采样、适合 replay，或可作为合规 audit record
MCP tools 契约，版本 2025-06-18	一份带日期的协议规范，用于发现和调用已暴露的工具，包括可选的工具列表变更通知（ MCP — Tools Specification ）	当前授权、不可绕过的强制执行、资源所有权或已经验证的外部结果
NIST AI RMF 1.0	已发布的自愿性风险管理框架；NIST 页面说明 1.0 正在修订（ NIST — AI Risk Management Framework ）	法律合规、产品认证或某一种强制的 agent 架构
OpenAI prompt caching	在所述日期核验的有效 provider 产品文档；匹配、breakpoint、retention 和计费因 provider/模型而异（ OpenAI — Prompt caching ）	可移植的 cache 契约，或跨用户/tenant 复用应用响应的权限
METR Time Horizon 1.1	最后更新于 2026-05-08 的研究测量页面，在其声明的软件任务分布上拟合可靠性与人类专家用时的关系（ METR — Task-Completion Time Horizons ）	任意已部署 agent 可以自治运行多久，或对所有职业与环境的预测
Anthropic 长时运行 harness 案例	一项带日期的厂商工程案例：随着被测模型和任务边界变化，任务分解与 evaluator 脚手架也随之调整（ Anthropic — Harness Design for Long-Running Application Development ）	未来模型必然会吸收某个特定 harness 组件这一普遍规律

当某一条带日期的记录发生变化时，更新该行及其核验日期。不要仅仅因为 provider 把某个机制移到了 API 之后，就改变这条不变的责任边界。

20.4 按责任分类的开放问题

这些是工程和研究问题，不是对某种未来方案必然胜出的断言。要证明取得了进展，应当给出明确契约、有代表性的证据和失败分析。

类别	开放问题	能够支持进展的证据
上下文	Harness 应当如何测量选择、compaction、reset、retrieval 和 memory consolidation 所丢失的信息？动态工具/上下文组装如何保留授权、provenance、invalidation 和 provider cache 的正确性？	Resume 测试、对抗性遗漏案例、provenance 覆盖率、跨 scope 负向测试，以及在声明任务切片上测得的质量/成本/延迟
状态	在旧 run 仍须可恢复时，长时运行配置应当如何迁移？在诚实的 replay 契约下，branch、并发 worker、补偿、删除和非确定性模型/工具结果应当如何组合？	跨版本恢复演练、invariant 检查、重复/未知效果测试、记录输出的确定性复用，以及明确的迁移/回滚证据
工具	Schema 如何可移植地表达 effect class、幂等性、审批重要性、postcondition、取消和 partial result？Computer use 与 peer-agent 接口应暴露多少状态，才能支持安全验证？	跨实现契约测试、注入的 timeout/race 案例、postcondition recall、安全重试测试，以及独立于模型文字的结果证据
安全	Identity、delegated authority、retrieval ACL、sandbox、egress、供应链 provenance、PDP 决策和分布式 PEP 如何无缺口地组合？如何测量 prompt injection 的残余风险，而不是宣称问题已经解决？	不可绕过测试、撤销延迟、最小权限分析、对抗性内容/工具套件、跨 tenant 测试，以及受保护的决策与效果记录
评估	随着任务、模型和用户分布变化，suite 如何保持代表性？Grader integrity、maker/checker 相关失败、污染、基础设施波动和延迟业务结果应如何影响发布主张？	带版本的任务与 grader、隔离的重复试验、calibration/adjudication set、按切片报告的不确定性、原始失败计数，以及与发布后结果的 join
运维	系统如何针对每个成功且符合 policy 的结果，联合优化成本、延迟、可靠性、隐私和人工注意力？完整 ledger 与 policy event 如何同采样 telemetry、retention、删除和事件响应共存？	完整且已对账的 counter、五类监控、预算强制测试、隐私生命周期证据、canary 对照、回滚演练和事件时间线
身份与标准	哪些可移植契约应当跨组织表达 agent version、sponsor、delegator、tenant、purpose、credential audience、approval、lineage、revocation 和 audit semantics？哪些部分属于协议，哪些属于本地 policy？	带版本的 conformance profile、多厂商互操作测试、明确的成熟度级别、负向授权测试、provenance 连续性，以及 revocation/kill-switch 演示

这些类别会相互作用。Semantic cache 同时涉及上下文、安全和运维；异步 peer-agent 任务同时涉及状态、工具、身份与评估。分类不会消除耦合，而是让每项缺失的保证变得可见。

20.5 两卷导航

下表给出了从《LLM 基础》中的模型机制走到《Agent Harness》工程责任的最短路径。仓库中的完整 [Foundations → Harness source map](#)维护了包含更充分边界说明的版本。

Foundations 机制	模型侧边界	Harness 责任	在 Harness 中继续阅读
1. Token 与模型输出	生成 token 概率或结构化输出，而不是外部效果	解析方案、关联调用、治理 dispatch 并验证效果	第 1 章、第 6 章
2. Tokenization	输入成本取决于表示方式和具体模型	测量真实契约，并强制执行上下文/视觉预算	第 3 章、第 15 章
3. Next-token prediction	流畅续写不能证明事实、权限、执行或完成	把方案与效果分开，并依据独立证据/结果评分	第 1 章、第 11 章
4. Attention	作用于当前输入；不会创建持久 memory	选择/排序上下文，并在外部持久化状态、memory、artifact 与 provenance	第 3 章、第 5 章、第 10 章
5. 训练数据与 scaling	广泛能力不能证明 workload readiness	评估并发布精确的模型加 harness 配置	第 11 章、第 18 章
6. Inference 与 sampling	每次尝试可能不同；constrained decoding 可保证形状，不能保证语义或权限	运行重复试验，并验证语义、权限与结果	第 6 章、第 11 章
7. Post-training	遵循指令是一种习得行为，不是访问控制	组合模型可见指导，但在 prompt 之外强制保护动作	第 2 章、第 7 章
8. Prompting 与 in-context learning	Prompt 结构会引导行为；不可信文字仍然能引导方案	追踪组装 provenance，分离数据与权限，并在 prompt 之外执行控制	第 2 章、第 4 章、第 7 章
9. Context window 与 cache	上下文有限；每次请求的 KV state 不等同于所有跨请求 cache	预算/转换上下文，并正确限定各 provider cache、response cache 和 semantic cache 的 scope	第 3 章、第 5 章、第 18 章
10. 知识、hallucination 与 uncertainty	合理性与口头置信不能充当已验证证据	检索受治理的来源、保留 citation/provenance、支持 abstention，并评价 grounding/结果	第 4 章、第 11 章、第 13 章
11. Embedding 与 retrieval	相似度和最小 RAG 不会运营生产数据生命周期	负责 ingestion、ACL、freshness、删除、index versioning、reranking 与 retrieval eval	第 4 章
12. Reasoning、工具与 agent	工具/computer action 是方案，其结果会作为上下文返回	完成调用、授权、重试、computer use、审批和结果生命周期	第 6 章、第 7 章、第 14 章、第 15 章
13. 模型行为评估	模型级任务与 grader 不能证明系统可靠性或外部状态	评估完整 trajectory、artifact、环境、基础设施、policy 与 release	第 11 章、第 16 章、第 17 章
14. Operational mental model	模型预测止于权限、持久化、执行和后果之前	在 harness、runtime、产品、control plane/PEP 与 evaluation harness 之间分配责任	第 1 章、第 18 章、第 19 章、第 20 章

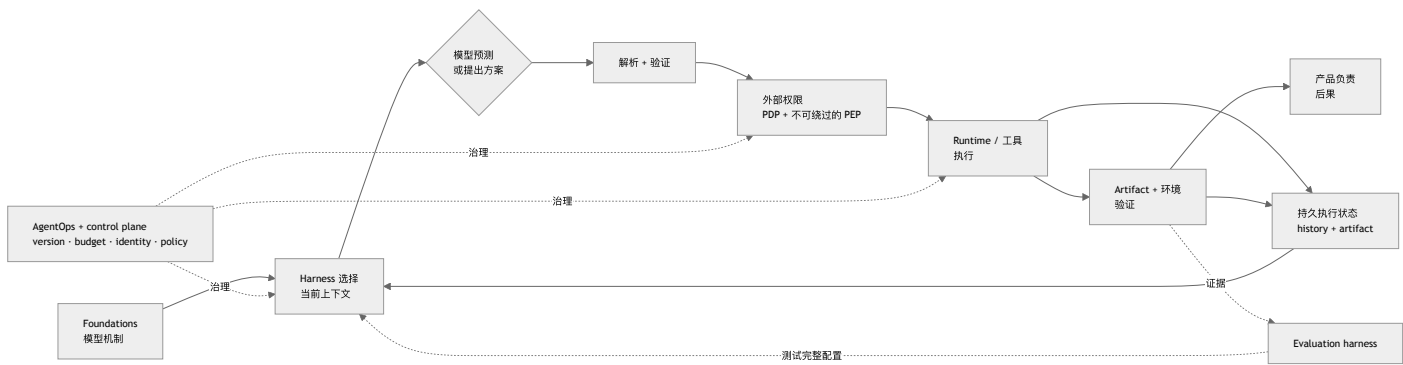
20.6 后续修订规则

当模型、产品、协议或 benchmark 发生变化时，提出四个问题：

- 1. 哪个被测组件变得更关键或更不关键？ 重新运行相关任务与风险切片；不要从 release announcement 推断答案。
- 2. 是实现位置移动了，还是保证消失了？ Provider 托管的机制仍然需要 owner、契约、证据和失败路径。
- 3. 哪项陈述已经过时？ 更新版本、状态、日期、scope 与来源；把历史案例结果保留为历史案例。
- 4. 责任边界仍然成立吗？ 如果模型仍然只负责预测或提出方案，而另一个系统负责效果、状态、权限、验证或后果，边界就没有移动。

这个领域可以快速变化，而本书的结论不必因此变复杂。让模型做它擅长的事。把权限与持久证据放在能够提供它们的系统里。测量完整配置，并且只陈述证据真正支持的主张。

图：一条边界，多个外部 owner



模型仍然是能力的核心。外围系统仍然要为把这种能力转化为受治理、可恢复、经过验证的工作承担责任。

要点

- **不变的责任边界具有持久性：模型负责预测或提出方案；外部系统负责上下文选择、持久状态、权限、执行、验证和后果。
- **外部责任分散在多个组件中：agent harness、runtime、产品、control plane/PEP 与 evaluation harness 提供不同保证。
- **原则、设计、标准、产品和测量的老化速度不同：相应标注版本、成熟度、scope、日期和证据。
- **开放问题归入七个责任组：上下文、状态、工具、安全、评估、运维和身份/标准。
- 两卷共同构成一张地图：《LLM 基础》解释模型机制；《Agent Harness》实现并检验该机制边界产生的责任。
- **能力提升不等于权限转移：只有当受控证据支持变更时才移除脚手架，同时保留外部保证。

延伸阅读

- OpenTelemetry, *Trace SDK*。 <https://opentelemetry.io/docs/specs/otel/trace/sdk/>
- Model Context Protocol, *Tools Specification*, 版本 2025-06-18。
<https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
- NIST, *AI Risk Management Framework*。 <https://www.nist.gov/itl/ai-risk-management-framework>
- OpenAI, *Prompt caching* (产品文档；核验于 2026-07-31)。
<https://developers.openai.com/api/docs/guides/prompt-caching>
- METR, *Task-Completion Time Horizons of Frontier AI Models*, Time Horizon 1.1, 更新于 2026-05-08。
<https://metr.org/time-horizons/>
- Anthropic, *Harness Design for Long-Running Application Development*, 2026 年 3 月。
<https://www.anthropic.com/engineering/harness-design-long-running-apps>

参考文献

Agent Harness：实践者教材的完整书目。

基础（前置）

- *LLM Foundations for Harness Engineering* — 本书所依赖的前置 stage-1 卷。../llm-foundations-zh/
-

基础

- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026.
<https://openai.com/index/harness-engineering/>
 - *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
 - Justin Young et al., *Effective Harnesses for Long-Running Agents*, Anthropic, Nov 2025.
<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
 - Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026.
<https://www.anthropic.com/engineering/harness-design-long-running-apps>
 - Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026.
<https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
 - Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026.
<https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
 - Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024.
<https://www.anthropic.com/engineering/building-effective-agents>
 - Shunyu Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, arXiv, Oct 2022.
<https://arxiv.org/abs/2210.03629>
 - Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026.
<https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
-

上下文、记忆与工作状态

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025.
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
 - Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025.
<https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
 - Charles Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv, Oct 2023.
<https://arxiv.org/abs/2310.08560>
 - Prateek Chhikara et al., *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.19413>
 - Kevin Lin et al., *Sleep-time Compute: Beyond Inference Scaling at Test-time*, arXiv, Apr 2025.
<https://arxiv.org/abs/2504.13171>
-

工具与约束

- Ken Aizawa, *Writing Effective Tools for Agents — with Agents*, Anthropic, Sep 2025.
<https://www.anthropic.com/engineering/writing-tools-for-agents>
- Adam Jones and Conor Kelly, *Code Execution with MCP: Building More Efficient Agents*, Anthropic, Nov 2025.
<https://www.anthropic.com/engineering/code-execution-with-mcp>
- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts: Making Claude Code More Secure and Autonomous*, Anthropic, Oct 2025.
<https://www.anthropic.com/engineering/claude-code-sandboxing>

- Simon Willison, *The lethal trifecta for AI agents: private data, untrusted content, and external communication*, Jun 2025. <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>
 - *Agent2Agent (A2A) Protocol*, Google / Linux Foundation, 2025. <https://github.com/a2aproject/A2A>
 - Linux Foundation, *Linux Foundation Launches the Agent2Agent Protocol Project*, Jun 2025. <https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents>
 - OpenAI, *Connect Private MCP Servers to OpenAI Products*, Jun 2026. <https://developers.openai.com/blog/connect-private-mcp-servers-to-openai-products>
 - OpenAI, *Programmatic Tool Calling*, 2026. <https://developers.openai.com/api/docs/guides/tools-programmatic-tool-calling>
-

规格与 workflow 设计

- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
-

推理与规划模式

(另见“基础”中的 *ReAct*, 即推理-行动的基础模式。)

- Noah Shinn et al., *Reflexion: Language Agents with Verbal Reinforcement Learning*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.11366>
 - Aman Madaan et al., *Self-Refine: Iterative Refinement with Self-Feedback*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.17651>
 - Zhibin Gou et al., *CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing*, arXiv, May 2023. <https://arxiv.org/abs/2305.11738>
 - Shunyu Yao et al., *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.10601>
 - Bin Feng Xu et al., *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.18323>
 - Andy Zhou et al., *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.04406>
-

多代理系统

- Mert Cemri et al., *Why Do Multi-Agent LLM Systems Fail?* (提出 MAST 失败分类法), UC Berkeley / arXiv, Mar 2025. <https://arxiv.org/abs/2503.13657>
-

可靠性与运营安全

- Temporal, *Durable Execution Meets AI: Why Temporal Is the Perfect Foundation for AI*, 2025. <https://temporal.io/blog/durable-execution-meets-ai-why-temporal-is-the-perfect-foundation-for-ai>
 - LangChain, *Durable Execution* (LangGraph documentation), 2025. <https://docs.langchain.com/oss/python/langgraph/durable-execution>
 - Martin Fowler, *CircuitBreaker*, martinfowler.com, Mar 2014. <https://martinfowler.com/bliki/CircuitBreaker.html>
 - Thinkst, *Canarytokens* (free tripwire tokens). <https://canarytokens.org/>
 - Anthropic Safeguards Research Team, *How We Contain Claude*, May 2026. <https://www.anthropic.com/engineering/how-we-contain-claude>
 - Google Cloud, *Agent Executor: Google's Distributed Agent Runtime*, 2026. <https://cloud.google.com/blog/products/ai-machine-learning/agent-executor-googles-distributed-agent-runtime/>
-

Agent 能力度量

- Thomas Kwa et al., *Measuring AI Ability to Complete Long Tasks*, METR / arXiv, Mar 2025. <https://arxiv.org/abs/2503.14499>
 - METR, *Task-Completion Time Horizons of Frontier AI Models*, 更新于 2026 年 5 月 8 日。 <https://metr.org/time-horizons/>
 - Joel Becker, *Clarifying Limitations of Time Horizon*, METR, 2026 年 1 月 22 日。 <https://metr.org/notes/2026-01-22-time-horizon-limitations/>
-

Loop Engineering (循环工程)

- Addy Osmani, *Loop Engineering*, addyosmani.com, Jun 2026. <https://addyosmani.com/blog/loop-engineering/>
 - *Loop Engineering*, O'Reilly Radar, 2026. <https://www.oreilly.com/radar/loop-engineering/>
 - Andrew Ng, *Three Loops for Building 0-to-1 AI Products*, The Batch, Jun 2026. <https://www.deeplearning.ai/the-batch/>
 - *The Anthropic leader who built Claude Code ditched prompting — now he writes loops*, The New Stack, 2026. <https://thenewstack.io/loop-engineering/>
 - *The Agentic Loop: A Practical Field Guide*, DEV Community, 2026. <https://dev.to/truongpx396/the-agentic-loop-a-practical-field-guide-mnc>
 - *Loop Engineering Guide (2026)*, AI Builder Club. <https://www.aibuilderclub.com/blog/loop-engineering-guide-2026>
 - *Loop Engineering Crash Course*, The AI Agent Factory (Panaversity). <https://agentfactory.panaversity.org/docs/loop-engineering-crash-course>
-

评估与可观测性

- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
 - Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026. <https://www.anthropic.com/engineering/infrastructure-noise>
 - Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
 - OpenTelemetry, *Semantic conventions for generative AI spans*. <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>
 - Anthropic Safeguards Research Team, *Agentic Misalignment: Summer 2026 Update*, 2026. <https://alignment.anthropic.com/2026/agentic-misalignment-summer-2026/>
 - OpenAI, *Trustworthy Third-Party Evaluations: Foundations*, 2026. <https://openai.com/index/trustworthy-third-party-evaluations-foundations/>
 - OpenAI, *Building Self-Improving Tax Agents with Codex*, 2026. <https://openai.com/index/building-self-improving-tax-agents-with-codex/>
-

运行时、Harness 与参考实现

- Harrison Chase, *Agent Frameworks, Runtimes, and Harnesses*, Oh My!, LangChain, Oct 2025. <https://blog.langchain.com/agent-frameworks-runtimes-and-harnesses-oh-my/>
 - Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
-

指令与 Prompting

- Eric Wallace et al., *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*, OpenAI, Apr 2024. <https://arxiv.org/abs/2404.13208>
 - OpenAI, *Custom Code Review Rules for Codex*, Jul 2026. <https://developers.openai.com/blog/custom-code-review-rules-for-codex>
-

模型选择、路由与推理

- Isaac Ong et al., *RouteLLM: Learning to Route LLMs with Preference Data*, 2024. <https://arxiv.org/abs/2406.18665>
 - Lingjiao Chen, Matei Zaharia, and James Zou, *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*, 2023. <https://arxiv.org/abs/2305.05176>
 - Charlie Snell et al., *Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters*, 2024. <https://arxiv.org/abs/2408.03314>
 - DeepSeek-AI, *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*, 2025. <https://arxiv.org/abs/2501.12948>
 - LiteLLM (BerriAI), *Python SDK and Proxy Server (AI Gateway)*. <https://github.com/BerriAI/litellm>
 - Portkey, *AI Gateway*. <https://github.com/Portkey-AI/gateway>
-

人-Agent 交互

- Saleema Amershi et al., *Guidelines for Human-AI Interaction*, CHI 2019. <https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>
 - Eric Horvitz, *Principles of Mixed-Initiative User Interfaces*, CHI 1999. <https://www.microsoft.com/en-us/research/publication/principles-of-mixed-initiative-user-interfaces/>
-

Computer-Use 与多模态 Agent

- Anthropic, *Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku*, Oct 2024. <https://www.anthropic.com/news/3-5-models-and-computer-use>
 - OpenAI, *Computer-Using Agent (Operator)*, Jan 2025. <https://openai.com/index/computer-using-agent/>
 - Jianwei Yang et al., *Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V*, 2023. <https://arxiv.org/abs/2310.11441>
 - Boyuan Zheng et al., *GPT-4V(ision) is a Generalist Web Agent, if Grounded (SeeAct)*, 2024. <https://arxiv.org/abs/2401.01614>
 - Shuyan Zhou et al., *WebArena: A Realistic Web Environment for Building Autonomous Agents*, 2023. <https://arxiv.org/abs/2307.13854>
 - Tianbao Xie et al., *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*, 2024. <https://arxiv.org/abs/2404.07972>
-

成本、隐私与治理框架

- AWS, *AgentOps: Operationalize Agentic AI at Scale with Amazon Bedrock AgentCore*, 2026. <https://aws.amazon.com/blogs/machine-learning/agentops-operationalize-agentic-ai-at-scale-with-amazon-bedrock-agentcore/>
- OWASP, *Top 10 for Large Language Model Applications*, 2025. <https://genai.owasp.org/llm-top-10/>
- NIST, *AI Risk Management Framework (AI RMF 1.0)*, 2023. <https://www.nist.gov/itl/ai-risk-management-framework>
- ISO/IEC 42001:2023 — *Information technology — Artificial intelligence — Management system*, ISO, 2023. <https://www.iso.org/standard/42001>
- *Regulation (EU) 2024/1689 (Artificial Intelligence Act)*, European Union, Jun 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>

- Fu Bang, *GPTCache: An Open-Source Semantic Cache for LLM Applications*, NLP-OSS @ EMNLP 2023. <https://github.com/zilliztech/GPTCache>
-

Agent Fleet、身份与控制平面

- NIST, *Identity and Authorization for Software Agents*, Feb 2026. <https://www.nist.gov/news-events/news/2026/02/new-concept-paper-identity-and-authority-software-agents>
 - Google Cloud, *Agent Registry Overview*, 2026. <https://docs.cloud.google.com/agent-registry/overview>
 - AWS, *AWS Agent Registry in Amazon Bedrock AgentCore (Preview)*, Apr 2026. <https://aws.amazon.com/about-aws/whats-new/2026/04/aws-agent-registry-in-agentcore-preview/>
 - AWS, *Introducing Amazon Bedrock AgentCore Identity: Securing Agentic AI at Scale*, 2026. <https://aws.amazon.com/blogs/machine-learning/introducing-amazon-bedrock-agentcore-identity-securing-agentic-ai-at-scale/>
 - Microsoft, *Agent Registry in the Microsoft 365 Admin Center*, 2026. <https://learn.microsoft.com/en-us/microsoft-365/admin/manage/agent-registry?view=o365-worldwide>
-

协议、标准与 Provider 契约

本节中可能变化的 provider 文档已于 **2026-07-31** 核验。产品行为、可用性、价格、保留时间与计量方式仍然取决于 provider 与日期。

- OpenAI, *Prompt Caching*. <https://developers.openai.com/api/docs/guides/prompt-caching>
 - Anthropic, *Prompt Caching*. <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>
 - Eric Wallace et al., *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*, 2024. <https://arxiv.org/abs/2404.13208>
 - Zuxin Liu et al., *IHEval: Evaluating Language Models on Following the Instruction Hierarchy*, 2025. <https://arxiv.org/abs/2502.08745>
 - Model Context Protocol, *Tools Specification*, 2025-06-18. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
 - OpenAI, *Computer Use*. <https://developers.openai.com/api/docs/guides/tools-computer-use>
 - Anthropic, *Computer Use Tool*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/computer-use-tool>
 - Anthropic, *How Tool Use Works*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>
 - Anthropic, *Vision*. <https://platform.claude.com/docs/en/build-with-claude/vision>
 - OpenTelemetry, *Trace API*. <https://opentelemetry.io/docs/specs/otel/trace/api/>
 - Temporal, *History Service Architecture*. <https://github.com/temporalio/temporal/blob/main/docs/architecture/history-service.md>
 - NIST, *SP 800-53 Rev. 5: Security and Privacy Controls for Information Systems and Organizations*. <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
 - NIST, *Audit — Glossary*. <https://csrc.nist.gov/glossary/term/audit>
 - NIST, *Zero Trust Architecture Glossary*. <https://pages.nist.gov/zero-trust-architecture/glossary.html>
 - Anthropic, *Contextual Retrieval*, 2024 年 9 月。 <https://www.anthropic.com/engineering/contextual-retrieval>
 - Microsoft Azure API Management, *Enable Semantic Caching for Azure OpenAI APIs*. <https://learn.microsoft.com/en-us/azure/api-management/azure-openai-enable-semantic-caching>
 - Microsoft Azure API Management, *Cache Lookup Policy*. <https://learn.microsoft.com/en-us/azure/api-management/cache-lookup-policy>
-

正文使用的其他一手资料

本节中可能变化的文档已于 **2026-07-31** 核验。

- A2A Protocol, *Specification*。 <https://a2a-protocol.org/latest/specification/>
- Patrick Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, 2020。 <https://arxiv.org/abs/2005.11401>
- Tianyu Gao et al., *Enabling Large Language Models to Generate Text with Citations*, 2023。 <https://arxiv.org/abs/2305.14627>
- Nelson F. Liu et al., *Lost in the Middle: How Language Models Use Long Contexts*, 2023。 <https://arxiv.org/abs/2307.03172>
- Mark Chen et al., *Evaluating Large Language Models Trained on Code*, 2021。 <https://arxiv.org/abs/2107.03374>
- OpenAI, *Function Calling*。 <https://developers.openai.com/api/docs/guides/function-calling>
- OpenAI, *Images and Vision*。 <https://developers.openai.com/api/docs/guides/images-vision>
- OpenAI, *Using the Latest Model*。 <https://developers.openai.com/api/docs/guides/latest-model>
- OpenAI, *Your Data / Data Residency Controls*。 <https://developers.openai.com/api/docs/guides/your-data>
- OpenAI, *Models*。 <https://developers.openai.com/api/docs/models>
- OpenAI, *Model Spec*, 2025-10-27。 <https://model-spec.openai.com/2025-10-27>
- Anthropic, *Handle Tool Calls*。 <https://platform.claude.com/docs/en/agents-and-tools/tool-use/handle-tool-calls>
- Anthropic, *Tool Reference*。 <https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-reference>
- Anthropic, *Tool Combinations*。 <https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-combinations>
- Anthropic, *Extended Thinking*。 <https://platform.claude.com/docs/en/build-with-claude/extended-thinking>
- Model Context Protocol, *Server Features Overview*, 2025-06-18。 <https://modelcontextprotocol.io/specification/2025-06-18/server/index>
- IETF, *RFC 9110: HTTP Semantics*, 2022 年 6 月。 <https://www.rfc-editor.org/rfc/rfc9110.html>
- Temporal, *Workflow Execution*。 <https://docs.temporal.io/workflow-execution>
- Temporal, *Events and Event History*。 <https://docs.temporal.io/workflow-execution/event>
- Temporal, *Activities*。 <https://docs.temporal.io/activities>
- Microsoft Azure Architecture Center, *Event Sourcing Pattern*。 <https://learn.microsoft.com/en-us/azure/architecture/patterns/event-sourcing>
- Microsoft Azure AI Search, *Security Filters for Trimming Results*。 <https://learn.microsoft.com/en-us/azure/search/search-security-trimming-for-azure-search>
- Elasticsearch, *Aliases*。 <https://www.elastic.co/guide/en/elasticsearch/reference/current/aliases.html>
- Kubernetes, *Components*。 <https://kubernetes.io/docs/concepts/overview/components/>
- Kubernetes, *Resource Management for Pods and Containers*。 <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>
- Docker, *Building Best Practices*。 <https://docs.docker.com/build/building/best-practices/>
- Microsoft Playwright, *Auto-Waiting and Actionability*。 <https://playwright.dev/docs/actionability>
- Jeffrey Dean and Luiz André Barroso, *The Tail at Scale*, 2013。 <https://research.google/pubs/the-tail-at-scale/>
- Geoffrey Huntley, *Ralph Wiggum as a “software engineer”*。 <https://ghuntley.com/ralph/>
- OpenTelemetry, *Tracing SDK*。 <https://opentelemetry.io/docs/specs/otel/trace/sdk/>
- OpenTelemetry, *Exceptions*。 <https://opentelemetry.io/docs/specs/otel/trace/exceptions/>
- W3C, *Trace Context*。 <https://www.w3.org/TR/trace-context/>
- W3C, *PROV-DM: The PROV Data Model*。 <https://www.w3.org/TR/prov-dm/>
- SPIFFE, *SPIFFE Concepts*。 <https://spiffe.io/docs/latest/spiffe-about/spiffe-concepts/>

- IETF, *RFC 8693: OAuth 2.0 Token Exchange*, 2020 年 1 月。 <https://www.rfc-editor.org/rfc/rfc8693.html>
 - IETF, *RFC 7009: OAuth 2.0 Token Revocation*, 2013 年 8 月。 <https://www.rfc-editor.org/rfc/rfc7009.html>
 - NIST, *AI RMF Core*。 <https://airc.nist.gov/airmf-resources/airmf/5-sec-core/>
 - NIST, *SP 800-61 Rev. 3: Incident Response Recommendations and Considerations for Cybersecurity Risk Management*。 <https://csrc.nist.gov/pubs/sp/800/61/r3/final>
 - NIST, *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*, AI 100-2e2025。 <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-2e2025.pdf>
 - NIST, *IR 7987: Policy Machine*。 <https://nvlpubs.nist.gov/nistpubs/ir/2014/NIST.IR.7987.pdf>
 - NIST, *IR 7987 Rev. 1: Policy Machine — Features, Architecture, and Specification*。 <https://nvlpubs.nist.gov/nistpubs/ir/2015/nist.ir.7987r1.pdf>
 - NIST, *Privacy Framework*。 <https://www.nist.gov/privacy-framework>
 - NIST/SEMATECH, *Confidence Intervals for a Proportion*。 <https://itl.nist.gov/div898/handbook/prc/section2/prc241.htm>
 - NIST/SEMATECH, *Analysis of Paired Observations*。 <https://www.itl.nist.gov/div898/handbook/prc/section3/prc311.htm>
 - NIST/SEMATECH, *Randomized Block Designs*。 <https://www.itl.nist.gov/div898/handbook/pri/section3/pri332.htm>
-

姊妹卷

- *LLM Foundations for Harness Engineering* (本书的模型内部姊妹卷)。 [../llm-foundations-zh/](https://llm-foundations-zh/)
-

阅读列表

- *Awesome Harness Engineering*, walkinglabs. <https://github.com/walkinglabs/awesome-harness-engineering>
- *Learn Harness Engineering* — 英文版, Walking Labs. <https://walkinglabs.github.io/learn-harness-engineering/en/>
- *Learn Harness Engineering* — 中文版, Walking Labs. <https://walkinglabs.github.io/learn-harness-engineering/zh/>
- *Learn Harness Engineering* — *Skills* (英文), Walking Labs. <https://walkinglabs.github.io/learn-harness-engineering/en/skills/>
- *Learn Harness Engineering* — *Skills* (中文), Walking Labs. <https://walkinglabs.github.io/learn-harness-engineering/zh/skills/>

术语表

本书采用的规范词汇。章节链接会指向完整论证及其正文引用；关键协议与标准定义也在此处直接给出来源。

系统边界

模型 (Model) — 接收当前输入 representation、生成 token 或结构化输出的概率组件。它负责提出，不拥有持久状态、credential、外部执行或后果 ([第 1 章](#))。

Agent — 模型参与的一种目标导向循环；周边软件负责组装 context、使用工具、维护状态并检查 outcome。

Agent harness — 紧邻模型、负责组装输入、解析模型输出并驱动一个或多个 agent loop 的系统。它的范围小于整个 product、runtime 或 fleet platform ([第 1 章](#))。

运行时 (Runtime) — 提供 scheduling、queue、durable step、checkpoint、sandbox process、cancellation 与 retry semantics 的执行底座 ([第 10 章](#))。

产品 / 应用 (Product / application) — Agent 工作获得业务意义的用户 workflow、domain logic、review surface 与 business state。

平台 / 控制平面 (Platform / control plane) — 为 registry、identity、policy administration/decision、lifecycle 与多租户治理提供的 fleet 共享基础设施。执行仍需要在每条受保护数据路径上或附近部署不可绕过的 PEP ([第 19 章](#))。

评估 Harness (Evaluation harness) — 创建任务环境、调用 agent harness 运行 trial、收集完整评分证据、执行 grader 并汇总结果的测试系统。它不同于被评估的生产 agent harness ([Anthropic - Demystifying Evals for AI Agents](#))。

Harness engineering — 把 prompt、context policy、tool、runtime control、state、verification 与 operations 当作一个系统迭代，而不是孤立优化某个 prompt。

上下文工程 (Context engineering) — 选择、排序、转换模型调用所需 representation，并追踪其 provenance ([第 3 章](#))。

ETCLOVG — 一种综述分类：Execution、Tools、Context、Lifecycle、Observability、Verification、Governance。本书把它用作组织视角，而不是行业标准 ([第 1 章](#))。

Context、检索、状态与证据

上下文 (Context) — 单次模型调用可见的 token 或多模态 representation。它是有限的，也不是持久 workflow state。

上下文窗口 (Context window) — 一次调用的 model/provider-specific 最大输入加输出跨度；实际可靠性可能在硬上限之前下降 ([Foundations 第 9 章](#))。

单次请求 KV 缓存 (Per-request KV cache) — 一次 generation 内为已经处理的 token 复用 key/value state。模型机制见 [Foundations 第 9 章](#)。

Provider Prompt Cache — Provider 跨独立请求复用合格 prompt prefix 的契约。匹配、TTL、billing 与 data control 都由 provider 决定 ([OpenAI](#)；[Anthropic](#))。它不是 per-request KV cache 的同义词。

应用响应缓存 (Application response cache) — 应用按照自己的 key 与 invalidation 规则，对完全相同且 scope-compatible 的请求复用最终响应。

语义缓存 (Semantic cache) — 对语义相似请求复用响应。Similarity 只是一个 gate；tenant、user/auth scope、model、prompt、tool、retrieval、policy version、freshness 与 risk 也必须兼容 ([Azure semantic caching](#)；[Azure cache policy](#))。

检索管线 (Retrieval pipeline) — Source ingestion → parse/chunk → metadata 与 ACL → versioned index → query → dense/lexical retrieval → fusion/reranking → context assembly → citation/provenance → outcome evaluation ([第 4 章](#))。

证据条目 (Evidence item) — 检索内容与 source_id、source_version、chunk_id、content_hash、index_version、acl_scope 等标识一起传递的单元，使 claim 可追溯、权限可复查。

Grounding — 回答或 action 与所提供 evidence 之间的关系。Retrieval correctness、grounding/citation correctness 与最终 task outcome 是不同测量层。

压缩 (Compaction) — 有损 context transformation: 把选定的 decision、constraint、open work、artifact pointer、provenance 与 uncertainty 保存在较小 representation 中 (第 5 章)。Foundations 第 9 章解释 context 为何有限, 但不定义 compaction algorithm。

记忆 (Memory) — 为未来调用可能使用而保存的信息产品。它需要 scope、provenance、authority、freshness、update、access 与 forgetting 规则。

执行状态 (Execution state) — 当前 step、pending action、budget、lease、approval status 等权威结构化 workflow state (第 10 章)。

事件历史 (Event history) — 在明确 replay contract 下, 足以恢复 workflow 的有序持久 accepted-event 记录。Temporal 把 Event History 作为 workflow 的恢复记录 (Temporal)。

检查点 (Checkpoint) — 位于已知 history position 的可恢复状态, 以及继续执行所需的引用、版本与 resume contract。它不是完整因果历史。

产物 (Artifact) — 文件、diff、报告、dataset、build、screenshot 或 test result 等可寻址工作产物。

Trace — 由 span、event、attribute、status、link 与 timestamp 组成的 observability 数据; 它可以被 sampling (OpenTelemetry Trace API)。Trace 不会自动成为 event history、eval trajectory、cost ledger 或 audit record。

Eval Transcript / Trajectory — 为一次 trial 评分而保存的完整 model、tool、observation、artifact 与 outcome 记录。只有 grader 所需证据完整时, 才能由 tracing 派生 (第 11 章)。

审计记录 (Audit record) — 具有明确 content、identity、timestamp、integrity、access、review 与 retention 的受保护问责记录。NIST 把这些属性分别列为控制要求 (NIST SP 800-53 Rev. 5)。

血缘 (Lineage) — 连接 identity、version、input、artifact、policy decision、approval、action 与 verified outcome 的 provenance graph。

上下文重置 (Context reset) — 从结构化 handoff 开始新的模型 context, 而 durable execution state 与 artifact 仍保存在模型外。

上下文防火墙 (Context firewall) — 把 worker 的中间 context 与 parent 隔离。它能减少 context pollution, 但不会提高 worker 的信任或权限; 返回 claim 仍需 evidence 与 verification。

工具、授权与运行时执行

工具调用 / 动作提议 (Tool call / action proposal) — 指明工具与参数的结构化模型输出。Application code 决定是否以及如何执行 (Anthropic - How Tool Use Works)。

调用生命周期 (Invocation lifecycle) — Proposal → stream completion → parse → schema validation → semantic validation → authorization → 必要时 mandatory approval → idempotency/dedup → execution → timeout/cancellation handling → result normalization → observation → outcome confirmation (第 6 章)。

Schema validity — Action 是否符合声明的 machine-readable shape。它不建立 semantic correctness、authorization 或安全后果。

Semantic validity — 规范化参数对当前 task、resource 与 state 是否合理。

执行结果 (Execution result) — Executor 对 attempted action 返回的内容。即使 intended environment outcome 没有发生, 它也可能返回成功; timeout 后也可能是 unknown。

Outcome — Action 或 trial 之后经过验证的 environment/business state; 不同于模型自述与 executor 的即时响应。

幂等键 (Idempotency key) — Caller 提供给兼容 service、用来 deduplicate 等价 mutation attempt 的标识。它可以减少重复 effect, 但不能替代 outcome verification。

MCP (Model Context Protocol) — 一种 client/server protocol; server 在自身实现与 access control 下暴露 tool 等 capability。Tool list 可以变化, 因此 discovery 不能替代 invocation-time authorization (MCP Tools Specification)。

A2A (Agent-to-Agent protocol) — Agentic application 之间进行 delegation 与 coordination 的协议边界; 它不同于向单个 agent runtime 暴露 tool 的边界。

沙箱 (Sandbox) — 具有明确 filesystem、network、credential、process 与 persistence 边界的隔离执行环境。它限制 blast radius, 并建立允许已授权 action 运行的区域 (第 7 章)。

能力授予 (Capability grant) — 在给定 identity、purpose、environment、audience 与 expiry 条件下, 对某项 resource operation 的 scoped authority。

委托授权 (Delegated authorization) — 从 user/service 派生的短期、audience-bound、purpose-bound authority。继续 delegation 可以收窄，但不能静默扩大权限。

指令优先级 (Instruction priority) — 模型可见 instruction 的预期行为顺序。它是 defense in depth，不是 authorization 或执行保证；hierarchy evaluation 研究仍会测量 conflict failure ([Instruction Hierarchy](#)；[IHEval](#))。

模型可见策略 (Model-visible policy) — 放入模型 context、用于引导行为并帮助模型提出合规 action 的 instruction 与 explanation。

可执行策略 (Executable policy) — 由外部软件在受保护 action path 上计算并执行的规则。

策略决策点 (Policy decision point, PDP) — 根据 subject、action、resource、purpose、environment 与 policy 计算决策的组件 ([NIST Zero Trust Glossary](#))。

策略执行点 (Policy enforcement point, PEP) — 对 protected-resource request 不可绕过地落实 policy decision 的组件。Prompt 不是 PEP ([NIST Zero Trust Glossary](#))。

建议型 Hook (Advisory hook) — 补充 context、告警或要求模型重新考虑，却不一定阻止 dispatch 的 lifecycle hook。

阻断执行 (Blocking enforcement) — 每个相关 action 必经、deny 会阻止 dispatch、且没有替代路径可以绕过的 control。

审批请求 / 咨询 (Approval request / consultation) — 模型或 workflow 提出的人类输入请求。它是有用的 orchestration，但本身不是硬安全 gate。

强制审批门 (Mandatory approval gate) — Policy 通过 runtime/PEP 在 protected dispatch 前强制插入的 gate，与模型是否主动请求无关。Approval 绑定 reviewer 看到的 identity、resource、normalized arguments、policy version、constraint 与 expiry ([MCP Tools Specification](#))。

致命三要素 (Lethal trifecta) — Private-data access、untrusted content 与 external communication 三者组合的实践者术语。它是 threat-model heuristic，不是正式安全标准 ([第 7 章](#))。

路由与 workflow 控制

路由器 (Router) — 在执行 target 前，按照声明的 quality、cost、latency、capability 与 governance feature 选择 route target 的 policy。

路由目标 (Route target) — Model + adapter + tool contract + reasoning policy 的组合，而不只是 model name。

质量级联 (Quality cascade) — 先尝试较低成本 target，评估结果后，在 acceptance signal 拒绝时升级到更强 target。

可靠性回退 (Reliability fallback) — Timeout、rate limit、provider error 等 availability failure 后切换 target；切换前必须检查 compatibility 与 side effect。

对冲 (Hedging) — 启动多个兼容 attempt 来降低 tail latency，并明确 winner、cancellation、cost 与 side-effect semantics。

人工升级 (Human escalation) — 把 uncertainty、incompatibility、policy conflict 或高 consequence 交给人，而不是静默换模型。

兼容性门 (Compatibility gate) — 检查 fallback target 的 modalities、context limit、tool protocol/schema、structured output、continuation state、safety policy、latency envelope 与 data region 是否兼容 ([第 8 章](#))。

推理策略 (Reasoning policy) — 对 reasoning effort、continuation、token accounting 与 allowed disclosure 的 provider-specific control/budget。它不是 hidden reasoning 的可移植 representation。

确定性工作流 (Deterministic workflow) — 外部 code 拥有 sequencing 与 state；model call 填充有界 step。

模型驱动循环 (Model-directed loop) — 模型选择下一项 proposed action；harness 负责 state、authorization、execution、stop rule 与 outcome check。

混合工作流 (Hybrid workflow) — 在确定性结构内部放置有边界的 model-selected branch 或 loop。

Orchestrator-workers — Orchestrator 提出分解并委托有边界工作；外部 state 与 handoff contract 保存 evidence 与 recovery 信息的模式 ([第 9 章](#))。

评估与验证

任务 / 试验 (Task / trial) — Task 定义 input、environment、constraint 与 success criteria；trial 是对该 task 的一次 attempt。

评分器 (Grader) — 测量 trial 某一项声明属性的 code-based、model-based 或 human component。

过程 / 轨迹评分 (Process / trajectory grading) — 检查 action、policy use、tool argument、budget 或 recovery behavior，不假定 final artifact 正确。

产物评分 (Artifact grading) — 检查 code、report、dataset 等可寻址 output。

环境状态评分 (Environment-state grading) — 检查执行后的实际外部状态，而不是相信 agent final message。

用户 / 业务结果评分 (User / business outcome grading) — 测量对用户或组织真正重要、可能延迟出现的产品结果。

pass@k / pass^k — pass@k 测量 k 次 attempt 中至少一次成功的概率；pass^k 测量全部 k 次一致成功。应按 product contract 选择指标并报告 uncertainty ([Foundations 第 13 章](#)；[第 11 章](#))。

验证器层级 (Verifier hierarchy) — Schema check → deterministic test/linter → environment outcome check → same-agent critique → independent model grader → human review。按 consequence、ambiguity、correlated failure、latency 与 cost 选择；并非所有任务都必须使用另一个 agent ([第 13 章](#))。

基础设施噪声 (Infrastructure noise) — Hardware、resource limit、disk/network behavior、image/dependency、cache、parallelism 或 timeout 导致的分数变化，而不是被测 agent change 的影响 ([Anthropic - Infrastructure Noise](#))。

发布证据包 (Release evidence packet) — 用于 release decision 的 pinned model-plus-harness configuration、per-slice trial result、uncertainty、grader version/calibration、failure、policy check 与 approval。

长运行任务、人类交互与 Computer Use

持久执行 (Durable execution) — 持久化 progress，使 run 能在 process、worker、network 或 infrastructure failure 后恢复。Replay 复用 recorded model/tool output，而不是重新生成 ([第 10 章](#))。

交接 (Handoff) — 包含 completed work、verified evidence、open risk、next action、artifact pointer 与 permission context 的 transfer package。

停止规则 (Stop rule) — 在 success、budget exhaustion、no progress、repeated failure、policy deny、human cancel 或 terminal environment state 时明确终止。

动作预算 (Action budget) — 同时覆盖 token、dollar、tool call、wall time、external spend 与 retry/fallback 的硬限制。

引导事件 (Steering event) — 在明确 acceptance point 后改变未来 planning 的 recorded instruction；它不会追溯取消已经 dispatch 的 action。

取消事件 (Cancellation event) — 停止未来工作并取消 compatible in-flight operation 的 recorded request；已经 commit 的 effect 可能需要 compensation。

Computer-use 工具循环 (Computer-use tool loop) — 模型提出 structured computer action；application 执行；screenshot 或其他 UI state 作为 tool result/observation 返回 ([OpenAI](#)；[Anthropic](#))。

截图观察 (Screenshot observation) — 某一时点捕获的 pixel evidence。执行时它可能已经过期，也不能单独证明 post-action outcome。

界面身份 (Surface identity) — Observation 与 proposed UI action 所绑定的 application、process、window/tab、frame、URL/origin、viewport、geometry 与 capture version。

观察 / 动作竞态 (Observation/action race) — Interface 在 observation 之后、dispatch 之前或期间发生变化，使基于状态 s_0 grounding 的 action 针对状态 s_1 执行。Freshness check 与 observation barrier 用于控制该竞态 ([第 15 章](#))。

视觉 / 坐标 Grounding (Visual / coordinate grounding) — 把 intended UI target 映射到当前 interface state 中的 coordinate、element 或 action。

Accessibility tree — UI role、label、state 与 relationship 的结构化视图；可用时很有帮助，但不保证覆盖全部可见或可执行状态。

运维、Fleet 与治理

熔断器 (Circuit breaker) — 在 failure 达到阈值后使不健康 dependency 的后续 call 快速失败、避免 retry storm 的 runtime control。

终止开关 (Kill switch) — 通过 run lifecycle、credential revocation 与 PEP denial 实现的人类或 policy stop，而不是只发送自然语言 instruction。

Span telemetry — Model call、tool call、retrieval、context assembly、policy、cost 与 outcome 的 trace span/attribute。

Trace-to-eval loop — 把经过 redaction、consent/licensing review 的 production failure 变成具有完整 outcome assertion 的可复现 regression task。

成本账本 (Cost ledger) — 与 run、action、provider、tool 与 external spend 关联的完整、未 sampling cost event。Sampled trace 本身不能支撑完整 ledger。

发布单元 (Release unit) — Model、prompt、tool schema/implementation、retrieval configuration/index、memory policy、sandbox image、policy、grader 与 budget 的版本化组合。

Agent 身份链 (Agent identity chain) — 相互连接的 agent version、sponsor、delegator、tenant、purpose、runtime/workload identity、credential audience、expiry 与 revocation state。

Agent Registry — Agent definition、version、owner、capability、requested authority、attestation、dependency 与 deployment state 的受治理 source of truth。

Agent Fleet — Agent definition、release、run、runtime、identity、tool 与 owner 的受治理集合，而不是单一 local agent instance。

AgentOps — 围绕 service health、agent outcome、safety/policy event、cost、privacy、release、incident 与 human escalation 的运营 discipline (第 18 章)。

不可否认性 (Non-repudiation) — 把 request 或 authorization 归因到 workload/policy identity 的 integrity-protected evidence。它不证明人类理解了 approval，也不证明模型 rationale 真实。

Foundations → Harness 知识衔接表

本表说明 *LLM Foundations* 中的模型侧概念，在本书中会转化成哪项工程责任。“交接”并不表示 harness 改变了模型机制，而是标出在真实系统中使用该机制时，周边系统必须承担的控制。

Foundations 概念	模型侧边界	Harness 工程责任	继续阅读
Token 与模型输出	模型生成 token 概率与结构化输出；它不会直接造成外部效果。	解析提议、保存 correlation ID、授权 dispatch、执行并验证 outcome。	第 1 章、第 6 章
Tokenization	文本与多模态输入消耗 model-specific budget；可见字符数或图片尺寸不是可移植的 token 估算。	按所选 provider/model 契约计量，并执行 context 与视觉输入预算。	第 3 章、第 15 章
Next-token prediction	流畅续写不能证明事实、授权、执行或任务完成。	把 proposal 与 effect 分开，按 evidence 或 environment state 评分，而不是按措辞自信度评分。	第 1 章、第 11 章
Attention	Attention 只作用于当前模型输入，不会形成持久记忆。	选择、排序、压缩、检索 context 并追踪 provenance；在模型外保存持久状态。	第 3 章、第 5 章、第 10 章
数据与 Scaling	训练覆盖与 scale 影响能力，但不能建立具体任务的 readiness。	在有代表性的任务与风险 slice 上评估精确的 model-plus-harness release。	第 11 章、第 18 章
推理与采样	Sampling 使不同 attempt 具有变异；constrained decoding 可以约束形状，但不能保证语义有效或获得权限。	运行重复 trial、验证语义，并在输出语法之外独立 gate 副作用。	第 6 章、第 11 章
Post-training	学得的交互惯例与 instruction following 属于行为属性，不是 access-control 保证。	在 context 中表达 instruction priority，但通过 runtime policy 与 PEP 执行受保护 action。	第 2 章、第 7 章
Prompting 与 In-context Learning	Prompt 结构可以引导行为；不可信文本仍可能通过 prompt injection 操纵模型。	追踪 assembly provenance，把检索内容与工具结果视为不可信输入，缩小权限，并在 prompt 外执行策略。	第 2 章、第 4 章、第 7 章
Context Window 与缓存	Context 有限；单次请求内的 KV state 不等于 provider 跨请求 prompt cache 的契约。OpenAI 与 Anthropic 都分别规定了 provider-specific eligibility 和 retention 行为（OpenAI、Anthropic）。	预算并转换 context，在有益时保留 cache-compatible prefix，并分别限定 response/semantic cache 的 scope 与失效条件。	第 3 章、第 5 章、第 18 章
知识、幻觉与不确定性	看似可信的回答、citation-shaped string 或口头不确定性都不是经过验证的 evidence。	检索有权限的 evidence，把 citation 绑定 source version，支持 abstain，并分别评估 grounding 与 outcome。	第 4 章、第 11 章、第 13 章
Embedding 与检索	Embedding similarity 与最小 RAG 不负责 ingestion、ACL、freshness、deletion 或 provenance。	负责生产检索数据路径，并把 retrieval、grounding、citation 与 task outcome 分层评估。	第 4 章
推理、工具与 Agent	模型可以提出结构化 tool/computer action；application 执行后把结果返回模型（Anthropic tool-use protocol）。	完成调用生命周期、执行权限、控制 retry 与副作用，并确认最终 environment state。	第 6 章、第 7 章、第 15 章
评估模型行为	模型级 task、trial、grader 与 pass metric 本身不能测量工具执行、环境状态或运营可靠性。	建设 evaluation harness：调用生产 agent harness、记录 transcript/outcome、隔离环境并报告系统级 slice。	第 11 章、第 16 章、第 17 章
运行时心智模型	模型负责预测；它不拥有 context selection、持久状态、权限、真实工具效果或后果。	在 harness、runtime、product、control plane、enforcement point 与 evaluation harness 之间明确分配这些责任。	第 1 章、第 19 章、第 20 章

如何理解边界

三条规则可以避免大多数类别错误：

1. **Model output** 可以提出 action；**execution result** 表示 executor 返回了什么；经过验证的 **outcome** 表示环境实际发生了什么变化。
 2. **Context** 是当前 call 的输入；**memory** 是供未来检索的信息产品；**execution state** 与 **event history** 支撑 workflow progress 与 recovery。
 3. 模型可见的 instruction 可以降低不良行为概率；只有不可绕过的 runtime control 才能提供执行保证。
- 双语版本采用的规范定义见[术语表](#)。